

LG AI Research Special Lecture

딥러닝 자연어처리 기초와 LLM 에이전트

Lecture 01 – AI의 첫걸음, 머신러닝과 딥러닝의 기초



중앙대학교 AI학과

이환희

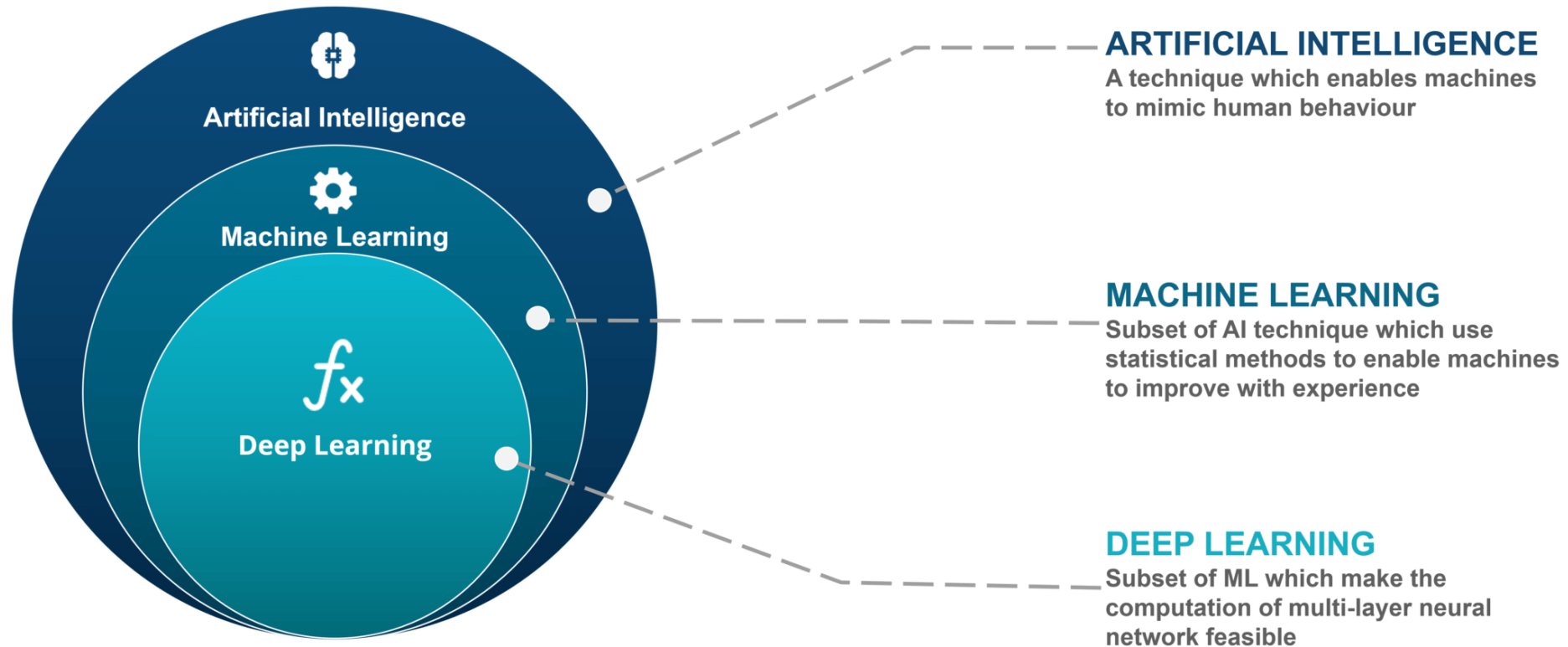
Outline

AI의 첫걸음, 머신러닝과 딥러닝의 기초

- The AI Landscape and Machine Learning
- Types of Machine Learning
- Deep Learning Fundamentals

Machine Learning

- **Machine Learning:** statistical algorithms that can effectively generalize and thus perform tasks without explicit instructions.

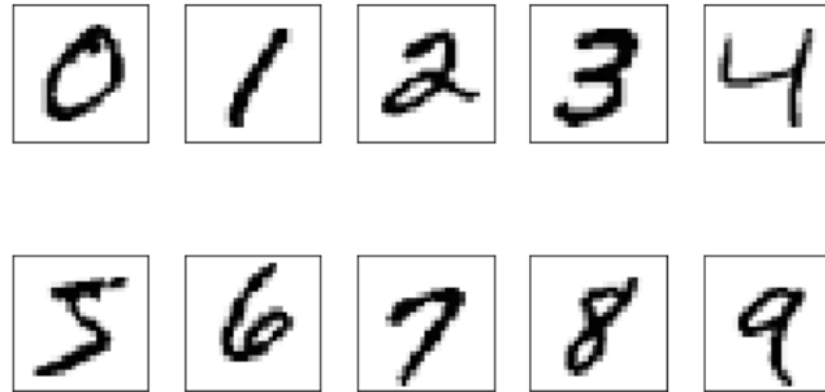


Learn from Experience

“A computer program is said to **learn** from experience E with respect to some class of tasks T and performance measure P , if its performance at tasks in T , as measured by P , improves with experience E .”

— Tom Mitchell, Professor at Carnegie Mellon University

Handwriting Recognition Example:



Task T : classifying handwritten digits from images

Performance measure P : percentage of digits classified correctly

Training experience E : dataset of digits given classifications, e.g., MNIST

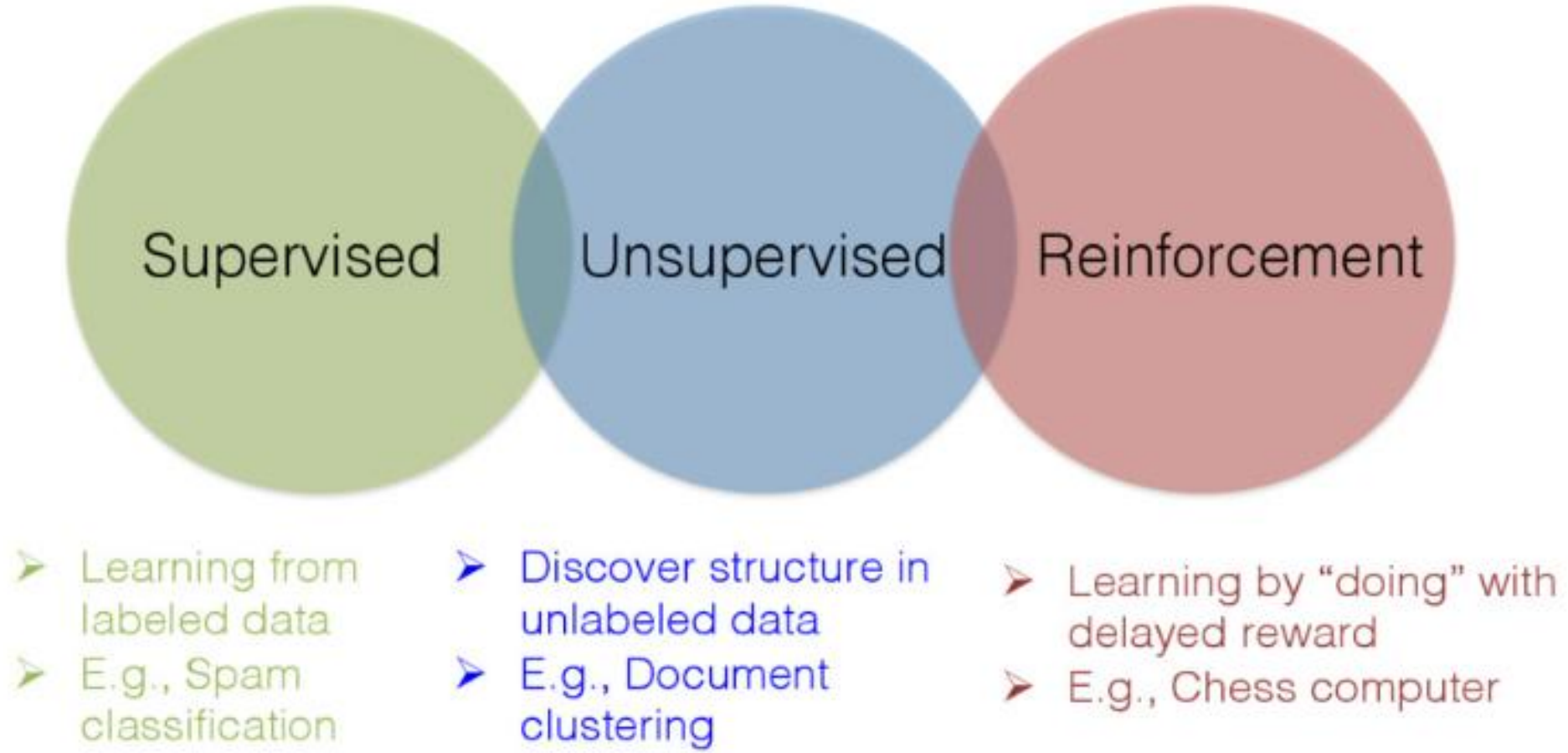
Machine Learning?

- Sub-field of AI that relates to making computers learn from experience.
- Requires:
 - **Data**
 - Used in learning (training) how to accomplish some task
 - Features
 - **Algorithm**
 - Improves automatically through experience (training)

Types of Machine Learning

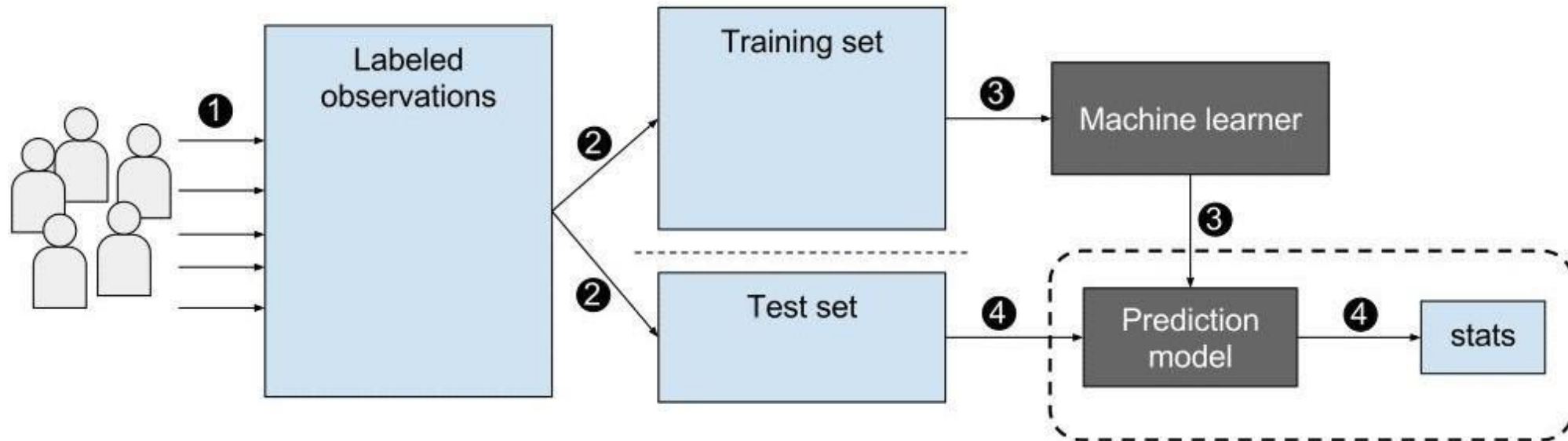
- Algorithm requires a **feedback** during training (**learning**) phase.
- Machine Learning types, by the nature of that feedback:
 - Supervised
 - Unsupervised
 - Semi-supervised
 - Reinforcement

Three Types of Machine Learning

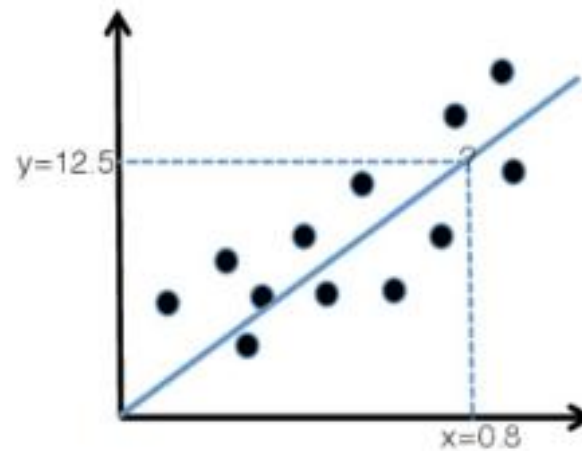
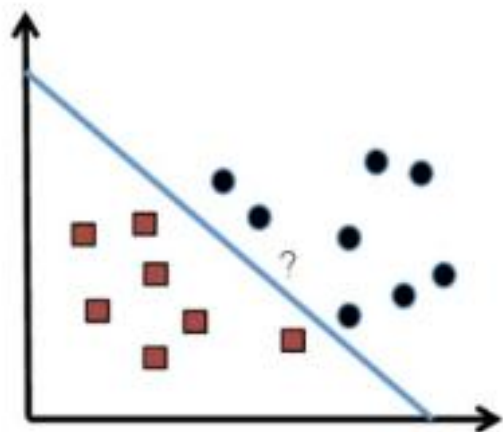
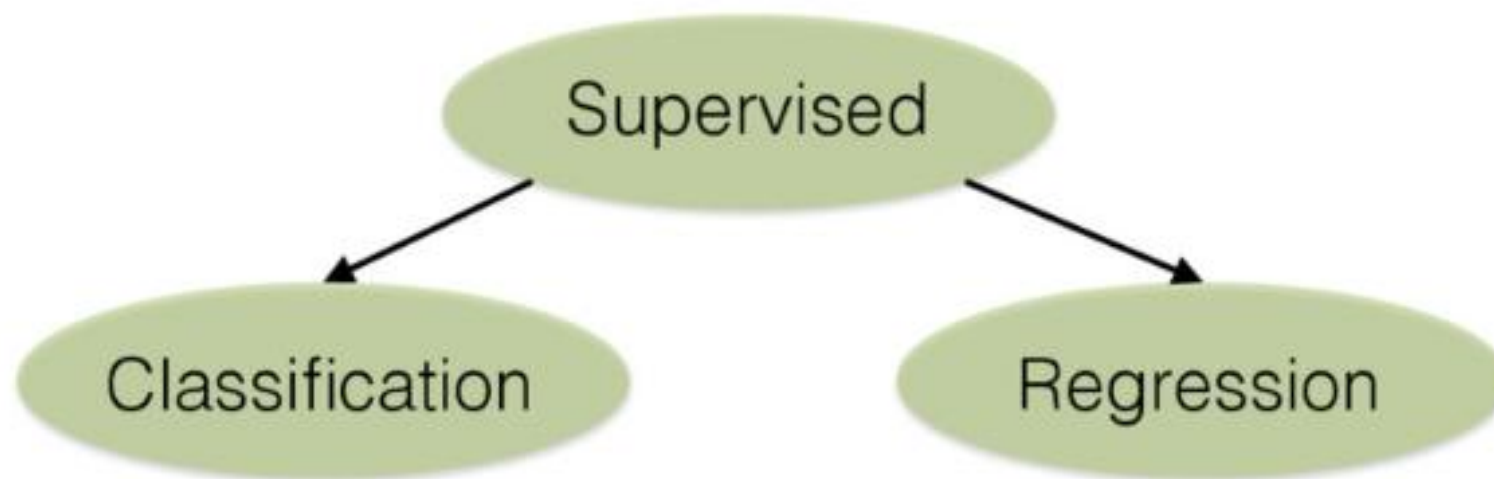


Supervised Machine Learning

- Training data: labeled (**ground truth** - expected algorithm result; expensive!)
- Training: Algorithm uses labels to evaluate its accuracy on training data.
- Risk of **overfitting**.



Supervised Learning



Supervised Machine Learning

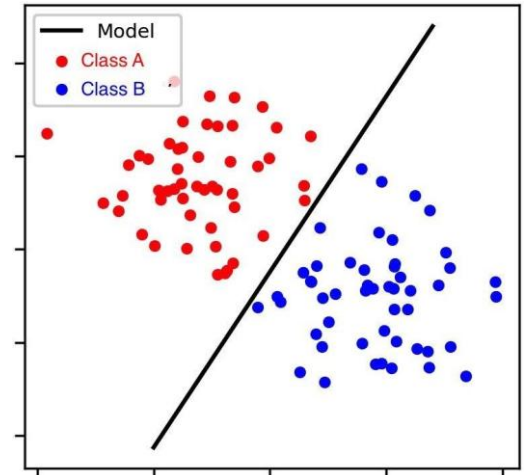
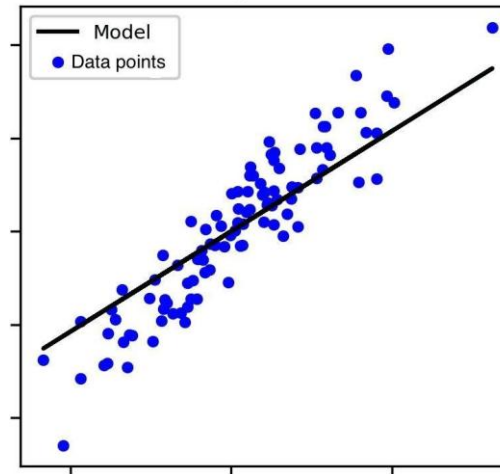
- Two types of problems it tries to solve:

- **Regression**

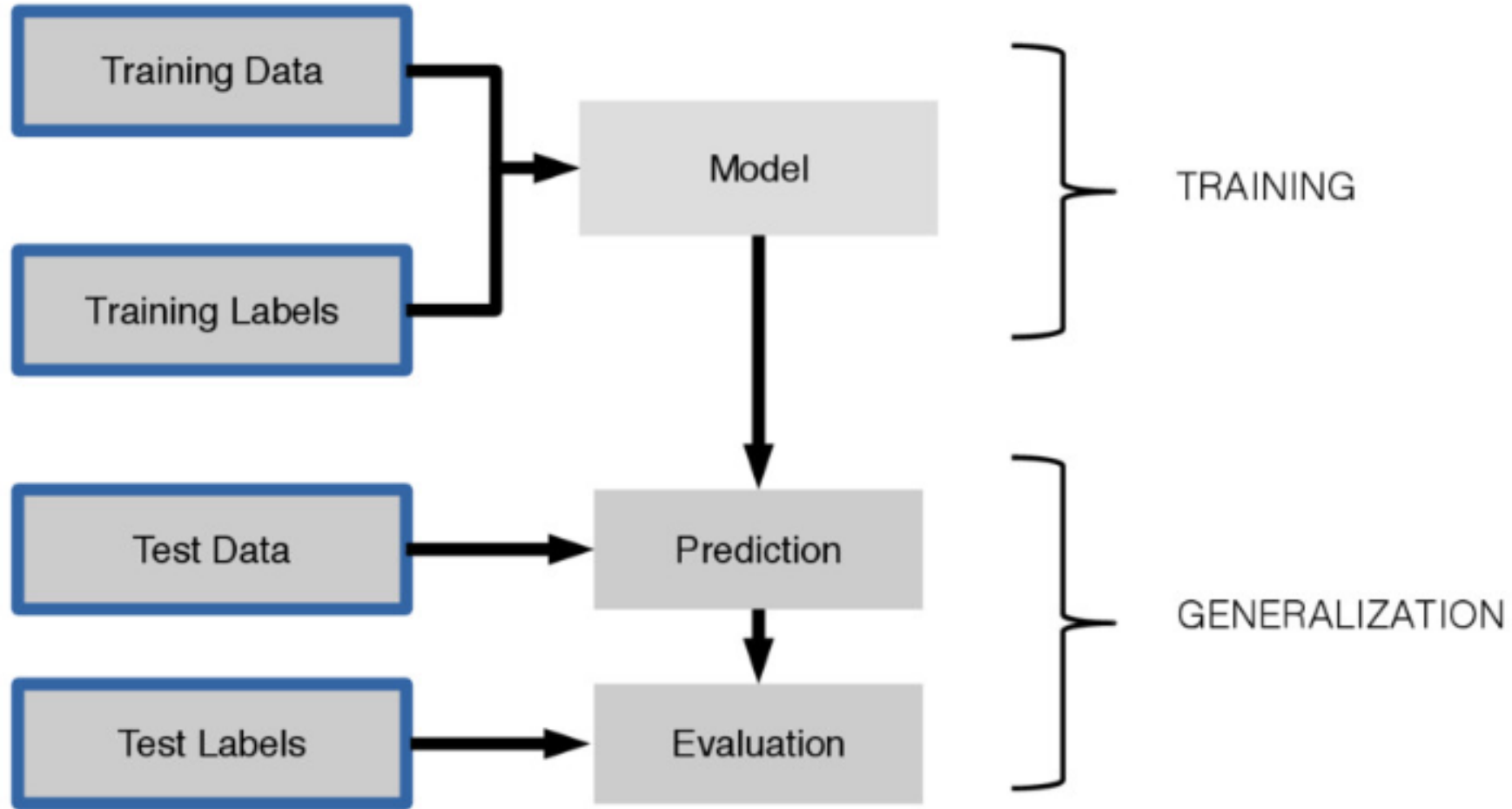
- Predict numerical (continuous) value
- Linear, Nonlinear Regression

- **Classification:**

- Predict categorical (discrete) value
- Naive Bayes Classifier, *Support Vector Machines*, Logistic Regression, ...
- Decision Tree, Random Forest, k-NN, Neural Networks, etc...can solve both problems



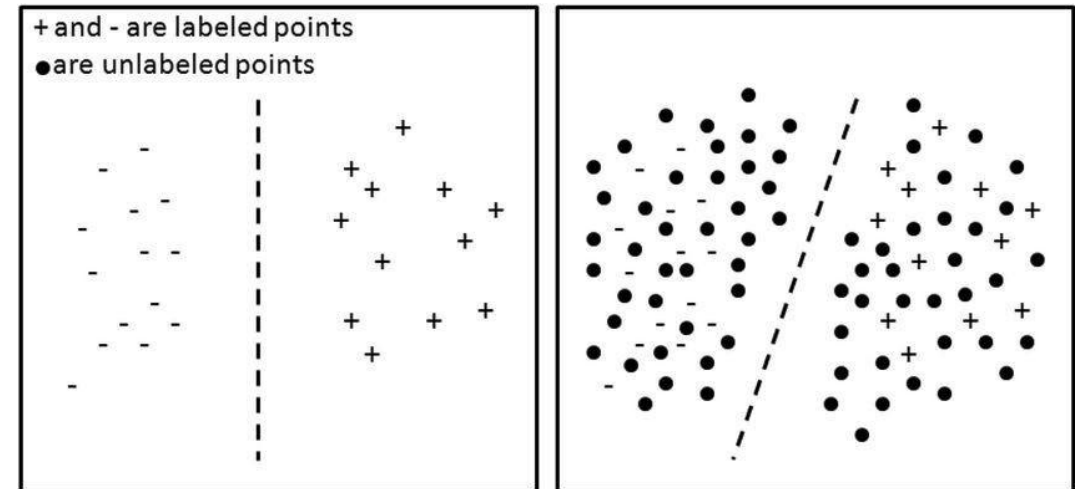
Supervised Learning Workflow



➤ Fit model on all data after evaluation

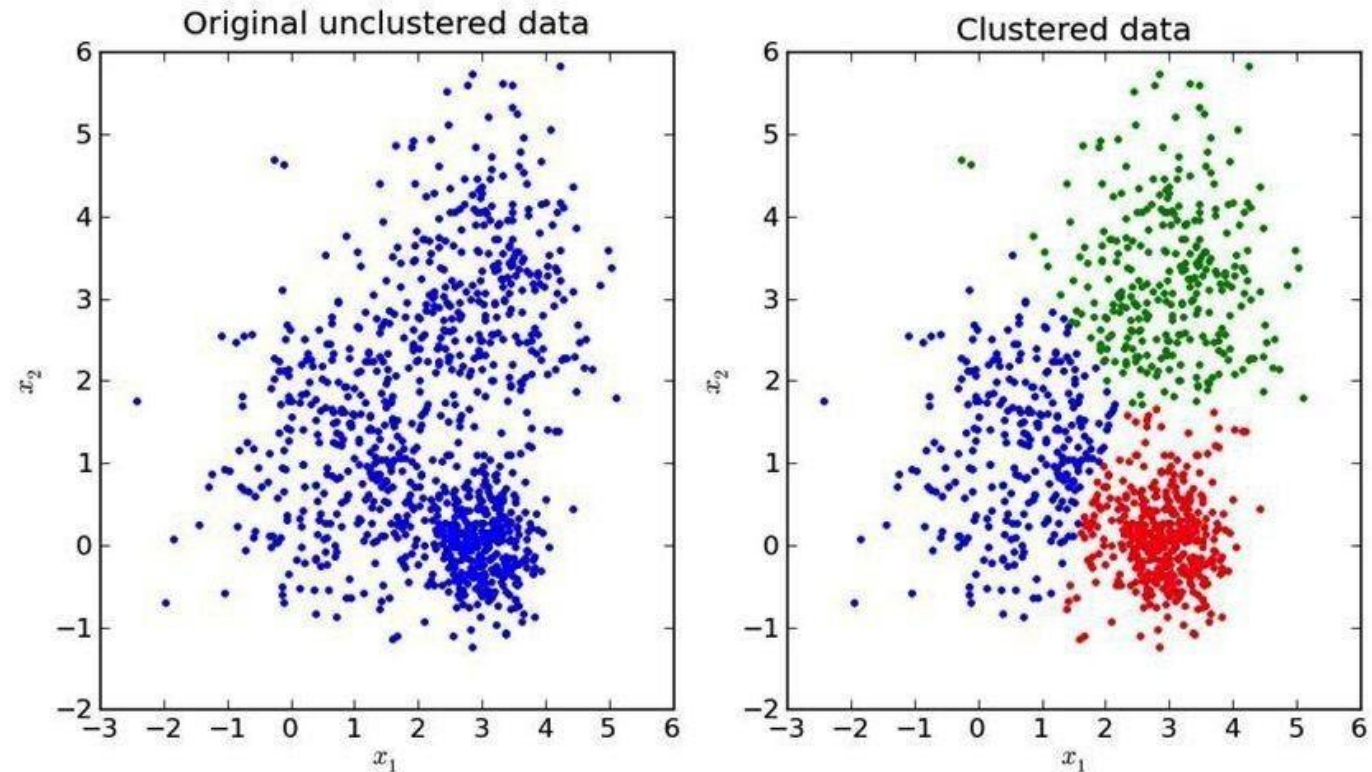
Semi-supervised Machine Learning

- Hybrid learning - between supervised and unsupervised
- Solves the problem of having not enough labeled data to train a supervised learning algorithm
- **Training data:** small labeled and large unlabeled data set
- **Training:**
 - Train model with labeled data
 - Trained model used to predict
 - labels for unlabeled data =>
 - **pseudo-labeled data**
 - Retrain model with both pseudo- and labeled data sets

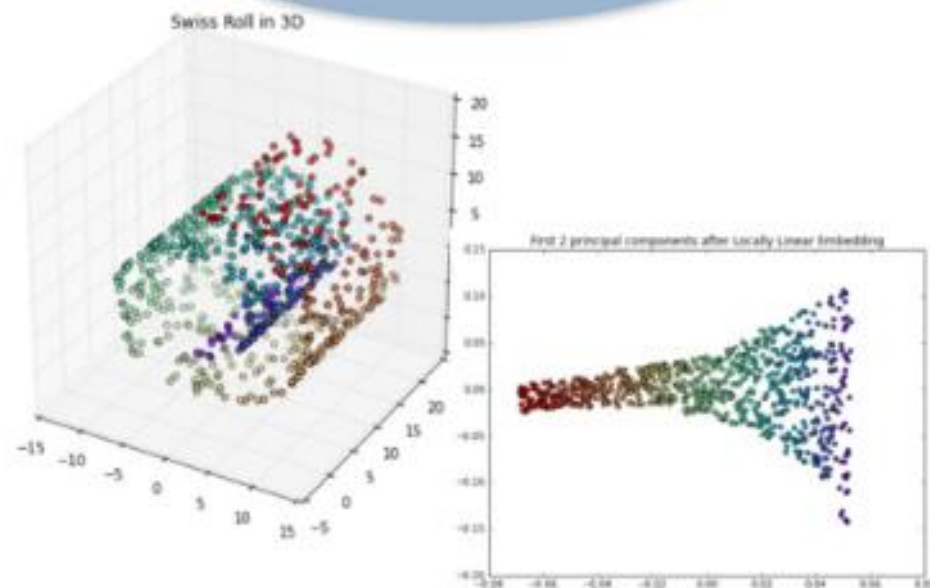
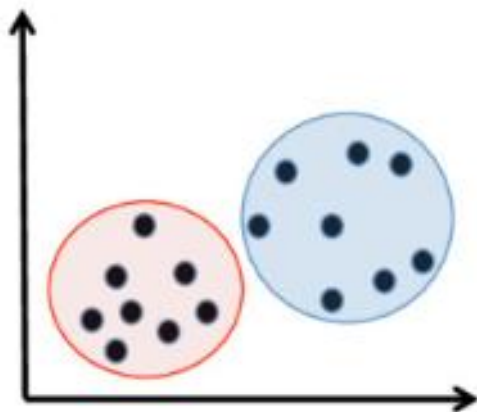
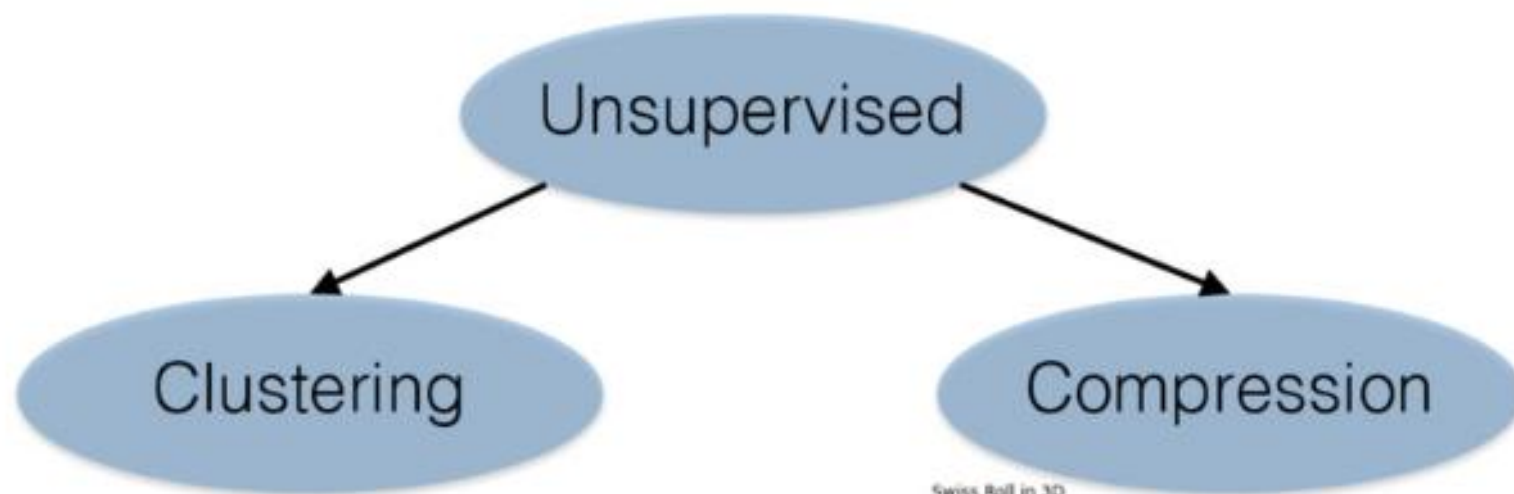


Unsupervised Machine Learning

- Training data: Unlabeled data
- Training:
 - Extract **features** and patterns from data itself
 - **Clustering**: these features used to label and classify the data into clusters
- Methods: k-Means clustering, ...

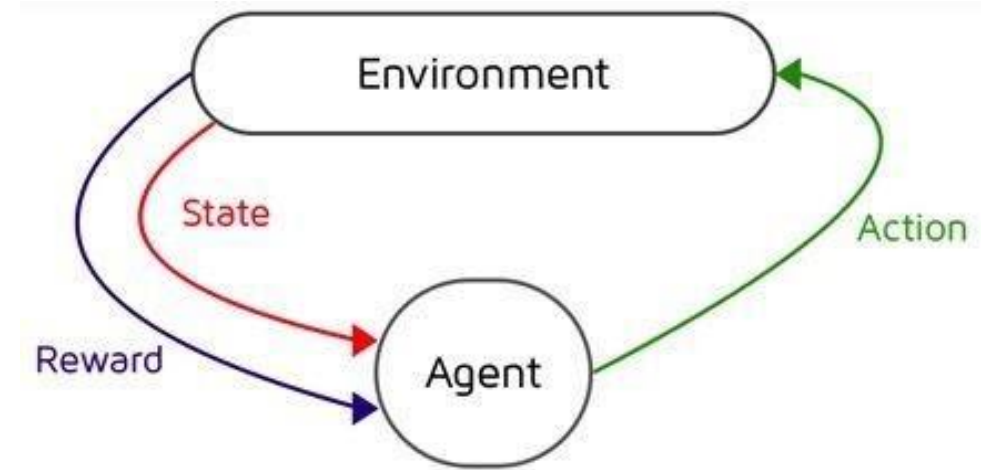


Unsupervised Learning



Reinforcement Machine Learning

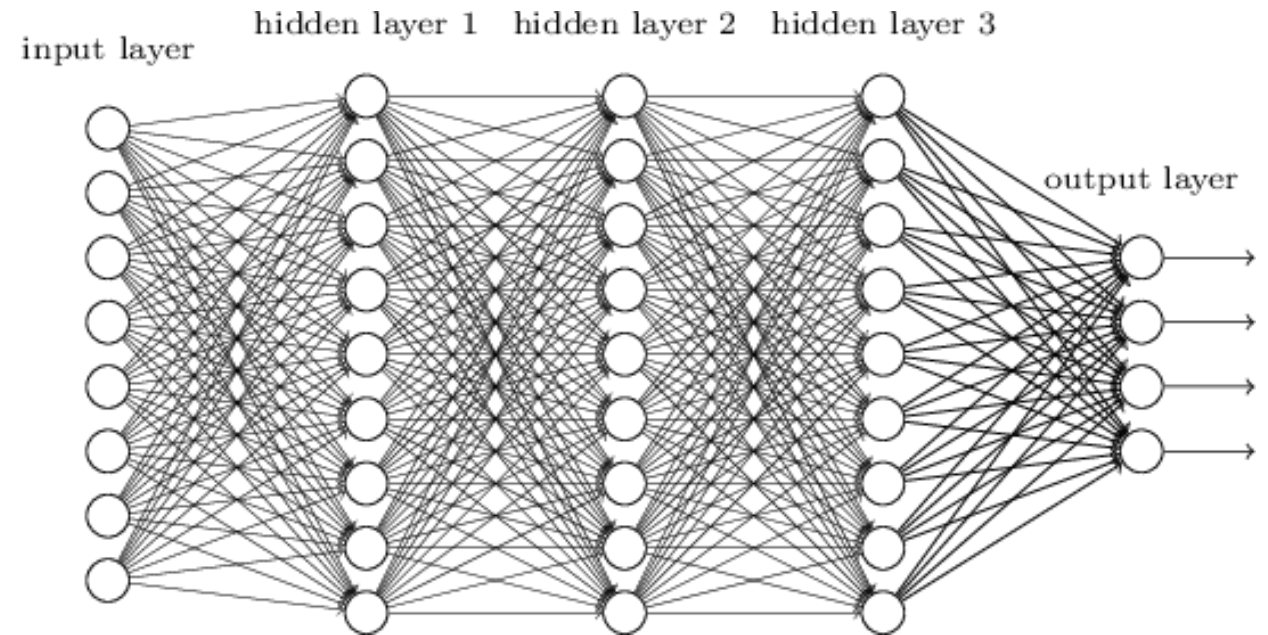
- Training data: none
- Training:
 - Machine trained to make specific decisions
 - Machine interacting with its environment
 - Trial and error
- **Reward system:** providing feedback when an artificial intelligence agent performs the best action in a particular situation
- Sequence of successful outcomes is reinforced to develop the best solution for a given problem.



What is Deep Learning?

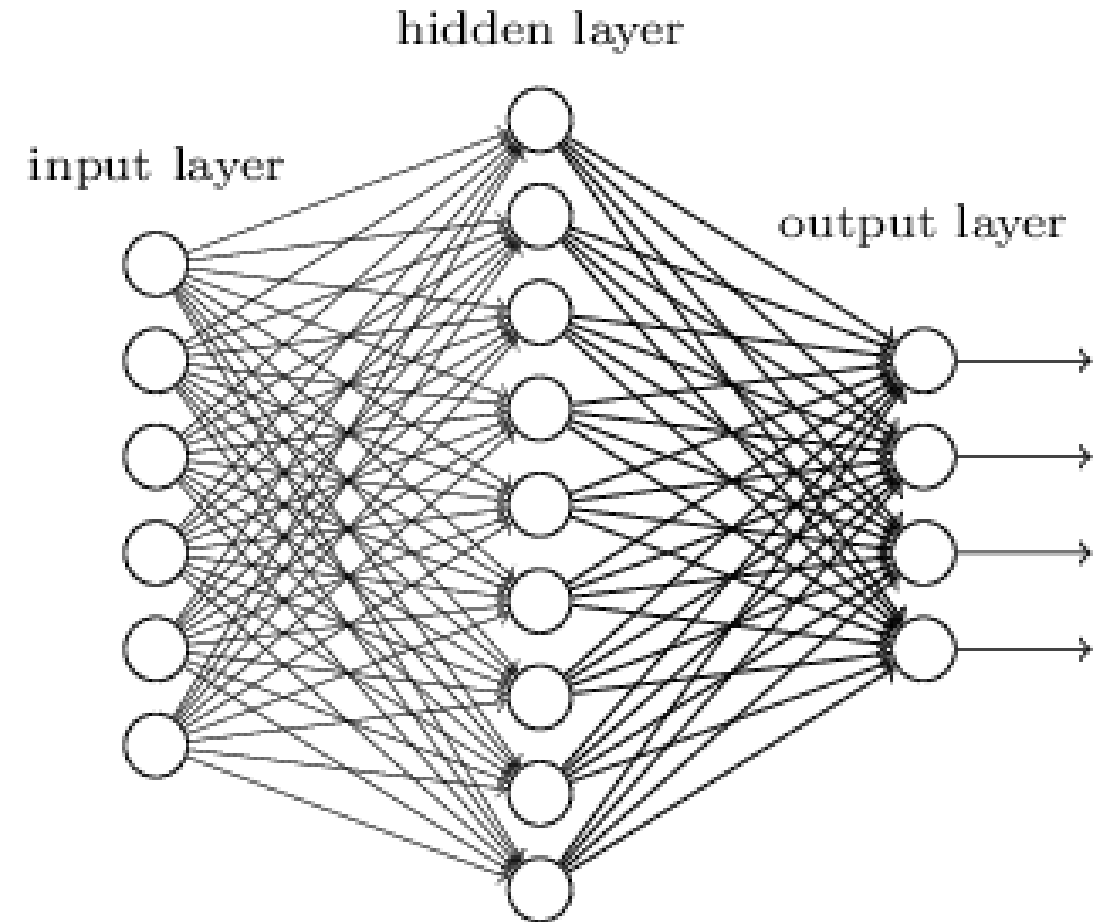
- **Deep Neural Networks** - more than one hidden layer (vs **Shallow Network**)

- With each new hidden layer system becomes
 - more intelligent;
 - increases capabilities to learn: **new** features and **more complex** features
- Each feature reflects one detail from the input



Artificial Neural Network

- Collection of simple, trainable, interconnected mathematical units (neurons) that collectively learn complex function
- System of interconnected **layers**:
 - **input**: numeric representation of the data
 - **hidden**: nonlinear function (**activation function**) of the sum of weighted inputs from the previous layer plus bias
 - **output**: prediction - set of values (continuous $\Rightarrow$ model solves **regression** problem; discrete $\Rightarrow$ **classification**)

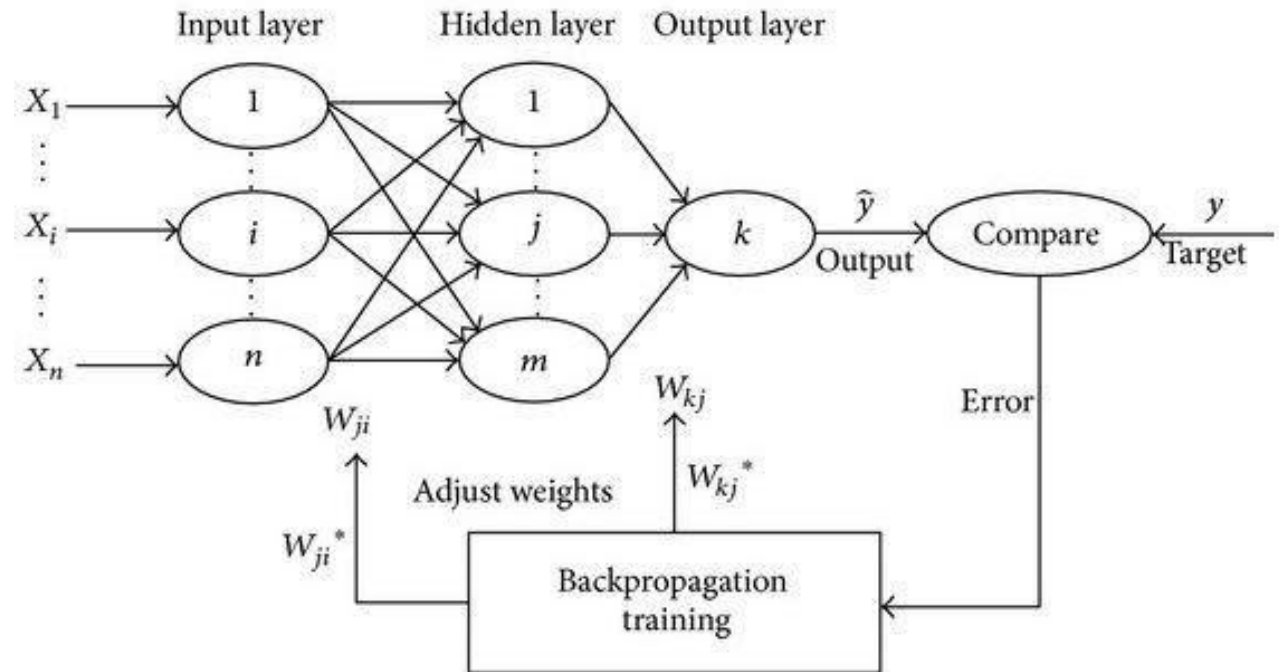


$$y = F(w_1 \times x_1 + w_2 \times x_2 + \dots w_N \times x_N + b);$$
$$F(x) = \max(0, x); (\text{ReLU})$$

x_i - input, w_i - weight

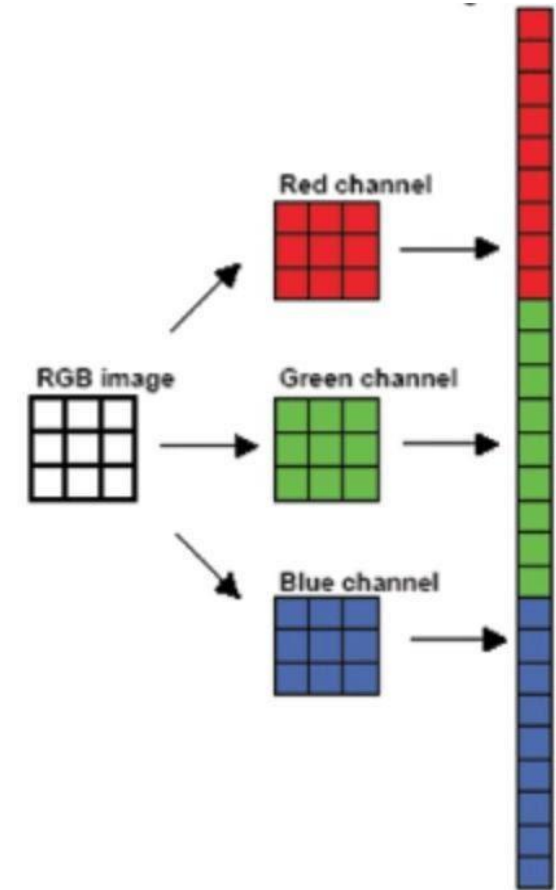
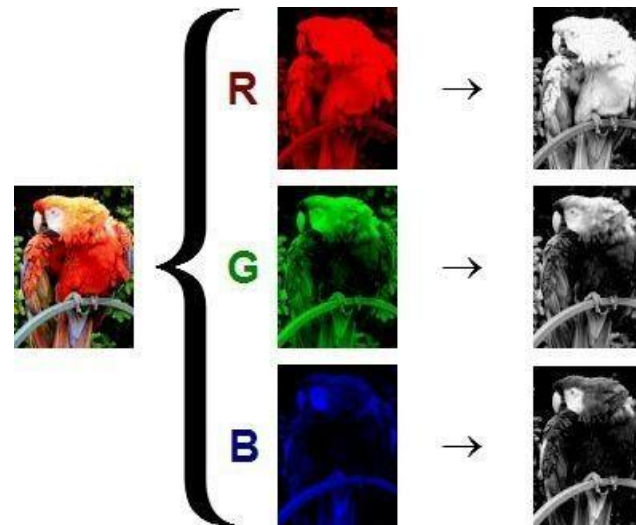
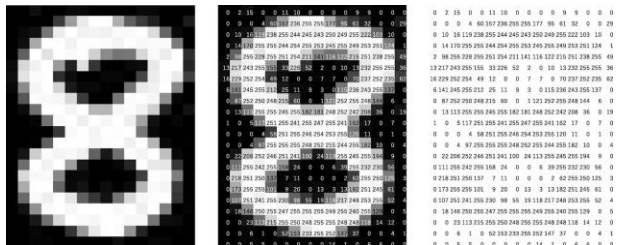
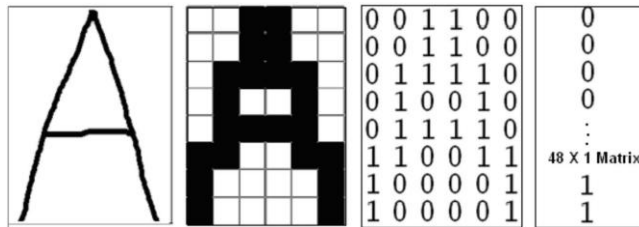
What is Learning in Neural Network?

- Learning (training):
 - Auto-adjusting model parameters with the each new input sample so the output (**prediction**) gets closer to expected values (**ground truth**)
- Forward-propagating activations for labeled input and back-propagating
- Errors to adjust parameters in each node in order to minimize loss function



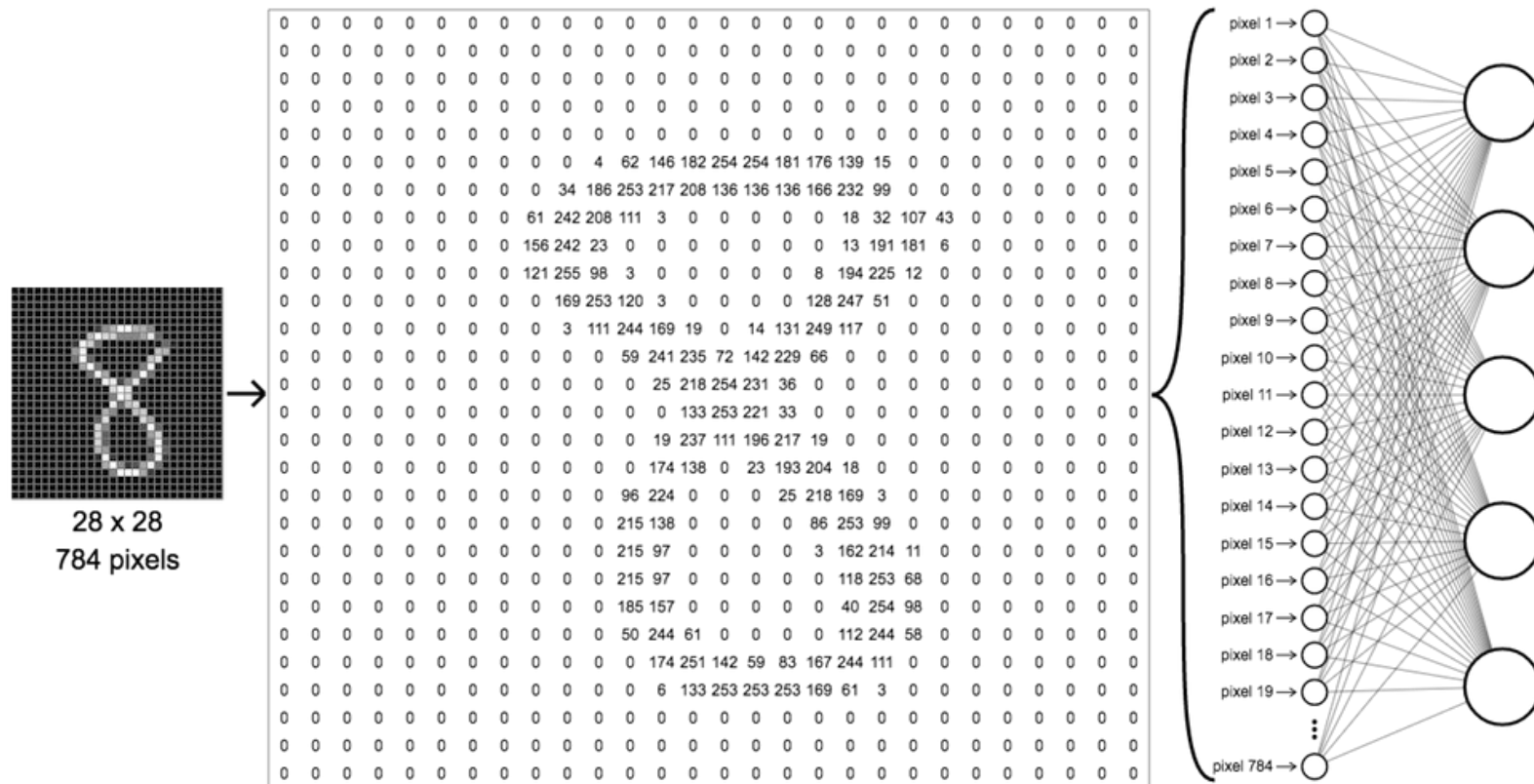
Neural Network Input: Digital Image

- **Digital image** - 2D set of pixels.
- Each **pixel** has numeric value(s) associated to it:
 - Monochrome: 0, 1
 - Grayscale: 0 (black) - 255 (white)
 - Colour: 0 - 25 (in each channel: R, G, B)
- Computer sees an array of numbers.



Fully-connected Input Layer

- Feeding input to a fully-connected input layer:
- 2D intensity (or 3D colour) matrix is collapsed into 1D vector



XOR Problem

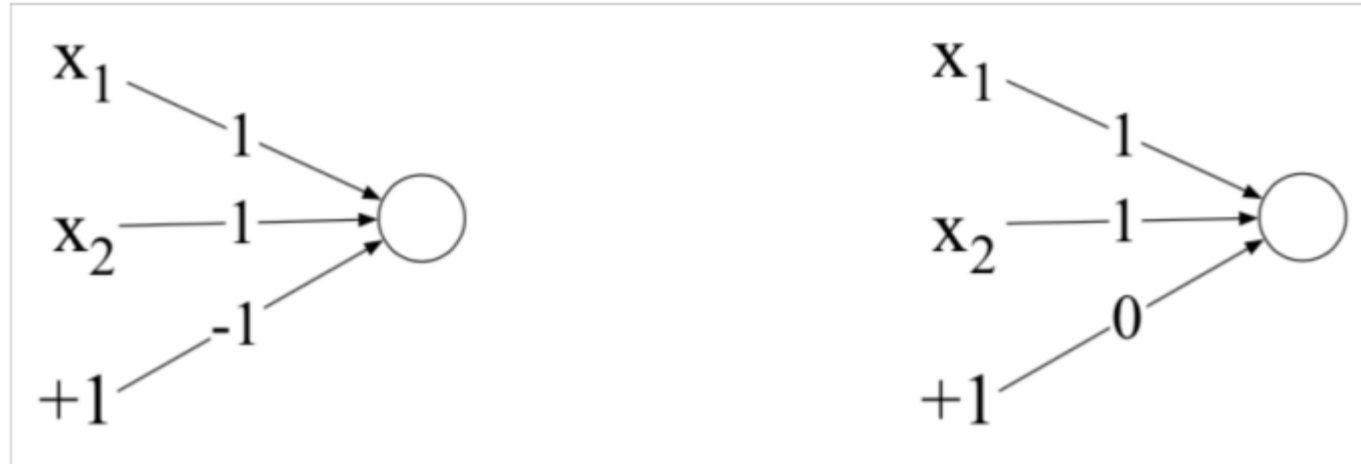
- Minsky-Papert proved perceptron can't compute XOR logical operation

$$y = \begin{cases} 0, & \text{if } w \cdot x + b \leq 0 \\ 1, & \text{if } w \cdot x + b > 0 \end{cases}$$

AND			OR			XOR		
x1	x2	y	x1	x2	y	x1	x2	y
0	0	0	0	0	0	0	0	0
0	1	0	0	1	1	0	1	1
1	0	0	1	0	1	1	0	1
1	1	1	1	1	1	1	1	0

XOR Problem

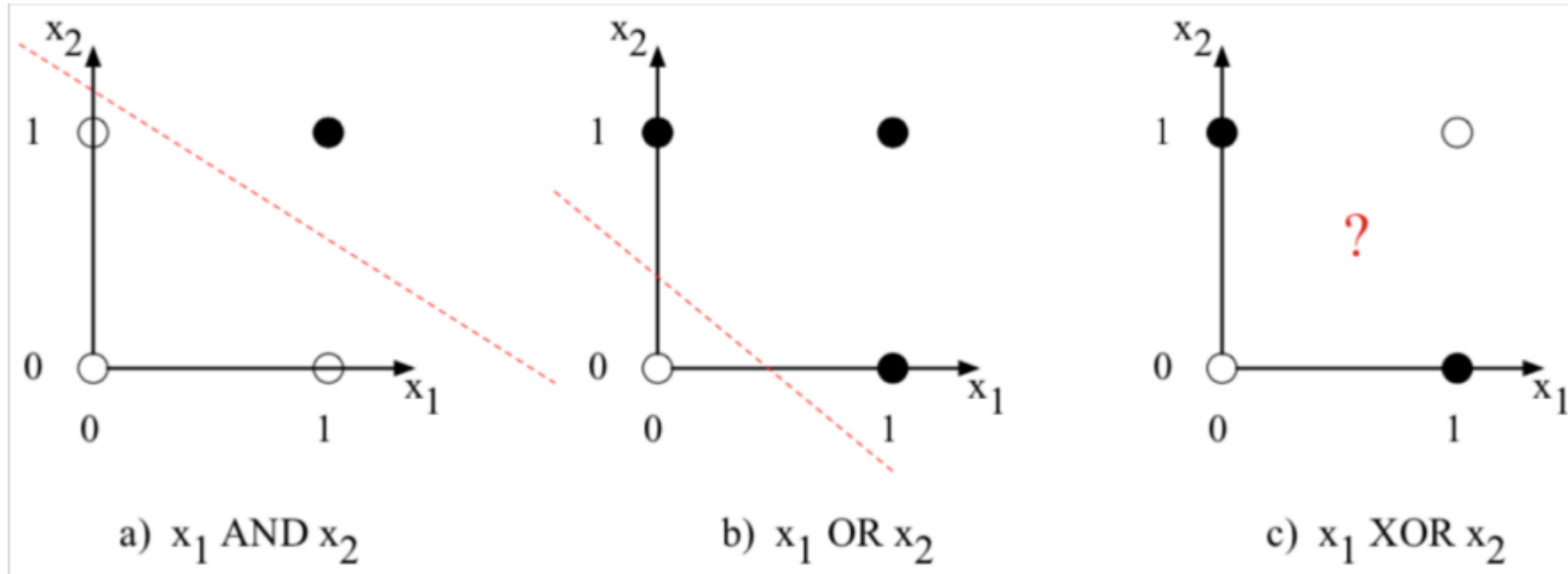
- Perceptron can compute the logical AND and OR functions easily



- But it's not possible to build a perceptron to compute logical XOR!

XOR Problem

- Perceptron is a linear classifier but XOR is not linearly separable



Why do we need non-linear activation functions?

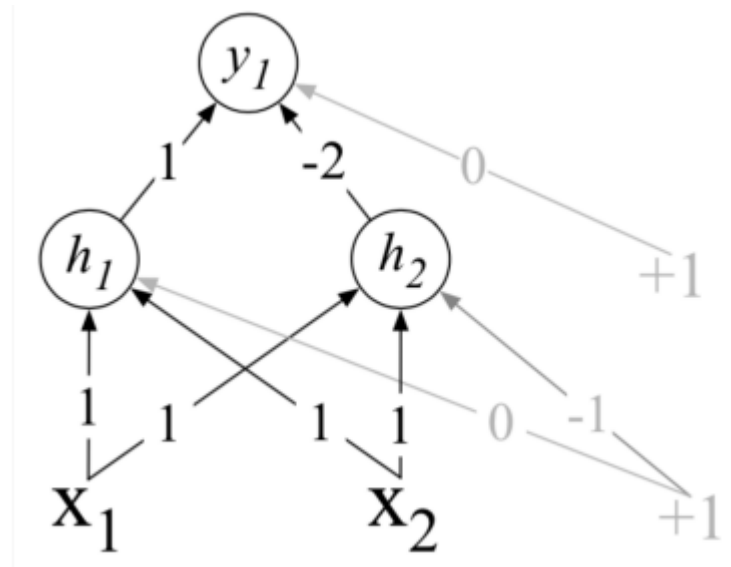
- Network of simple linear (perceptron) units cannot solve XOR problem
 - A network formed by many layers of purely linear units can always be reduced to a single layer of linear units

$$\begin{aligned}a^{[1]} &= z^{[1]} = W^{[1]} \cdot x + b^{[1]} \\a^{[2]} &= z^{[2]} = W^{[2]} \cdot a^{[1]} + b^{[2]} \\&= W^{[2]} \cdot (W^{[1]} \cdot x + b^{[1]}) + b^{[2]} \\&= (W^{[2]} \cdot W^{[1]}) \cdot x + (W^{[2]} \cdot b^{[1]} + b^{[2]}) \\&= W' \cdot x + b'\end{aligned}$$

- We've already shown that a single unit cannot solve the XOR problem

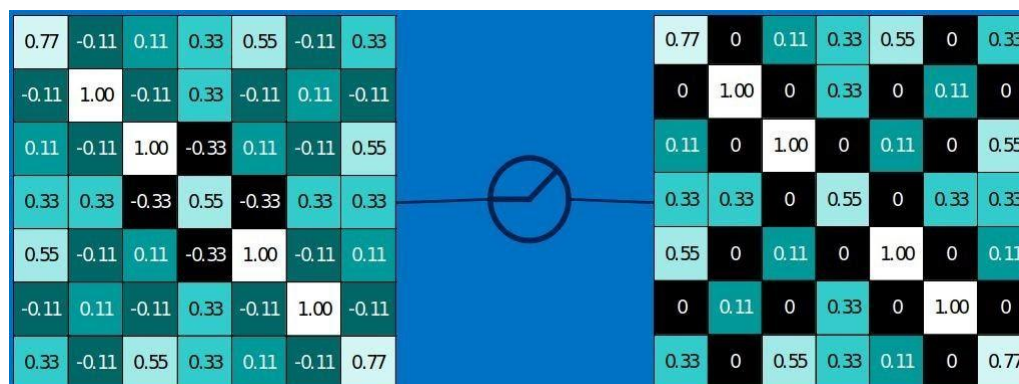
XOR Problem with Non-linear Functions

- XOR function can be computed using two layers of ReLU-based units
- XOR problem demonstrates need for multi-layer networks

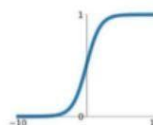


Nonlinearity Layer: Rectified Linear Unit (ReLu)

- **Activation function: nonlinear** function converts **feature map** into the **activation map**



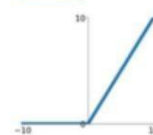
Sigmoid
 $\sigma(x) = \frac{1}{1+e^{-x}}$



tanh
 $\tanh(x)$



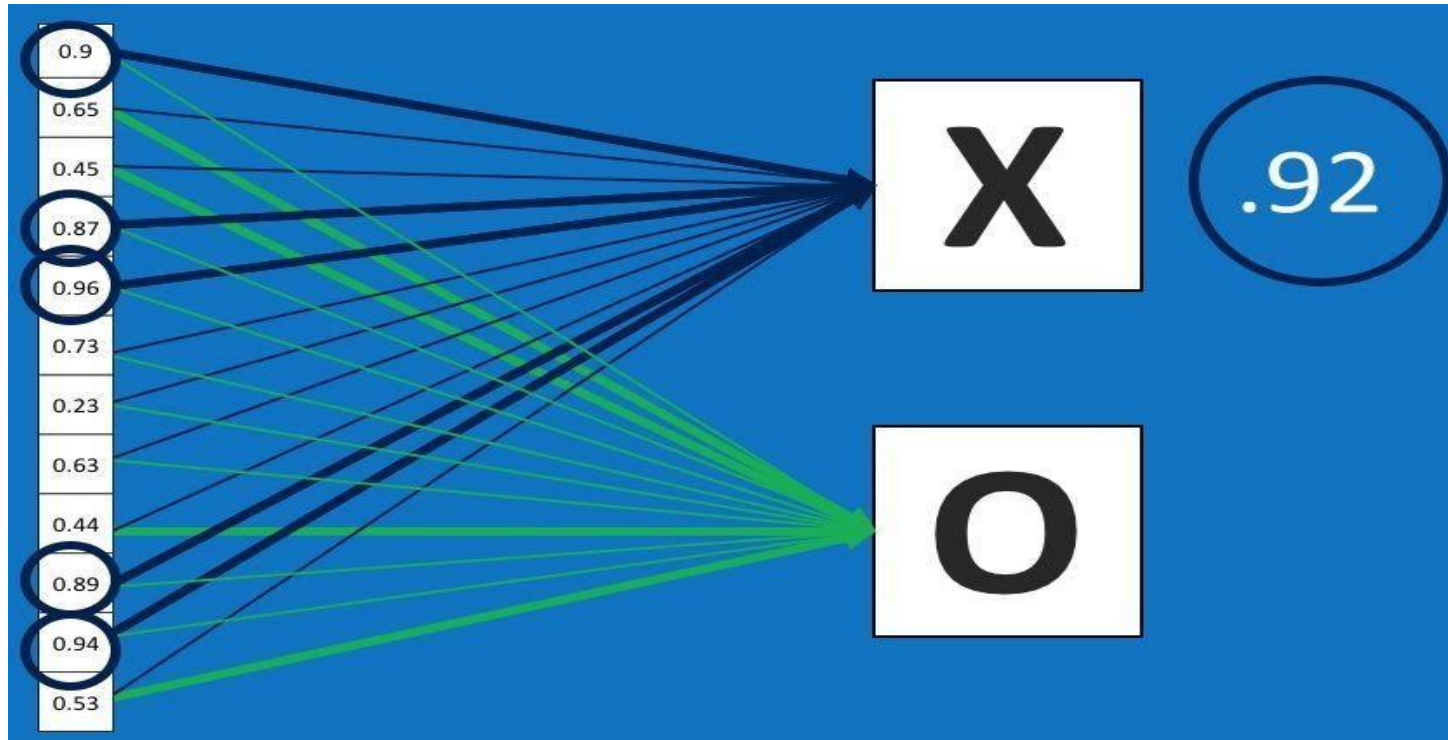
ReLU
 $\max(0, x)$



- ReLU makes CNNs perform best.
- CNN data input is usually normalized to range $[-1, 1]$ => improves learning speed and accuracy

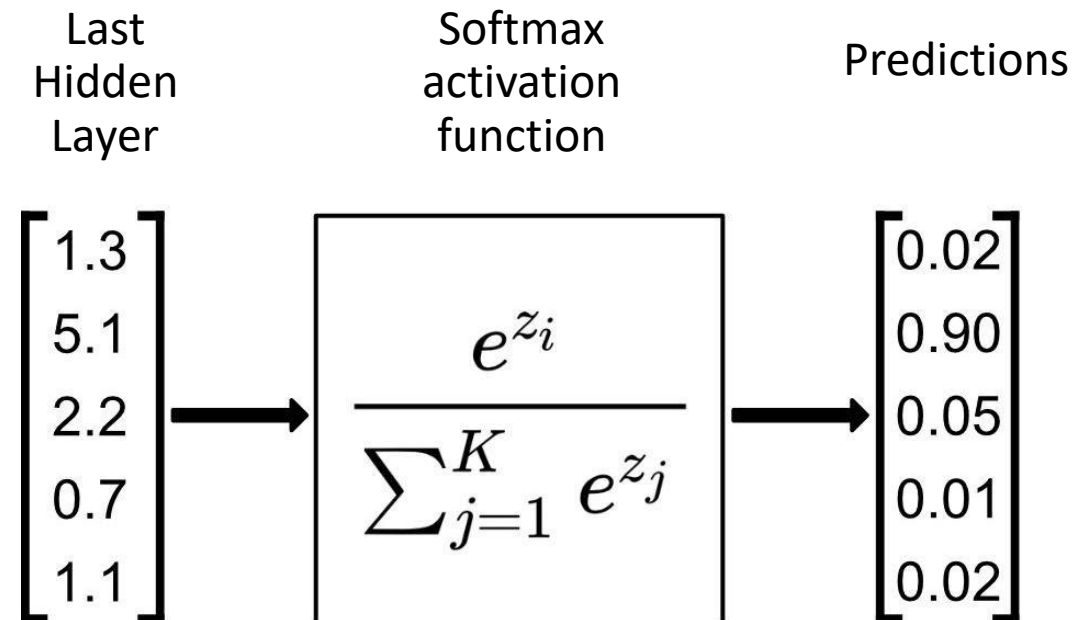
Fully-connected layers

- They learn non-linear combinations of the high-level features outputted by the convolutional layers.
- Features are assigned weights which describe their contribution in recognizing particular object.



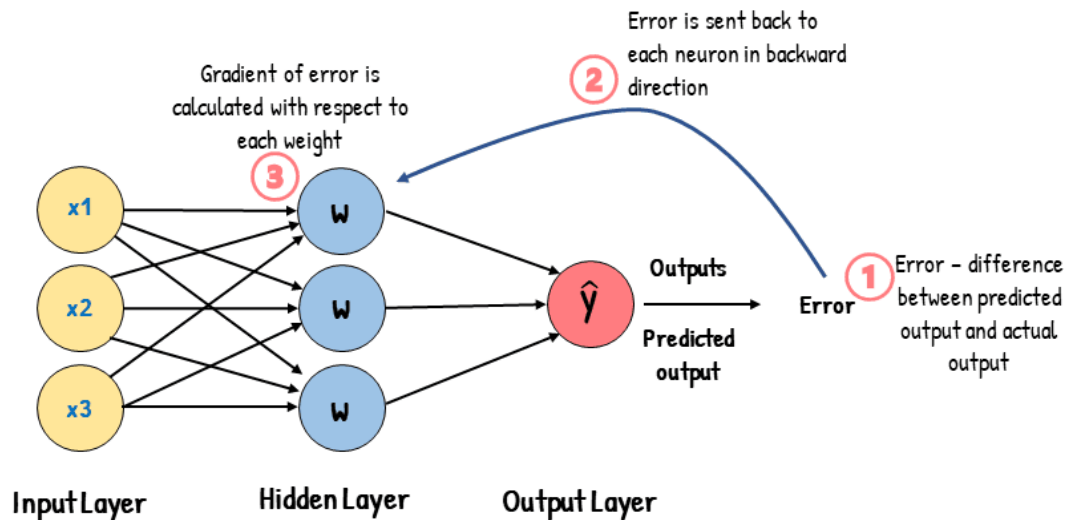
Softmax Layer

- Serving as the final layer after the Fully Connected layer, the Softmax function converts the network's raw scores (logits) into a meaningful probability distribution (summing to 1) for each class.
- This probability output is essential for calculating the "error" (Loss), which is then used by the backpropagation algorithm to update the network's weights during training.



How Do Neural Networks Learn? Backpropagation

- To minimize the "**Error**" (or "Loss") by adjusting the network's internal parameters (weights). The error is the difference between the network's **Prediction** and the **Actual Target** (the correct answer).



1. Forward Pass (Make a Guess)

- Input data is fed *forward* through the network (Input Layer → Hidden Layers → Output Layer).
- The network makes a **Prediction**.
- The **Error** is calculated by comparing the prediction to the correct answer.

2. Backward Pass (Learn from the Mistake)

- The **Error** is propagated *backward* from the Output Layer to the Input Layer.
- Backpropagation algorithm** calculates how much each **Weight** in the network contributed to the total error.
- These weights are then **Adjusted** (updated) slightly in a direction that will reduce the error.
- "Forward Pass → Backward Pass" cycle is repeated many times, allowing the network to get progressively better at its task.

Summary

- **Artificial Intelligence** is the broad concept of machines mimicking human behavior, **Machine Learning** is a subset of AI that learns from data, and **Deep Learning** is a subset of ML using multi-layer neural networks.
- **Supervised Learning** learns from **labeled data** to solve problems like **Classification** (predicting a category) and **Regression** (predicting a continuous value).
- **Unsupervised Learning** discovers hidden patterns and structures in **unlabeled data**, with a primary task being **Clustering**.
- **Reinforcement Learning** involves an "agent" learning by maximizing a **reward** signal, while **Semi-supervised Learning** uses a mix of labeled and unlabeled data.
- The classic XOR problem proved the limitations of single-layer linear models (Perceptrons), demonstrating the need for **multi-layer networks** equipped with **non-linear activation functions (e.g., ReLU)** to solve complex problems.
- **Backpropagation** is a core training algorithm for neural networks; it's a cycle of making a prediction (**Forward Pass**), calculating the **Error (Loss)**, and then adjusting the network's weights to minimize that error (**Backward Pass**).
- **Softmax Layer** is used as the final layer in classification tasks, it converts the network's raw scores (logits) into a meaningful **probability distribution** (summing to 1) across all classes.

LG AI Research Special Lecture

딥러닝 자연어처리 기초와 LLM 에이전트

Lecture 02 – 자연어처리의 기초와 RNN



중앙대학교 AI학과

이환희

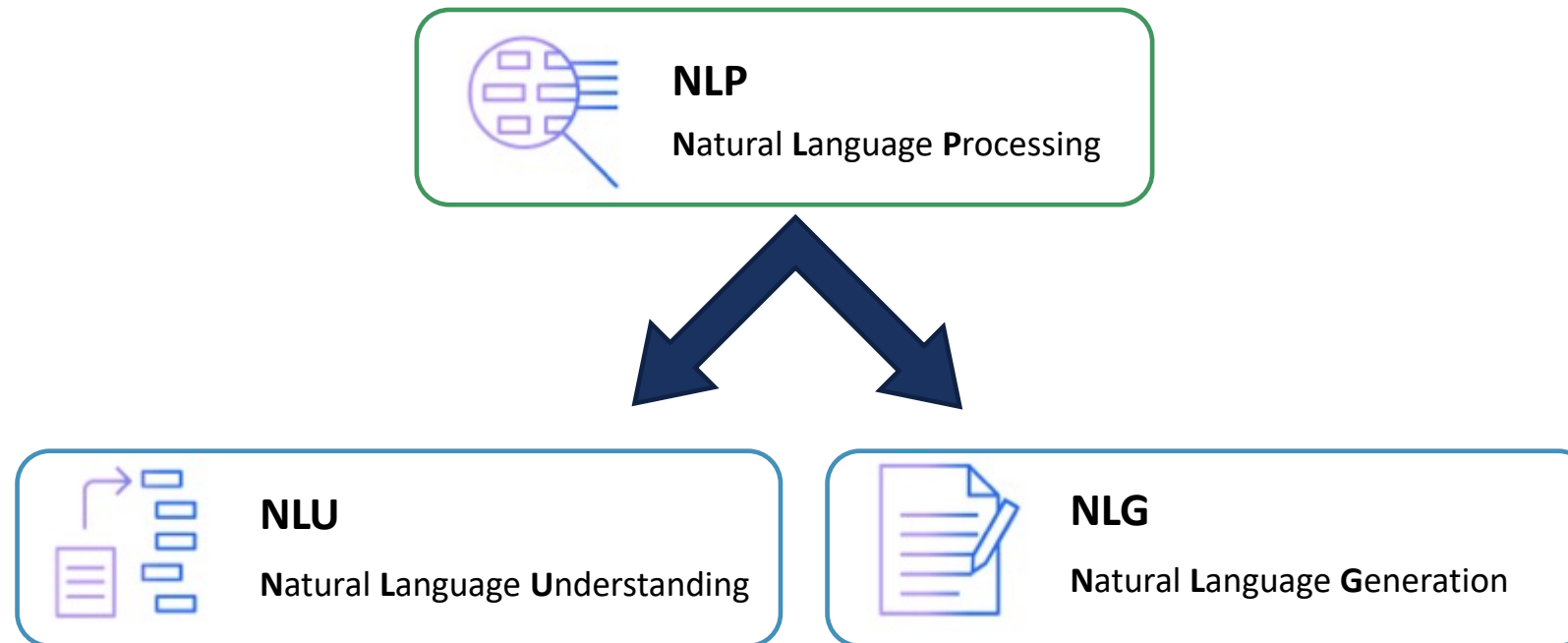
Outline

자연어처리의 기초와 RNN

- What is Natural Language Processing?
- Tokenization
- Word Embedding
- Language Modeling
- Recurrent Neural Networks

Natural Language Processing

- Giving computers the ability to understand text in much the same way human beings can.



Natural Language Processing Tasks



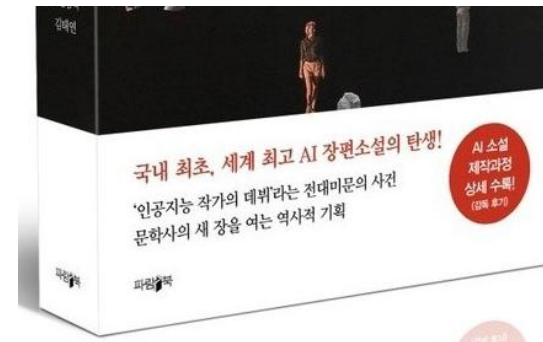
Dialog System



Summarization

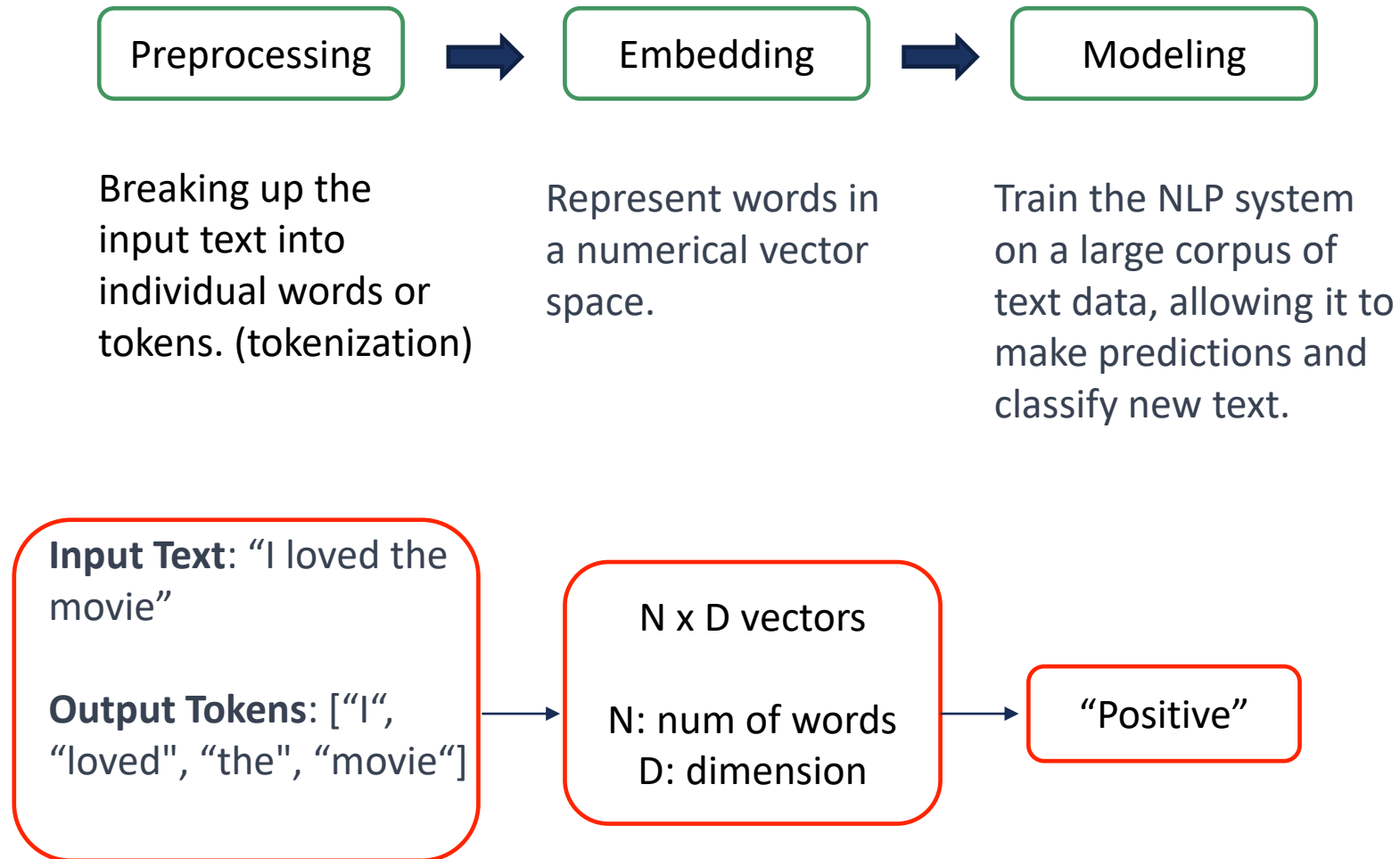


Machine Translation

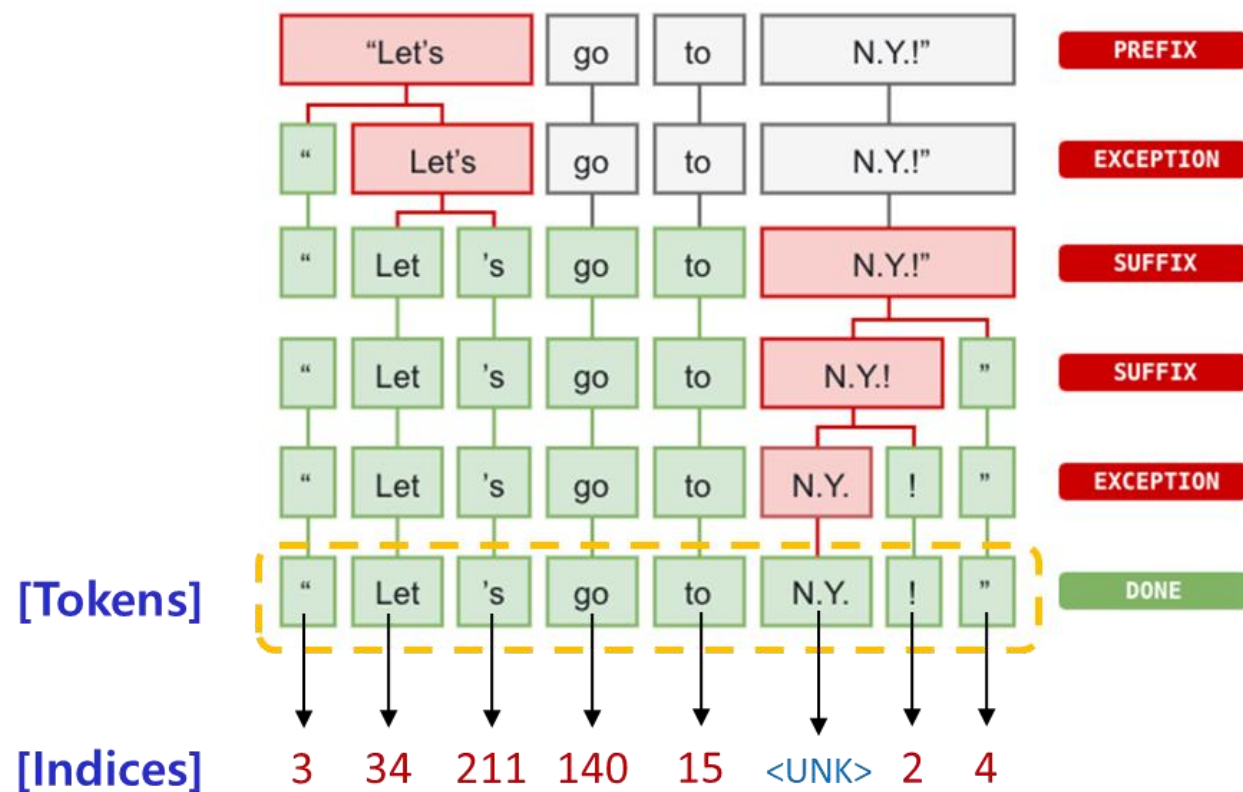


Story Generation

Building NLP Systems



Tokenization: Example

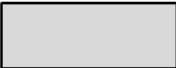


Tokenization: Padding

Batching and Padding

I	love	Mom	'	s	cooking
I	love	you	too	!	
This	is	the	shit		
No	way				
Yes					

Batch size : [5, 4, 3, 3, 2, 1]

 Padding

Tokenization: Beyond Word Level – Vocab Size?

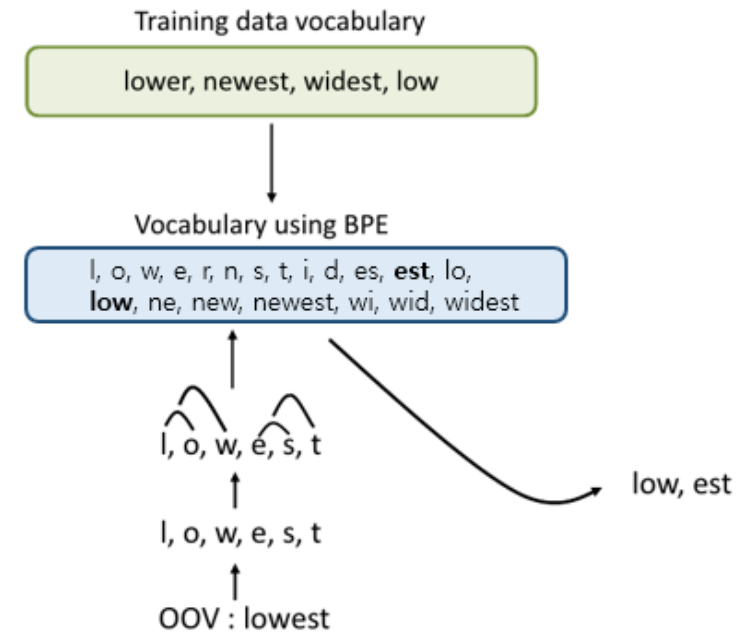
- What happens if the vocab size is too big? Or too small?

Big: Too-much computation!,
sparsity

Small: Out-of-Vocabulary

“not present in a system's dictionary or
database of known terms”

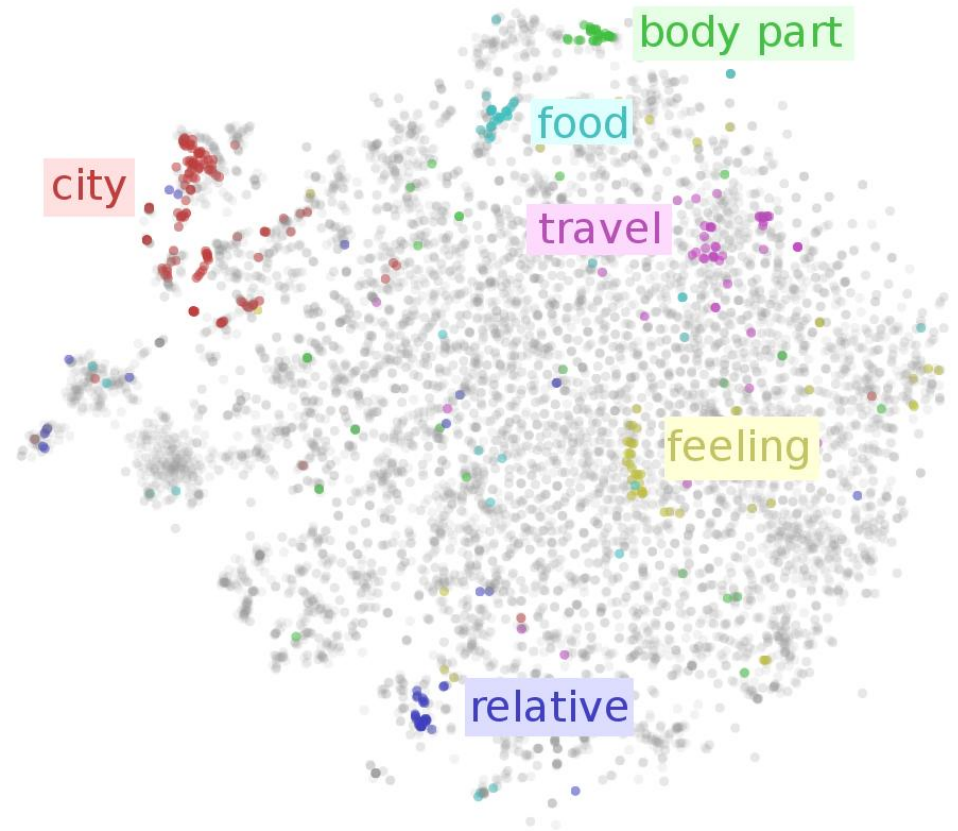
Solution 1: Character Level



Solution 2: Byte Pair Encoding (3rd week's class)

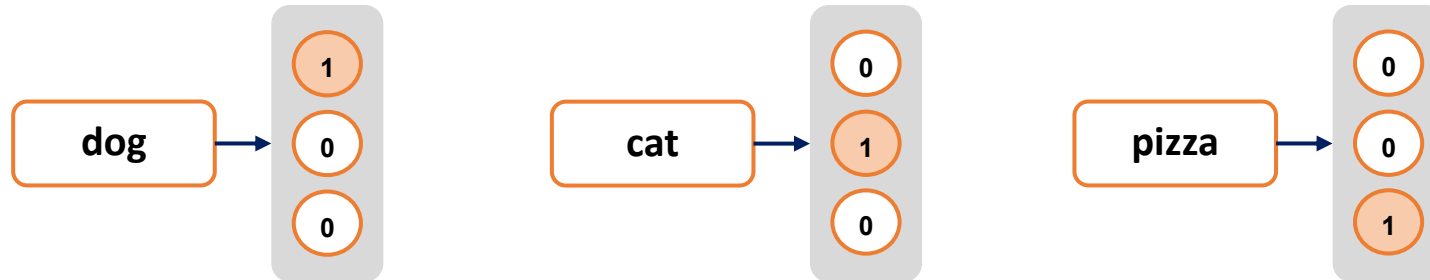
Word Embedding

- A representation that maps words to real-valued vectors



Word Embedding

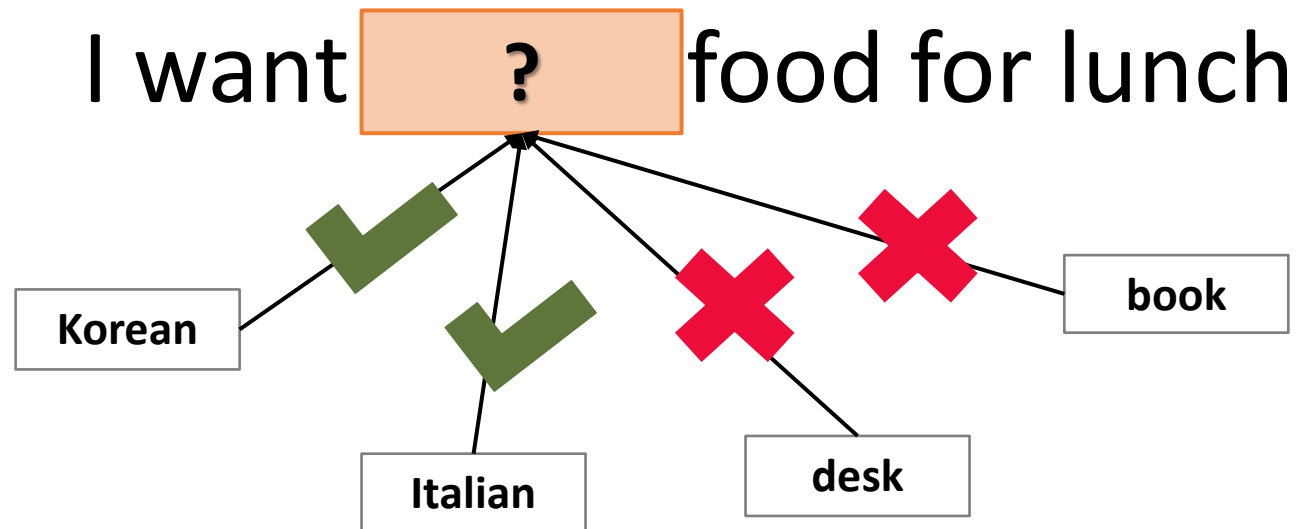
- Bag-of-Words(BoW)
 - A text is represented as the bag of its words
 - ex) One-hot encoding



Word Embedding

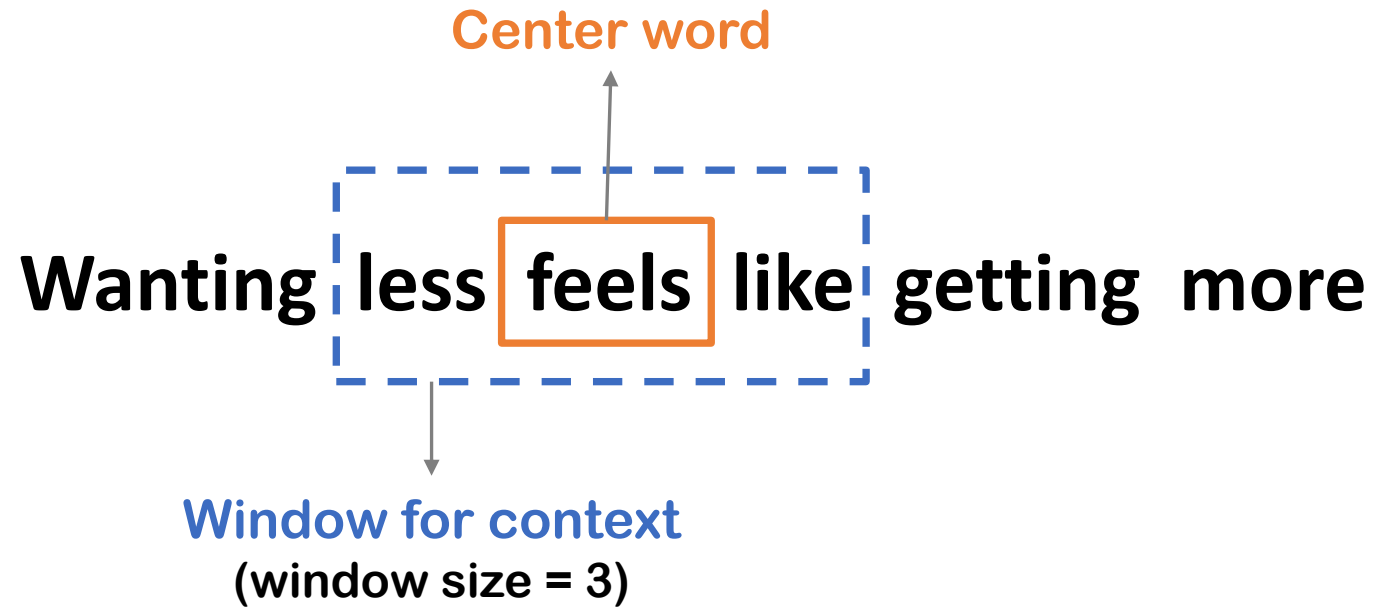
- Word2Vec

- One of the most popular techniques (Tomas Mikolov, 2013)
- Constructs word embeddings where words with similar context are embedded close to each other
- CBOW and Skip-gram models



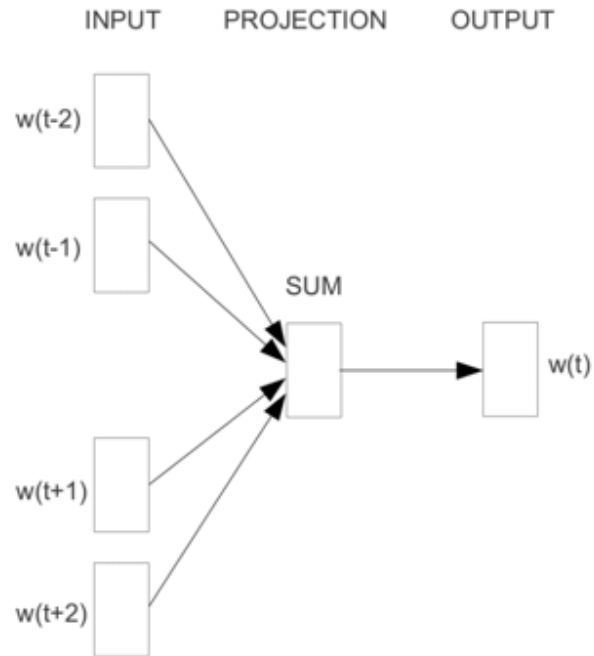
Word Embedding

- How to generate word embedding in Word2Vec
 - Center word and its context words



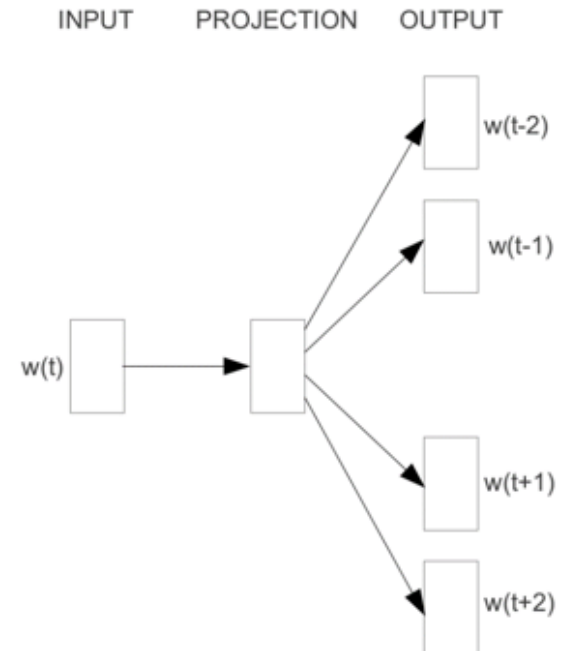
Word Embedding

■ CBOW(Continuous Bag-of-Words)



$$E = -\log p(w_t | w_{t-c}, \dots, w_{t+c})$$

■ Skip-gram

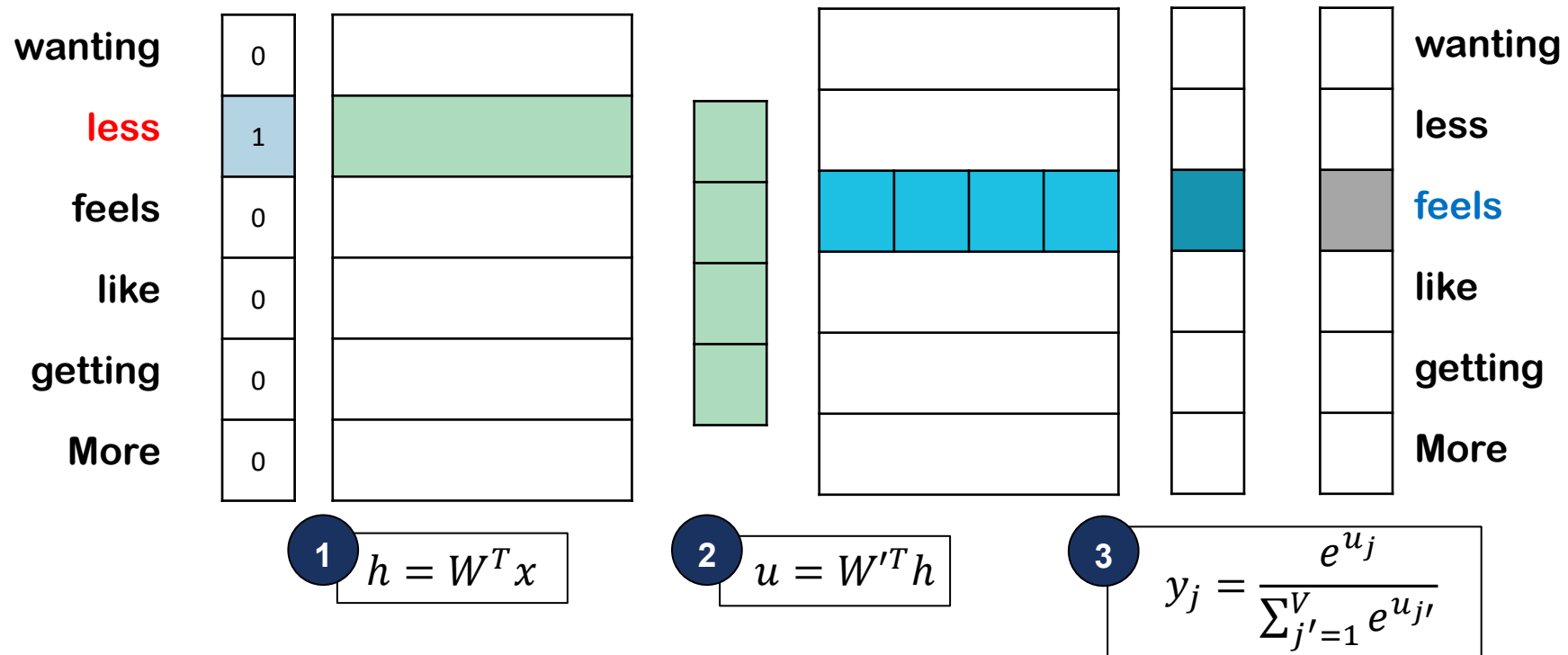


$$E = -\log p(w_{t-c}, \dots, w_{t+c} | w_t)$$

Word Embedding

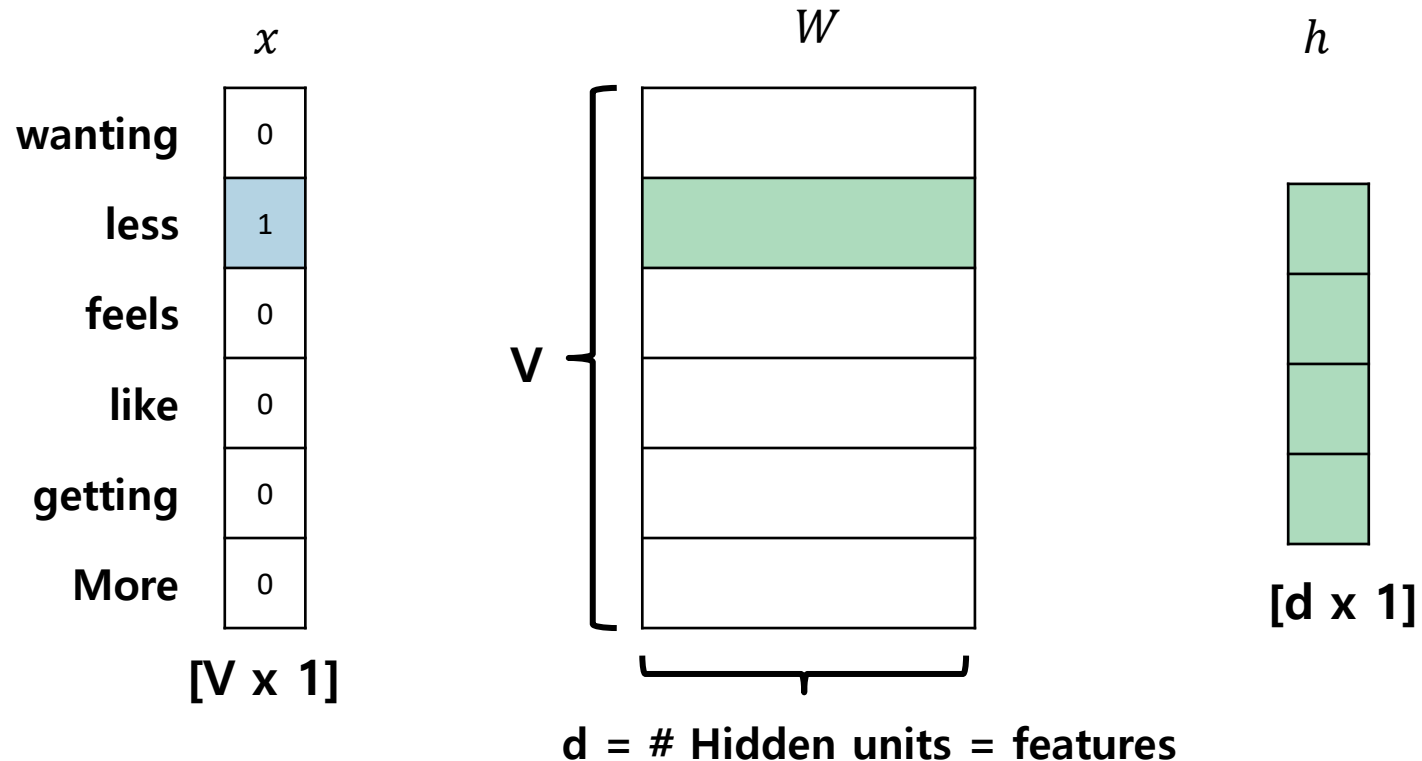
- Skip-gram model

- $y_j = p(w_j|w_i)$ is the probability that w_j is the context word, given the input w_i



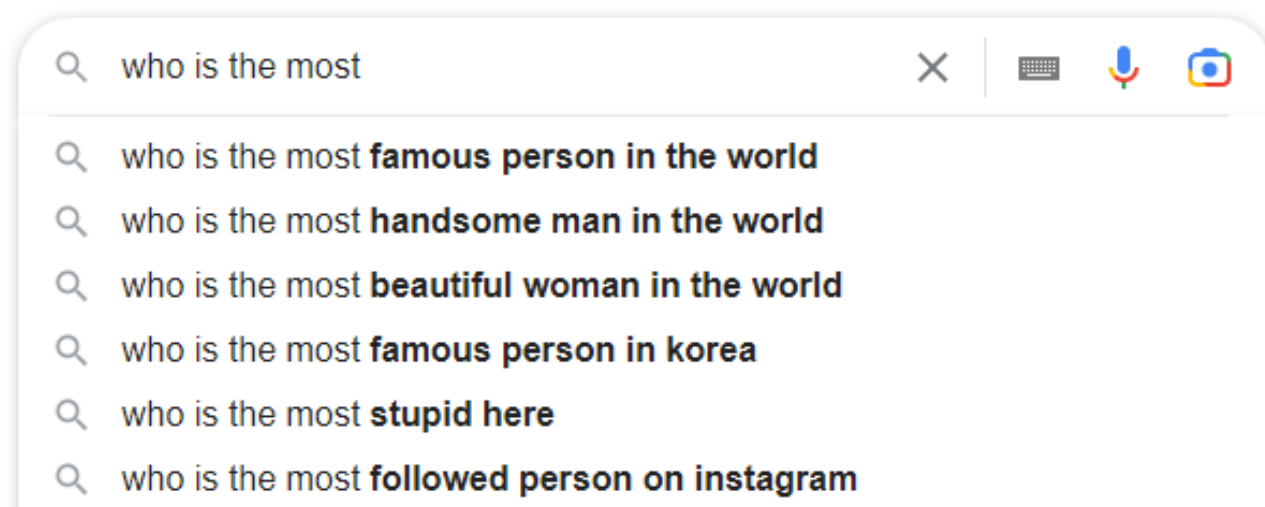
Word Embedding

- Skip-gram model
 - Rows in hidden layer weight matrix become word vectors (word vector look-up table)



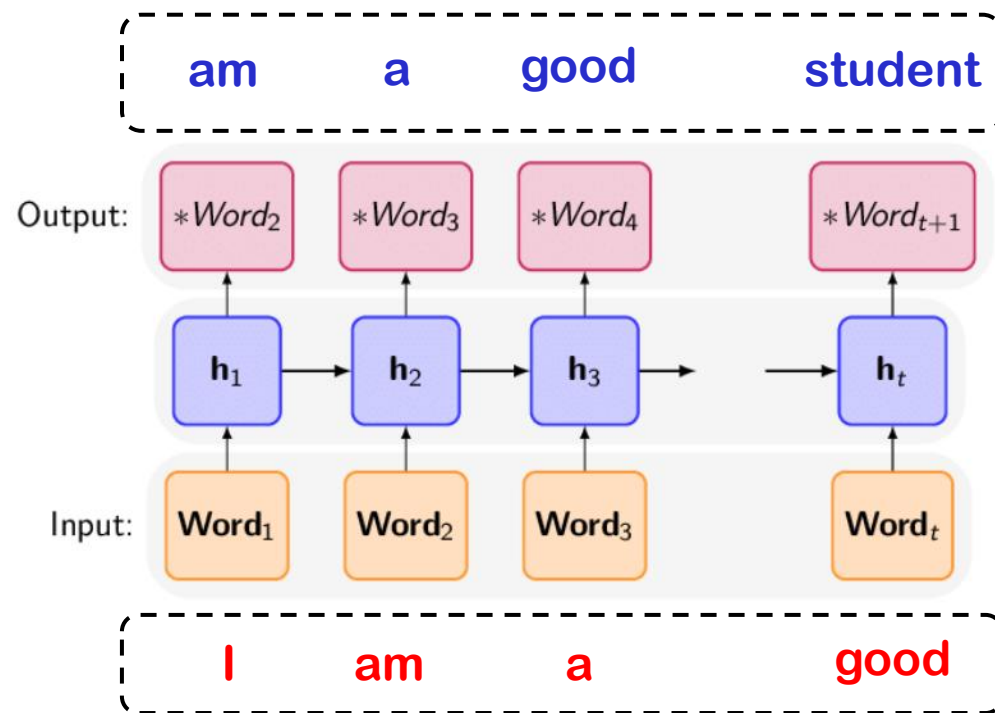
Language Model

- A model that has learned the **probability distribution of sentences or words**.
- By learning from a large amount of text data, the model understands the patterns and characteristics of sentences, and based on this, it can **generate new sentences** or **predict the next word**.



Language Model

- **Language Modeling:** Task of understanding the probability distribution over a sequence of words



Language Model

- A language model computes a probability for a sequence of words:

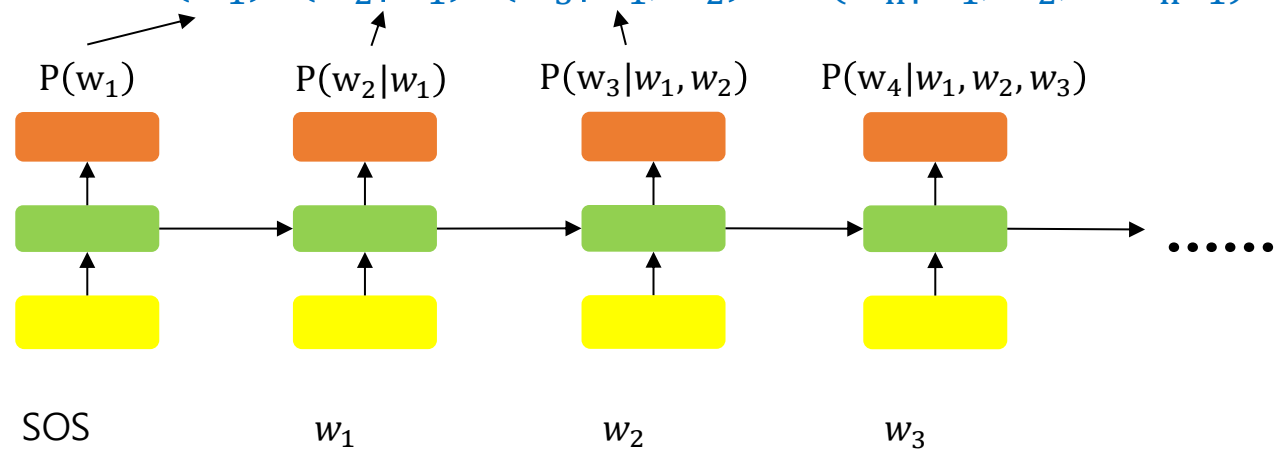
$$P(w_1, \dots, w_T)$$

Word ordering: $P(\text{the cat is small}) > P(\text{small the is cat})$

Word choice: $P(\text{walking home after school}) > P(\text{walking house after school})$

To compute $P(w_1, \dots, w_T)$

$$= P(w_1)P(w_2|w_1)P(w_3|w_1, w_2) \dots P(w_n|w_1, w_2, \dots, w_{n-1})$$



Building Neural Language Model: Fixed Language Model

- **Input:** sequence of words - $x^{(1)}, x^{(2)}, \dots, x^{(t)}$ **Output:** prob. dist. of the next word $P(x^{(t+1)} | x^{(t)}, \dots, x^{(1)})$
- A simple fixed-length language model predicts the probability of the next word by considering only a constant, predefined number of preceding words (a "fixed window"), rather than the entire sequence history.

the students opened their _____
fixed window

*"We need a neural architecture
that can process any length input"*

output distribution

$$\hat{y} = \text{softmax}(Uh + b_2) \in \mathbb{R}^{|V|}$$

hidden layer

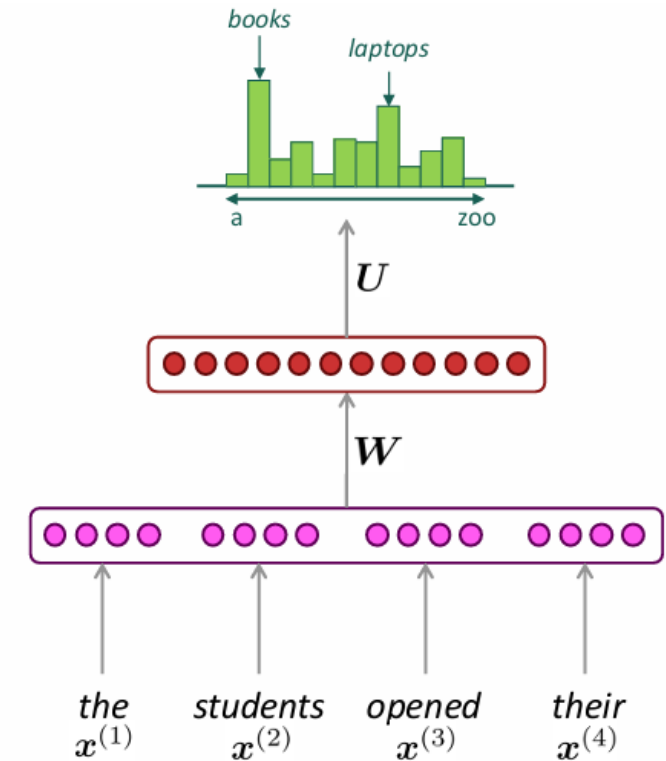
$$h = f(We + b_1)$$

concatenated word embeddings

$$e = [e^{(1)}; e^{(2)}; e^{(3)}; e^{(4)}]$$

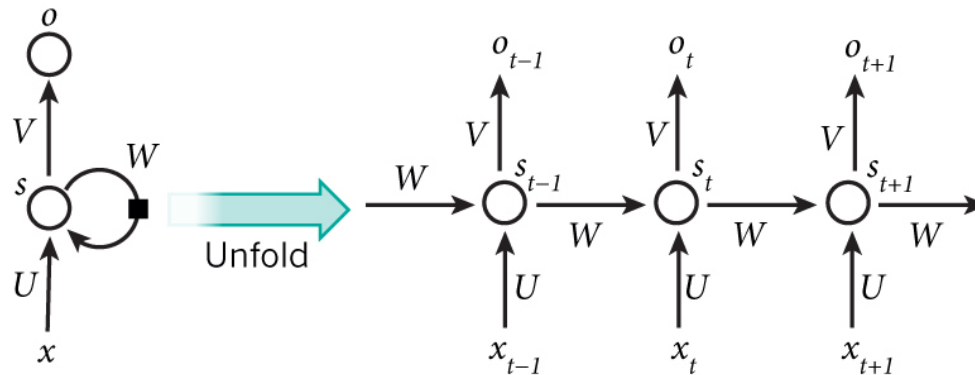
words / one-hot vectors

$$x^{(1)}, x^{(2)}, x^{(3)}, x^{(4)}$$



Recurrent Neural Network

- RNNs are a very natural way to model sequential data:
- RNNs consider input sequence over discrete time.
- RNNs have the ability to remember information in their hidden states for a long time.
- They are equivalent to very deep nets with a hidden layer per one time step.
- Except that they use the same weights at every time step and they get input at every time step.



Apply the same weights repeatedly!

$$s_t = f(Ux_t + Ws_{t-1})$$

$$o_t = Vh_t$$

s_t : hidden state

f : activation function
(Sigmoid, Tanh, ReLU)

Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

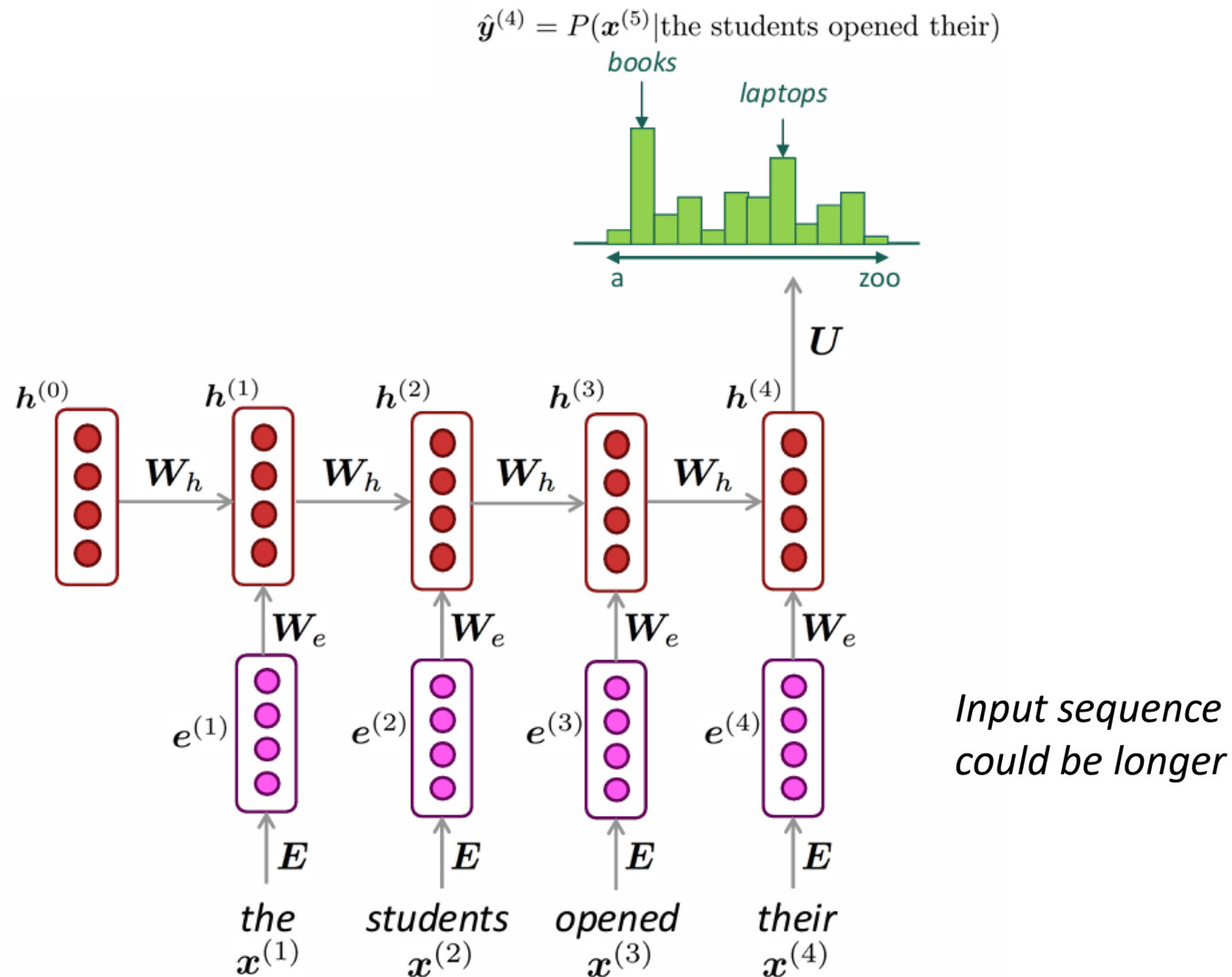
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Slides adapted from CS224n

Pros and Cons of RNN

- **Advantages of RNN:**

- Can process any length input laptops
- Computation for step t can use information from many steps back
- Model size doesn't increase for longer input context
- Same weights applied on every timestep, so there is symmetry in how inputs are processed.

- **Disadvantages of RNN**

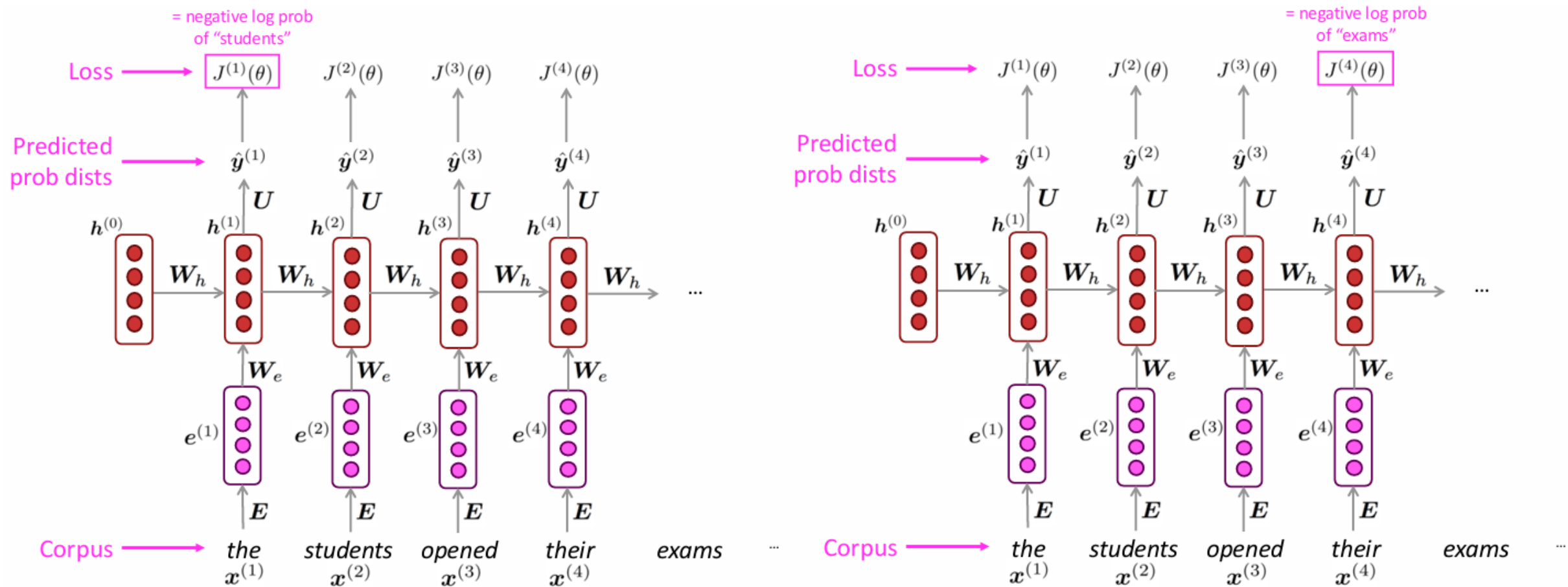
- Recurrent computation is slow
- In practice, difficult to access information from many steps back

Training an RNN Language Model

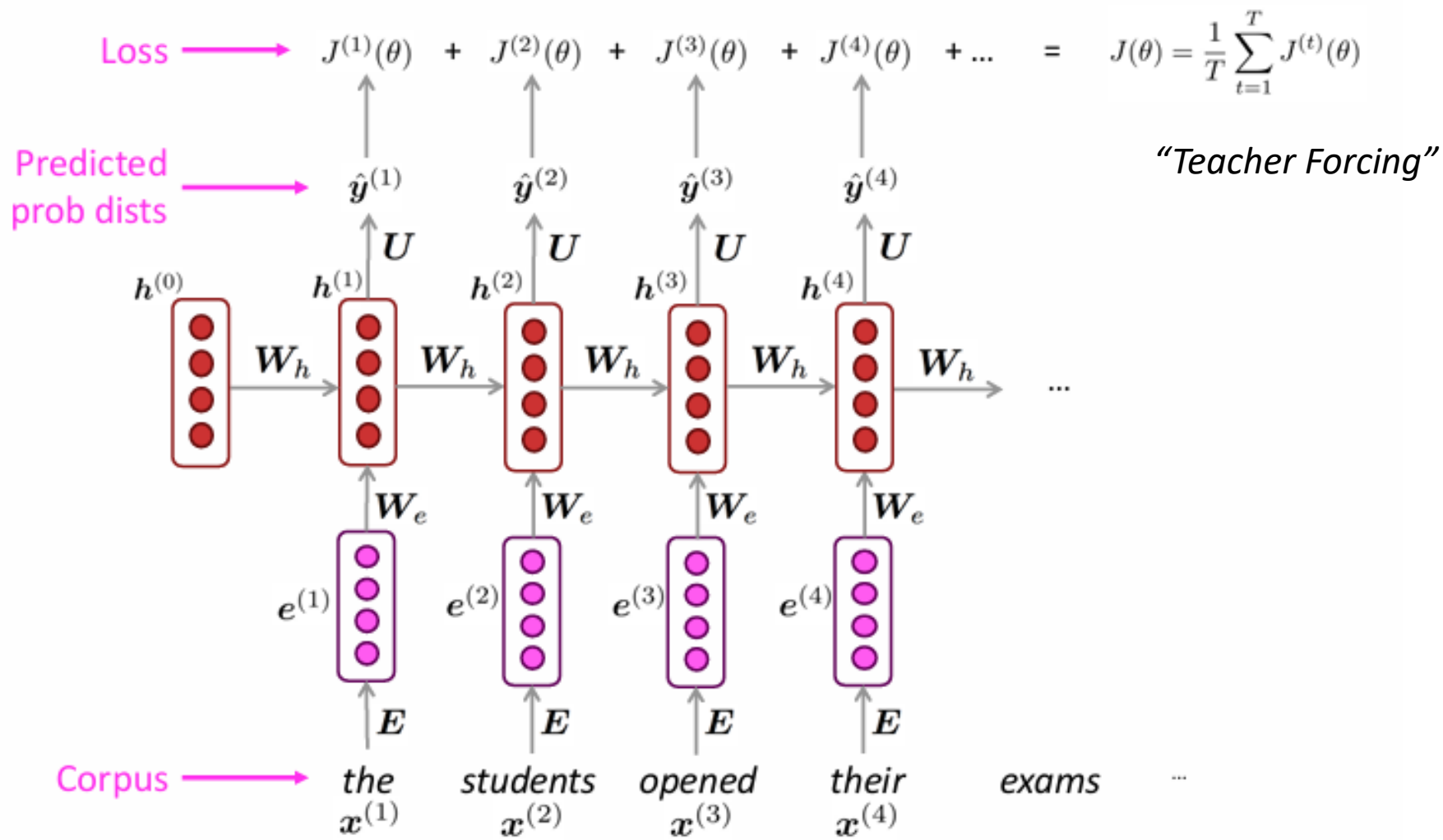
- Get a big corpus of text which is a sequence of words $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$
- Feed into RNN-LM and compute output distribution $\hat{\mathbf{y}}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of every word, given words so far
- Loss function on step t is **cross-entropy** between predicted probability distribution , and the true next word (one-hot for):
$$J^{(t)}(\theta) = CE(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) = - \sum_{w \in V} \mathbf{y}_w^{(t)} \log \hat{\mathbf{y}}_w^{(t)} = - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$
- Average this to get **overall loss** for entire training set

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{\mathbf{y}}_{\mathbf{x}_{t+1}}^{(t)}$$

Training an RNN Language Model



Training an RNN Language Model



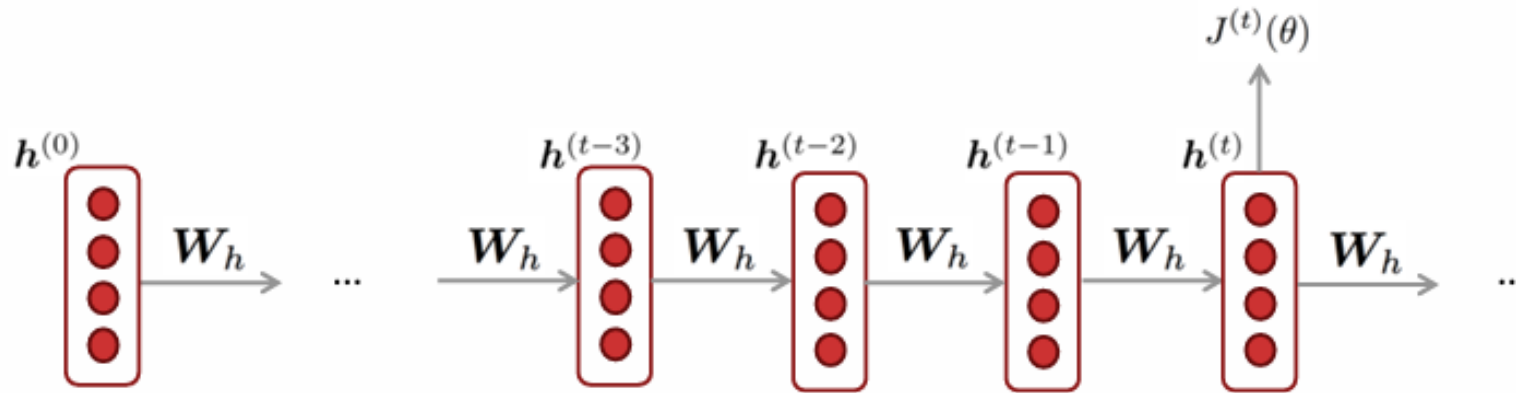
Training an RNN Language Model

- Computing loss and gradients across entire corpus $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$ at once is too expensive!

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta)$$

- In practice, consider $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(T)}$ as a sentence (or a document)
- Recall: Stochastic Gradient Descent allows us to compute loss and gradients for small chunk of data, and update.
- Compute loss $J(\theta)$ for a sentence (actually, a batch of sentences), compute gradients and update weights.
- Repeat on a new batch of sentences.

Backpropagation of RNNs

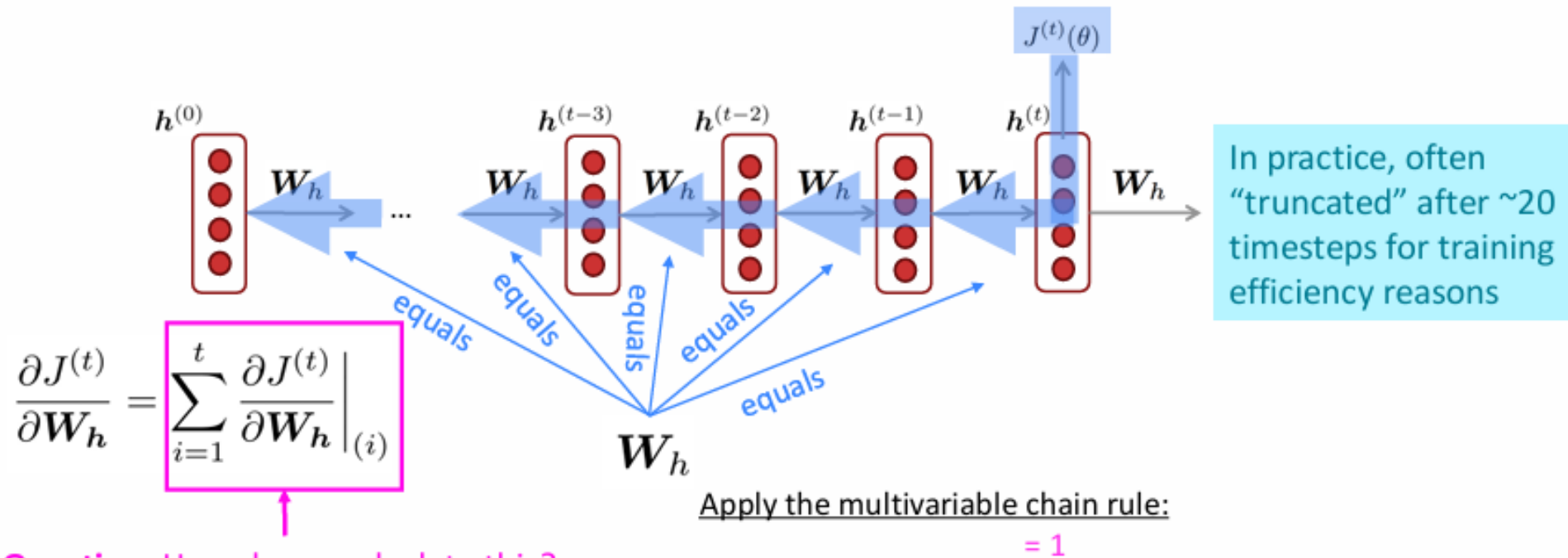


- Derivative of $J^{(t)}(\theta)$ with respect to the repeated weight matrix:

$$\frac{\partial J^{(t)}}{\partial W_h} = \sum_{i=1}^t \left. \frac{\partial J^{(t)}}{\partial W_h} \right|_{(i)}$$

- The gradient w.r.t. a repeated weight is the sum of the gradient w.r.t. each time it appears.

Training the parameters of RNNs: Backpropagation for RNNs



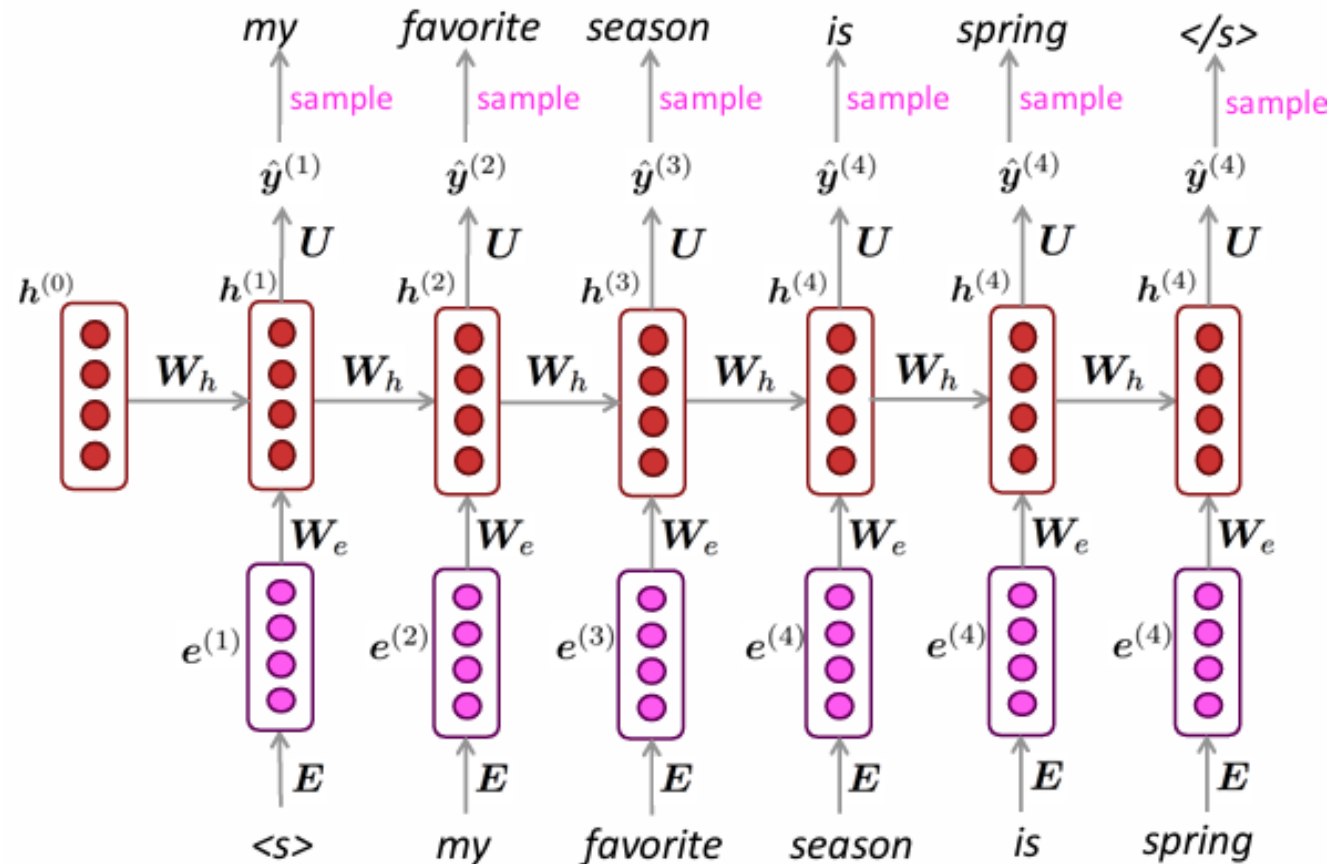
Question: How do we calculate this?

Answer: Backpropagate over timesteps $i = t, \dots, 0$, summing gradients as you go. This algorithm is called “**backpropagation through time**” [Werbos, P.G., 1988, *Neural Networks 1*, and others]

$$\begin{aligned} \frac{\partial J^{(t)}}{\partial \mathbf{W}_h} &= \sum_{i=1}^t \frac{\partial J^{(t)}}{\partial \mathbf{W}_h} \bigg|_{(i)} \frac{\partial \mathbf{W}_h|_{(i)}}{\partial \mathbf{W}_h} \\ &= \sum_{i=1}^t \frac{\partial J^{(t)}}{\partial \mathbf{W}_h} \bigg|_{(i)} \end{aligned}$$

Generating with an RNN Language Model (“Generating roll outs”)

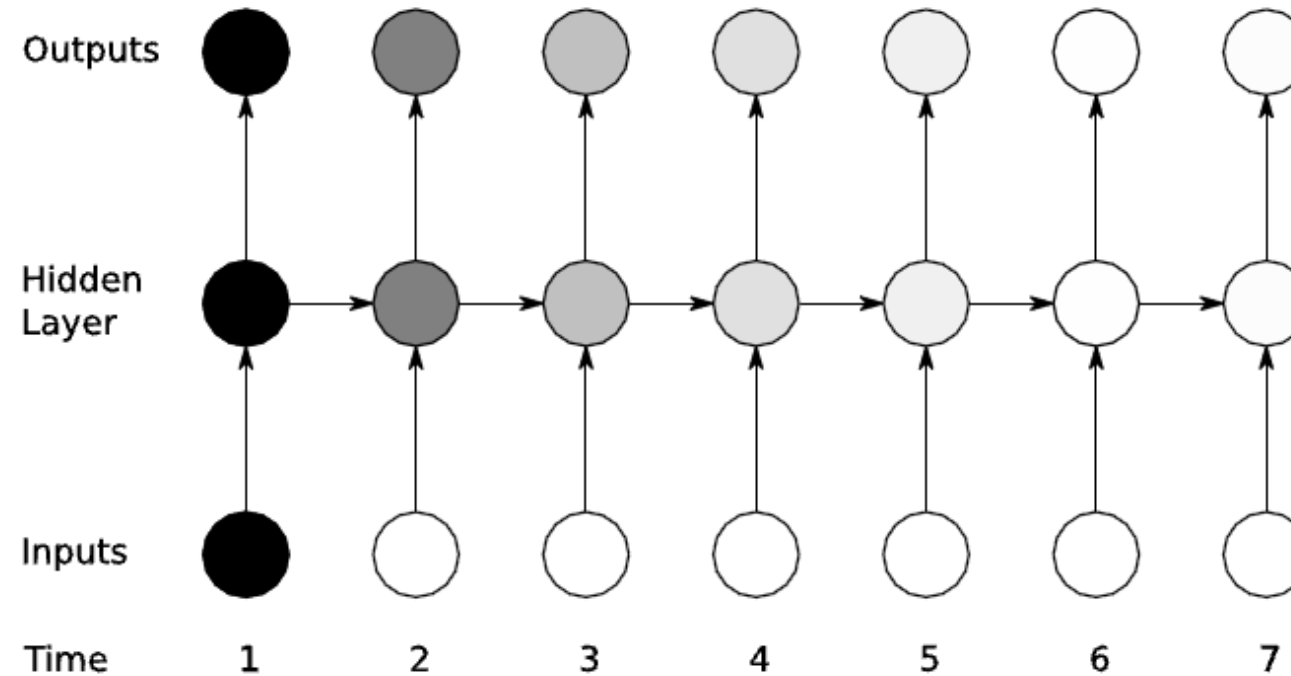
- We can use a RNN language model to generate text by repeated sampling.
- Sampled output becomes next step’s input.



Slides adapted from CS224n

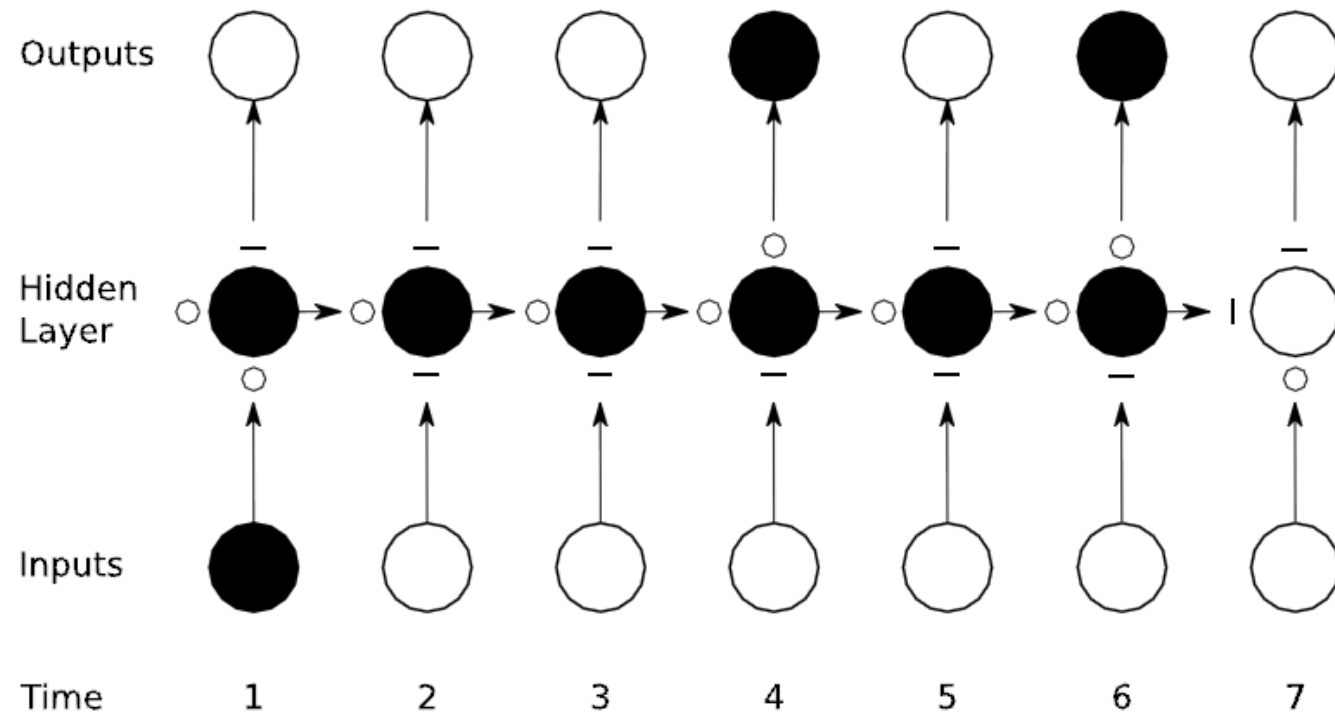
Problem of RNN: Vanishing Gradients

- **Vanishing Gradients problem for basic RNNs**
 - Influence of the inputs at time t decreases and vanishes over time



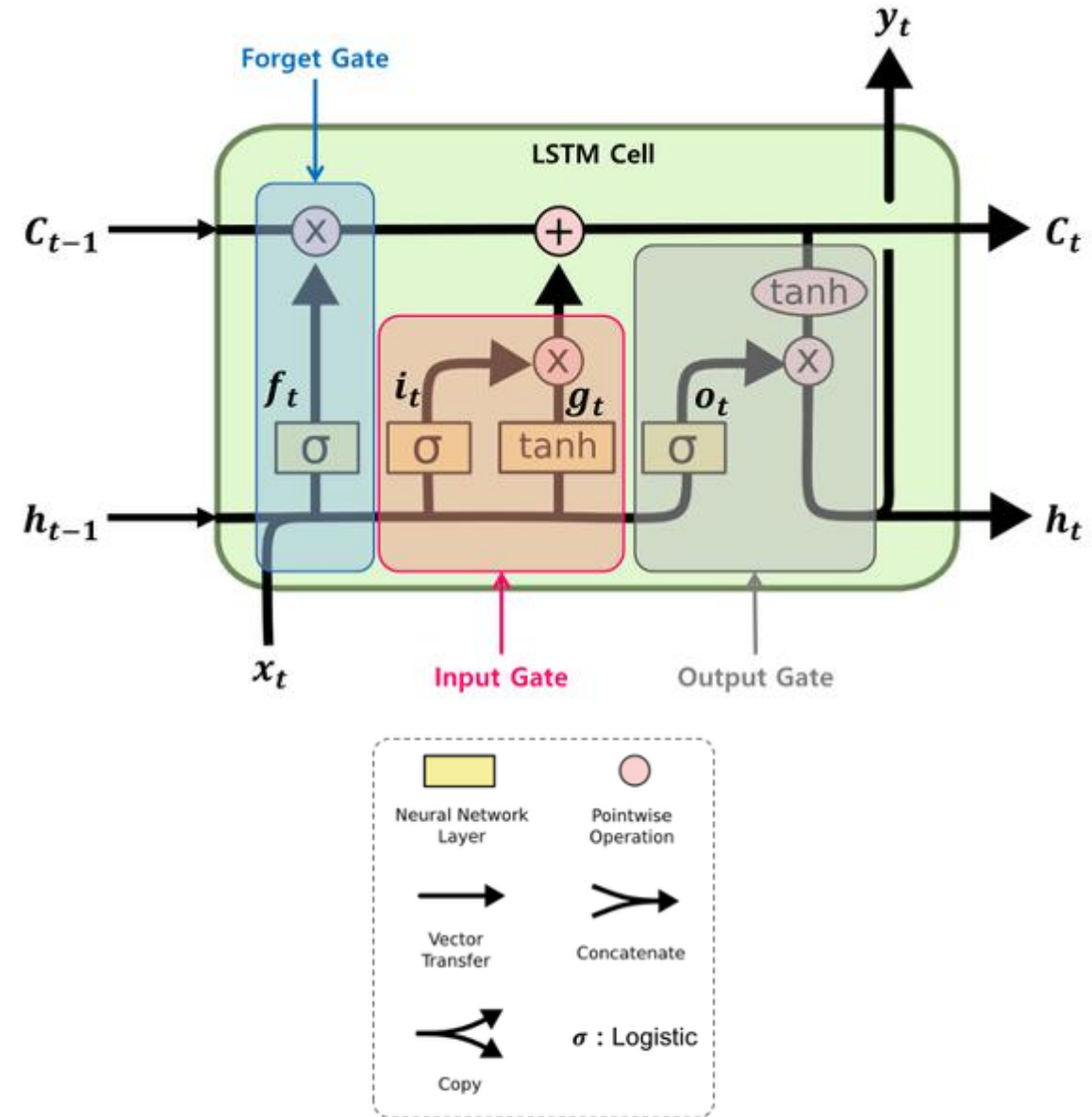
LSTM-RNNs

- **LSTM can preserve gradient information**
 - Hidden layer units formed with Long Short-Term Memory (LSTM) cells can store and access information over longer periods of time.



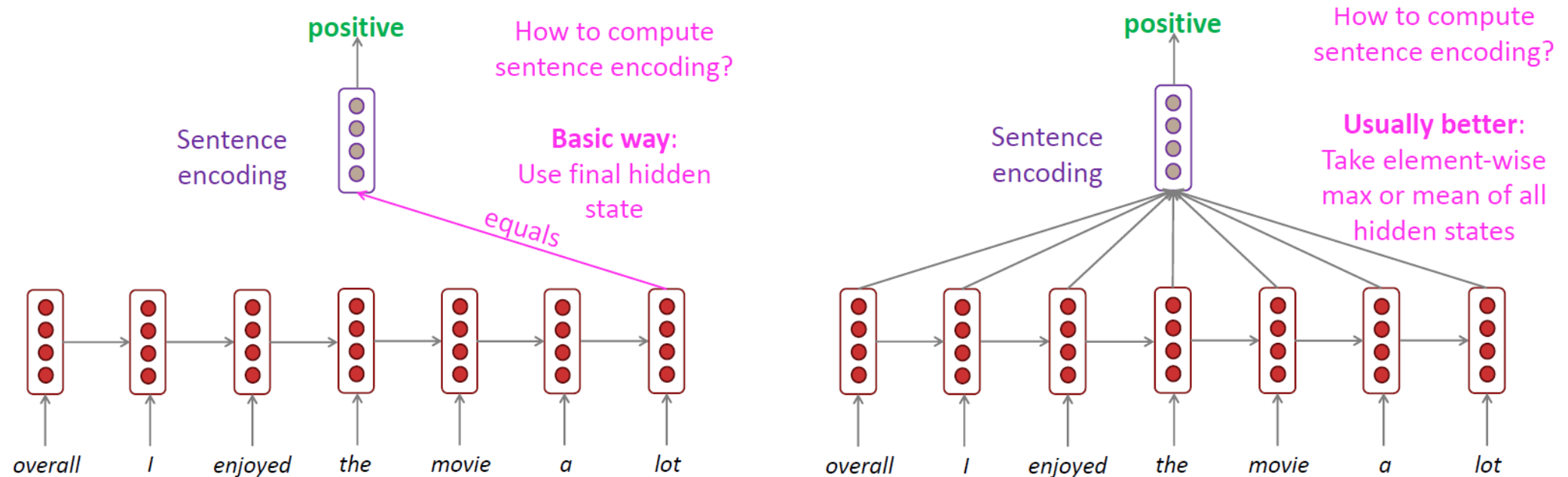
LSTM-RNNs

- LSTM block architecture
 - 3 gates
 - Input gate adjust the influence from input to cell
 - Forget gate adjust the influence from cell to cell over time
 - Output gate adjust the influence from cell to output



RNN for Encoding Sentence

- Usage on sentiment classification



Summary

- NLP involves preprocessing text, representing words as numerical vectors, and training models to understand and generate language.
- **Tokenization** breaks raw text into meaningful units (tokens) like words or sub-words, which are then mapped to indices in a vocabulary.
- **Word Embeddings** (like Word2Vec) learn dense, low-dimensional vectors for these tokens, capturing semantic meaning by placing words with similar contexts close together in the vector space.
- **Language Modeling** is the task of learning the probability of a sequence of words, which is fundamental for predicting the next word or generating new text.
- **Recurrent Neural Networks (RNNs)** are designed to process sequences of any length by using an internal hidden state that acts as a memory, carrying information from one timestep to the next.
- RNNs are trained using **Backpropagation Through Time**, which unrolls the network and sums gradients at each timestep to update the shared weights.
- Simple RNNs struggle with long-term dependencies due to the **vanishing gradient problem**, which is why more complex architectures like **LSTMs** use internal gates to better control the flow of information and preserve memory.

LG AI Research Special Lecture

딥러닝 자연어처리 기초와 LLM 에이전트

Lecture 03 – LLM의 핵심, 트랜스포머와 어텐션 메커니즘



중앙대학교 AI학과

이환희

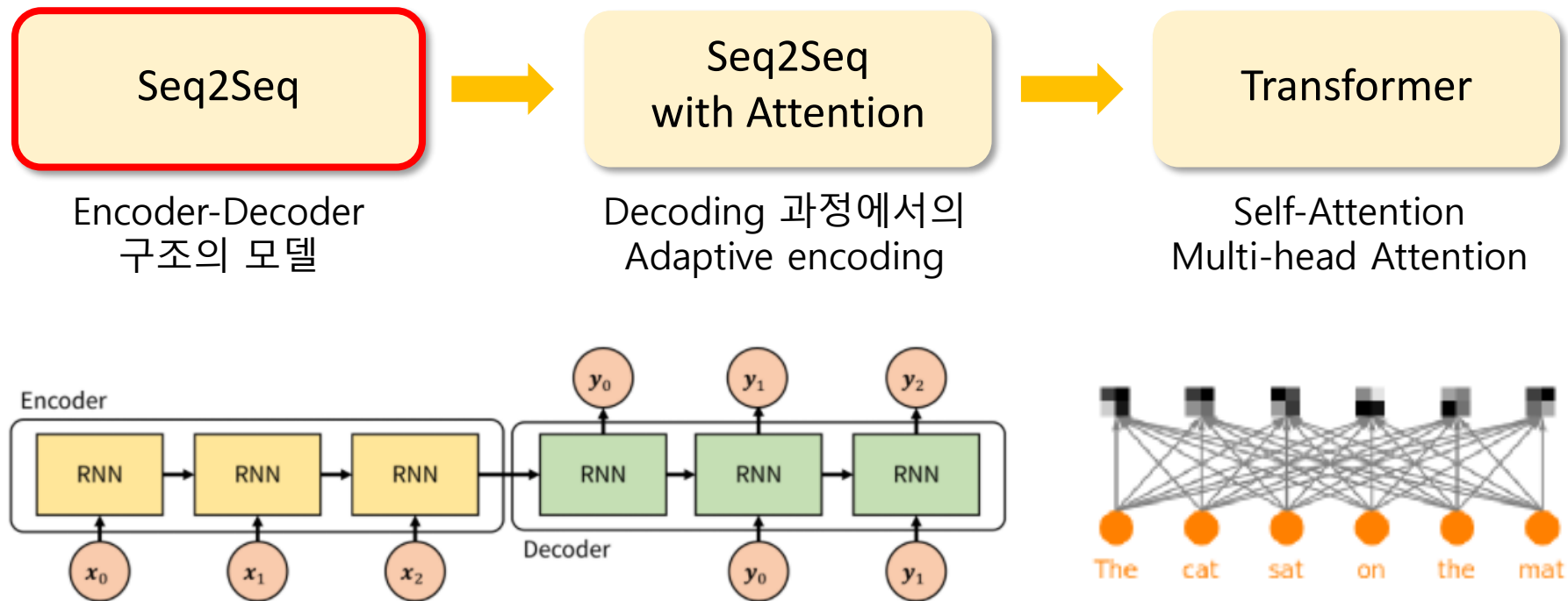
Outline

LLM의 핵심, 트랜스포머와 어텐션 메커니즘

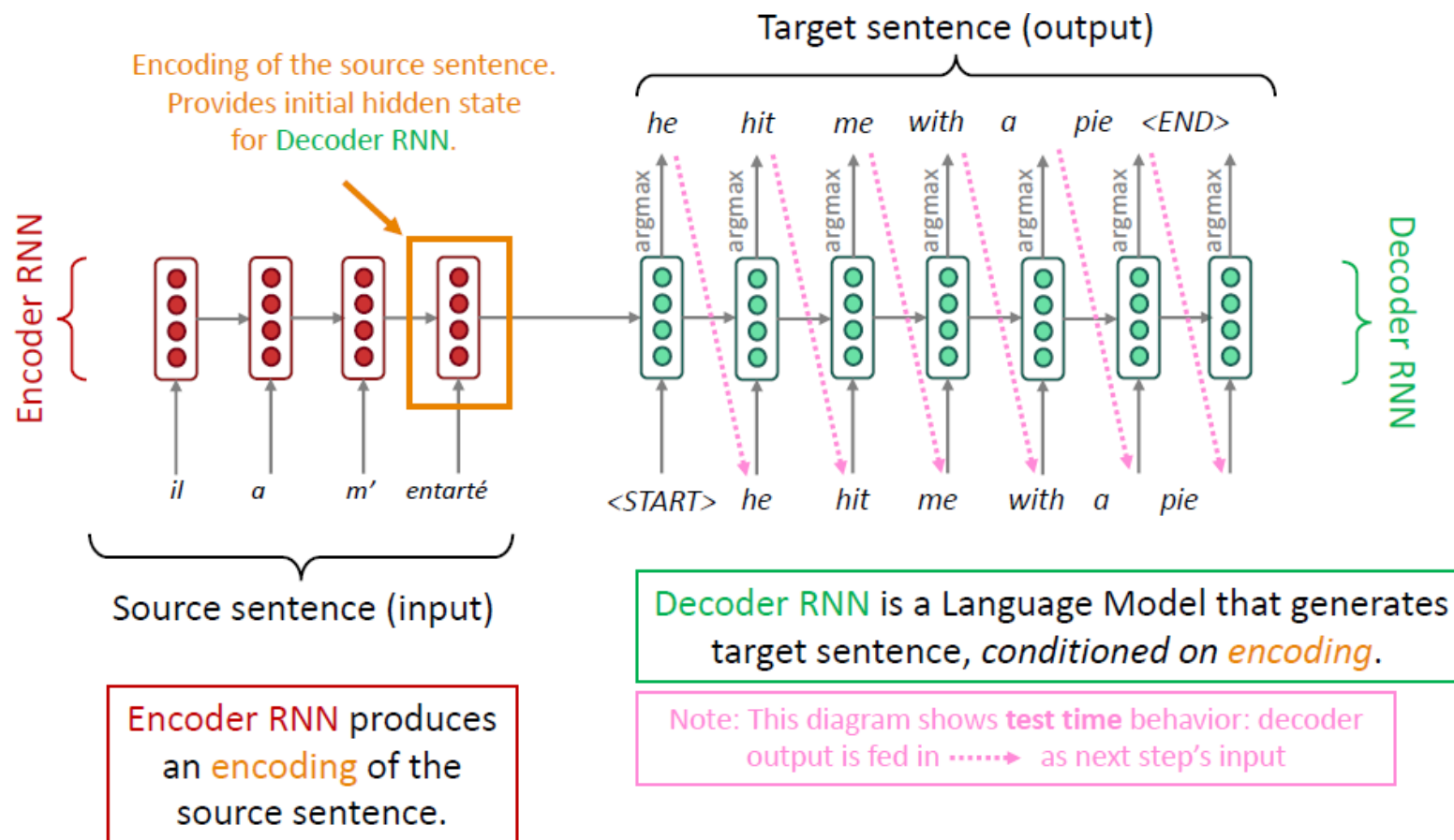
- Seq2seq Model
- Attention Mechanism
- Self-Attention and Transformer
- Language Generation with Transformer

The Evolutionary History of Language Modeling

- Source sequence → Target sequence
 - Machine translation / Dialog generation
 - Parsing sentences into grammar trees

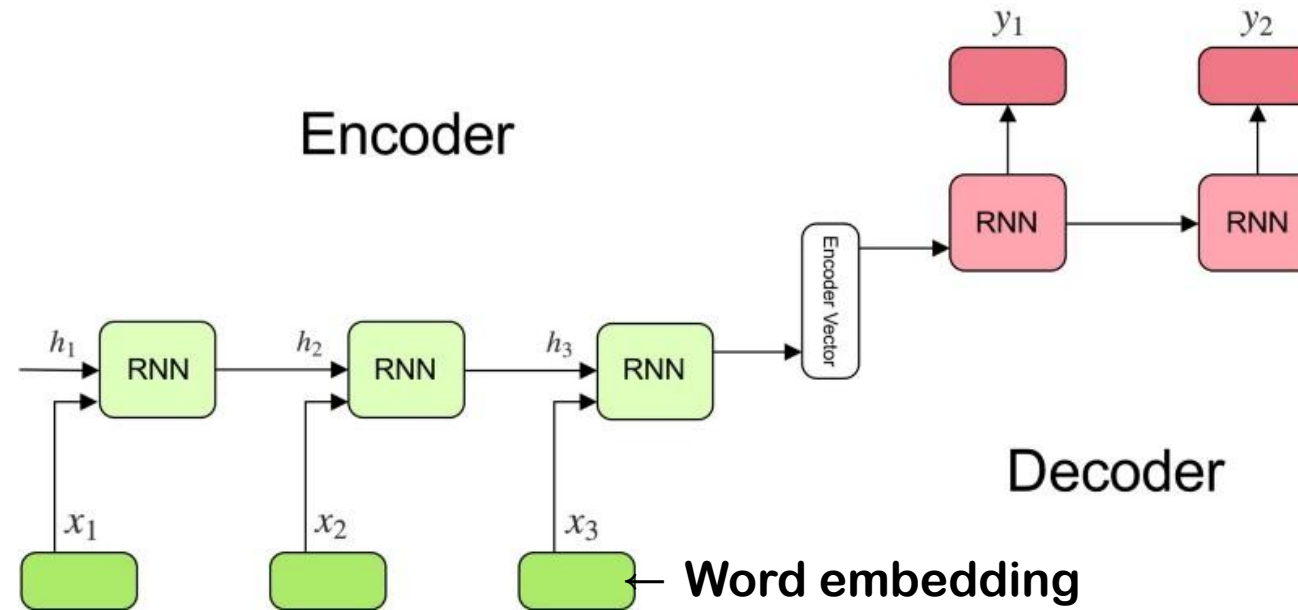


Seq2Seq Model



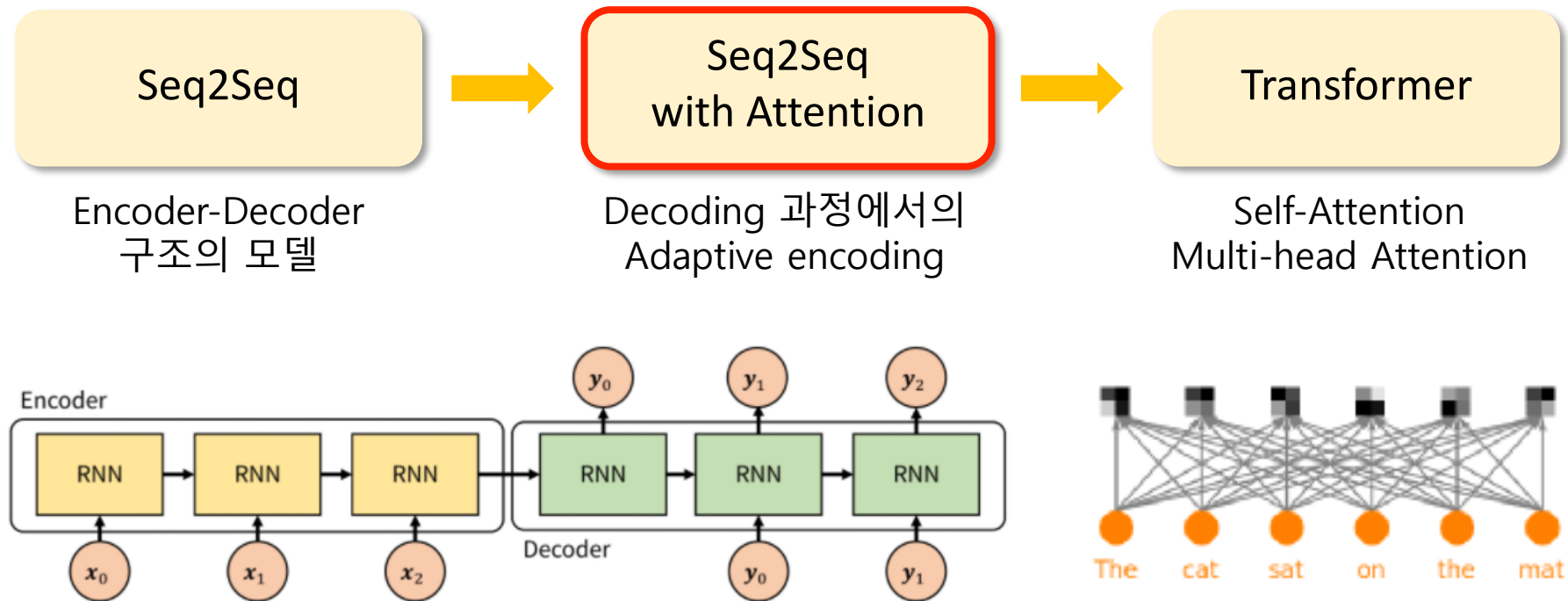
Seq2Seq Encoder-Decoder using RNN

- We need to understand seq2seq encoder-decoder model to know the motivation of 'attention mechanism'.
- Encoder: from word sequence to sentence representation (a real-valued vector).
- Decoder: from representation to word sequence distribution



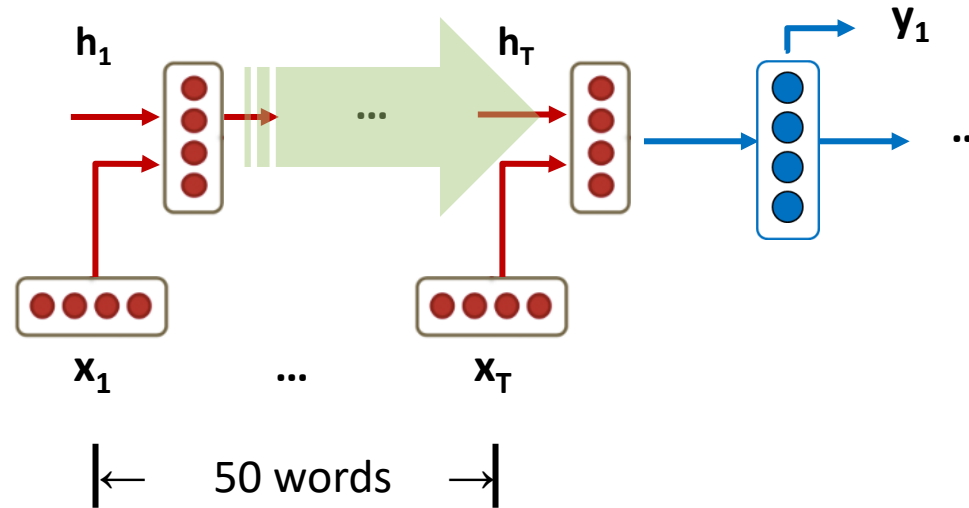
The Evolutionary History of Language Modeling

- Source sequence → Target sequence
 - Machine translation / Dialog generation
 - Parsing sentences into grammar trees



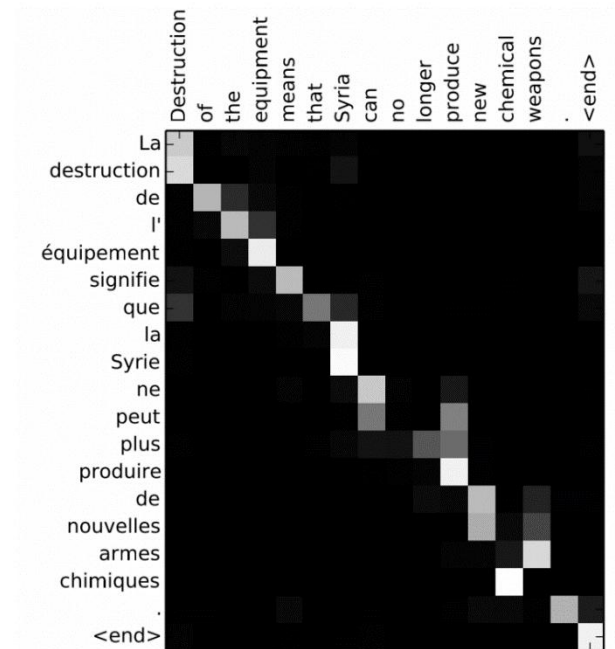
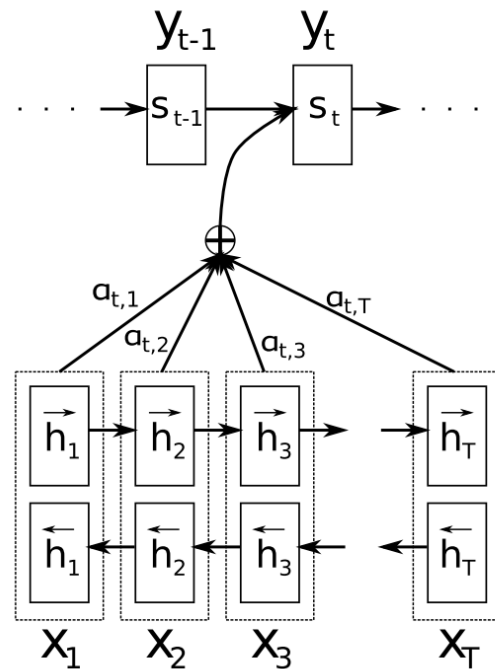
Attention Model – Motivation

- Challenge in vanilla seq2seq for long sentences
- Decoder generates a translation solely based on the last hidden state.
- Information about the first word needs to be encoded in the last hidden state.



Intuition of Attention Mechanism

- Attention mechanism in decoder
- The decoder decides which different parts of the source sentence to pay “attention” at each step of the output generation.



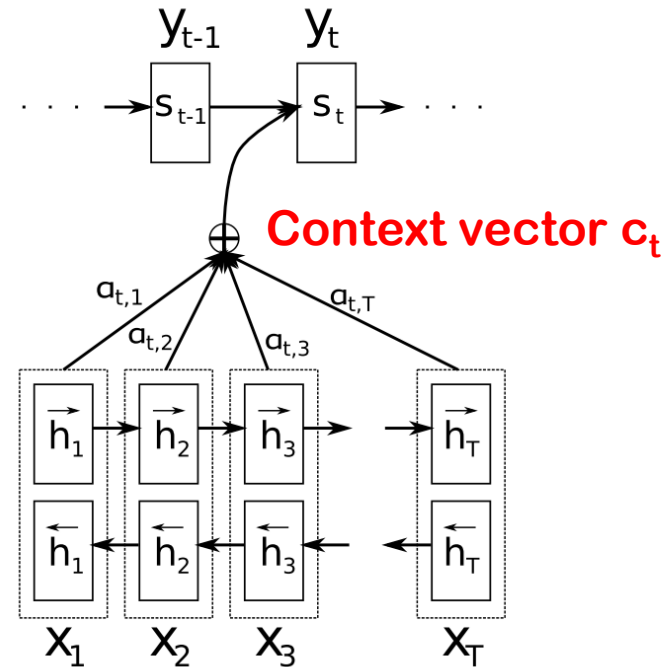
Attention Mechanism with Seq2Seq

- RNN hidden state of the decoder at i : $s_i = f(s_{i-1}, y_{i-1}, c_i)$
- The context vector c_i is computed as a weighted sum of annotations $(h_1, \dots, h_T)$:

$$c_i = \sum_{j=1}^T \alpha_{ij} h_j$$

- How to get attention weight α_{ij} :

Alignment score function



Attention Mechanism – Scoring

- Alignment score function $e_{ij} = \text{score}(s_{i-1}, h_j)$

where s_{i-1} is the RNN hidden state just before emitting i th word,
and h_j is the j th RNN hidden state of the input sentence.

- It scores how well the inputs around position j and the output at position i match.

$$\text{score}(s_{i-1}, h_j) = \begin{cases} s_{i-1}^\top h_j \\ s_{i-1}^\top W_a h_j \\ v_a^\top \tanh(W_a [s_{i-1}; h_j]) \\ \text{single hidden layer network} \end{cases}$$

Attention Mechanism – Normalization

- Let α_{ij} be the probability that the target word y_i is aligned to (or translated from) a source word x_j .
- α_{ij} is computed by normalizing the probabilities with a softmax:

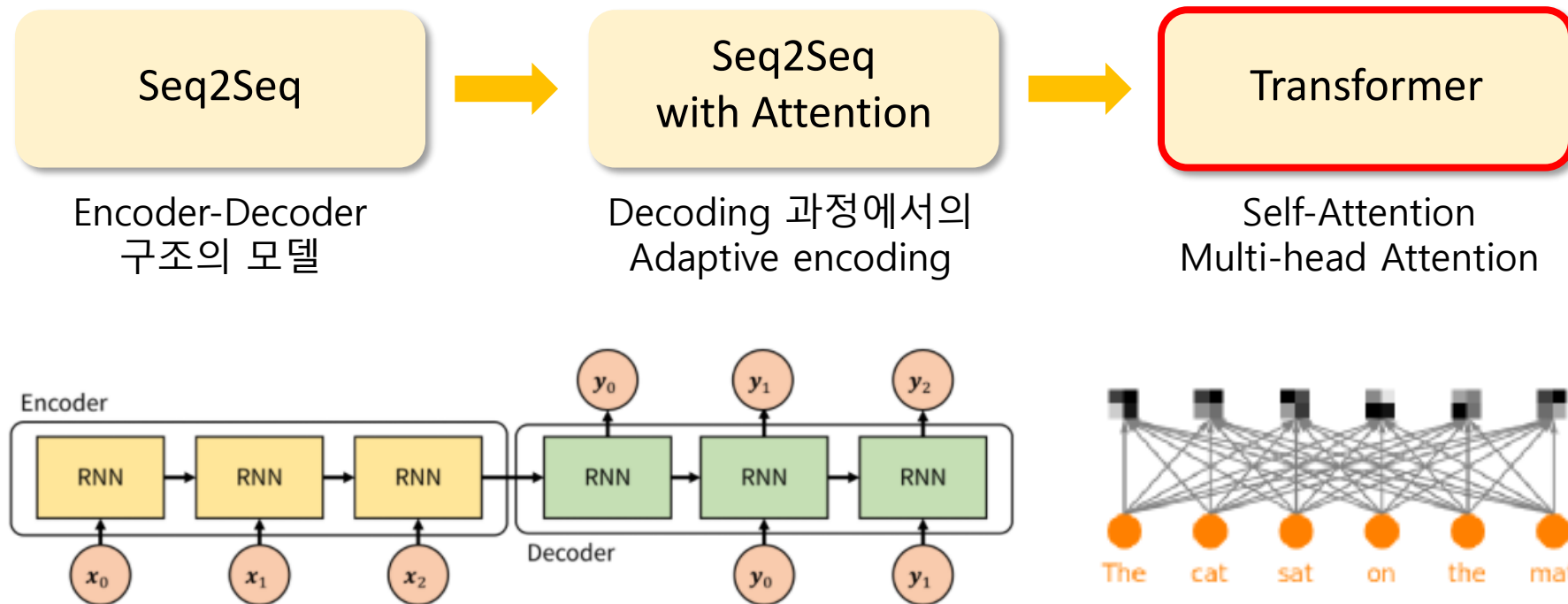
$$\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k=1}^T \exp(e_{ik})}$$

Limitation of RNN

- Difficulty in capturing long-term dependencies:
 - Suffer from the vanishing gradient problem
- Computationally expensive
 - Difficult to train
 - Very long gradient paths
- Inability to process parallel sequences
 - Can only process one input at a time.

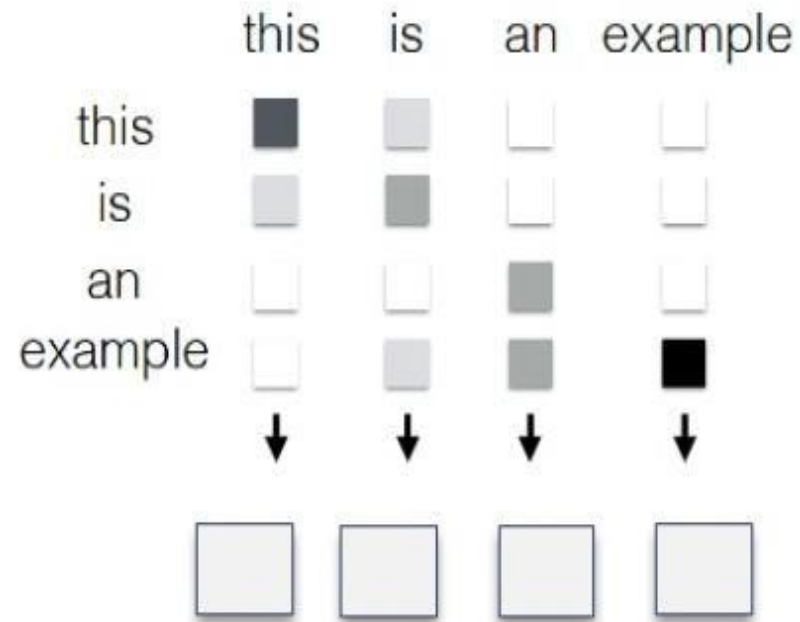
The Evolutionary History of Language Modeling

- Source sequence → Target sequence
 - Machine translation / Dialog generation
 - Parsing sentences into grammar trees



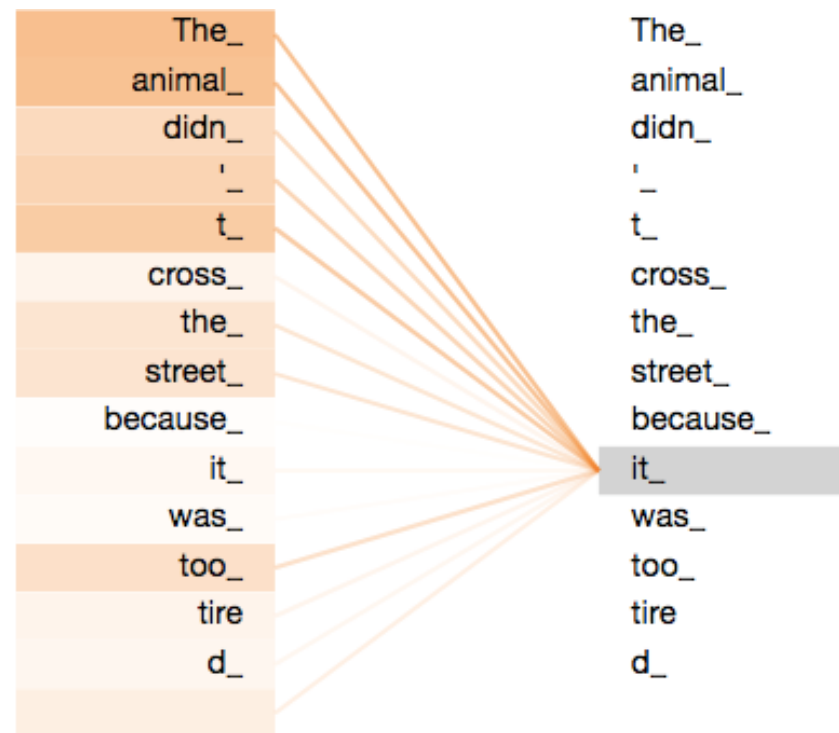
Self Attention

- **Self-Attention:** A technique that calculates the influence of each word on each other within an input sentence and weights it accordingly, and uses it to obtain a new presentation for each word
- Each element in the sentence attends to other elements from the same sentence -> context-sensitive encodings
- Self-attention enhances the automatic understanding of text



Self Attention & Transformer

- **Self-Attention:** A technique that calculates the influence of each word on each other within an input sentence and weights it accordingly, and uses it to obtain a new presentation for each word
- **Transformer:** Multi-layered artificial neural network model using *Self-Attention* technique



Representation for "it" is obtained by intensive reference to "animal."

RNN vs Transformer

Better
Faster
Stronger

- **RNN**

- **Recurrent module** : Suitable for processing variable length representations

Hard to parallelize efficiently (sequential nature)

No explicit modeling of long-range dependencies

Maximum path length : $O(n)$
 n : sequence length

- **Transformer**

- **Self-attention mechanism** : Consists of fixed-size matrix multiplications

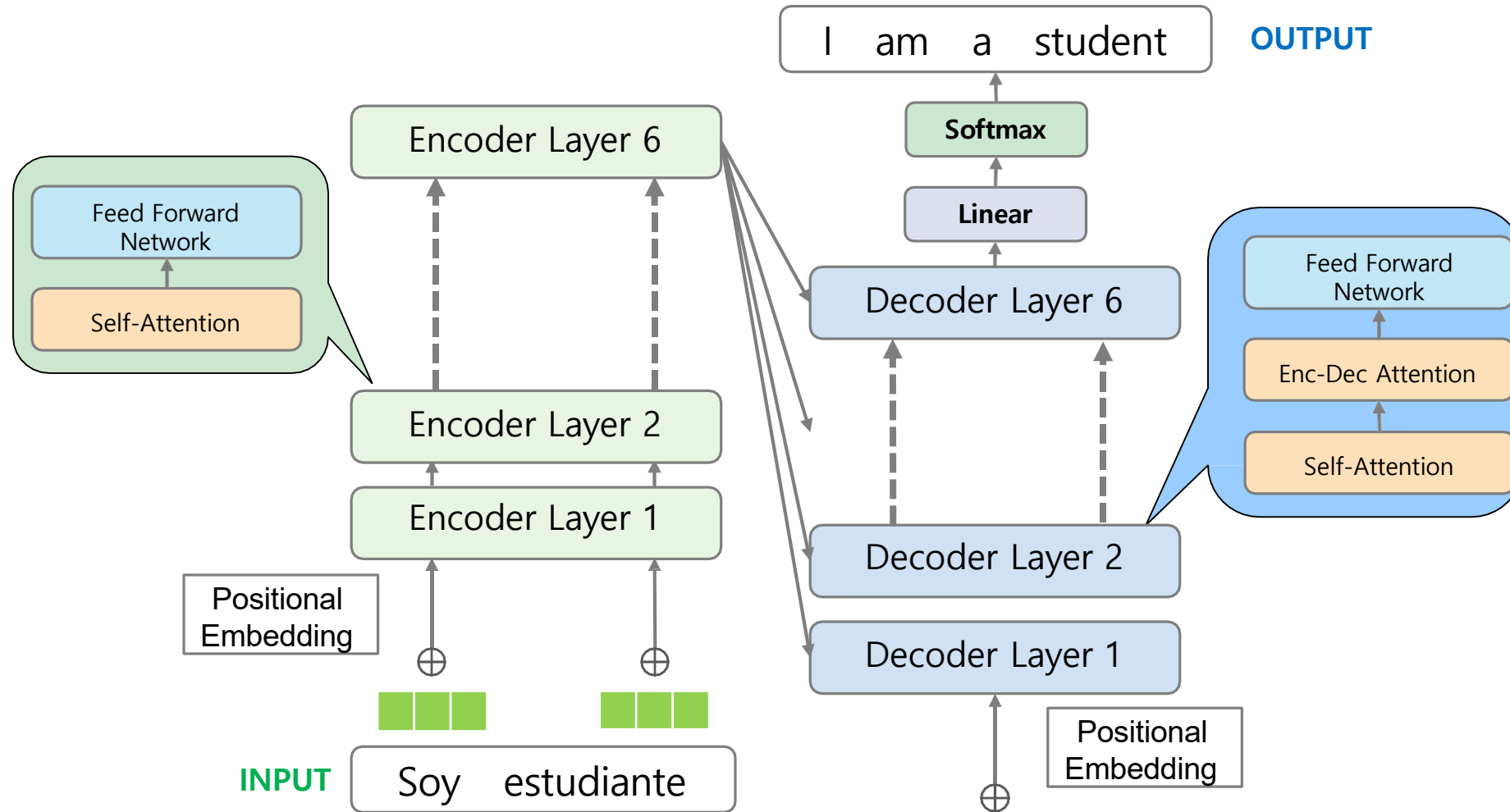
Easy to parallelize → **Fast**

Complete connections between consecutive layers

Maximum path length : $O(1)$

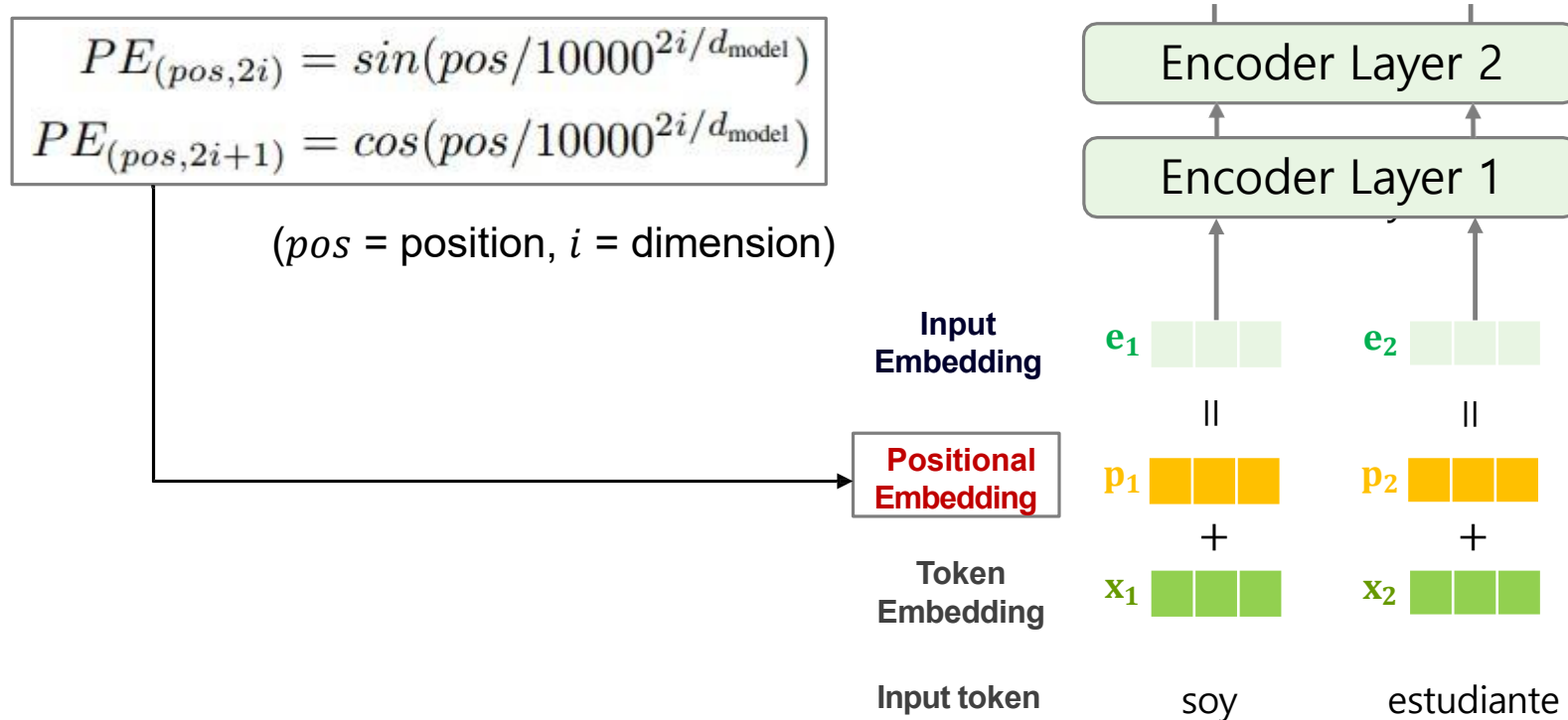
distance between any pair of input and output positions in networks

Transformer: Model Overview



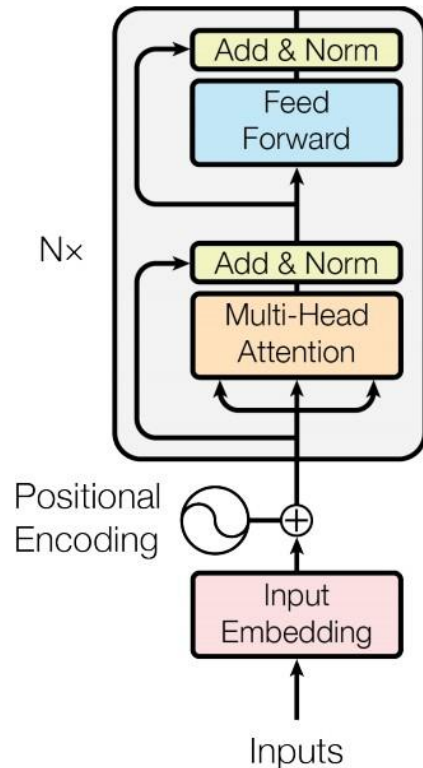
Transformer: Positional Embedding

- **Positional Embedding**
 - Information about the relative or absolute position of the tokens in the sequence
(to utilize the order of the sequence)



Transformer: Encoder

- Encoder



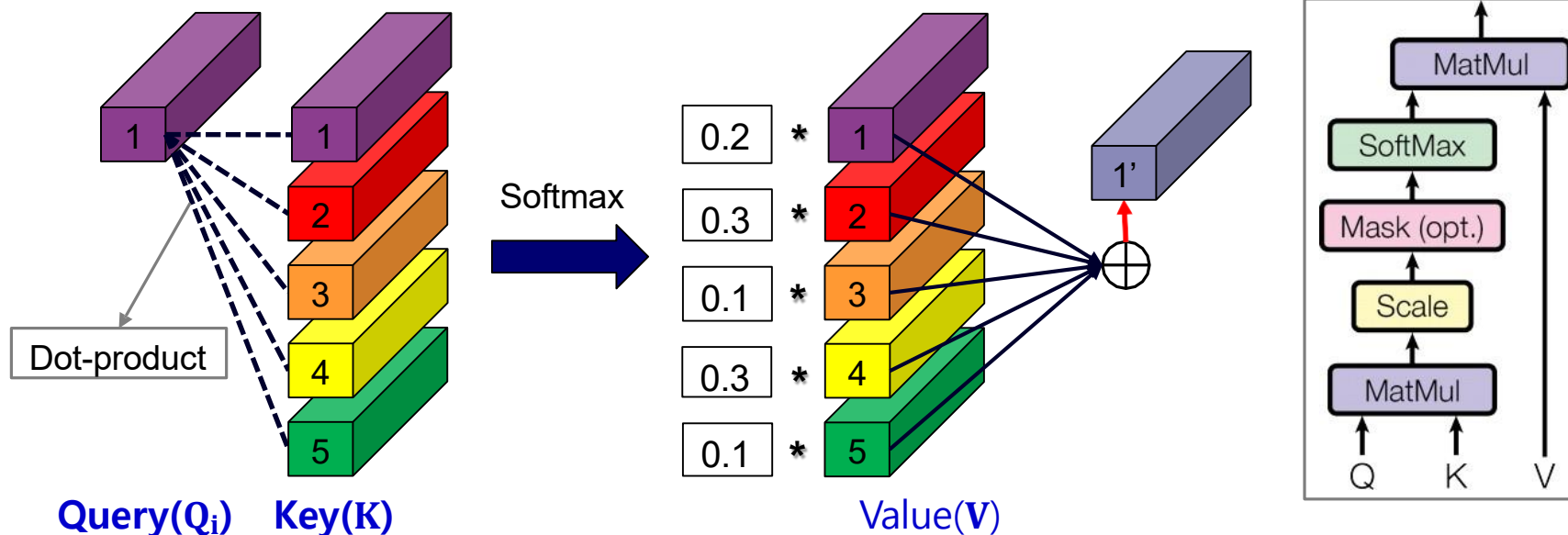
- A stack of $N=6$ identical layers
- Each layer has two sub-layers
 - (1) Multi-Head Self-attention
 - (2) Position-wise Feed Forward Network
- The output of each sub-layer
= $\text{LayerNorm}(\underbrace{x}_{\text{residual connection}} + \text{Sublayer}(x))$
- All of the queries, keys, and values come from the output of the previous layer ($Q=K=V$)

Transformer: Self-Attention

- **Self-attention** (Scaled Dot-product Attention)
 - Computing a representation of a sequence considering the relationship between different positions

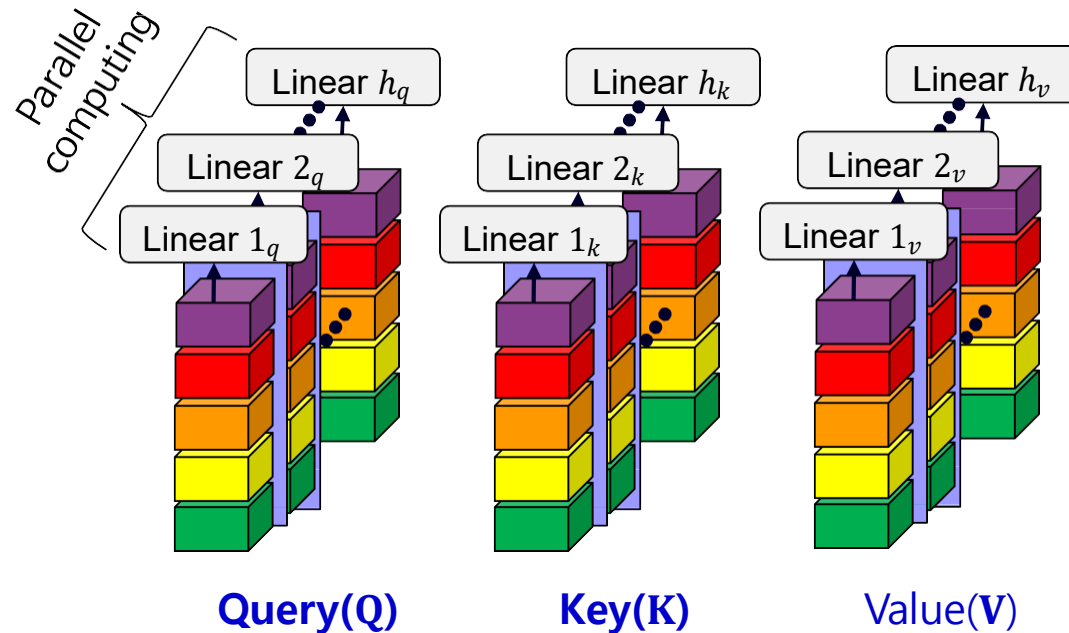
The animal didn't cross the ...
[1] [2] [3] [4] [5]

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



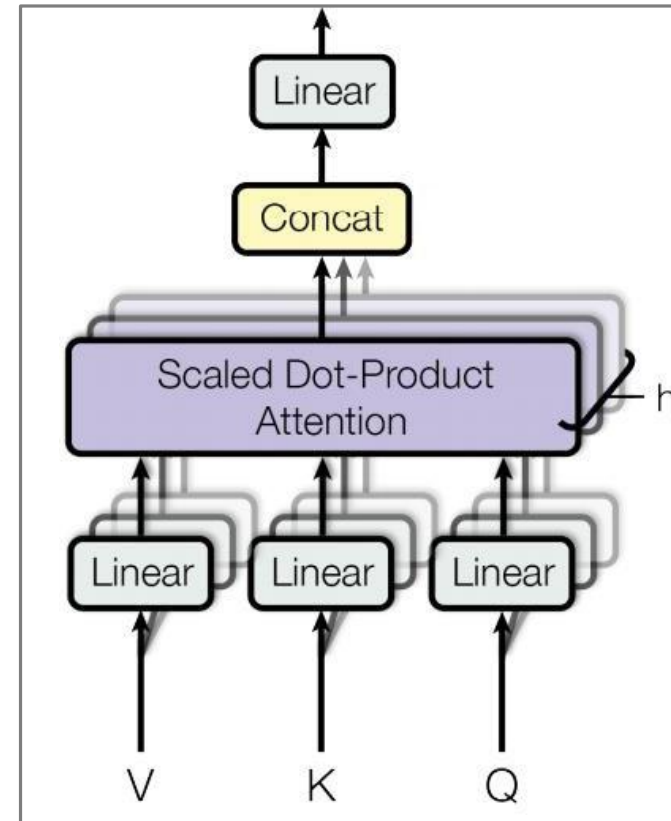
Transformer: Self-Attention

- **Multi-Head Attention**
 - Apply the “Scaled Dot-Product Attention” several times with different linear projections



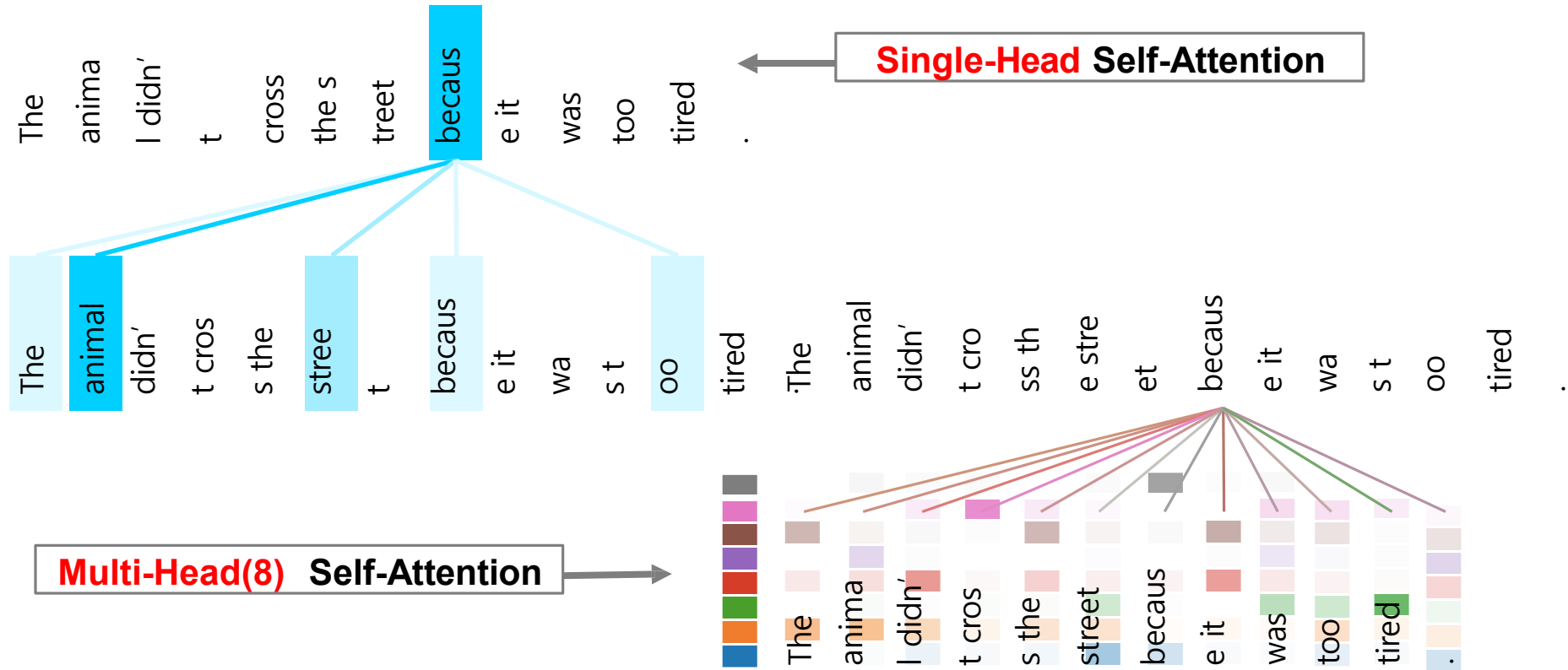
$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, \dots, \text{head}_h) W^O$$

where $\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V)$



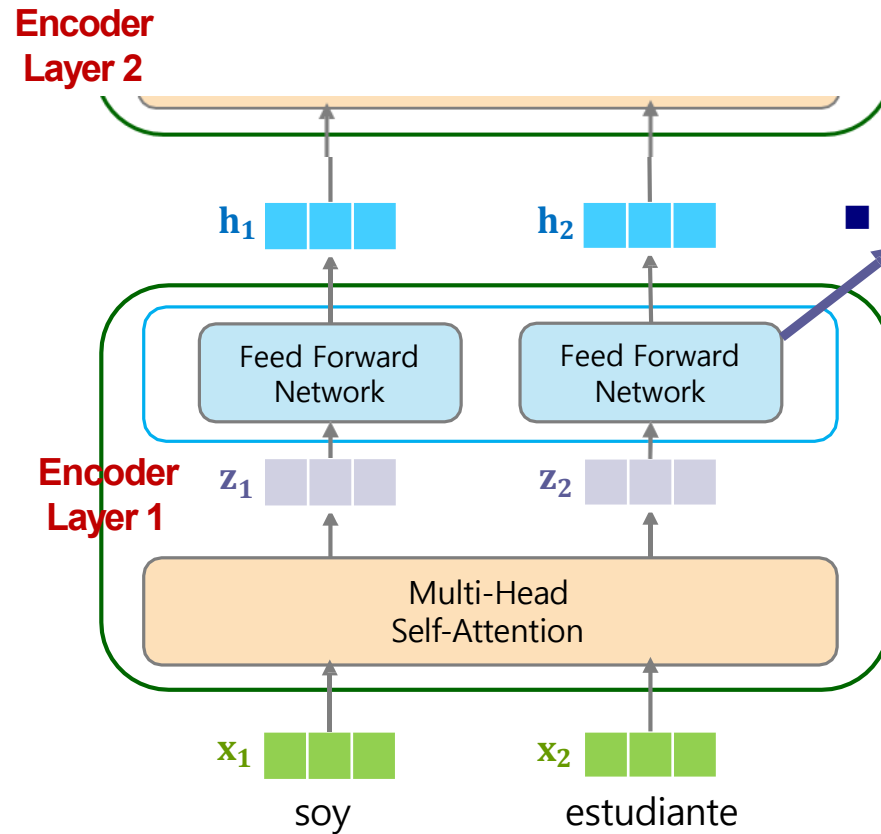
Transformer: Self-Attention

- Self-Attention Visualization



Transformer: Encoder Architecture

- Encoder



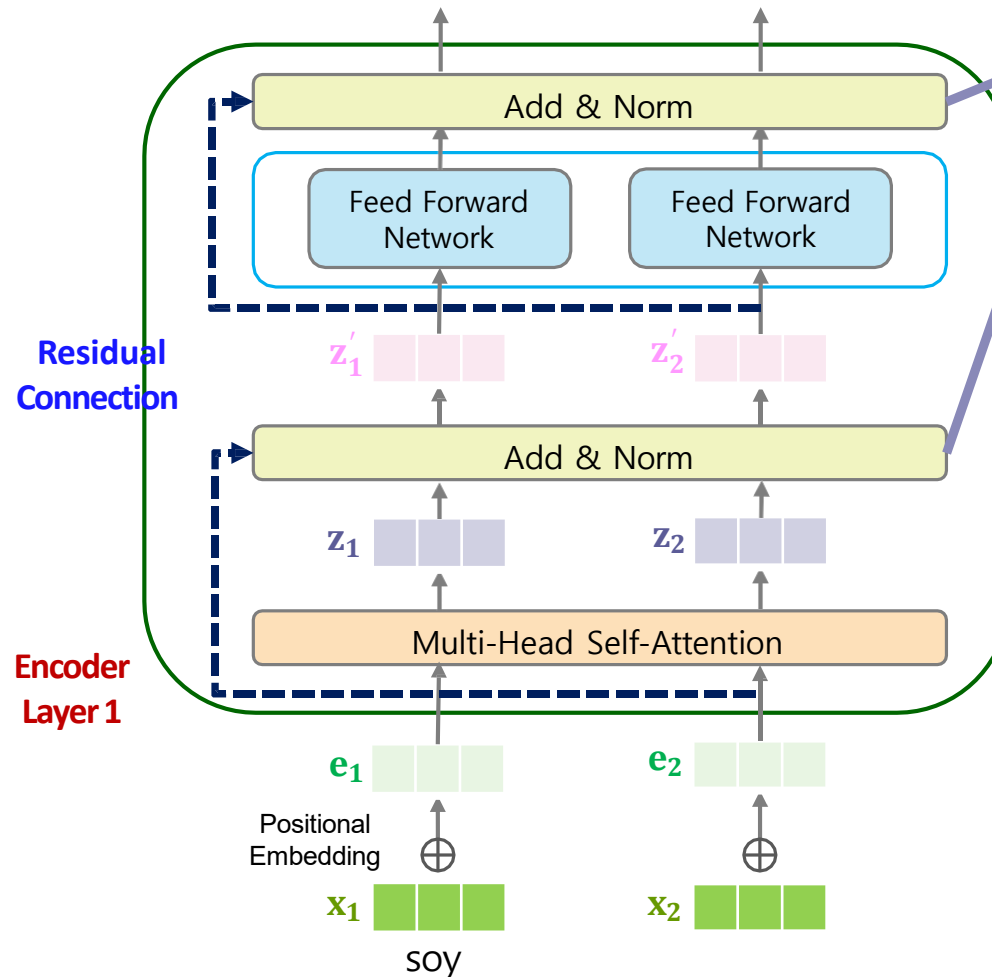
Point-wise Feed Forward Network

- Applied to each position separately and identically (different parameters from layer t to layer $t+1$)
- Two linear transformations with ReLU activation in between

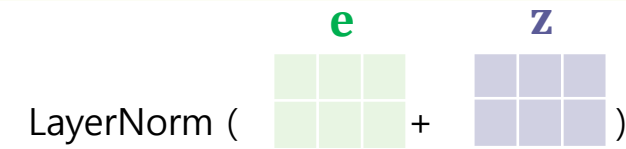
$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Transformer: Residual Connection

- Residual Connection

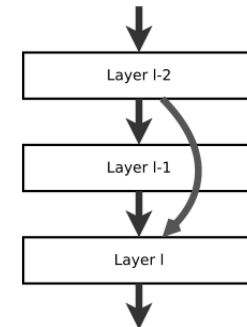


LayerNorm ($x + \text{Sublayer}(x)$)



- * Residual Connection

- Skip connection
- Short-cuts to jump over some layers



- * Layer Normalization

- Normalizes the inputs across the features

batch		
1	3	6
2	2	2
0	1	5
4	6	1
5	2	3
1	0	1
mean		
2	3	3
std		
2	2	2

$$\mu_i = \frac{1}{m} \sum_j x_{ij}$$

$$\sigma_i = \frac{1}{m} \sum_j (x_{ij} - \mu_i)^2$$

$$\hat{x}_{ij} = \frac{x_{ij} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}}$$

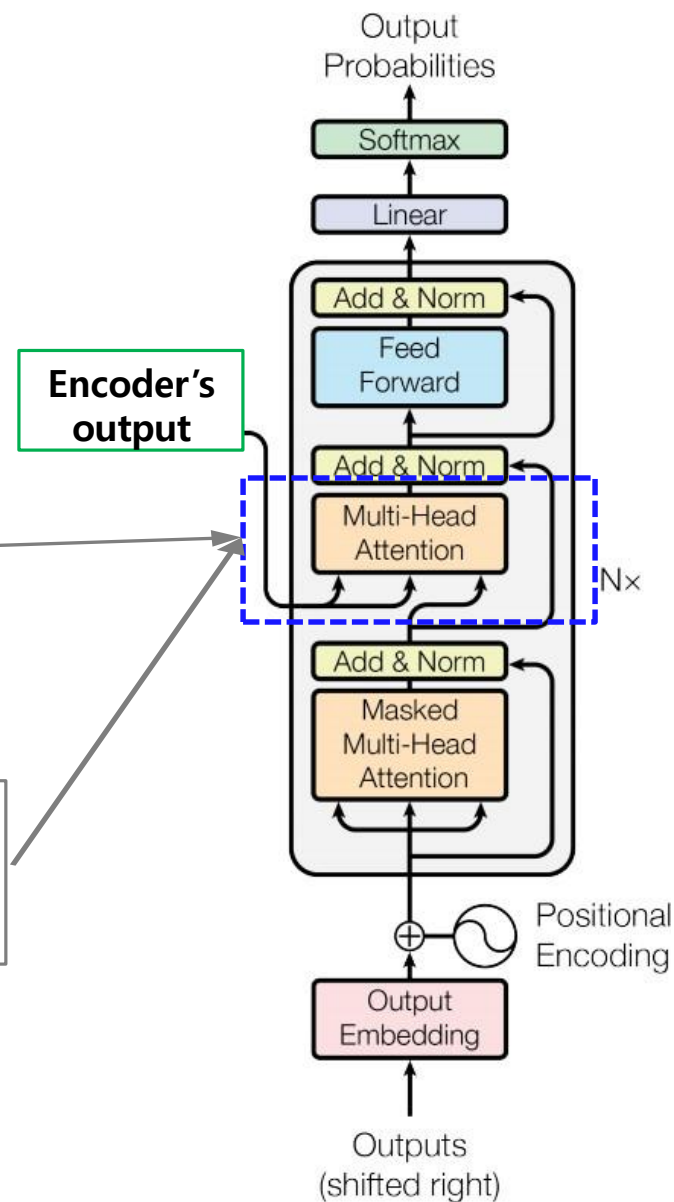
$\nwarrow$ batch-dim
 $\swarrow$ feature-dim

Transformer: Decoder

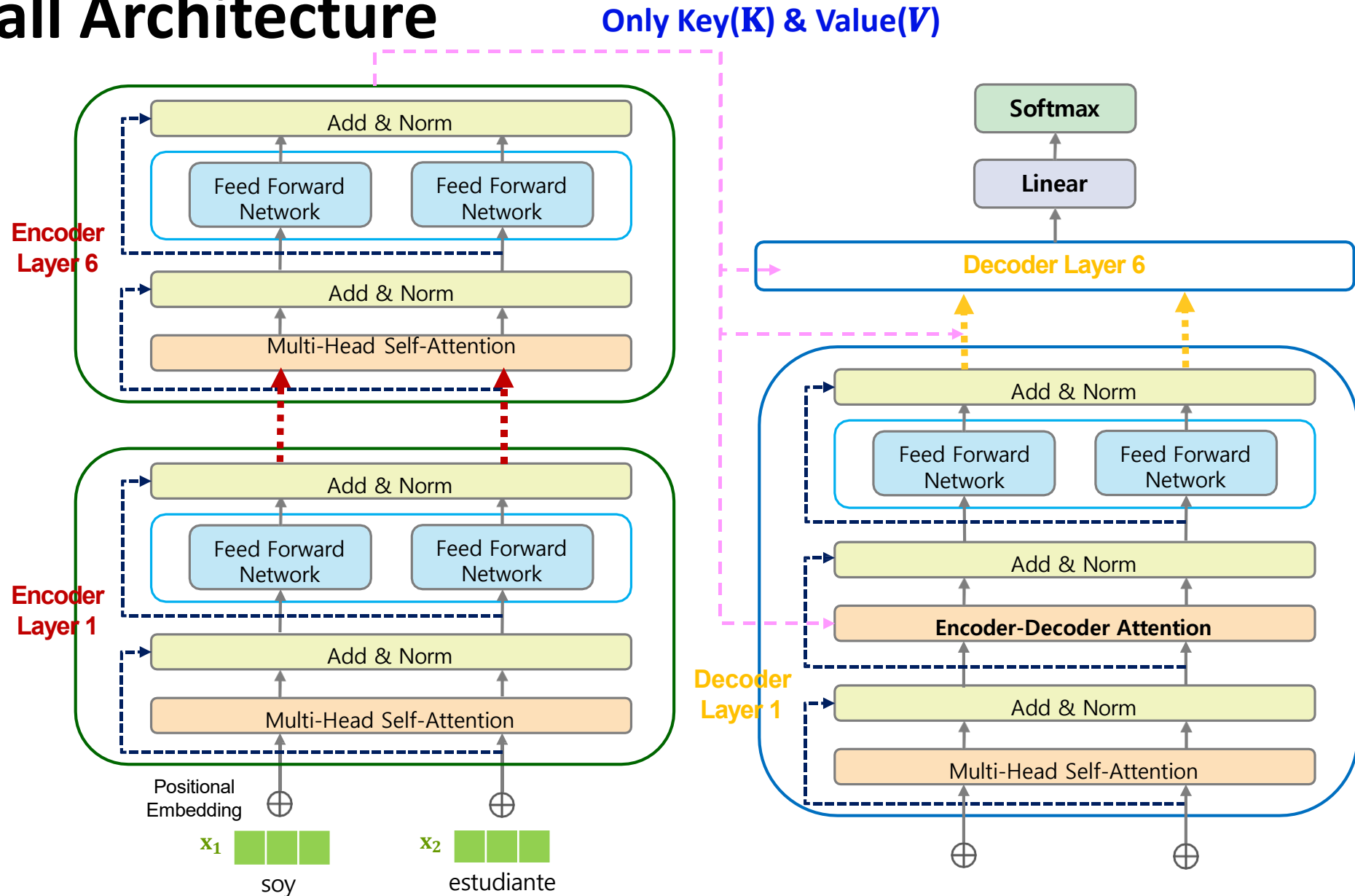
- **Decoder** (almost the same as encoder)

- A stack of $N=6$ identical layers
- Each layer has three sub-layers
 - (1) Multi-Head Self-attention
 - (2) Multi-Head Attention over the output of the encoder stack (enc-dec attention)
 - (3) Position-wise Feed Forward Network

The queries Q come from the previous decoder layer, and the key K and value V come from the output of the encoder

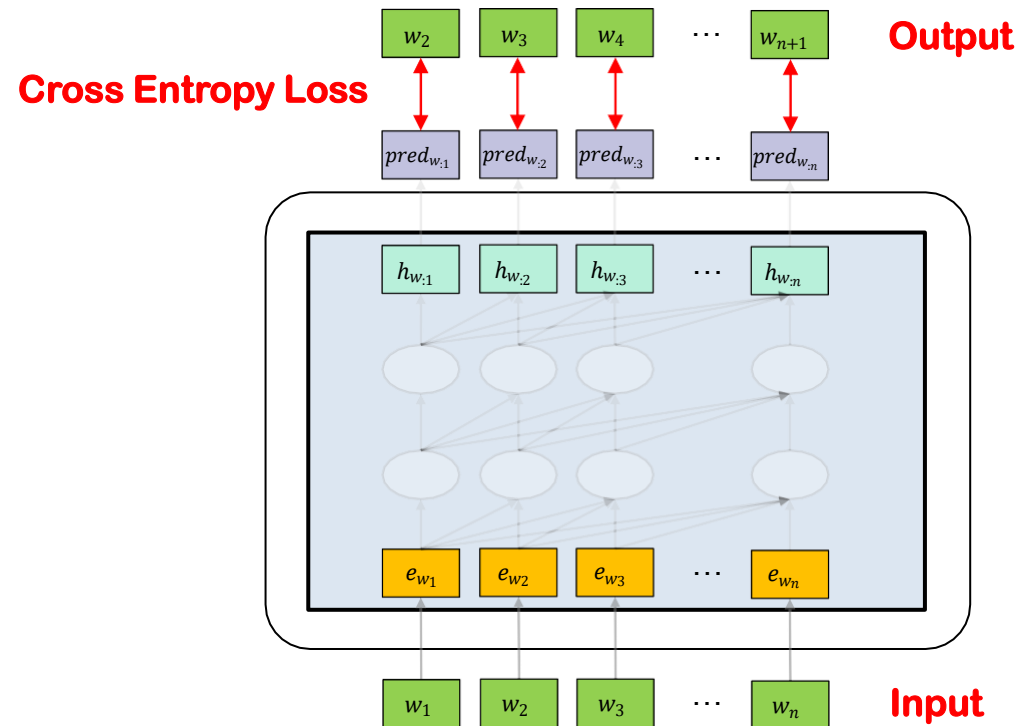


Overall Architecture



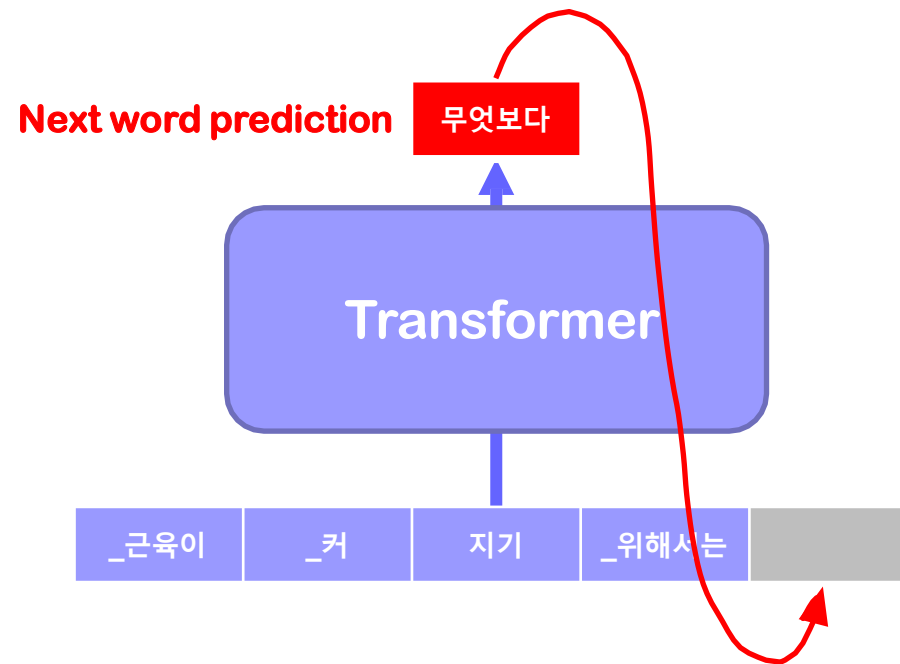
Natural Language Generation with Transformer: Training

- We can train transformer using cross entropy loss
- We can understand the process of transformer training as a classification problem



Natural Language Generation with Transformer: Inference

- The generation of text is performed by repeatedly predicting the next word
- The predicted word is inserted at the end of the sentence and used as the input of the GPT model



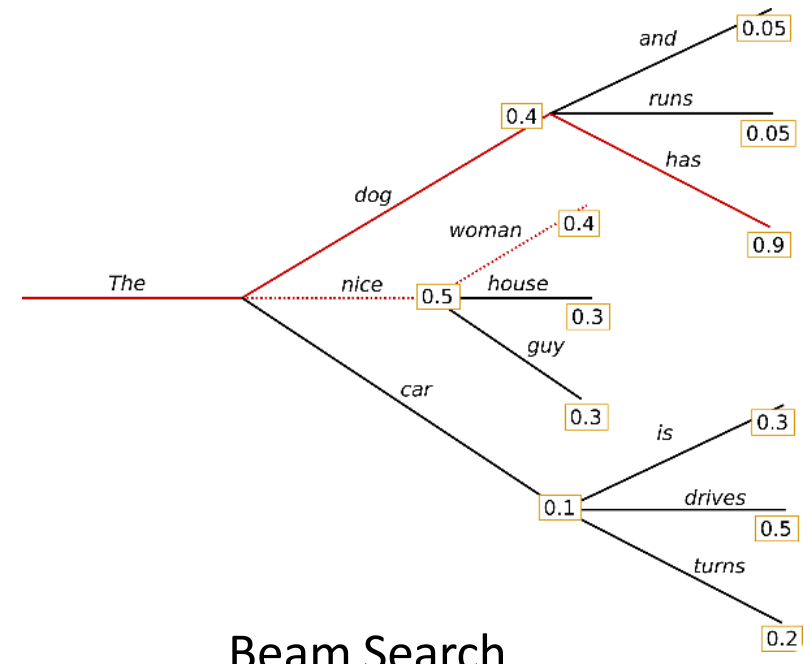
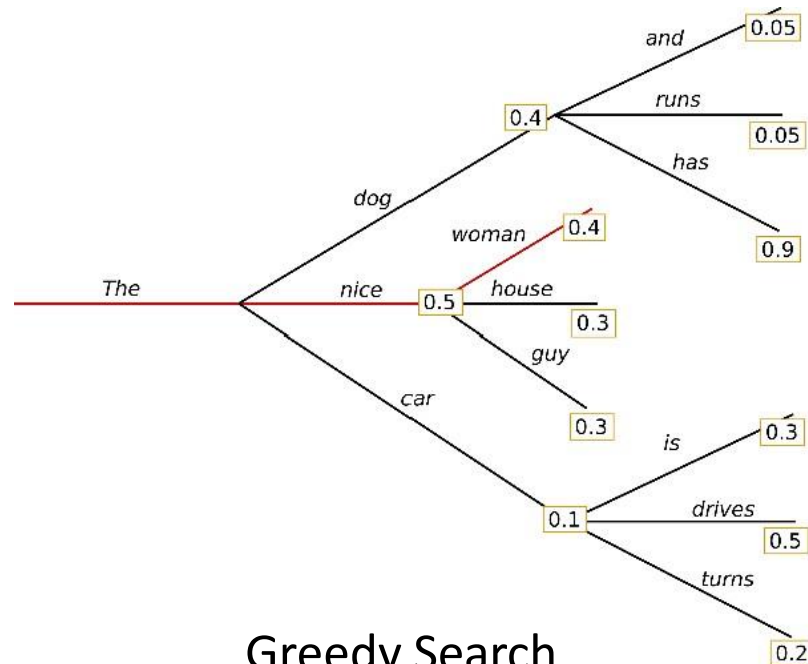
Natural Language Generation with Transformer: Inference

- The generation of text is performed by repeatedly predicting the next word
- The predicted word is inserted at the end of the sentence and used as the input of the GPT model



Natural Language Generation: Search Algorithm

- Greedy search just selects the word with the highest probability
- Beam search keeps the most likely *num_beams* of hypotheses at each time step and eventually choosing the highest probability path



Summary

- We trace the evolution from RNN-based models to the Transformer, driven by the need to solve long-range dependencies and enable parallel processing.
- **Seq2Seq (Sequence-to-Sequence)** models use an **Encoder** RNN to compress an entire input sequence into a single "context vector," and a **Decoder** RNN to generate an output sequence from that vector.
- The **information bottleneck** of this single vector was solved by the **Attention Mechanism**, allowing the Decoder to "look back" at all input hidden states and assign "attention scores" to focus on the most relevant parts of the input for each step.
- Despite this improvement, RNNs remain slow due to their sequential nature, leading to the **Transformer** architecture, which removes RNNs entirely.
- The Transformer's core component is **Self-Attention**, a mechanism that allows each word in a sequence to look at and weigh the importance of all other words *in the same sequence*, creating a deeply context-aware representation.
- The full **Transformer** model is an Encoder-Decoder stack built from **Multi-Head Attention** (running self-attention in parallel) and **Feed-Forward Networks**, using **Positional Embeddings** to retain word order information.
- During inference, this model generates text **autoregressively** by predicting one word at a time, feeding that prediction back into the input, and then predicting the next word, often using a **Beam Search** algorithm to find the most probable sequence.

LG AI Research Special Lecture

딥러닝 자연어처리 기초와 LLM 에이전트

Lecture 04 – 거대 언어 모델의 사전학습과 진화



중앙대학교 AI학과

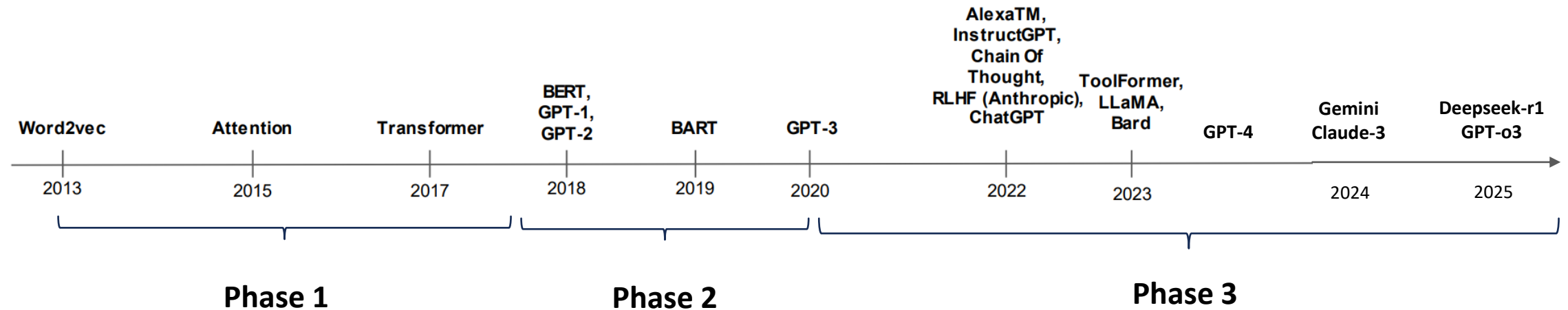
이환희

Outline

거대 언어 모델의 사전학습과 진화

- Pre-trained Word Embeddings
- Pre-training Whole Models
- In-context Learning
- Reinforcement Learning from Human Feed back
- InstructGPT
- ChatGPT

History of Large Language Models



NLP Milestone Papers

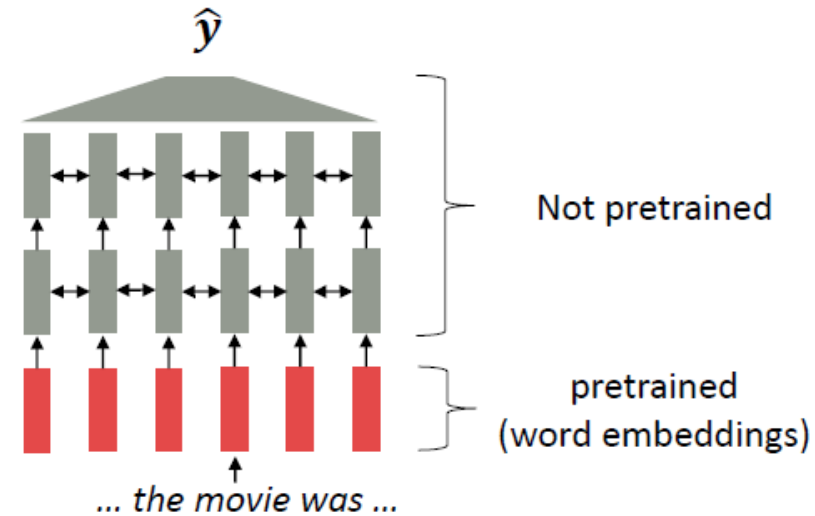
- **Word2Vec**: Mikolov et al. "Distributed representations of words and phrases and their compositionality." NeurIPS 2013
 - **Attention**: Bahdanau et al. "Neural machine translation by jointly learning to align and translate." ICLR 2015
 - **Transformer**: Vaswani et al. "Attention is all you need." NIPS 2017.
-
- **BERT**: Devlin et al. "Bert: Pre training of deep bidirectional transformers for language understanding." NAACL 2019 *Pre-training LM*
 - **GPT-1**: Radford et al. "Improving language understanding by generative pre training." (2018).
 - **GPT-2**: Radford et al. "Language models are unsupervised multitask learners." OpenAI blog 1.8 (2019): 9.
 - **BART**: Lewis et al. "Bart: Denoising sequence to sequence pre training for natural language generation, translation, and comprehension." ACL 2020
-
- **GPT-3**: Brown et al. "Language models are few shot learners, NeurIPS 2020 *Pre-training LLM*
 - **InstructGPT**: Ouyang et al. "Training language models to follow instructions with human feedback.", arXiv
 - **Chain Of Thought**: Wei et al. "Chain of thought prompting elicits reasoning in large language models." arXiv:2201.11903
 - **RLHF** (Anthropic): Bai et al. "Training a helpful and harmless assistant with reinforcement learning from human feedback ." arXiv:2204.05862
 - **ChatGPT**: No paper, still secrets from OpenAI
 - **LLaMA**: Touvron et al. "LLaMA: Open and Efficient Foundation Language Models." arXiv:2302.13971
 - **GPT-4**: GPT-4 Technical Report
 - **DeepSeek-R1**: DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning, arXiv:2501

Pre-trained Word Embeddings

- Start with pre-trained word embeddings (no context!)
- Learn how to incorporate context in an LSTM or Transformer while training on the task.

Some issues to think about:

- The training data we have for our **downstream task** (like question answering) must be sufficient to teach all contextual aspects of language.
- Most of the parameters in our network are randomly initialized!

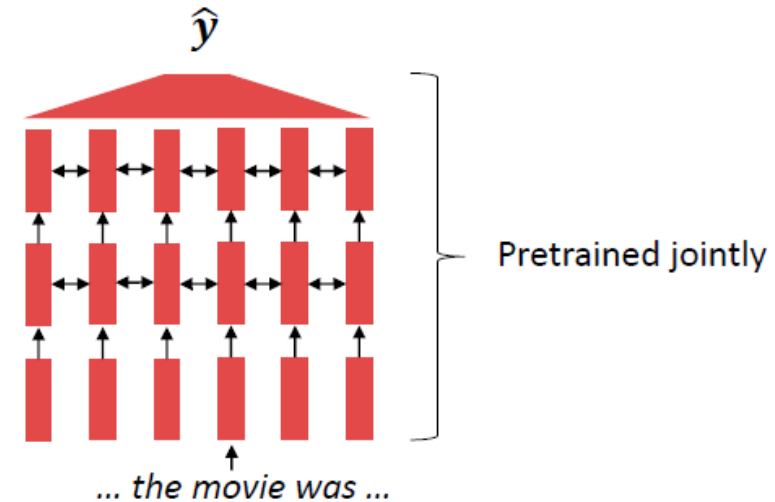


[Recall, *movie* gets the same word embedding, no matter what sentence it shows up in]

Pre-trained Whole Models

In modern NLP:

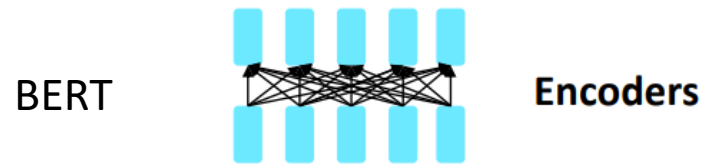
- All (or almost all) parameters in NLP networks are initialized via **pretraining**.
- Pretraining methods hide parts of the input from the model, and train the model to reconstruct those parts .
- This has been exceptionally effective at building strong:
 - **representations of language**
 - **parameter initializations** for strong NLP models.
 - **Probability distributions** over language that we can sample from



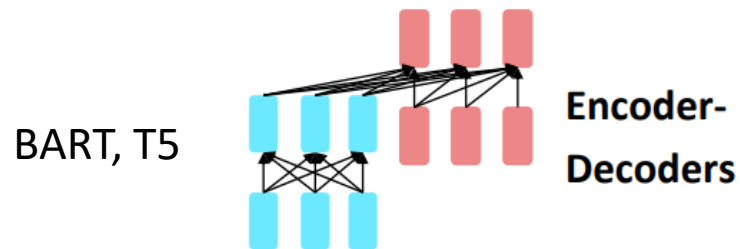
[This model has learned how to represent entire sentences through pretraining]

Pre-training for Three Types of Architectures

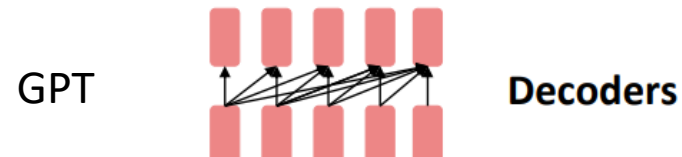
- The neural architecture influences the type of pre-training, and natural use cases.



- Gets bidirectional context – can condition on future!
- How do we train them to build strong representations?



- Good parts of decoders and encoders?
- What's the best way to pretrain them?



- Language models! What we've seen so far.
- Nice to generate from; can't condition on future words

What Can we Learn from Reconstructing the Input?

There's increasing evidence that pretrained models learn a wide variety of things about the statistical properties of language.

- *Chung-Ang University is located in ____, Seoul.* [Trivia]
- *I put ____fork down on the table.* [syntax]
- *The woman walked across the street, checking for traffic over ____shoulder.* [coreference]
- *I went to the ocean to see the fish, turtles, seals, and ____.* [lexical semantics/topic]
- *Overall, the value I got from the two hours watching it was the sum total of the popcorn and the drink. The movie was ____.*
[sentiment]
- *Iroh went into the kitchen to make some tea. Standing next to Iroh, Zuko pondered his destiny. Zuko left the ____.*
[some reasoning – this is harder]
- *I was thinking about the sequence that goes 1, 1, 2, 3, 5, 8, 13, 21, ____*
[some basic arithmetic; they don't learn the Fibonacci sequence]

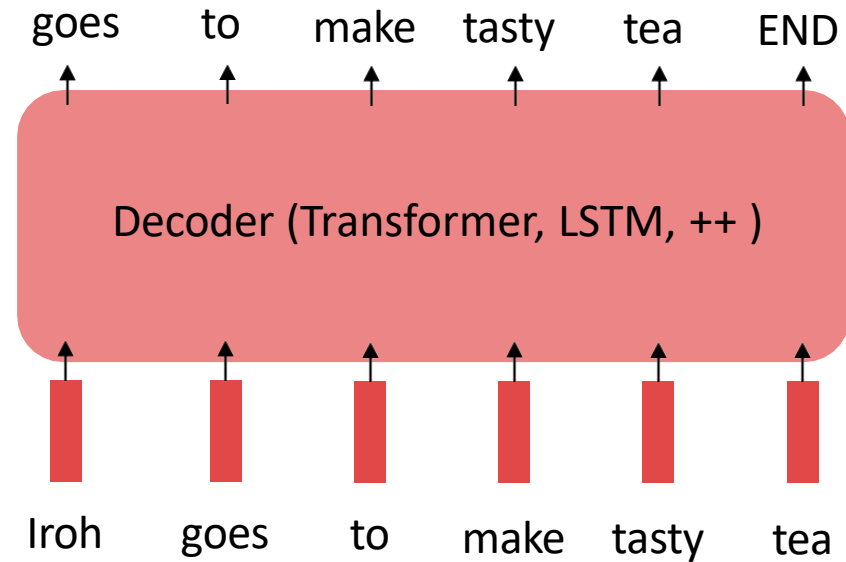
Pre-training through language modeling

Recall the **language modeling** task:

- Model $p_{\theta}(w_t | w_{1:t-1})$, the probability distribution over words given their past contexts.
- There's lots of data for this!

Pretraining through language modeling:

- Train a neural network to perform language modeling on a large amount of text.
- Save the network parameters.

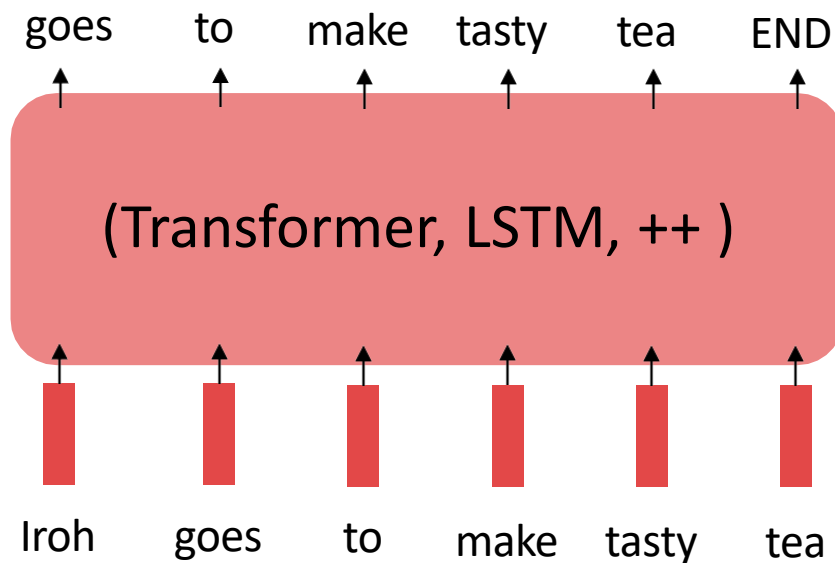


Pre-training / Fine-tuning Paradigm

- Pretraining can improve NLP applications by serving as parameter initialization.

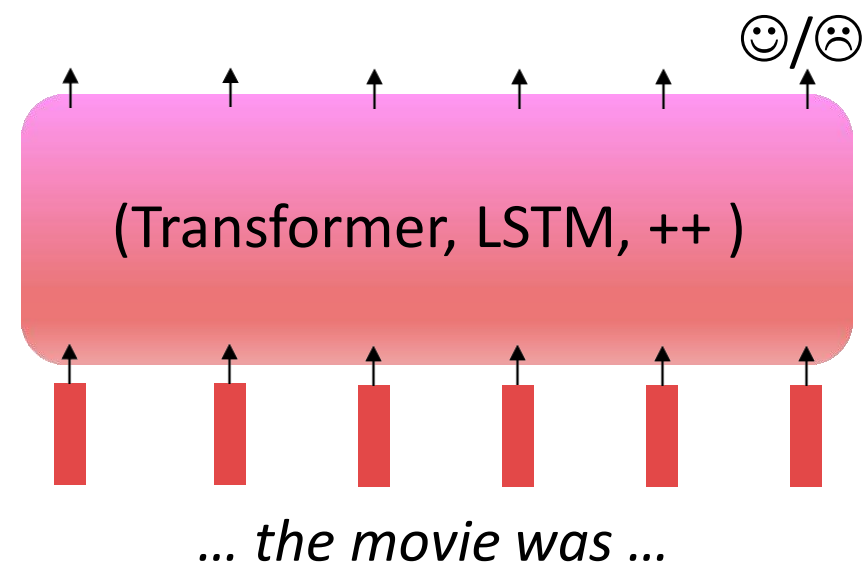
Step 1: Pretrain (on language modeling)

Lots of text; learn general things!



Step 2: Finetune (on your task)

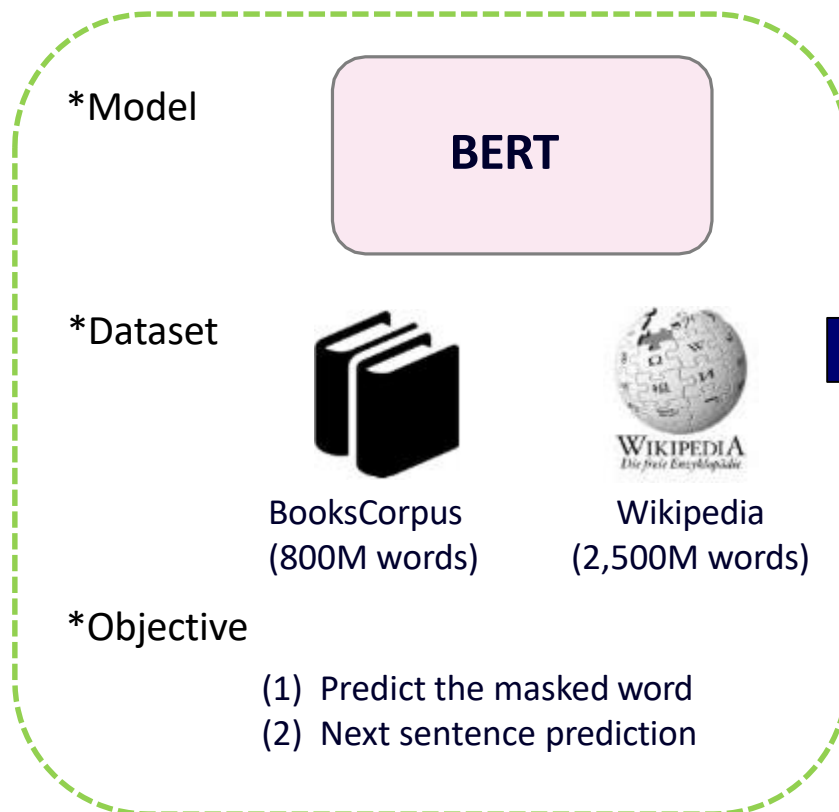
Not many labels; adapt to the task!



Pre-training / Fine-tuning Paradigm

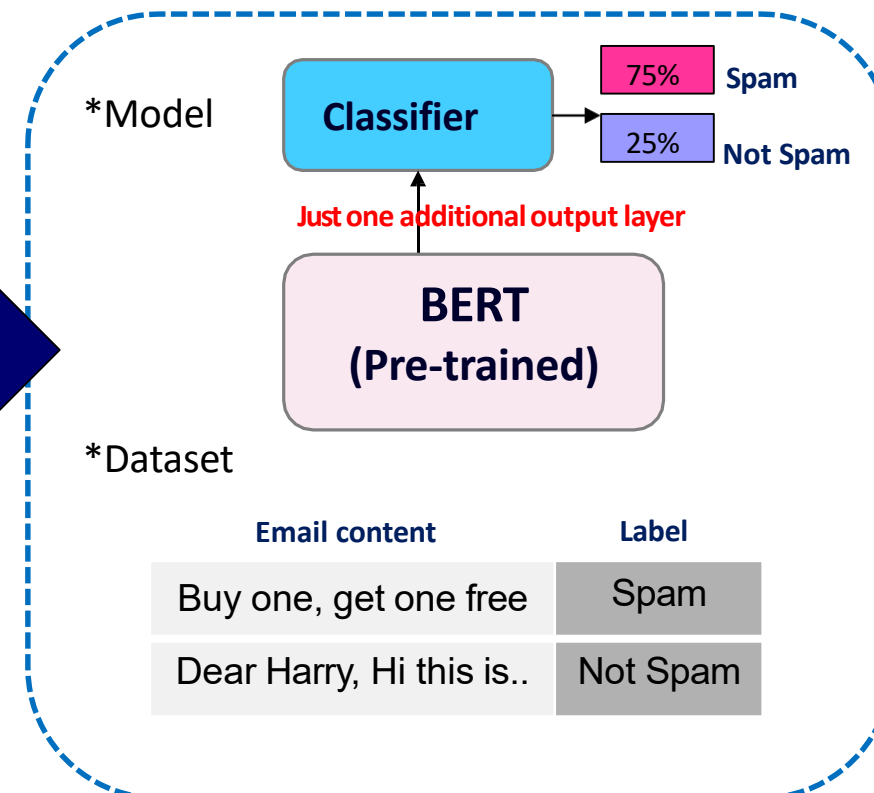
(1) Pre-training (self-supervised)

Training on large amounts of data
(Language modeling)



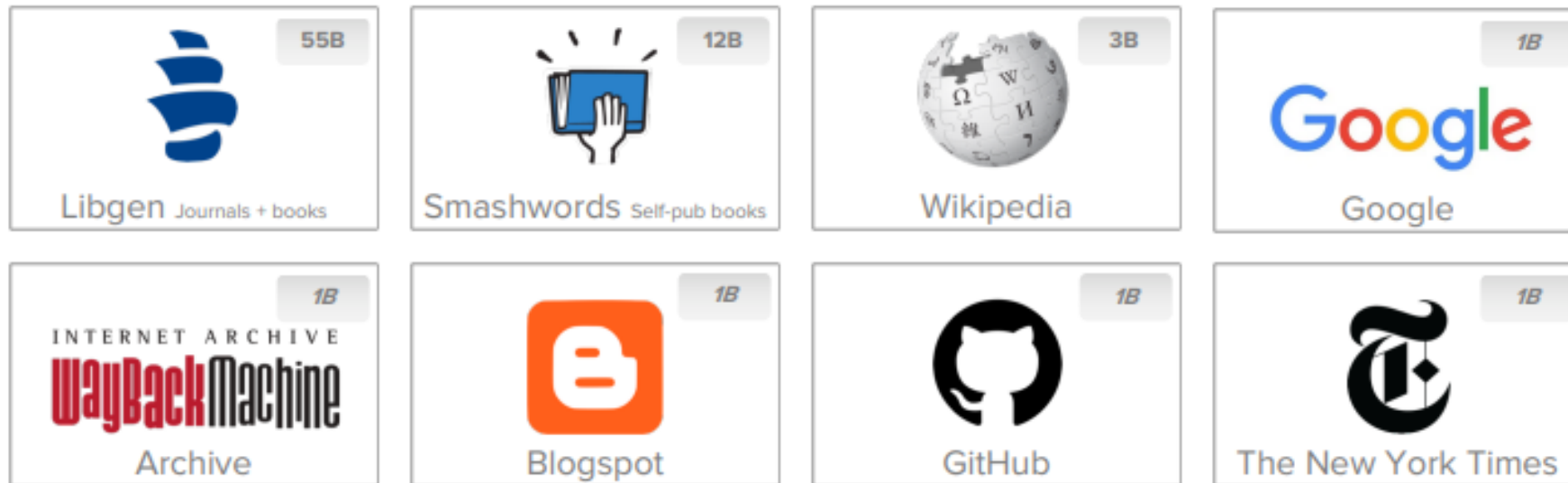
(2) Fine-tuning (supervised)

Training on a specific downstream task
with a labeled dataset



Pre-training “Large” Language Models

- Pre-training a very large model using massive amounts of data on the web by predicting the next word.
- Looks like reading all the documents in the world as if reading books.
- Through this process, various "**common sense**" and "**prior knowledge**" are acquired.



GPT-3: Initialization of Large-scale “Pre-training”

- Large-scale language model pre-training
 - Published in June 2020
 - Architecture: 175 Billion parameter Transformer (96 layers, 12k hidden dim, 96 heads)
 - Dense attentions & sparse attentions used
 - Dataset: 300 Billion tokens (60% CommonCrawl + 22% WebText2 + 16% Books + 3% Wikipedia)
 - Model not available, Commercial API available

Pre-training LLM GPT-3 and In-Context Learning

- In-context Learning – kind of Few-shot Learning

1) Zero-shot

```
1 Translate English to French:
2 cheese => .....
```

2) Few-shot

```
1 Translate English to French:
2 sea otter => loutre de mer
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => .....
```

Advantages of GPT-3: In-Context Learning

- **Versatility:** One model can solve multiple problems
 - Summarization, Programming, Translation, Sentiment Analysis, etc.
- **No need** for large amounts of human-labeled learning data for *fine-tuning*
- Allows to solve new problems that are not solved due to the lack of dataset

Limitation of GPT-3 & Instruct GPT

- GPT-3 generates the next words/sentences well
- GPT-3 cannot generate sentences well for a given instruction
- **InstructGPT:** Language model more aligned with their users (sibling models for ChatGPT)

PROMPT *Explain the moon landing to a 6 year old in a few sentences.*

COMPLETION GPT-3

Explain the theory of gravity to a 6 year old.

Explain the theory of relativity to a 6 year old in a few sentences.

Explain the big bang theory to a 6 year old.

Explain evolution to a 6 year old.

InstructGPT

People went to the moon, and they took pictures of what they saw, and sent them back to the earth so we could all see them.

Challenges in Developing better Language Model

“The three H’s of Model Desiderata”

1) Helpful:

- The AI should help the user solve their task

2) Honest:

- The AI should give accurate information

3) Harmless:

- The AI should not cause physical, psychological, or social harm to people or the environment

Challenges in Developing better Language Model

- **Misalignment:** When the training objective does not capture the desiderata we want from models

$$p(x) = \prod_{i=1}^n p(s_n | s_1, \dots, s_{n-1})$$

Training: Predict the next token



The three H's of Model Desiderata

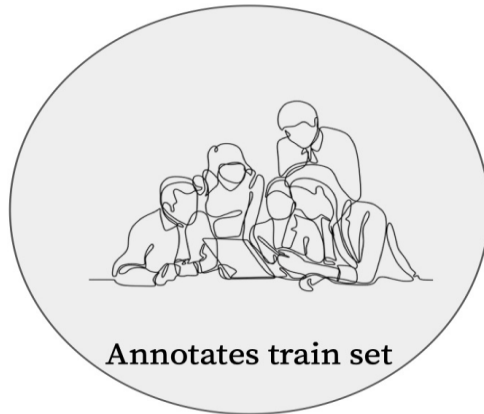
“Misalignment”

Training: Next-token prediction

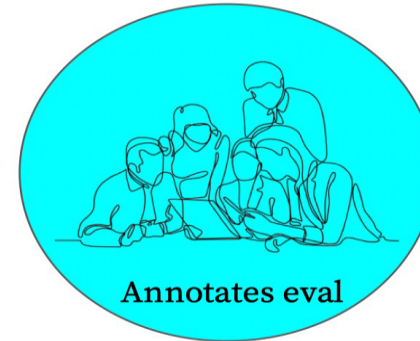
Evaluation: Follow instructions (e.g. summarize this)

Solution: Human Feedback

- Solution to misalignment: learn directly from human feedback



40 Annotators from Upwork/ScaleAI
- Screened/Onboarded/Diverse etc etc etc



Different annotators from Upwork/ScaleAI
- Not screened, to better mirror real-world

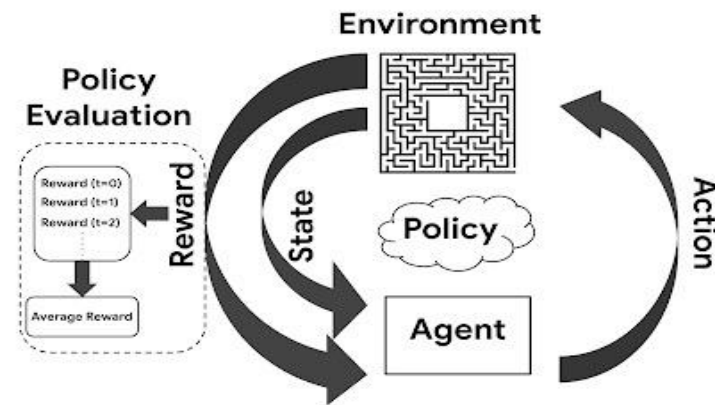
Training LMs to Follow Instructions with Human Feedback

1. Supervised Fine-Tuning (SFT)

- Using human-written demonstrations

2. Reinforcement Learning from Human Feedback (RLHF)

- Agent: Language Model
- Environment: Human users
- State: Human Inputs
- Action: Model Output
- Policy: Language Model Generation given Input
- **Reward: Human Feedback**



Supervised Fine-tuning (SFT)

- Collect demonstrations for prompt from humans
- Train the model to generate the demonstrations from the prompt



Supervised Fine-tuning (SFT)

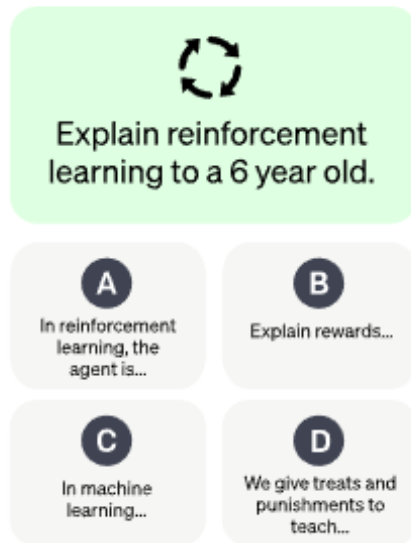
- Prompt example and statistics

Use-case	Prompt	Use-case	(%)	Number of Prompts		
Brainstorming	List five ideas for how to regain enthusiasm for my career	Generation	45.6%	SFT Data		
Generation	Write a short story where a bear goes to the beach, makes friends with a seal, and then returns home.	Open QA	12.4%	split	source	size
Rewrite	This is the summary of a Broadway play: "" {summary} "" This is the outline of the commercial for that play: ""	Brainstorming	11.2%	train	labeler	11,295
		Chat	8.4%	train	customer	1,430
		Rewrite	6.6%	valid	labeler	1,550
		Summarization	4.2%	valid	customer	103
		Classification	3.5%			
		Other	3.5%			
		Closed QA	2.6%			
		Extract	1.9%			

Reinforcement Learning from Human Feedback (RLHF)

① Collect comparison data and train a reward model

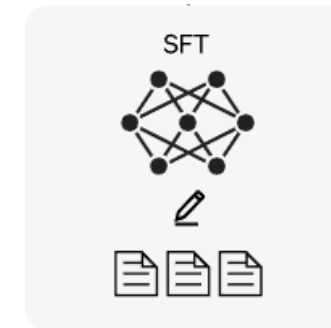
1) Collect a prompt and model outputs



2) Rank the outputs by humans

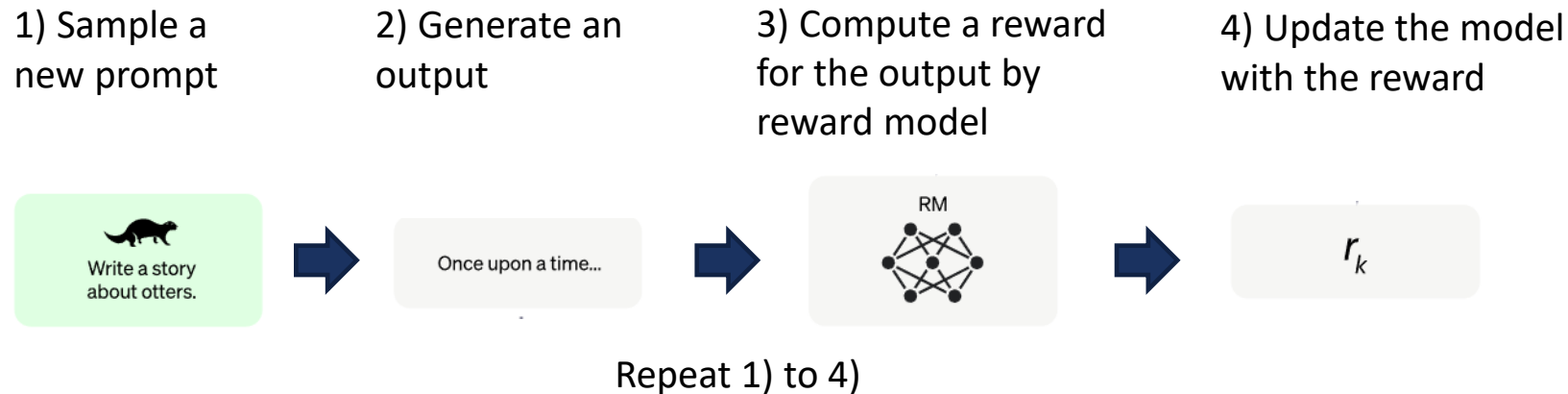


3) Train reward model



Reinforcement Learning from Human Feedback (RLHF)

② Optimize a policy against the reward model using reinforcement learning



- During training, following loss function is used

$$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} \left[r_{\theta}(x, y) - \beta \log \left(\frac{\pi_{\phi}^{\text{RL}}(y | x)}{\pi^{\text{SFT}}(y | x)} \right) \right] + \gamma E_{x \sim D_{\text{pretrain}}} \left[\log(\pi_{\phi}^{\text{RL}}(x)) \right]$$

↑ makes sure PPO model output does not deviate too far from SFT

← auxiliary LM objective on the pre-training data

Why is RLHF Good Compared to just SFT?

1) **Reward is a more nuanced training signal** than autoregressive loss

- If the correct next token is “great”, the AR loss penalizes the prediction “amazing” the same as “sandwiches”. The RM assigns similar rewards to sequences with similar quality

2) The **RM “critiques” actual completions generated from the model** itself, whereas SFT training does not use model generations, since it is completely offline.

- This means the RM may provide more “tailored” feedback to the model

Why is RLHF Good Compared to just SFT?

3) The **RM** more directly captures the notion of “preference”

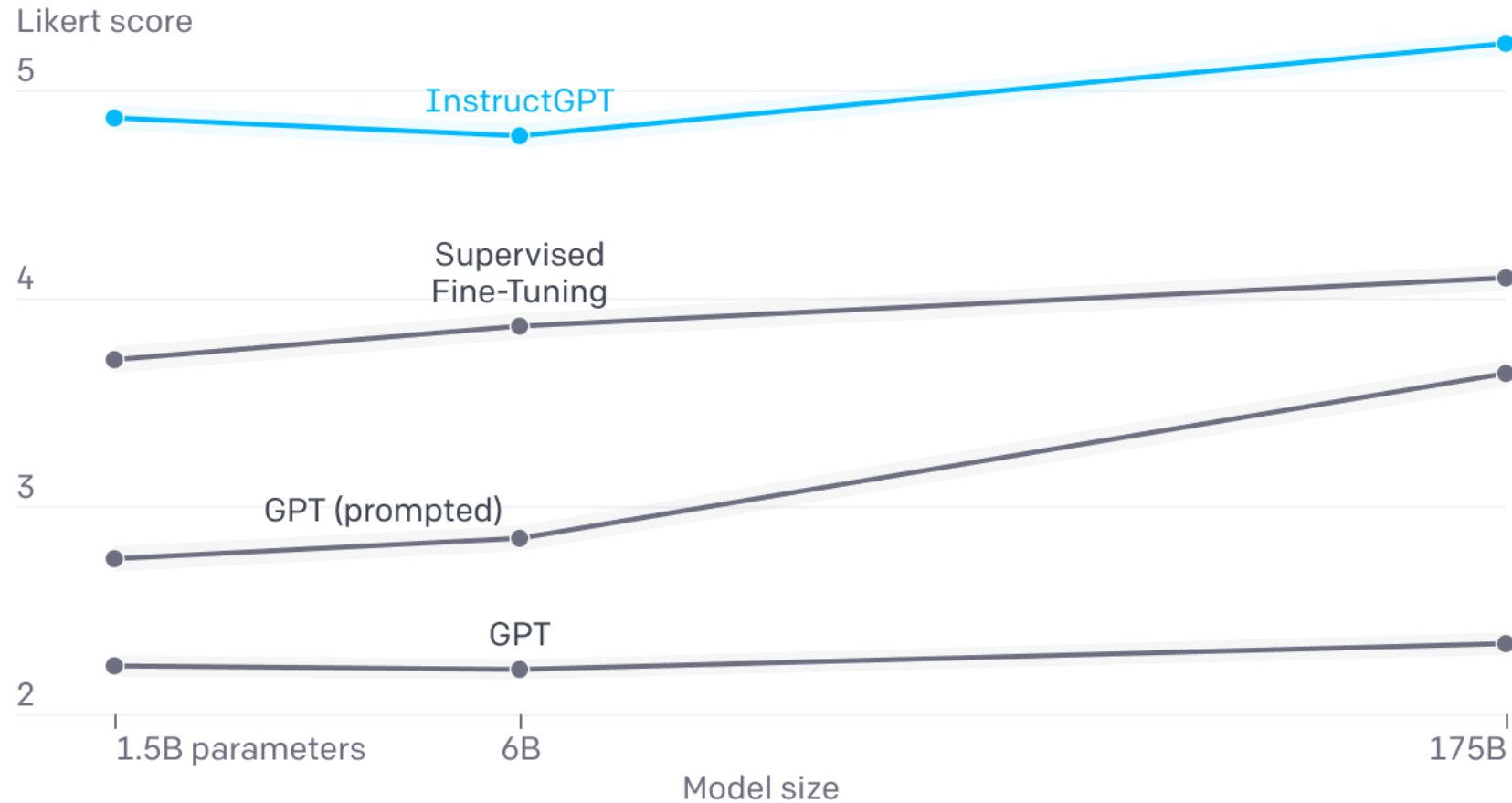
- Preferences induce rankings, and rankings can be used to infer preferences
- Ranking is very naturally captured by the reward signal, better sequences = higher reward
- In SFT, preference is not explicitly captured, since we only train to regurgitate “the best” example

4) The **RM** is more data efficient

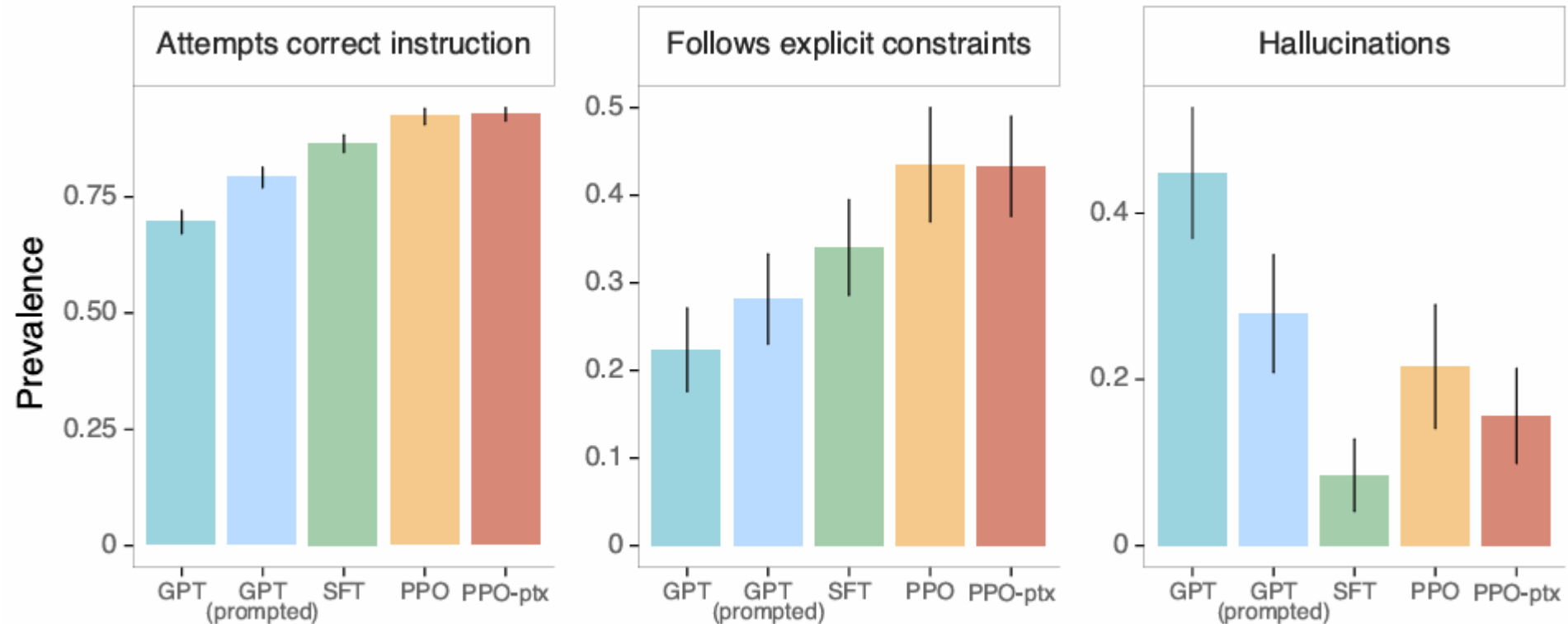
- There is a reason step 1 uses 13k prompts, but step 3 can use 31k prompts.
- For SFT, we need humans to generate target. Once we train the RM, it can be used to score any output

InstructGPT: Human Preference Test

- GPT-3 + SFT + RLHF (InstructGPT) shows best results!



InstructGPT: Preference Breakdown



- InstructGPT models follow the instruction more correctly (helpful)
- InstructGPT models less likely to hallucinate (honest)

From InstructGPT to ChatGPT

- “...same method as InstructGPT, but with **slight differences in the data collection setup**”
 - Similar to **SFT** model of InstructGPT
 - Aggregate NLP datasets, write prompt templates, and fine-tune model
- **SFT data**
 - human AI trainers provided conversations in which they played both sides—the user and an AI assistant
 - mixed this new dialogue dataset with the InstructGPT dataset, which we transformed into a dialogue format
- **Reward Model data**
 - took conversations that AI trainers had with the chatbot. We randomly selected a model-written message,
 - sampled several alternative completions, and had AI trainers rank them

Limitation of Naïve ChatGPT: Lack of up-to-date Information

ChatGPT 3.5 ▾



You

한국 프로야구에서 2023년에 어느 팀이 우승했어?



ChatGPT

죄송하지만, 제가 현재 시점에서 실시간 정보를 업데이트할 수는 없어요. 그러나 한국 프로야구의 최근 시즌 우승팀은 한국야구위원회의 웹사이트나 스포츠 뉴스 사이트 등을 통해 확인할 수 있을 거예요. 2023년 시즌의 우승팀은 해당 시기에 가장 우수한 성적을 거두었을 것입니다.



ChatGPT 3.5 ▾



You

대한민국의 20대 대통령은 누구야?



ChatGPT

2024년 현재로서는 대한민국의 20대 대통령은 누구인지는 알 수 없어. 현재로서는 그 정보가 없거든. 하지만 20대 대통령이 누가 될지에 대한 예상이나 가정을 해볼 수는 있어.



Retrieval Augmented Generation: Accurate Response using Search Engine

ChatGPT 5.1 Instant ▾

↑ 공유하기

👤 사람 추가

웹 찾아보는 중

한국 프로야구에서 2023년에 어느 팀이 우승했어?

LG 트윈스가 2023년 2023년 한국시리즈에서 우승했습니다.

ko.wikipedia.org +1

📄 👍 🗨️ ↗️ ↻️ ... 📄 출처

Information
Retrieval



Text Generation



Close-book exam
(Hard mode)



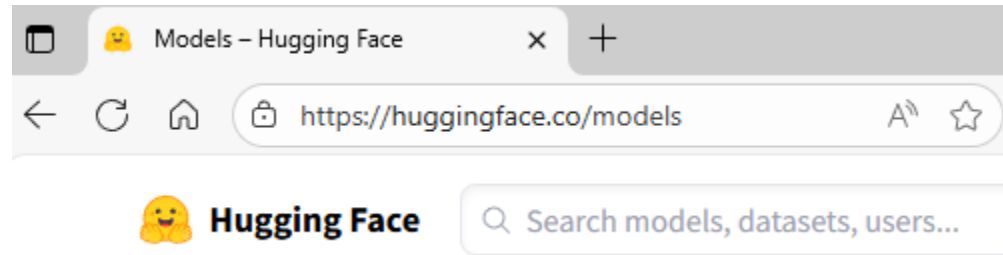
Retrieval-Augmented
Text Generation



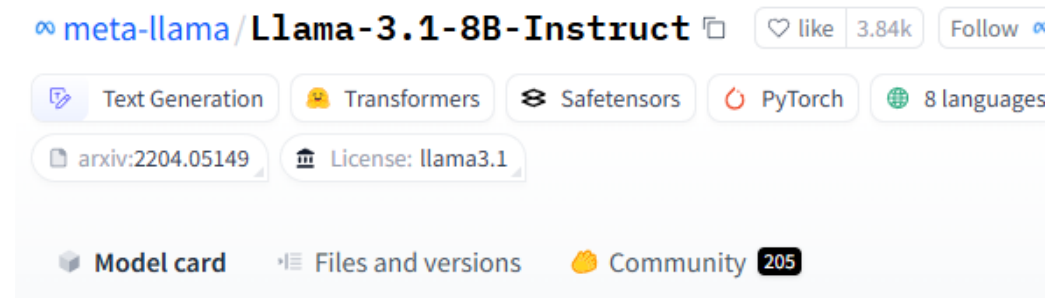
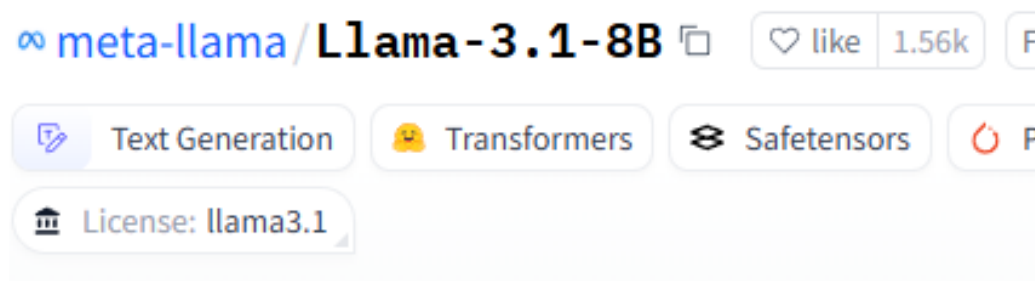
Open-book exam
(Easy mode)



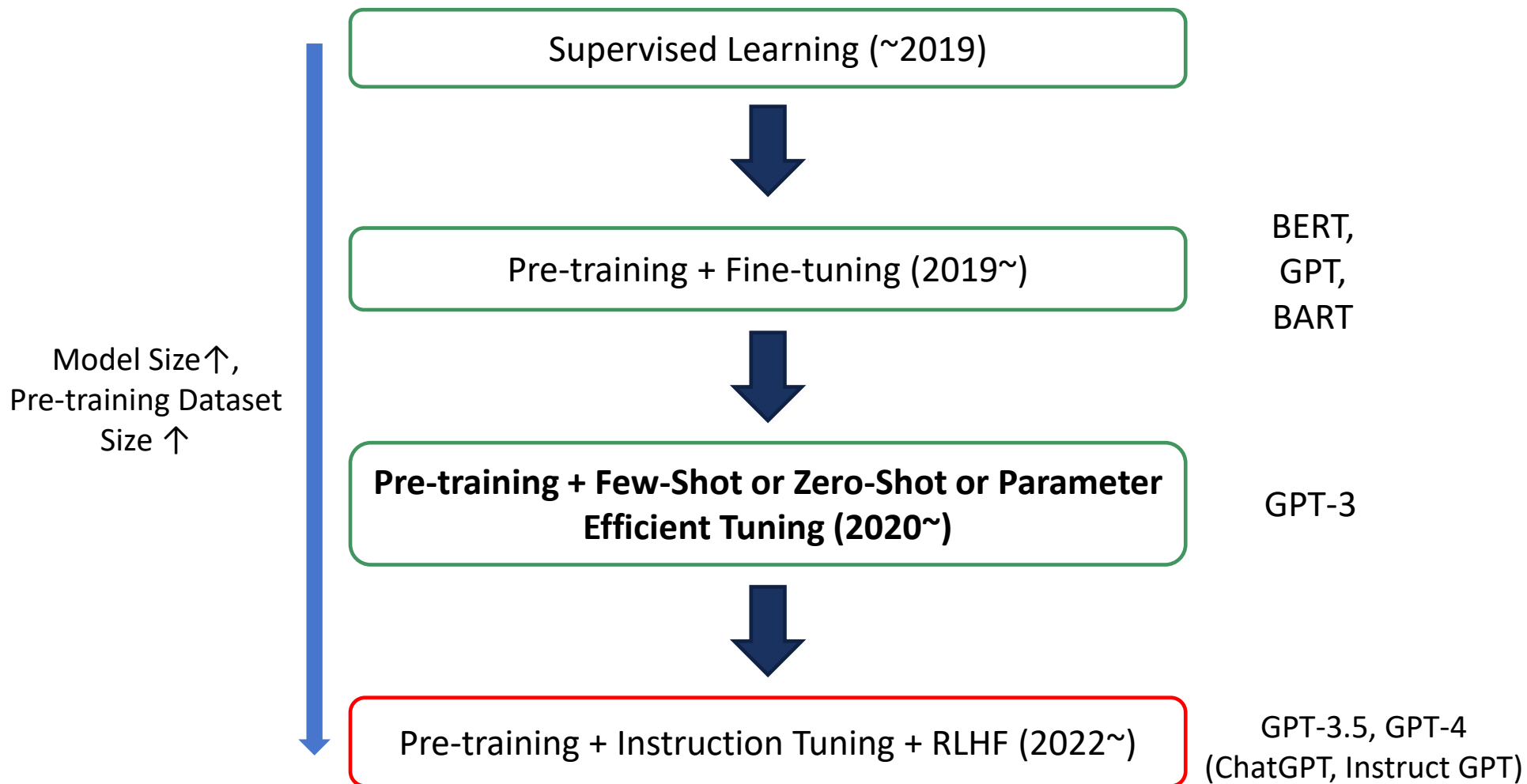
Downloading Pre-trained Models: Instruct Model vs Base Model



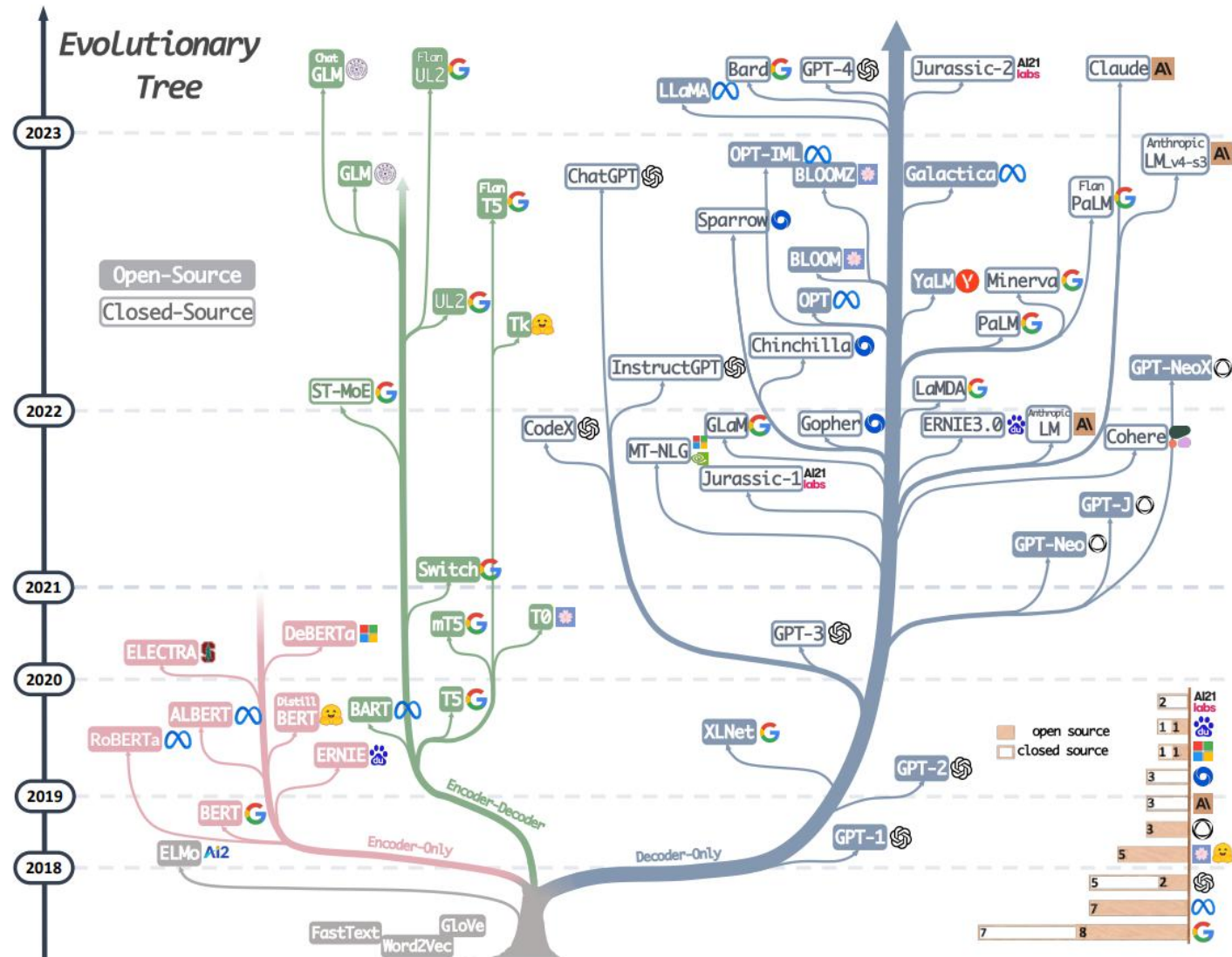
- A **Base Model** is the result of the **initial pre-training** phase, excelling at **text completion**.
- An **Instruct Model** is created by **fine-tuning a Base Model** to align with user intent, making it **optimized for answering questions and following instructions**.



Training Paradigm in NLP



Evolutionary Tree of Modern LLMs until 2023



Yang et al., Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond, arXiv

Summary

- We trace the evolution of pre-training, from learning static *Word Embeddings* to **Pre-training Whole Models** like BERT and GPT, which learn deep contextual representations from massive, unlabeled text.
- This established the dominant **Pre-training / Fine-tuning paradigm**, where a model is first trained on a general, self-supervised task and then adapted with a small amount of labeled data for a specific downstream task.
- The scaling of this approach led to **Large Language Models (LLMs)** like **GPT-3**, which demonstrated **In-context Learning** (zero-shot and few-shot), allowing the model to perform new tasks from just a prompt without any gradient updates.
- However, this scaling also revealed a "Misalignment" problem: a model trained to *predict* the next word isn't necessarily helpful, honest, or harmless.
- **InstructGPT** aligns models to user intent through two-stage: first, **Supervised Fine-Tuning (SFT)** on human-written examples, and second, **Reinforcement Learning from Human Feedback (RLHF)** to "reward" outputs that humans prefer.
- **ChatGPT** was then developed by applying a similar instruction-tuning and RLHF methodology, specializing the model for a dialogue format.
- Finally, to overcome the static knowledge of pre-trained models, **Retrieval-Augmented Generation (RAG)** is used to provide the model with up-to-date, external information (e.g., from a search) before it generates an answer.

LG AI Research Special Lecture

딥러닝 자연어처리 기초와 LLM 에이전트

Lecture 05 – 스스로 계획하고 실행하는 AI, LLM 에이전트



중앙대학교 AI학과

이환희

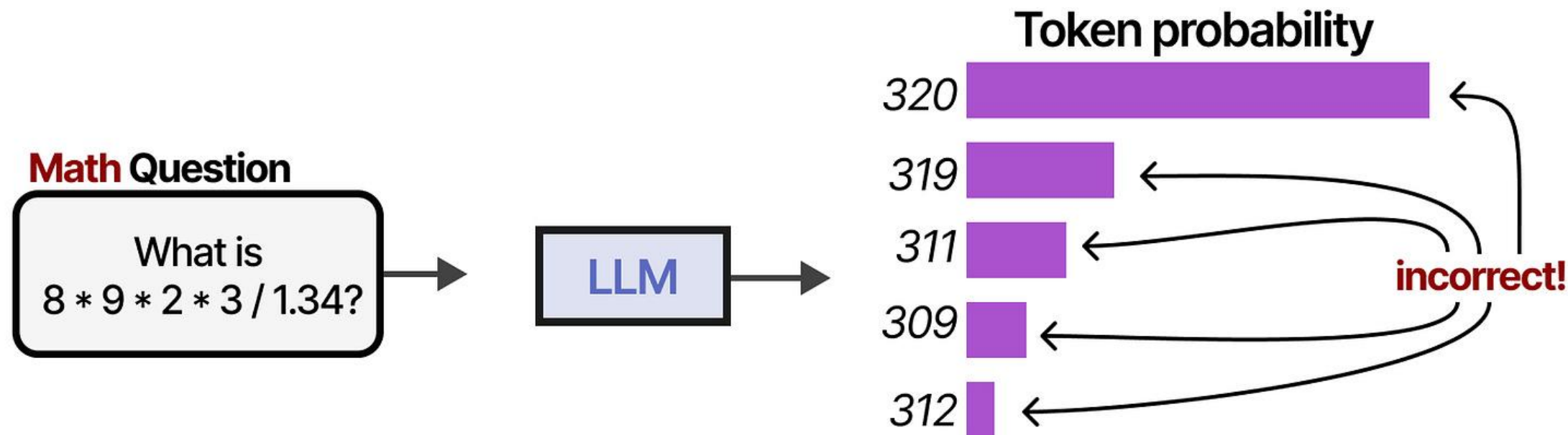
Outline

스스로 계획하고 실행하는 AI, LLM 에이전트

- Definition of LLM Agents
- Memory of LLM
- Using Tools for LLM & MCP
- Planning, ReAct
- Multi-agent Collaboration

What are LLM Agents?

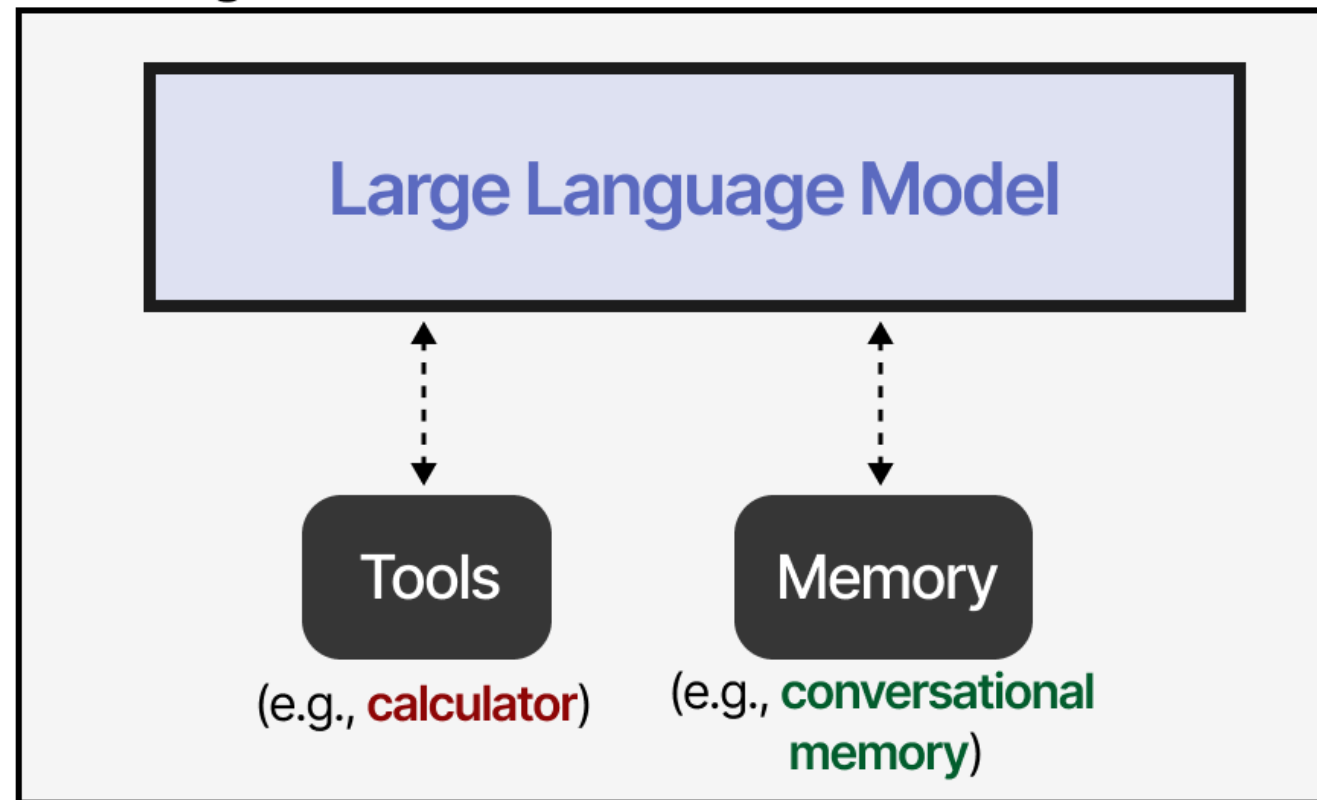
- There are many other tasks that LLMs often fail at, including basic math like multiplication and division



What are LLM Agents?

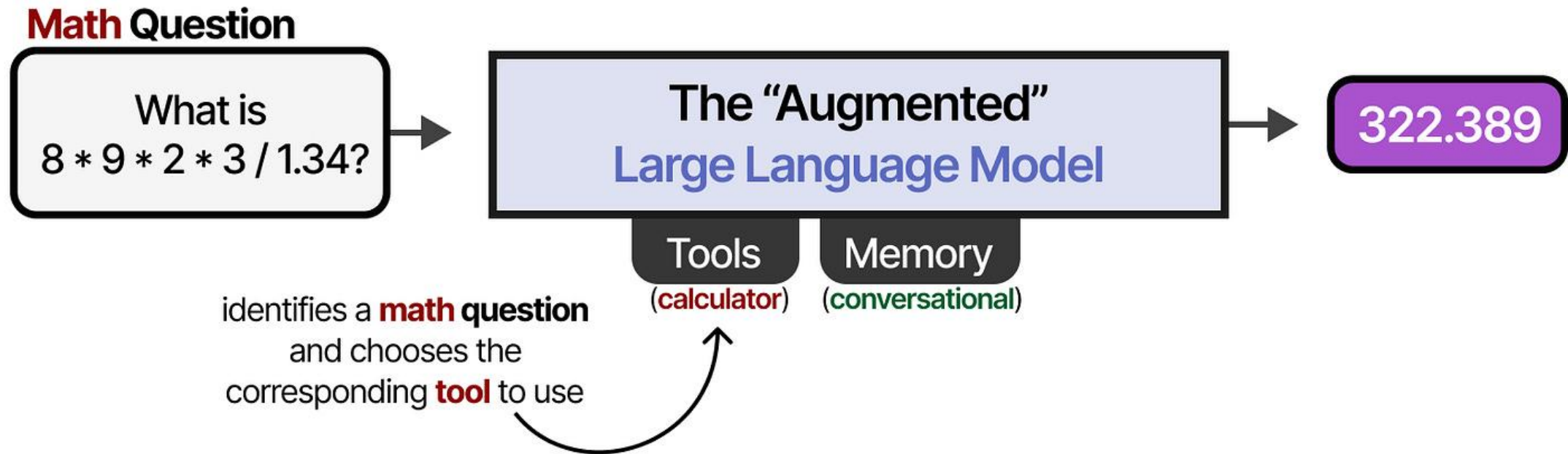
- Through external systems, the capabilities of the LLM can be enhanced. Anthropic calls this “The Augmented LLM”.

The Augmented LLM



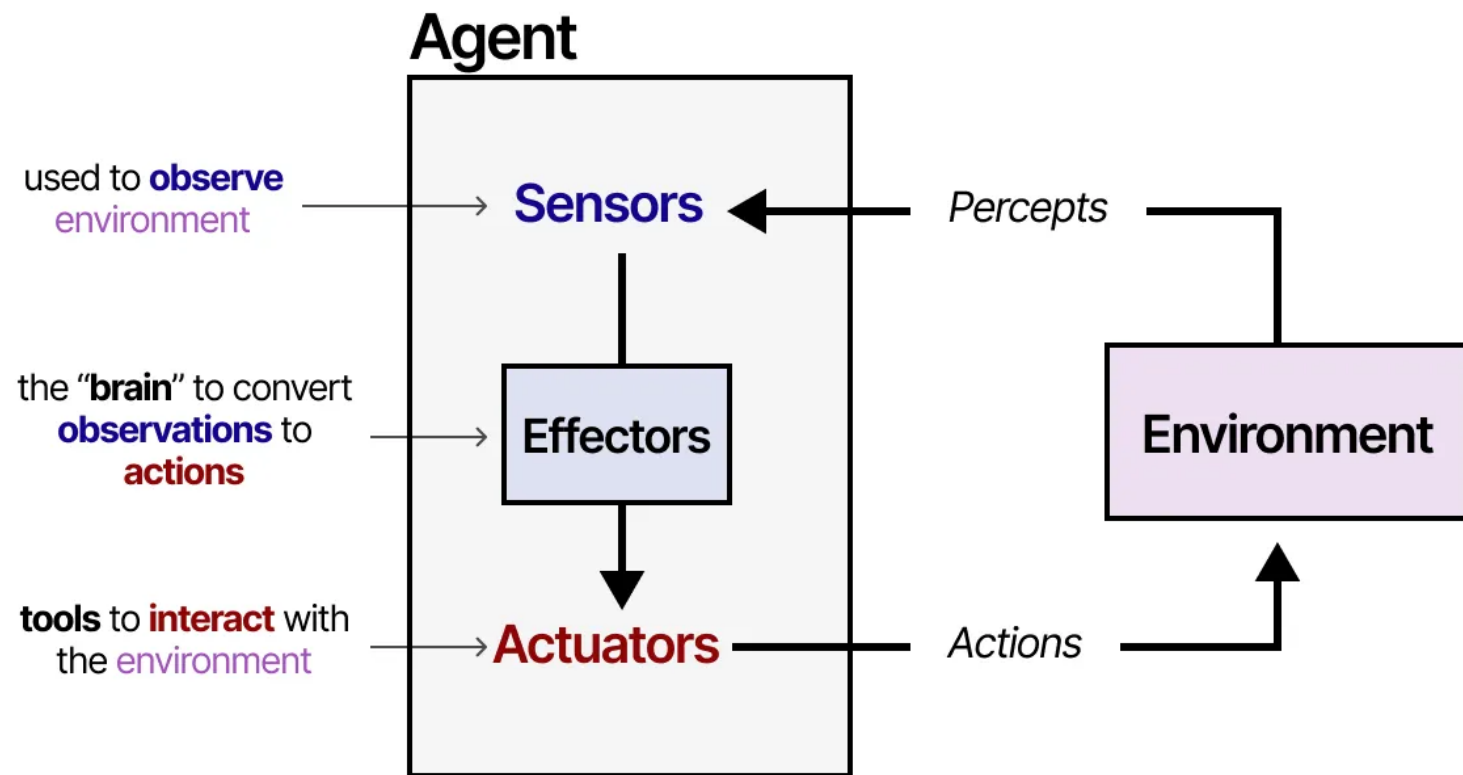
What are LLM Agents?

- For instance, when faced with a math question, the LLM may decide to use the appropriate tool (a calculator).



What are LLM Agents?

- An agent is anything that can be viewed as perceiving its environment through **sensors** and acting upon that environment through **actuators**.



Environments — The world the agent interacts with

Sensors — Used to observe the environment

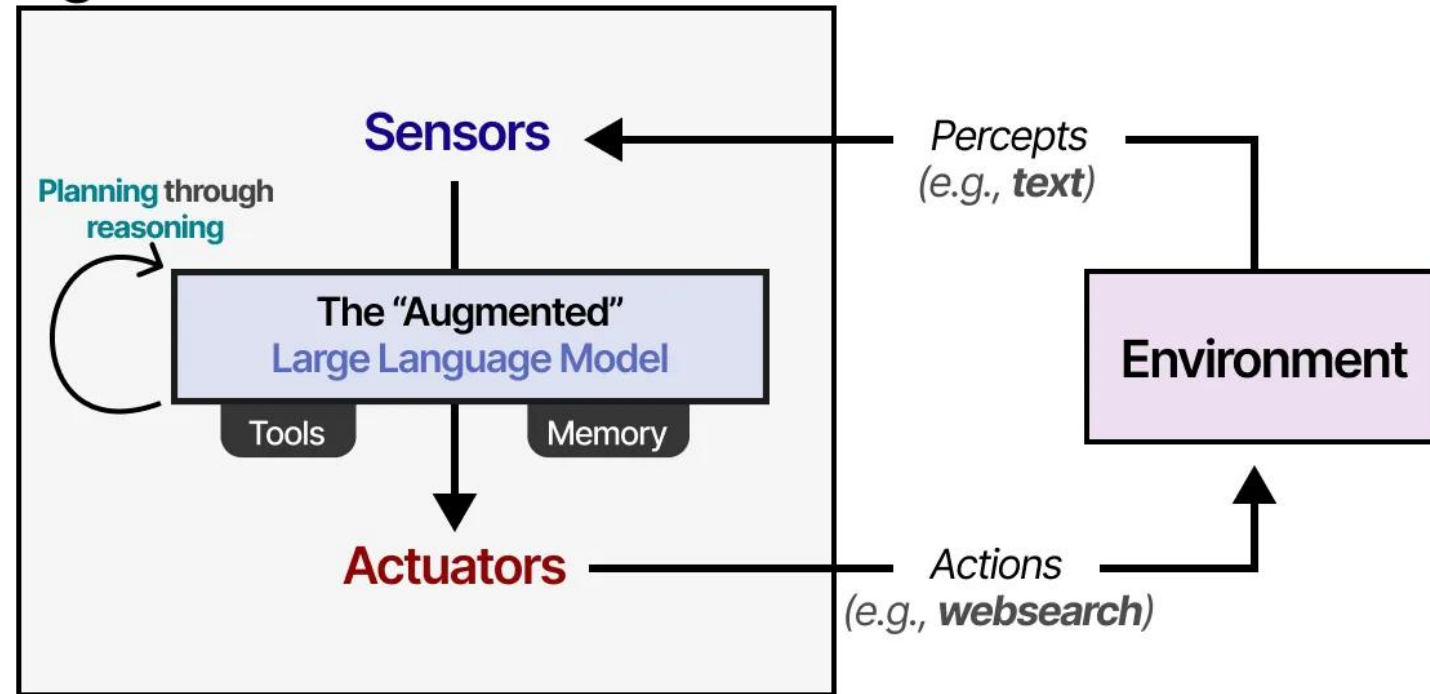
Actuators — Tools used to interact with the environment

Effectors — The “brain” or rules deciding how to go from observations to actions

What are LLM Agents?

- We can generalize this framework a bit to make it suitable for the “Augmented LLM” as below.

Agent



Environments — The world the agent interacts with

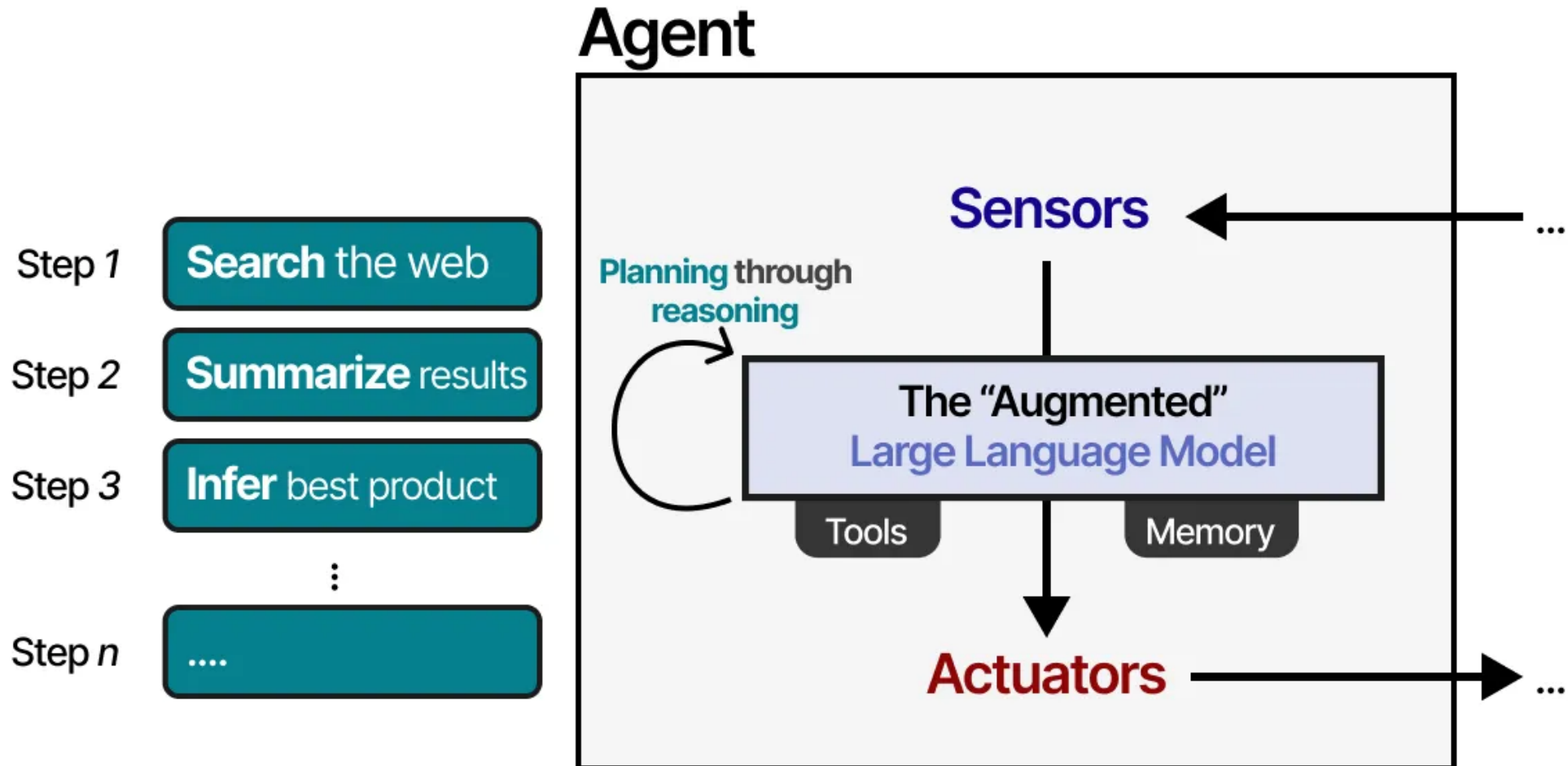
Sensors — Used to observe the environment

Actuators — Tools used to interact with the environment

Effectors — The “brain” or rules deciding how to go from observations to actions

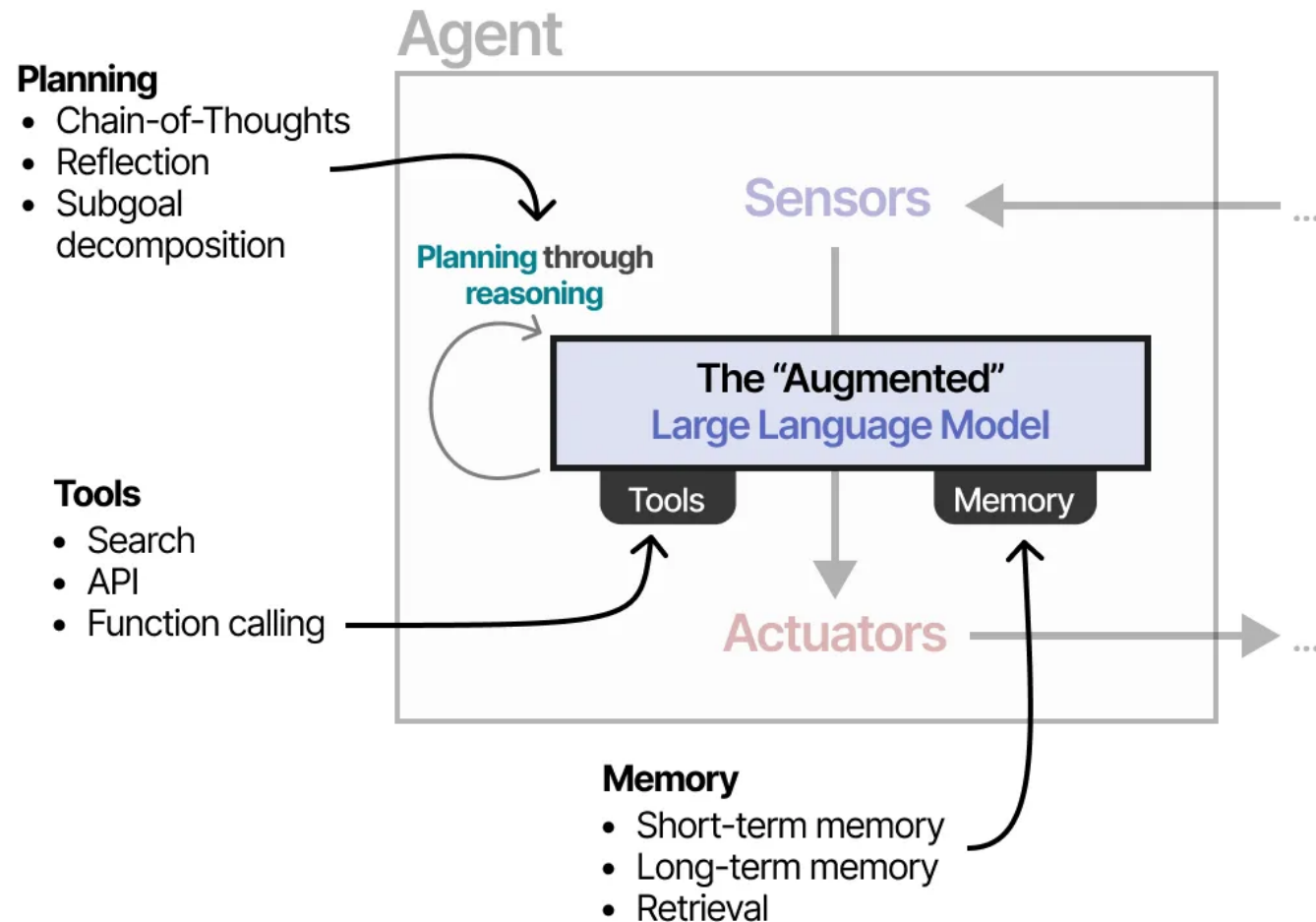
What are LLM Agents?

- Using the reasoning behavior, LLM Agents will plan out the necessary actions to take.



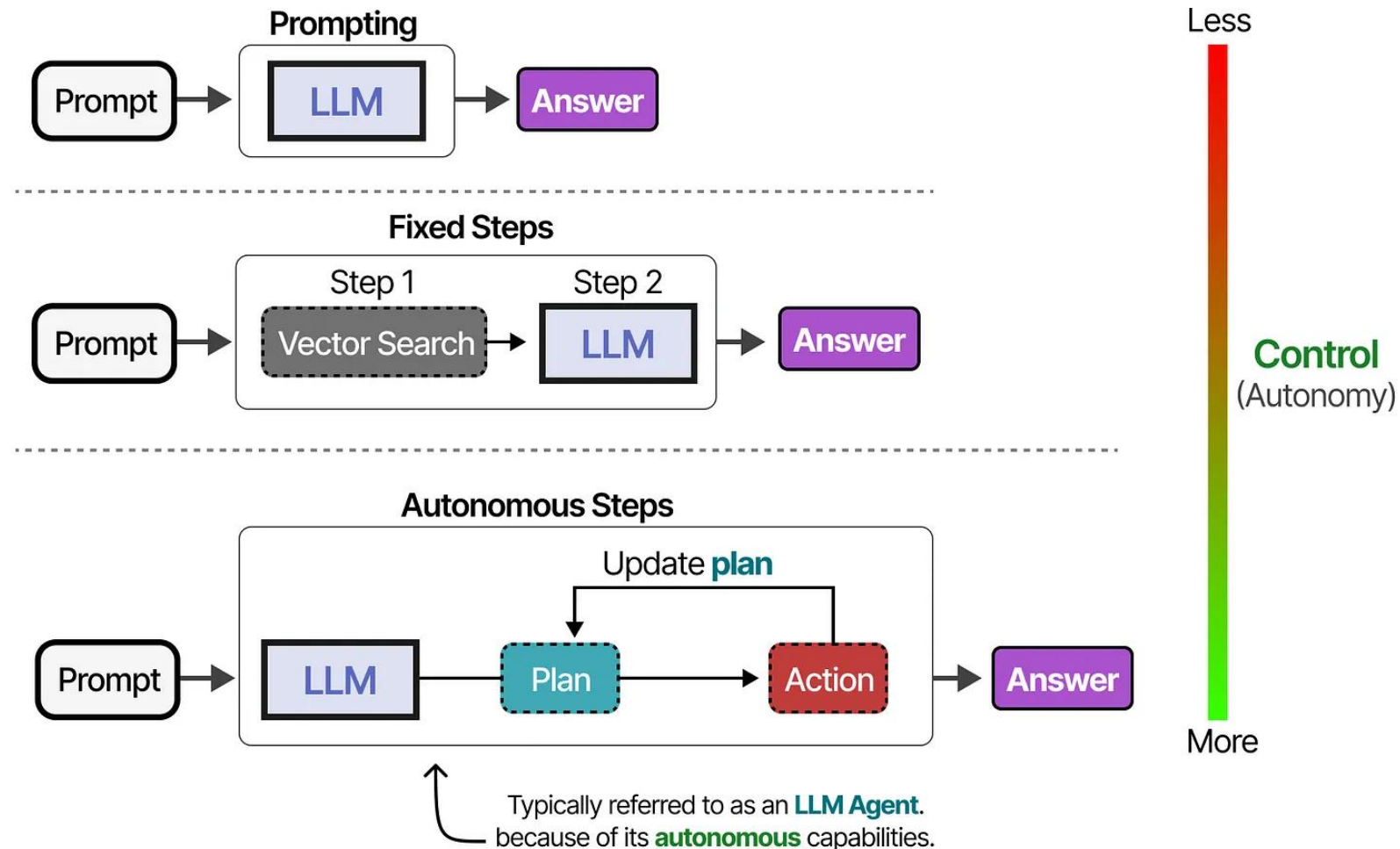
What are LLM Agents?

- This planning behavior allows the Agent to understand the situation (LLM), plan next steps (planning), take actions (tools), and keep track of the taken actions (memory).



What are LLM Agents?

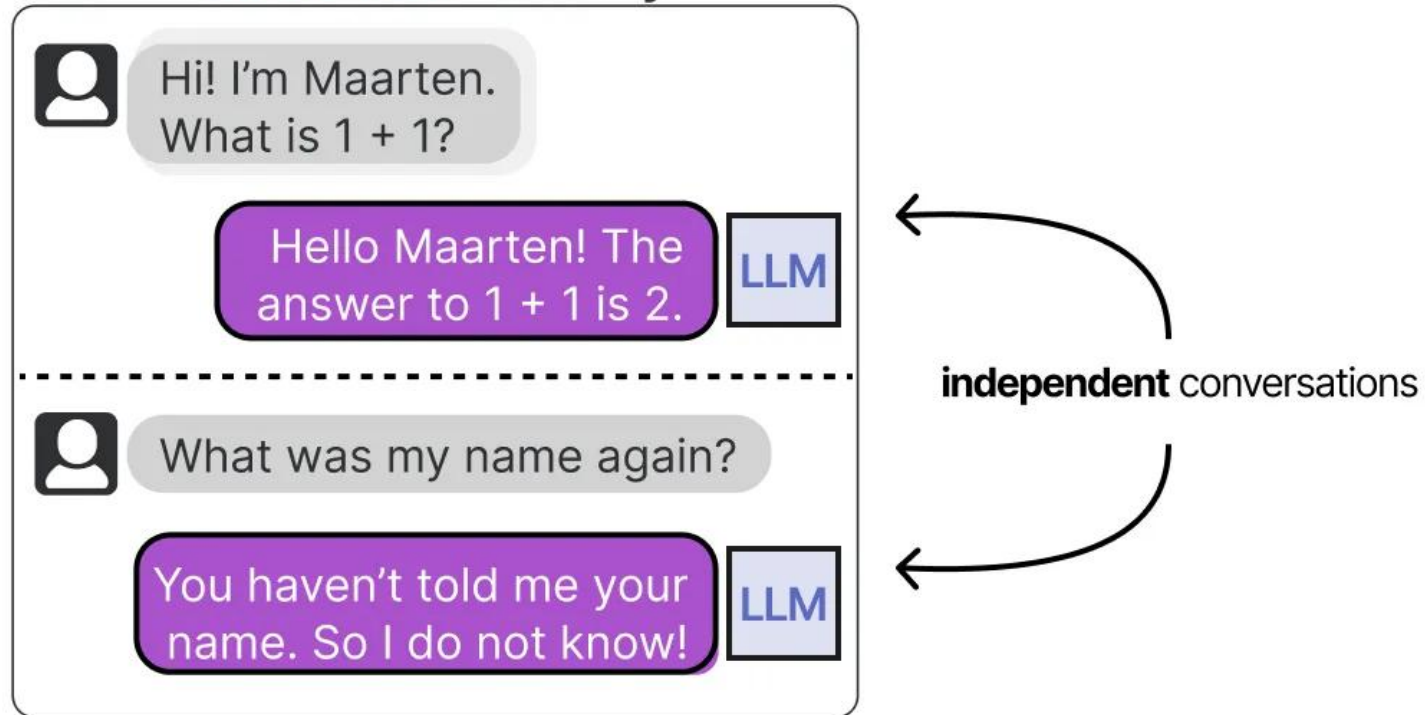
- Depending on the system, LLM Agents have varying degrees of autonomy.



Short-term Memory

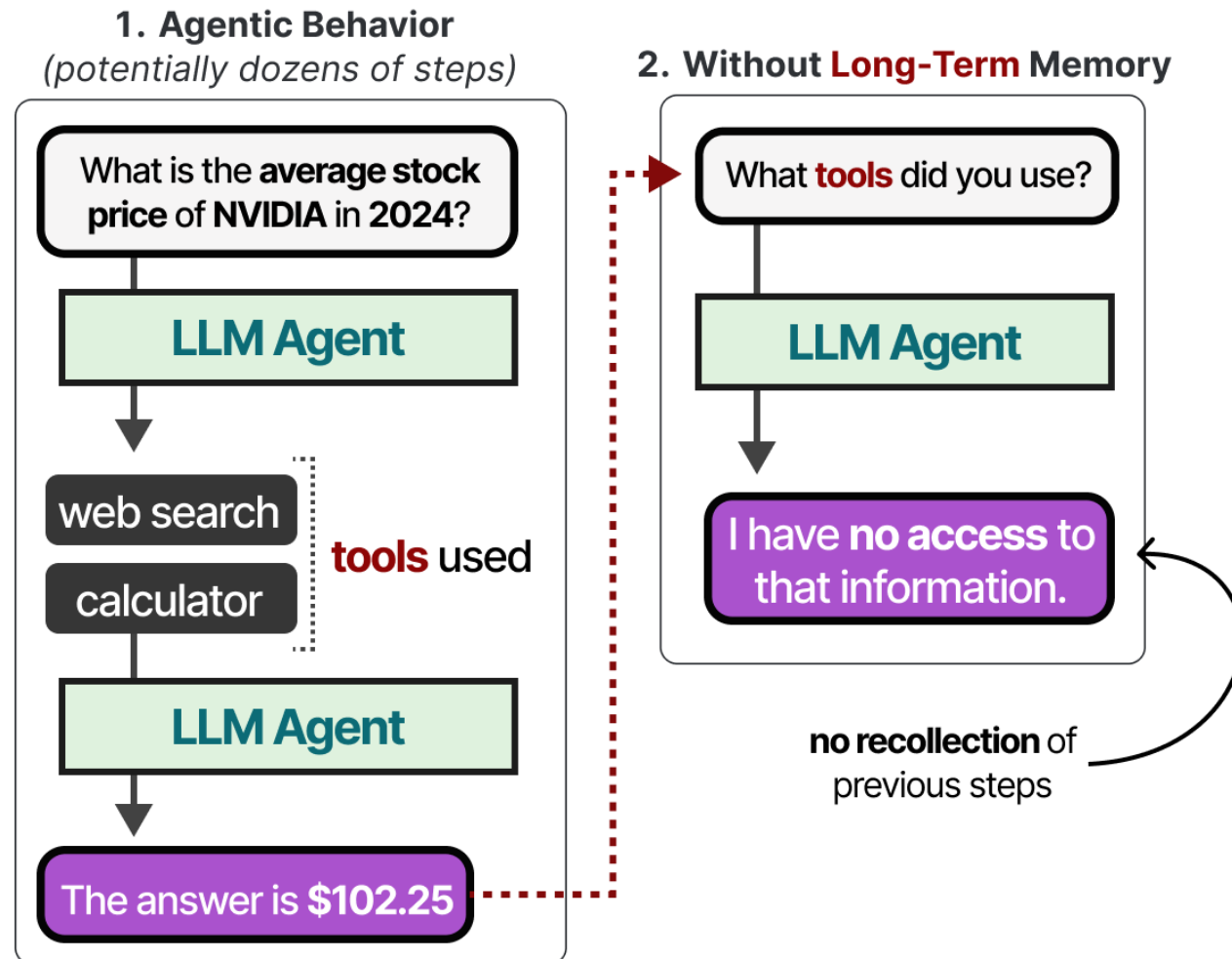
- LLMs are forgetful systems, or more accurately, do not perform any memorization at all when interacting with them.
- When you ask an LLM a question and then follow it up with another question, it will not remember the former.

Without **Short-Term** Memory



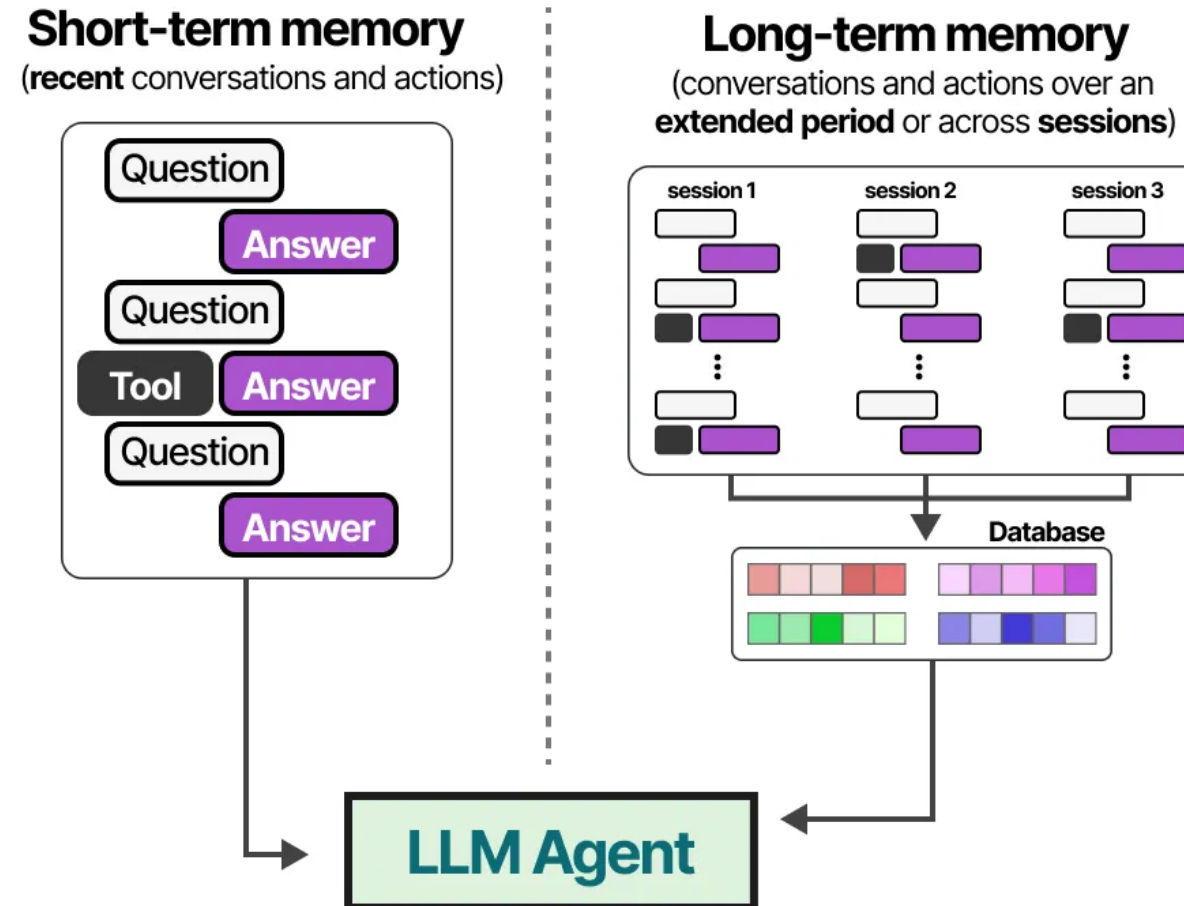
Long-term Memory

- LLM Agent also needs to keep track of potentially dozens of steps, not only the most recent actions.



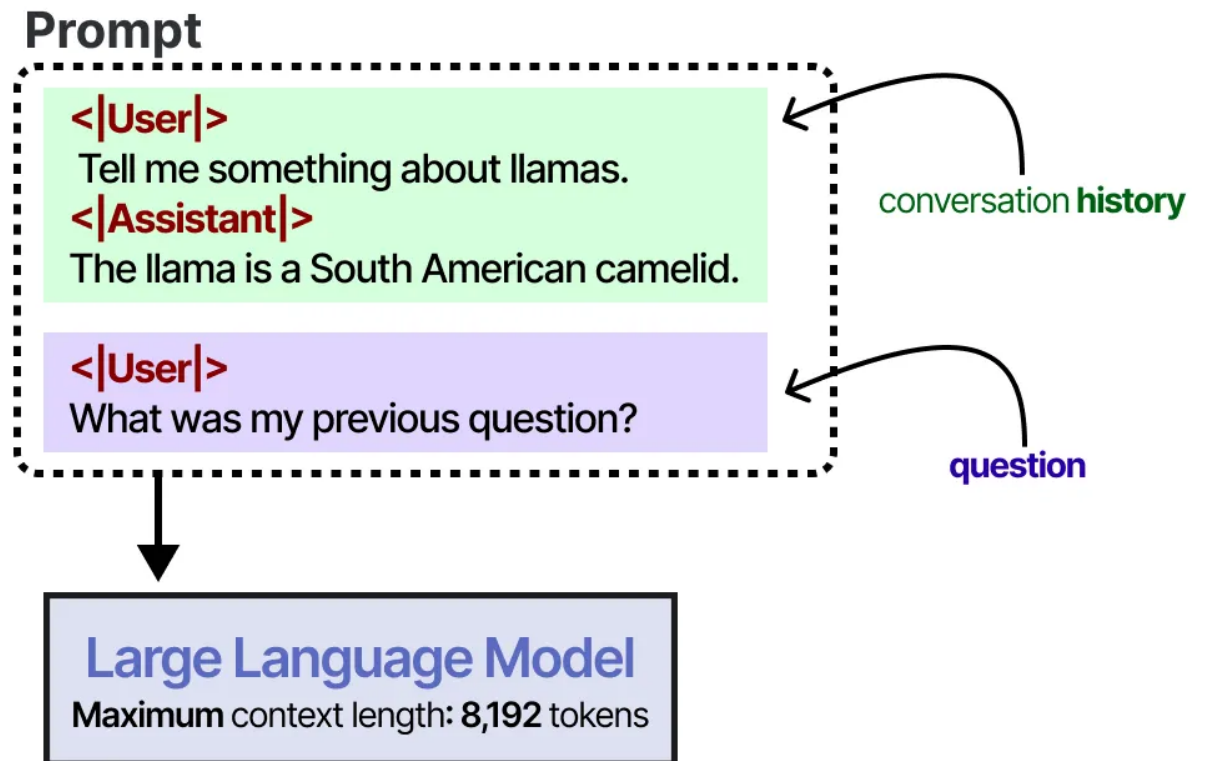
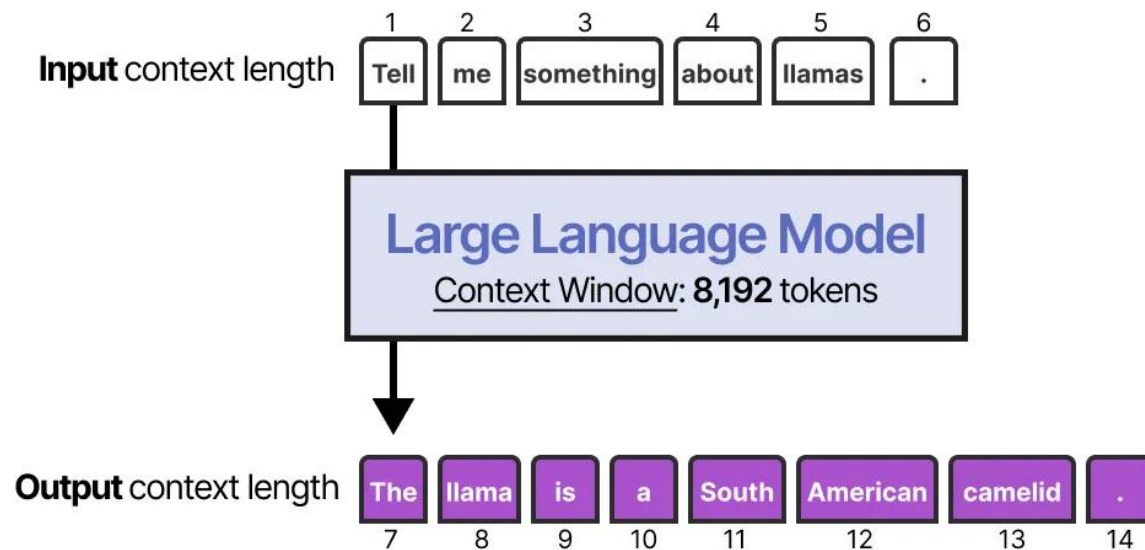
Long-term Memory

- This is referred to as long-term memory as the LLM Agent could theoretically take dozens or even hundreds of steps that need to be memorized.



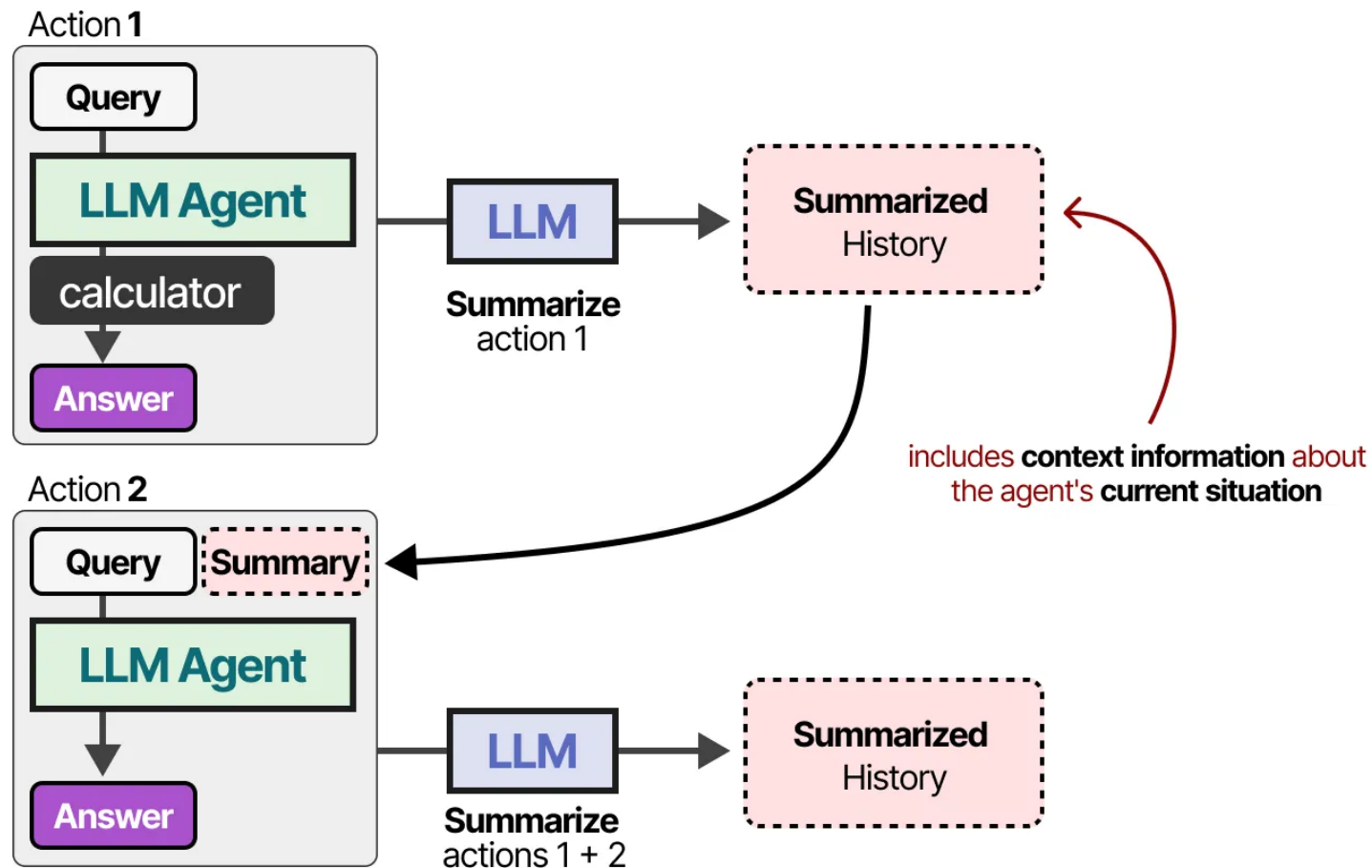
Implementing Short-term Memory

- Model's context window enables short-term memory
- This works as long as the conversation history fits within the LLM's context window.
- However, instead of actually memorizing a conversation, we essentially “tell” the LLM what that conversation was.



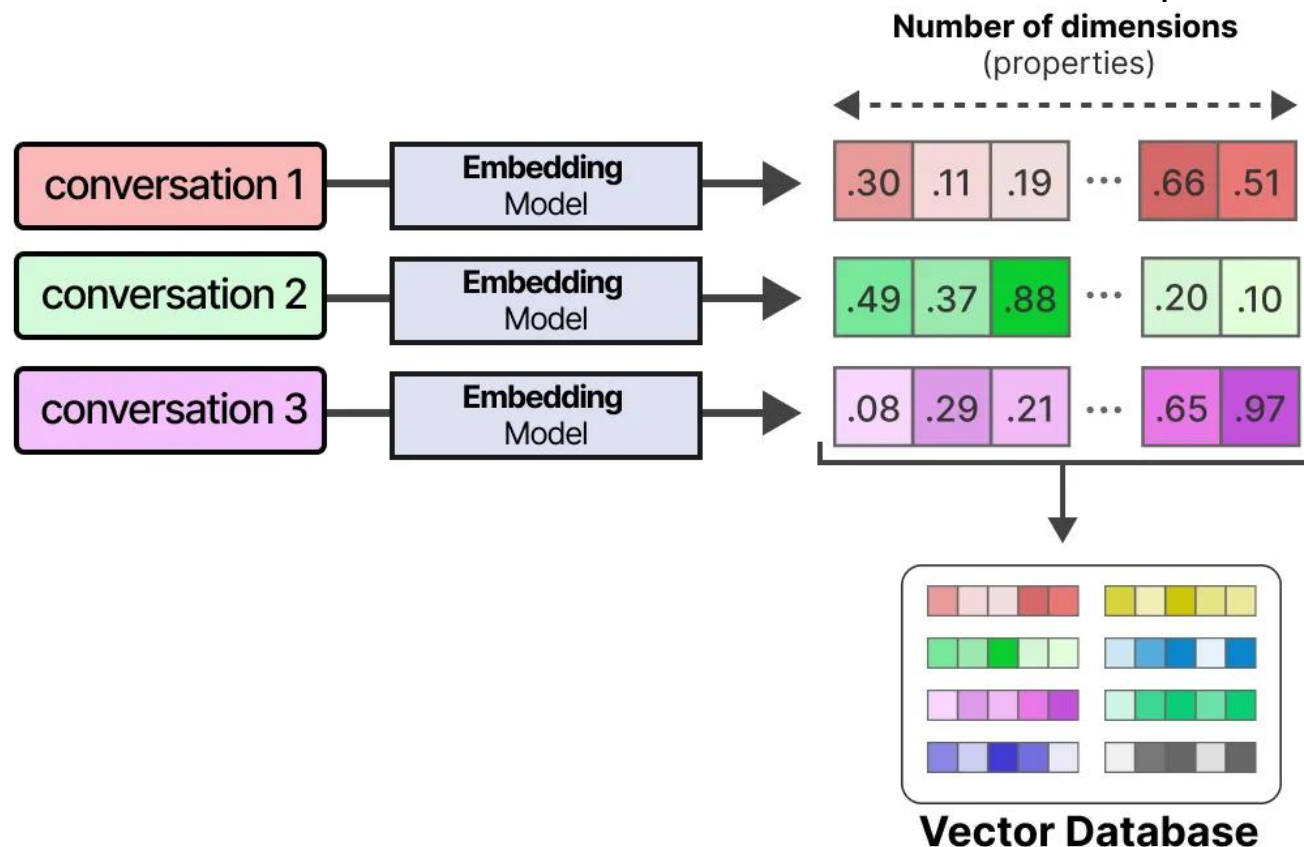
Implementing Short-term Memory

- For models with a smaller context window, or when the conversation history is large, we can instead use another LLM to summarize the conversations that happened thus far.



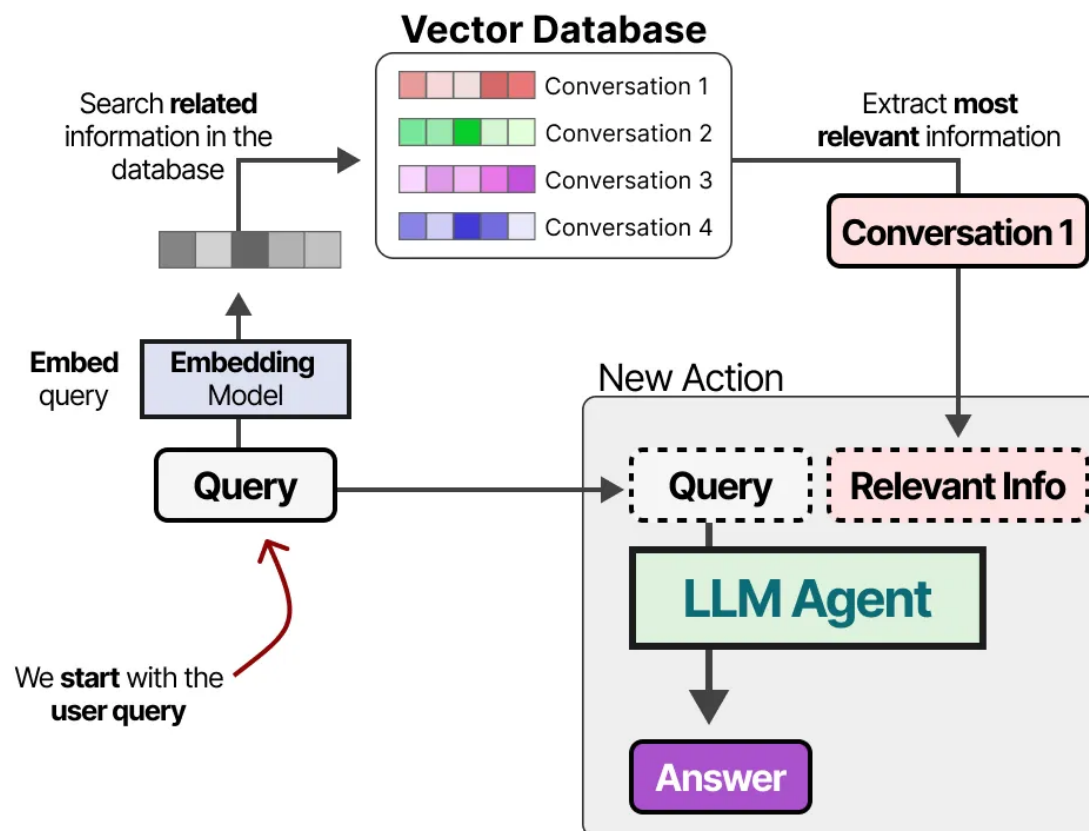
Implementing Long-term Memory

- A common technique to enable long-term memory is to store all previous interactions, actions, and conversations in an external vector database.
- To build such a database, conversations are first embedded into numerical representations that capture their meaning.



Implementing Long-term Memory

- After building the database, we can embed any given prompt and find the most relevant information in the vector database by comparing the prompt embedding with the database embeddings.
- This method is often referred to as Retrieval-Augmented Generation (RAG).



Memory Types

- Different types of information can also be related to different types of memory to be stored.
- In psychology, there are numerous types of memory to differentiate, but the Cognitive Architectures for Language Agents paper coupled four of them to LLM Agents.

<i>Memory Type</i>		<i>Human example</i>	<i>Agent example</i>
Working	Agent's current and recent circumstances	Shopping List	Context
Procedural	Instructions to determine the agent's behavior	Tying Shoes	System Prompt
Semantic	Facts about the world	Dog Breeds	User Information
Episodic	Sequences of the agent's past behaviors	7th Birthday	Past Actions

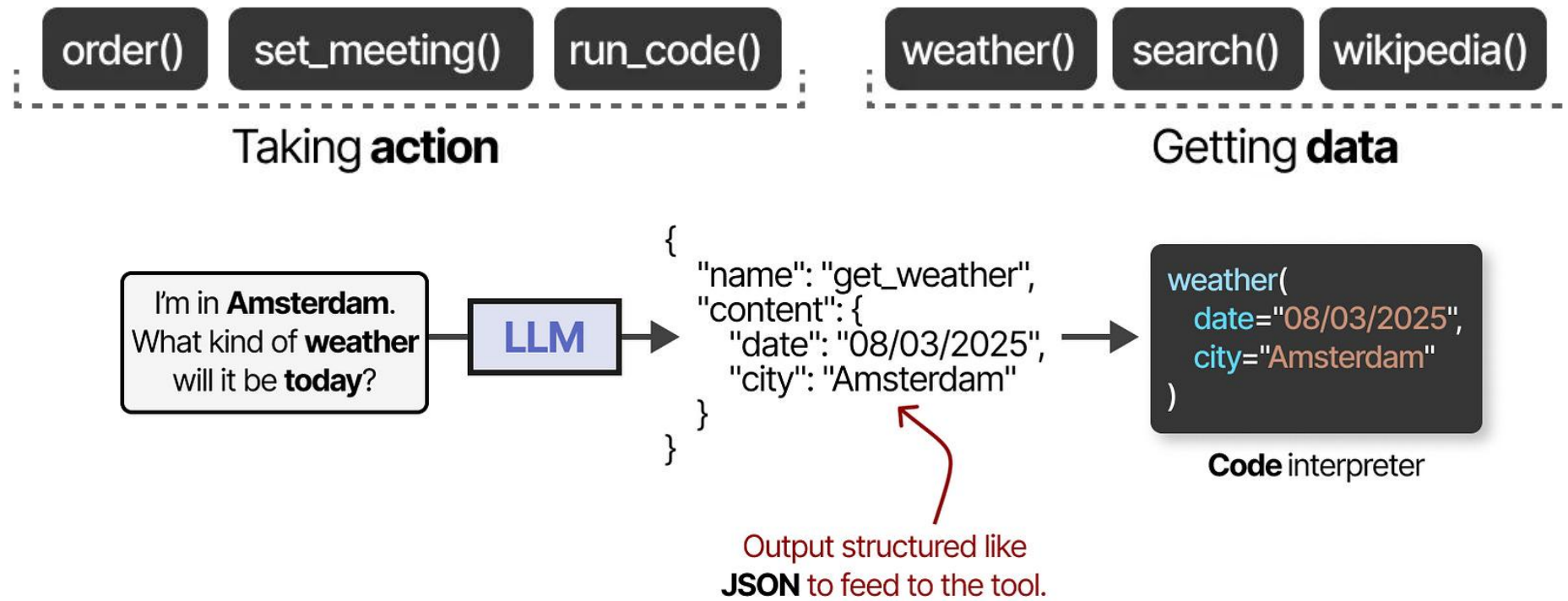
 **Short-term** memory

 **Long-term** memory

Semantic memory (facts about the world) might be stored in a different database than working memory (current and recent circumstances).

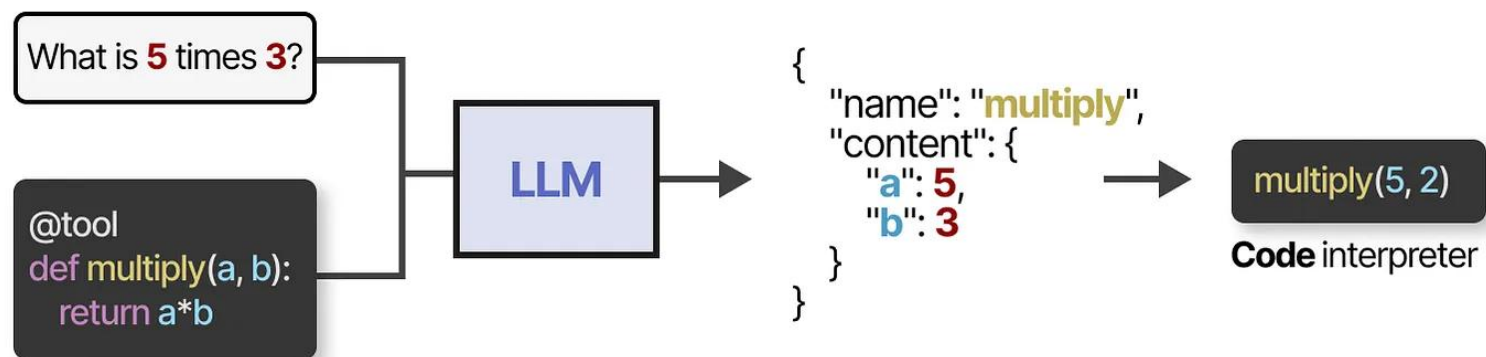
Using Tools to LLM

- Tools allow a given LLM to either interact with an external environment (such as databases) or use external applications (such as custom code to run).
- Tools generally have two use cases: **fetching data** to retrieve up-to-date information and **taking action** like setting a meeting or ordering food.

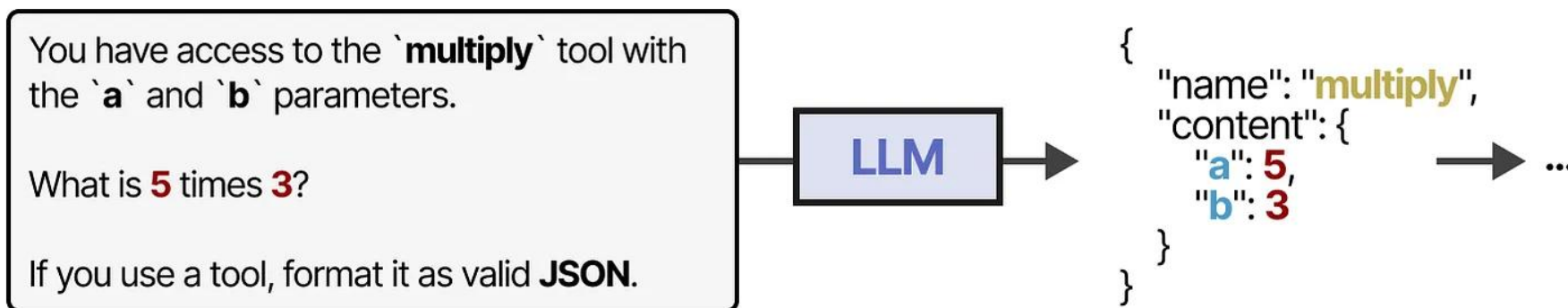


Using Tools to LLM

- **Function calling:** generate custom functions that the LLM can use, like a basic multiplication function.

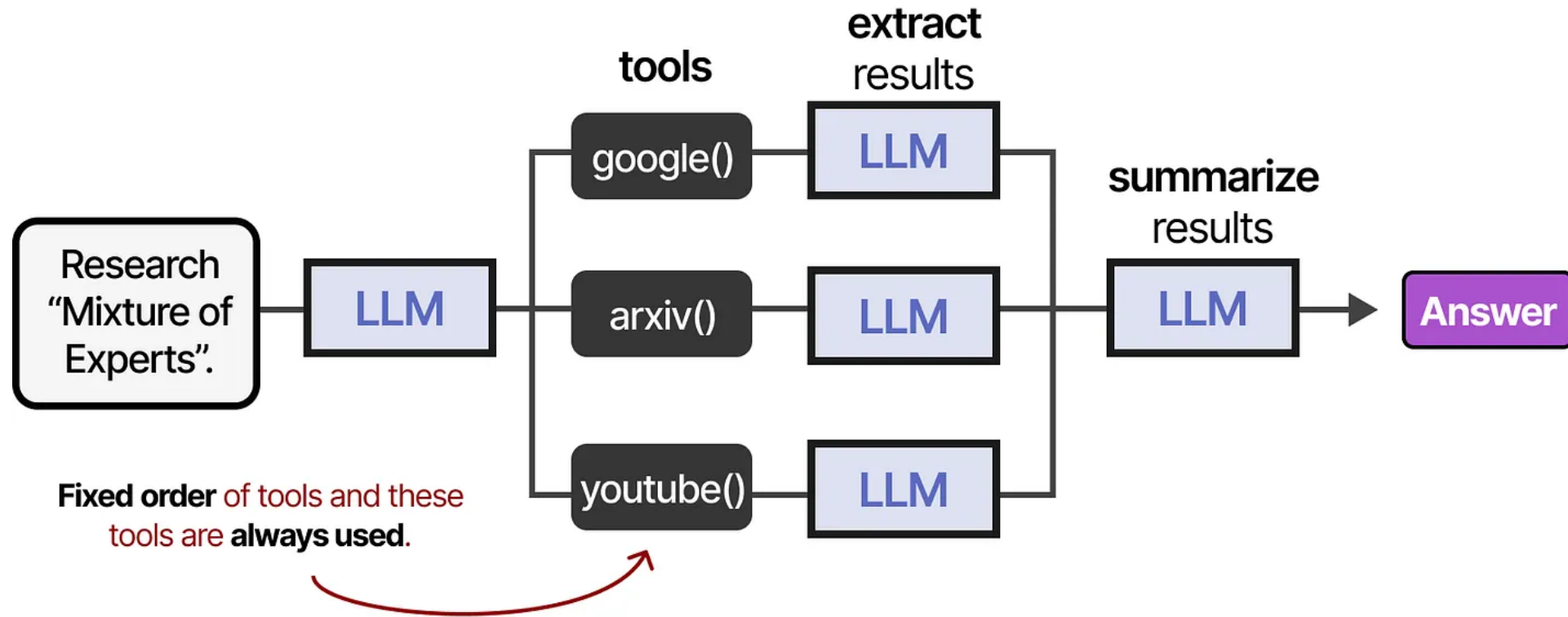


- Some LLMs can use any tools if they are prompted correctly and extensively. Tool-use is something that most current LLMs are capable of.



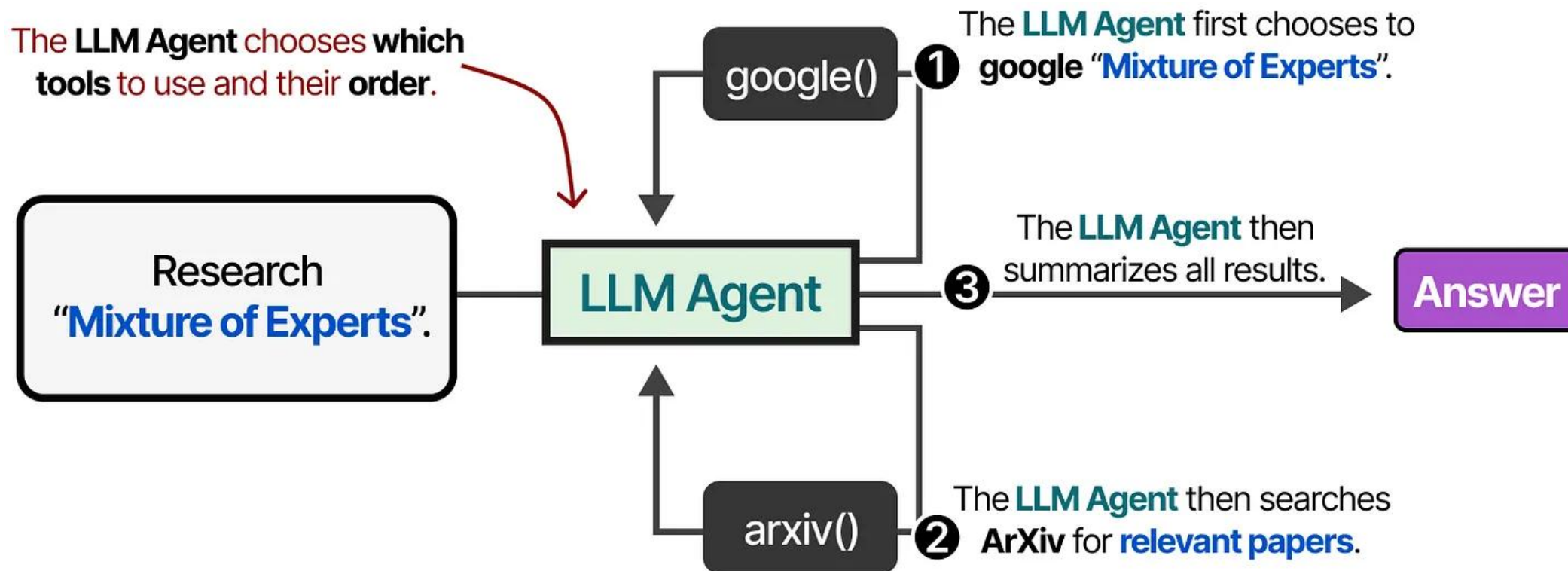
Using Tools to LLM

- Tools can either be used in a given order if the agentic framework is fixed...



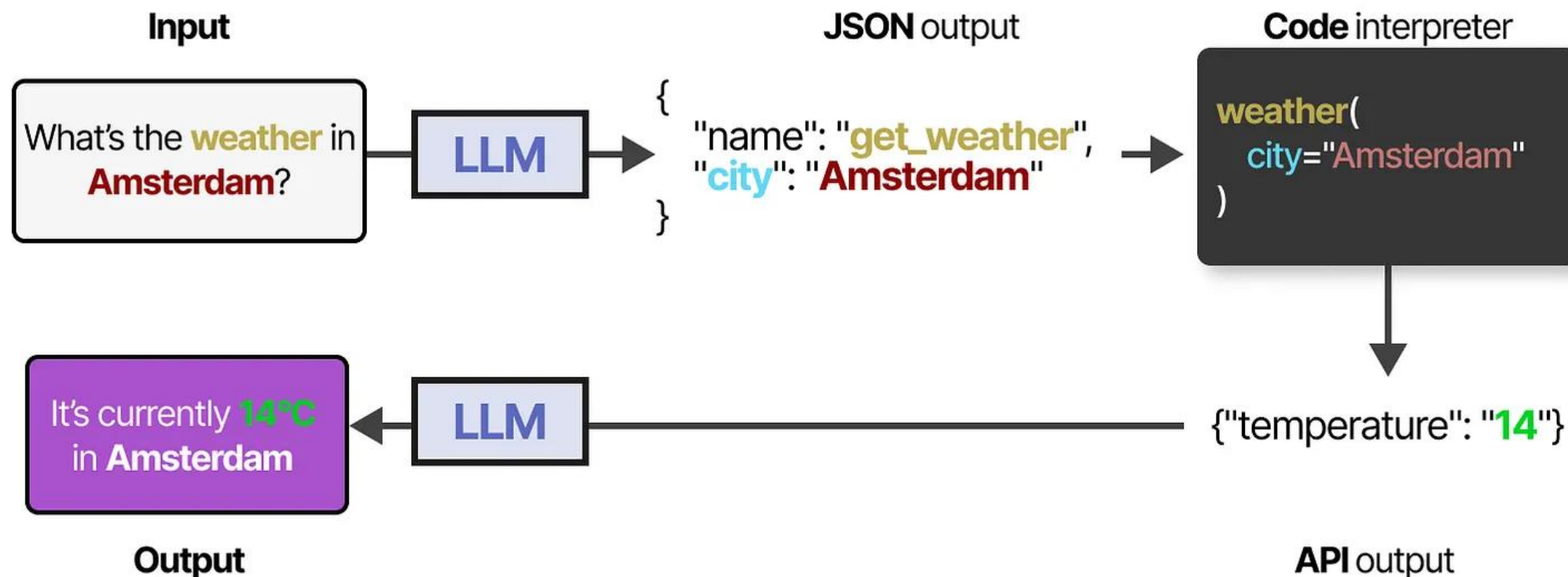
Using Tools to LLM

- LLM can autonomously choose which tool to use and when.



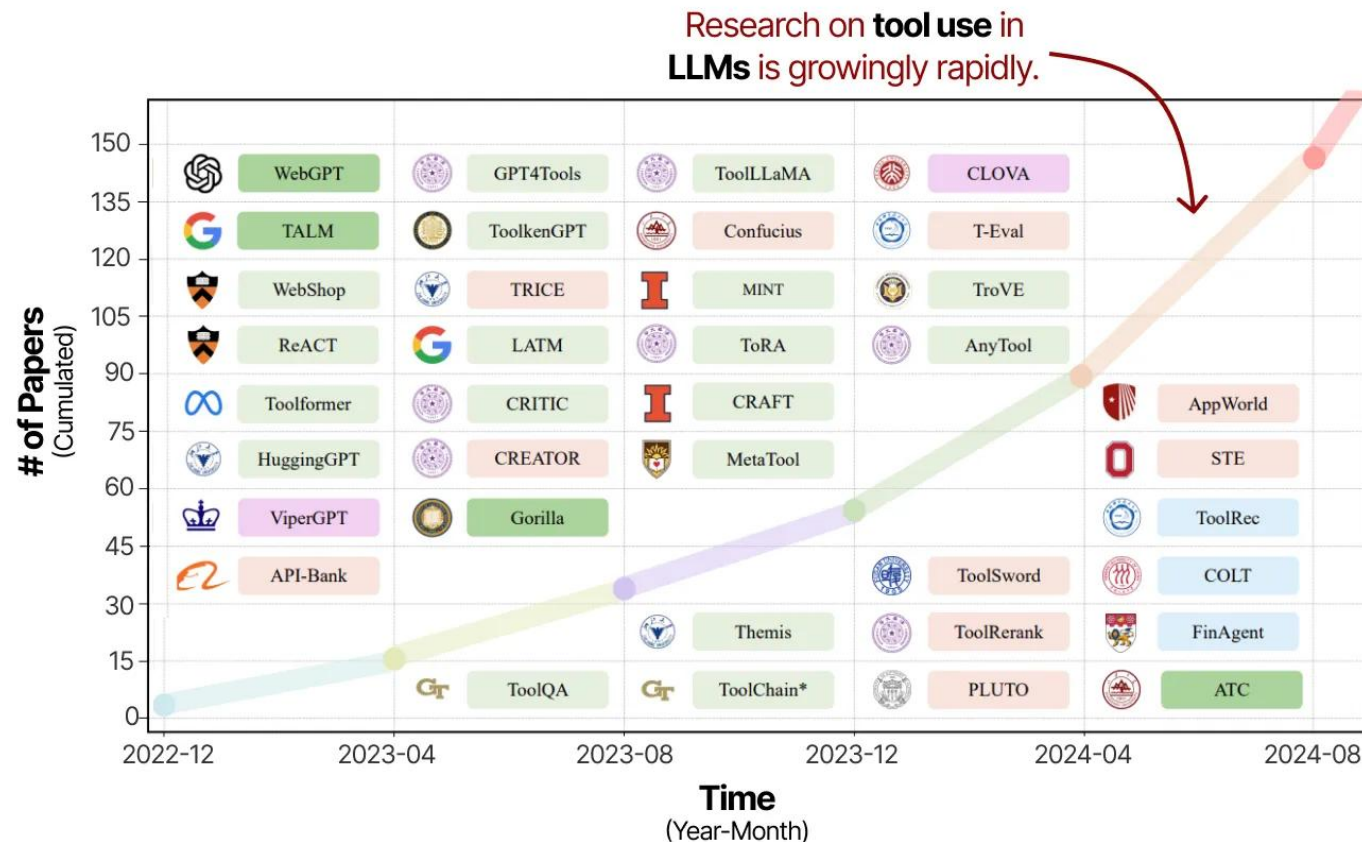
Using Tools to LLM

- In other words, the output of intermediate steps is fed back into the LLM to continue processing.



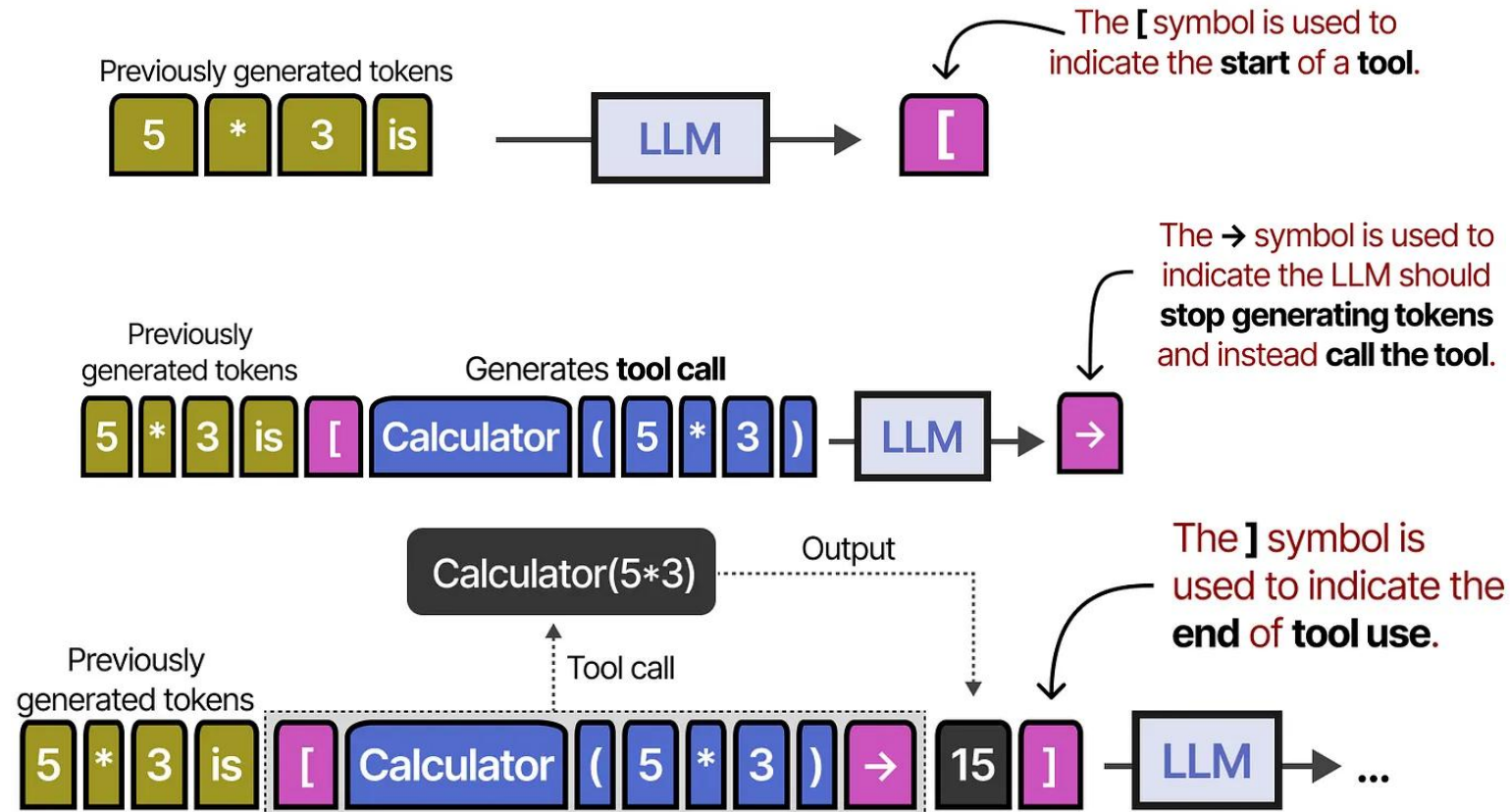
Toolformer

- Tool use is a powerful technique for strengthening LLMs' capabilities and compensating for their disadvantages. As such, research efforts on tool use and learning have seen a rapid surge in the last few years.
- Toolformer is a model trained to decide which APIs to call and how.



Toolformer

- [and] tokens to indicate the start and end of calling a tool.
- LLM generates tokens until it reaches the → token which indicates that the LLM stops generating tokens.
- Then, the tool will be called, and the output will be added to the tokens generated thus far.
- The] symbol indicates that the LLM can now continue generating if necessary.



Toolformer

- Toolformer creates this behavior by carefully generating a dataset with many tool uses the model can train on. For each tool, a few-shot prompt is manually created and used to sample outputs that use these tools.

Few-shot prompt per tool (**Calculator**)

You can add calls to a calculator API. You can call the API by writing "[Calculator(formula)]".

Input: 5 times 3 is 15.

Output: 5 times 3 is [Calculator(5*3)].

Input: 12 divided by 6 is 2.

Output: 12 divided by 6 is [Calculator(6/2)].

examples
(manually created)

Input: 5 times 3 is 15.

data
(questions from large datasets)

LLM

incorrect tool

Output: 5 times 3 is [Multiply(5-3)].

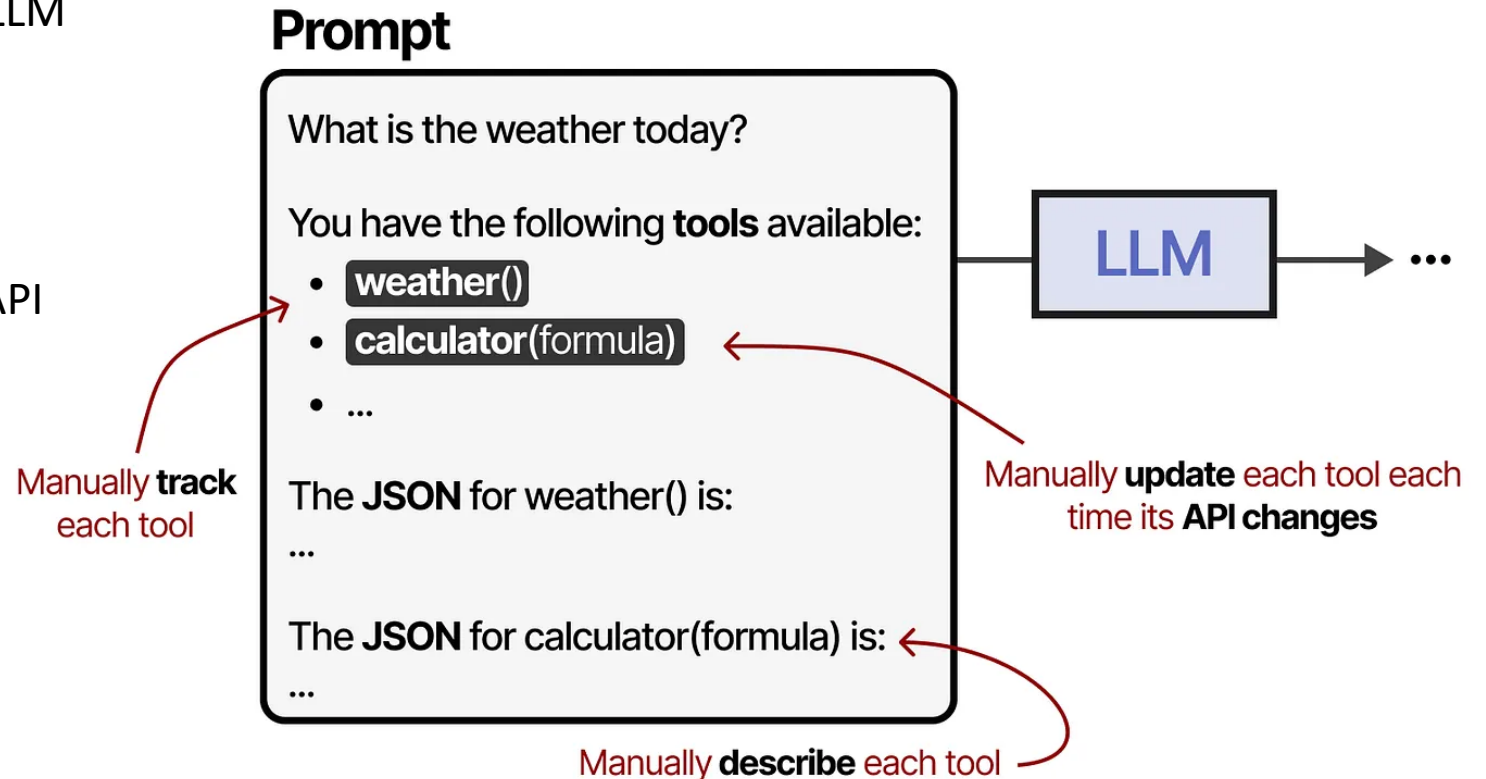
Output: 5 times 3 is [Calculator(5*3)].

Output: 5 times 3 is Calculator(5/12).

filtered
(the best outputs are selected based on correctness of tool use, output, and loss decrease)

Model Context Protocol (MCP)

- Tools are an important component of Agentic frameworks, allowing LLMs to interact with the world and extend their capabilities.
- However, enabling tool use when you have many different API becomes troublesome as any tool needs to be:
 - Manually **tracked** and fed to the LLM
 - Manually **described** (including its expected JSON schema)
 - Manually **updated** whenever its API changes

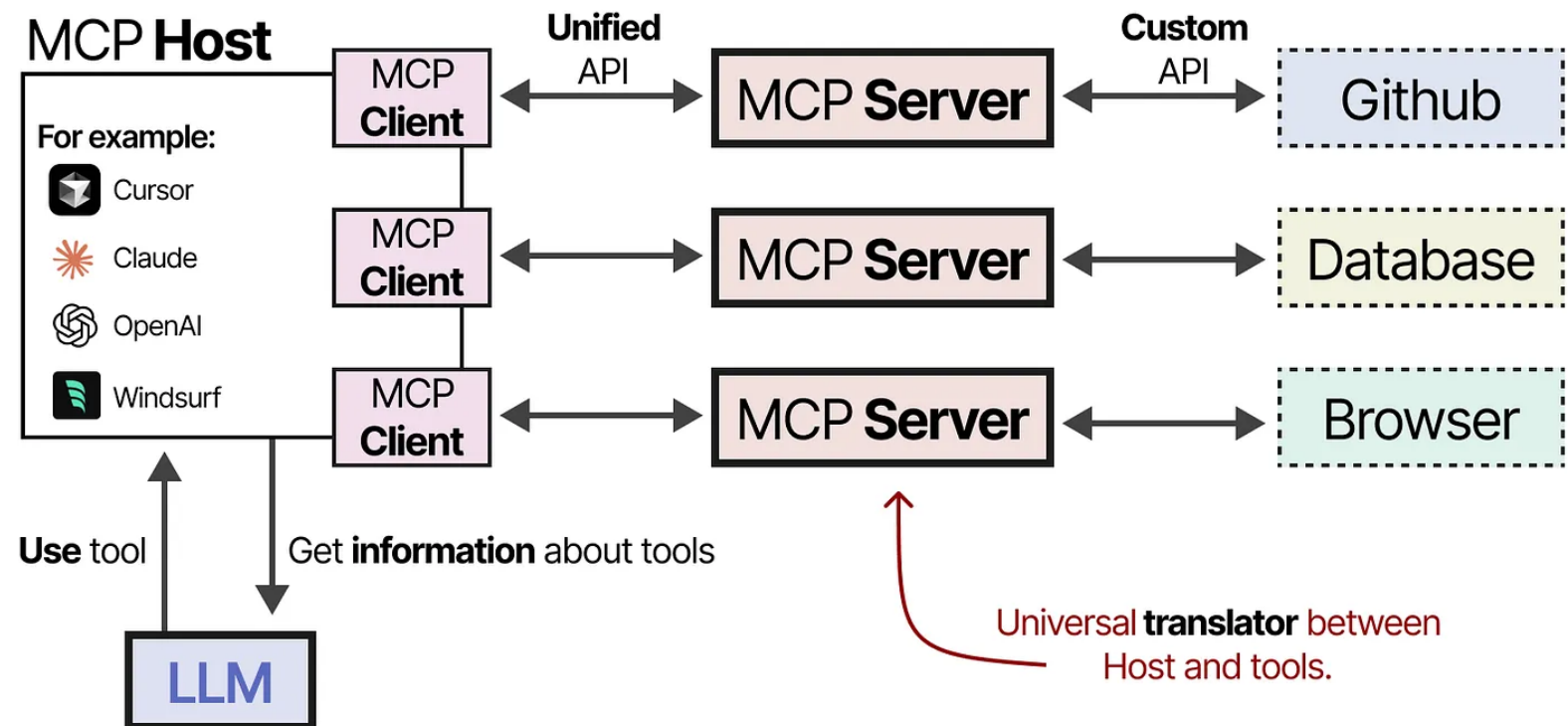


Model Context Protocol (MCP)

- For easier implementation any given Agentic framework, Anthropic developed the Model Context Protocol (MCP).
- MCP standardizes API access for services like weather apps and GitHub.

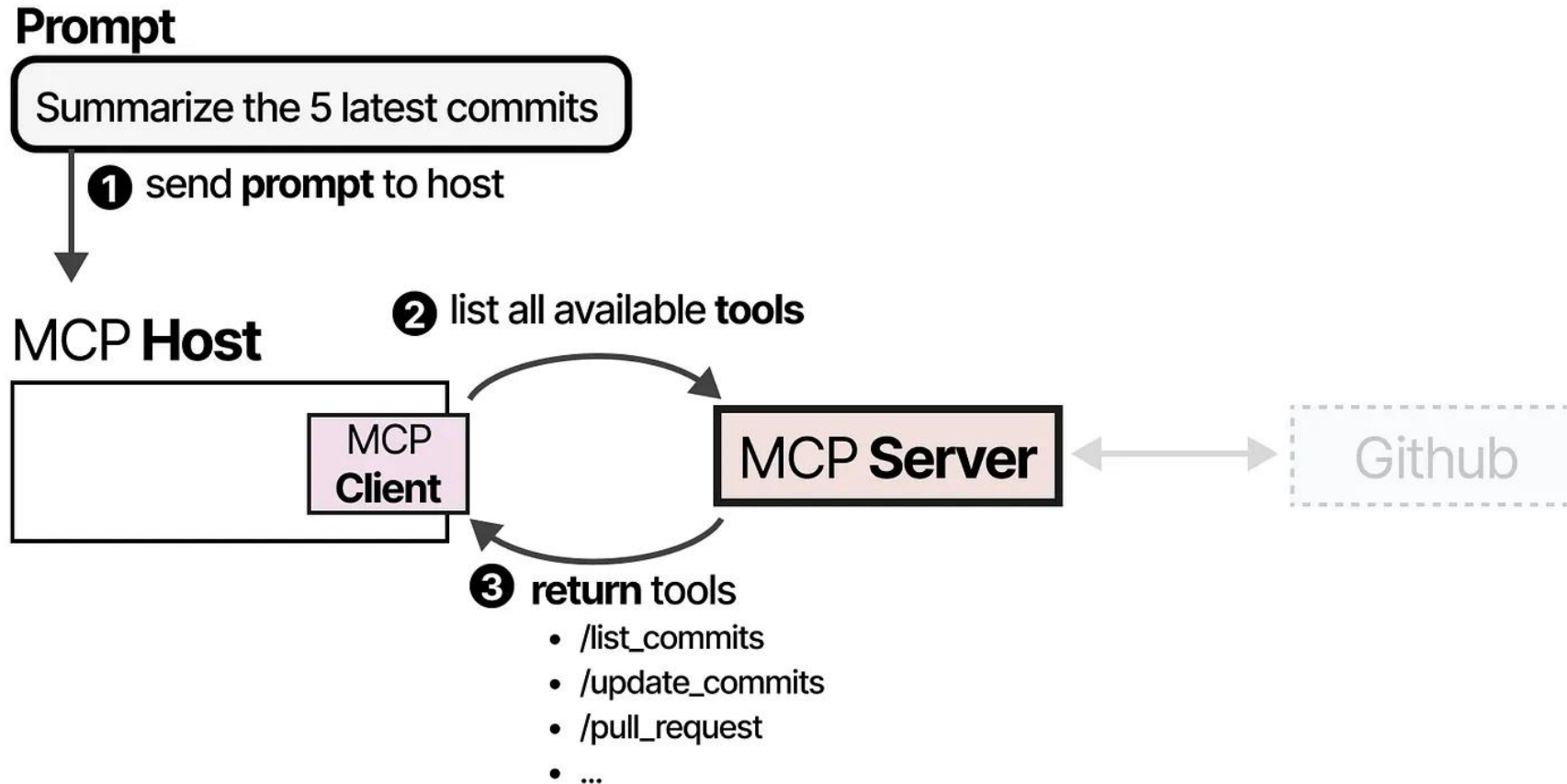
Three components of MCP

- **MCP Host** — LLM application (such as Cursor) that manages connections
- **MCP Client** — Maintains 1:1 connections with MCP servers
- **MCP Server** — Provides context, tools, and capabilities to the LLMs



Model Context Protocol (MCP)

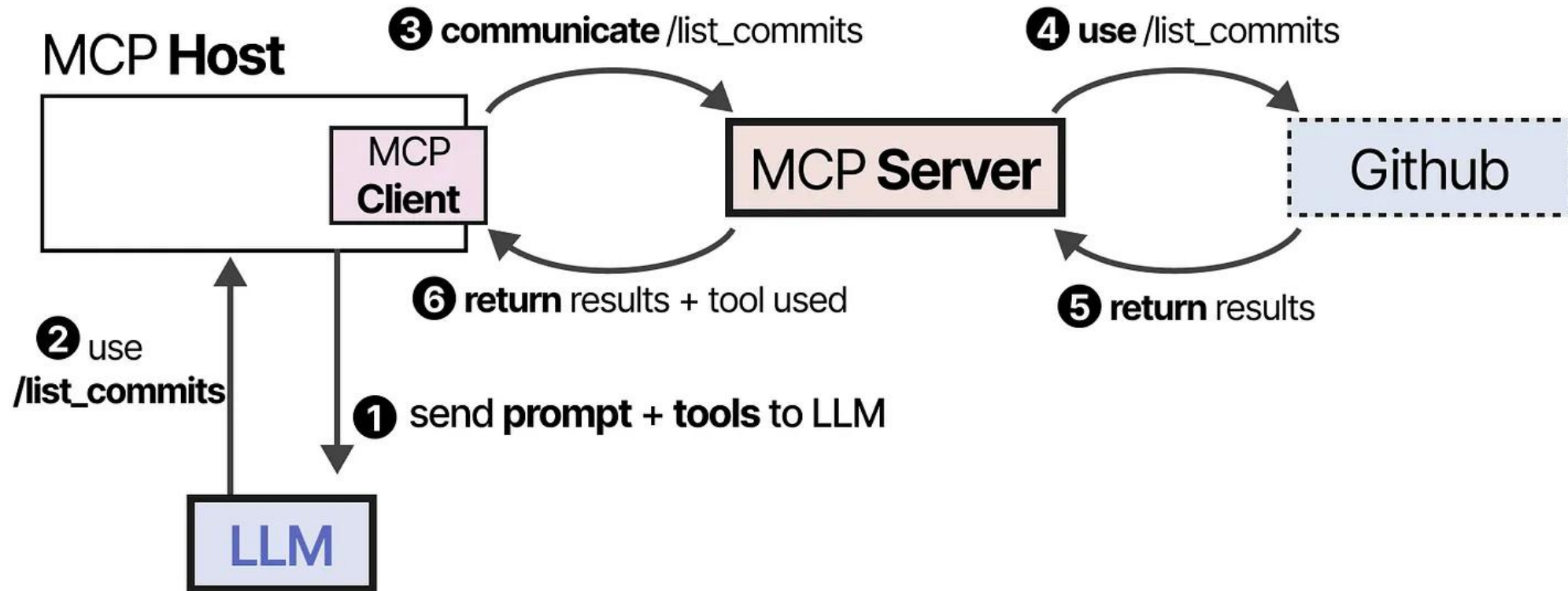
- Let's assume you want a given LLM application to summarize the 5 latest commits from your repository.



- The MCP Host (together with the client) would first call the MCP Server to ask which tools are available.

Model Context Protocol (MCP)

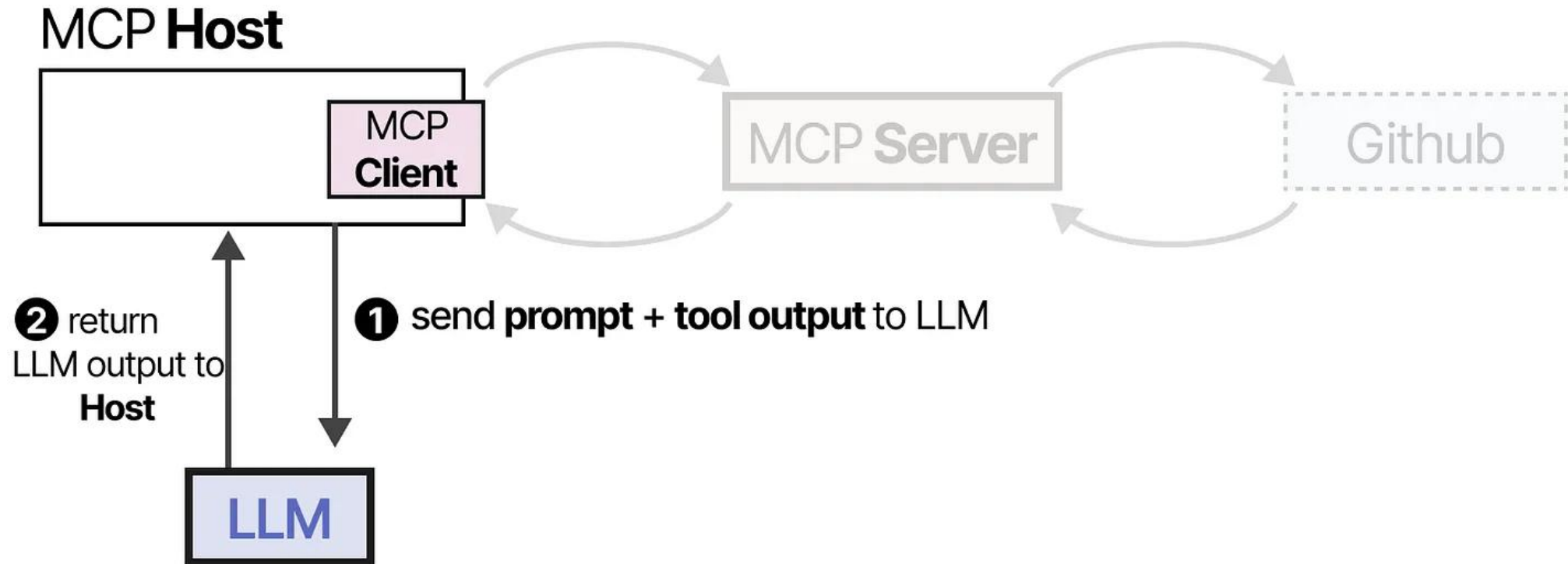
- Let's assume you want a given LLM application to summarize the 5 latest commits from your repository.



- The LLM receives the information and may choose to use a tool. It sends a request to the MCP Server via the Host, then receives the results, including the tool used.

Model Context Protocol (MCP)

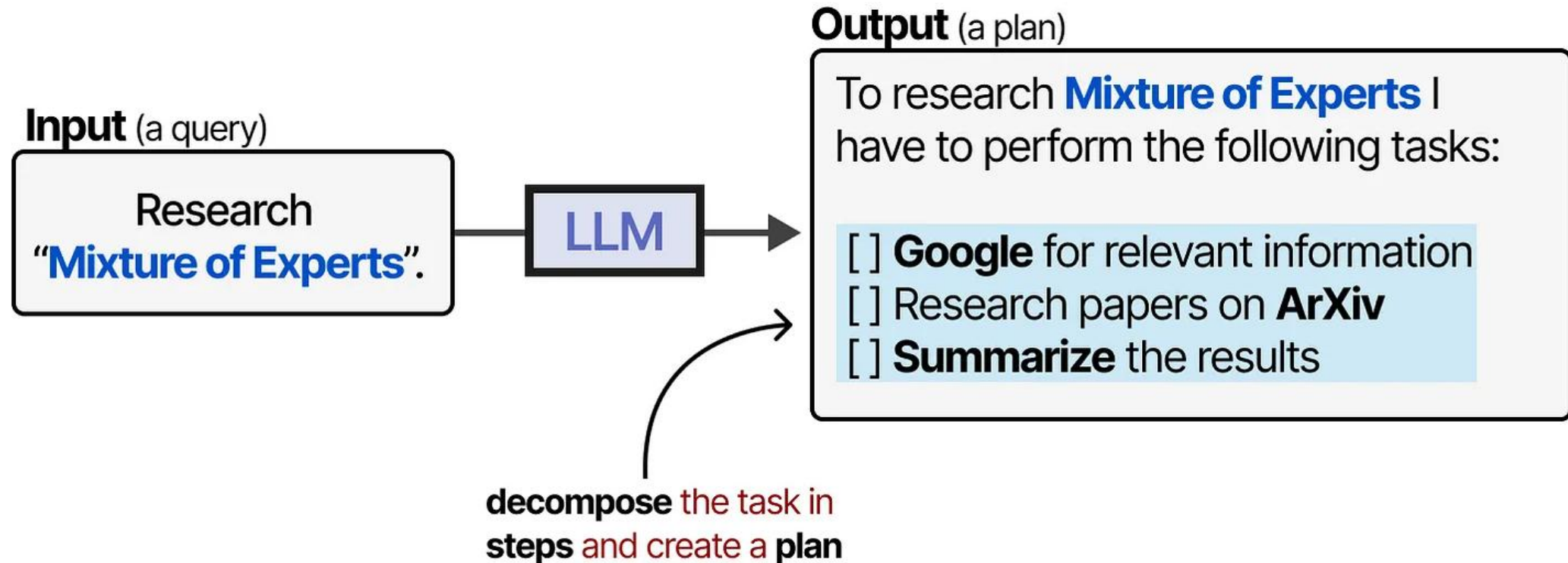
- Let's assume you want a given LLM application to summarize the 5 latest commits from your repository.



- Finally, the LLM receives the results and can parse an answer to the user.

Planning

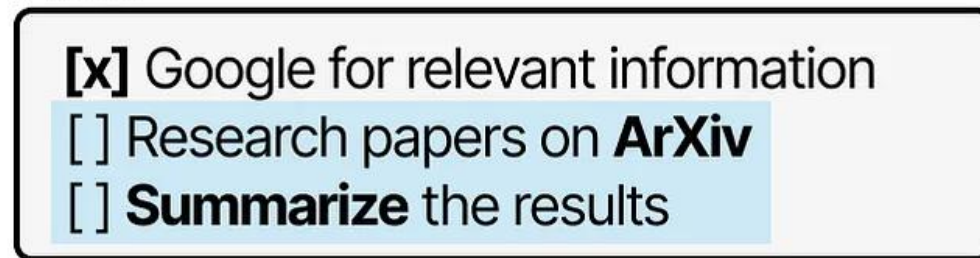
- Tool use allows an LLM to increase its capabilities. They are typically called using JSON-like requests.
- But how does the LLM, in an agentic system, decide which tool to use and when?
- This is where planning comes in. Planning in LLM Agents involves breaking a given task up into actionable steps.



Planning

- Plan allows the model to iteratively reflect on past behavior and update the current plan if necessary.

Plan



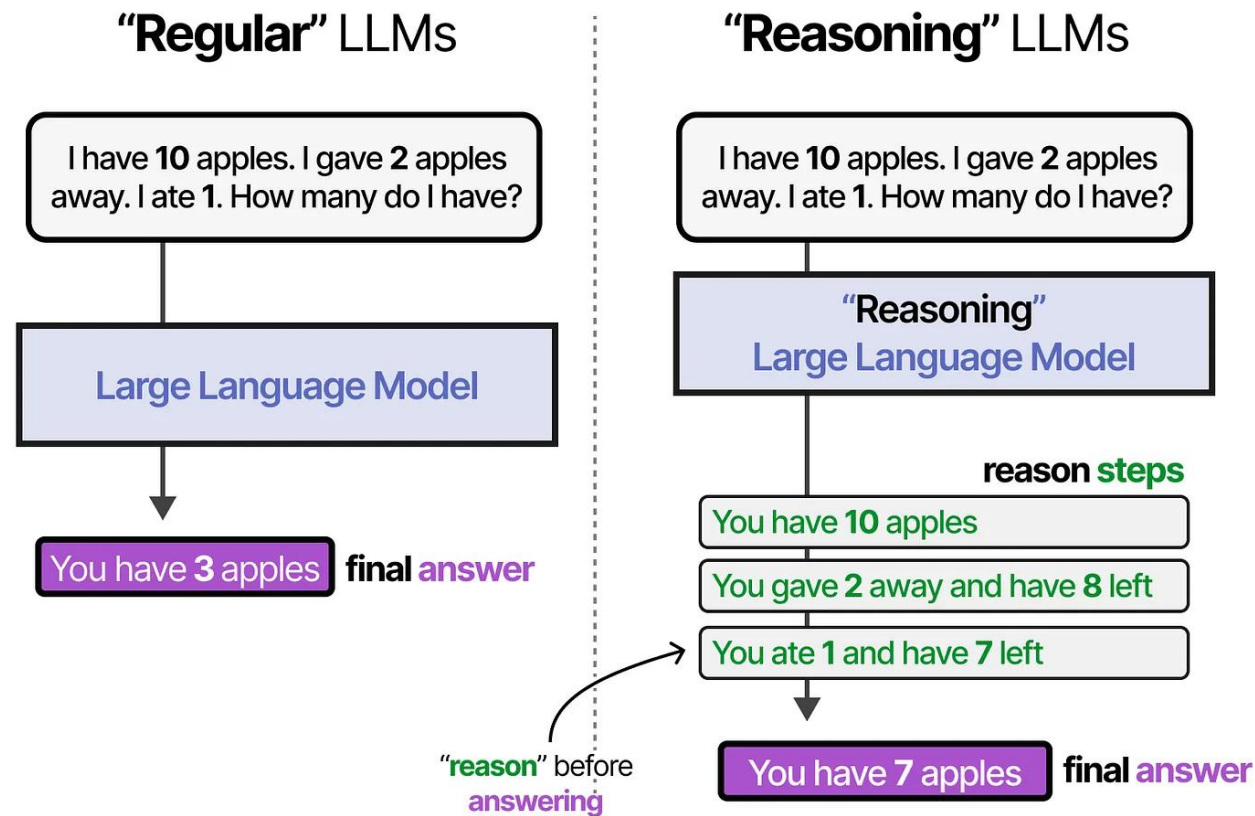
Next Steps



- To enable planning in LLM Agents, let's first look at the foundation of this technique, namely reasoning.

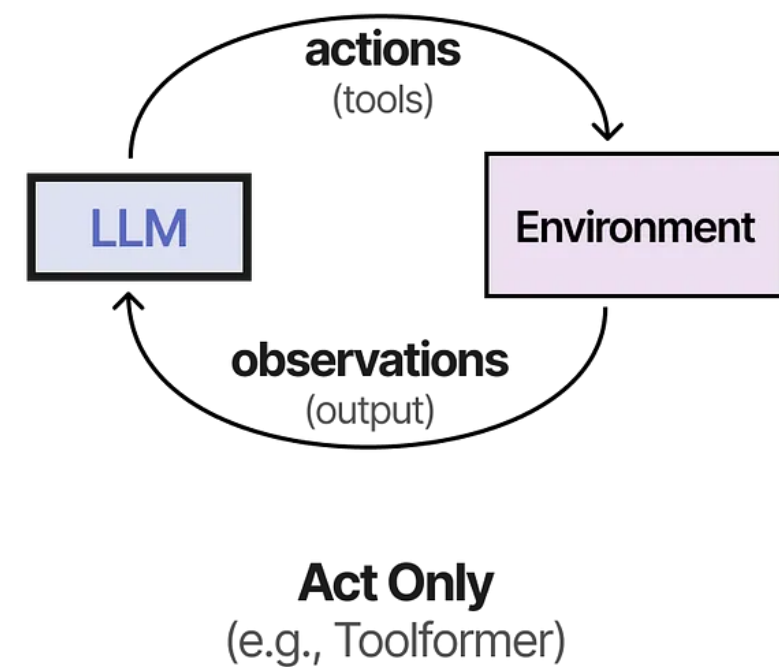
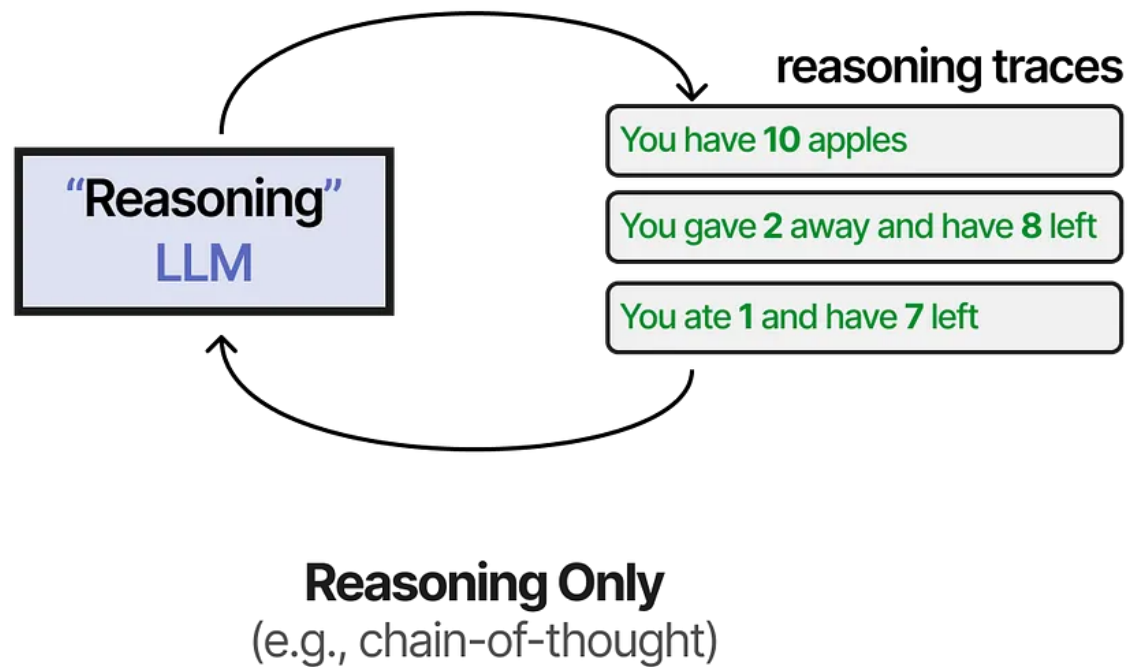
Reasoning

- Planning actionable steps requires complex reasoning behavior. As such, the LLM must be able to showcase this behavior before taking the next step in planning out the task.
- “Reasoning” LLMs are those that tend to “think” before answering a question.



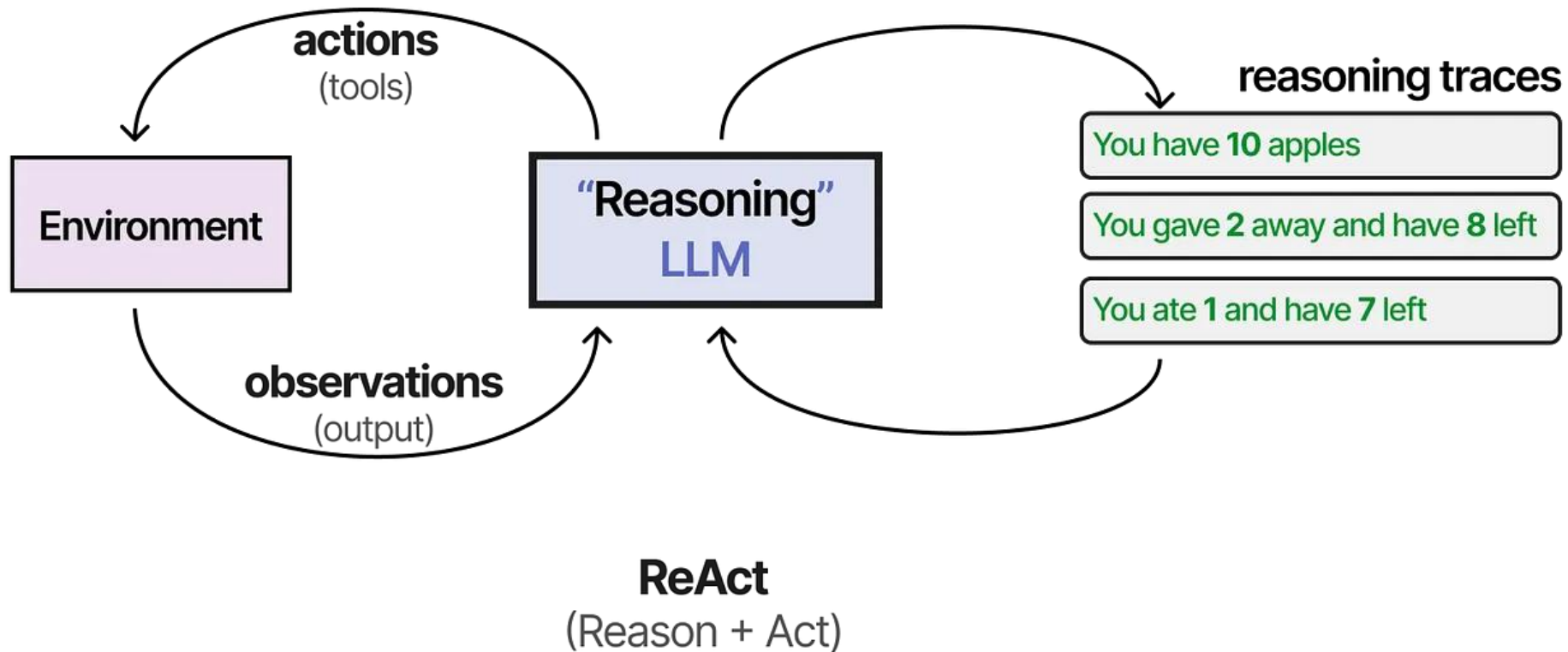
Reasoning

- Enabling reasoning behavior in LLMs is great but does not necessarily make it capable of planning actionable steps.
- Chain-of-Thought, for instance, is focused purely on reasoning.



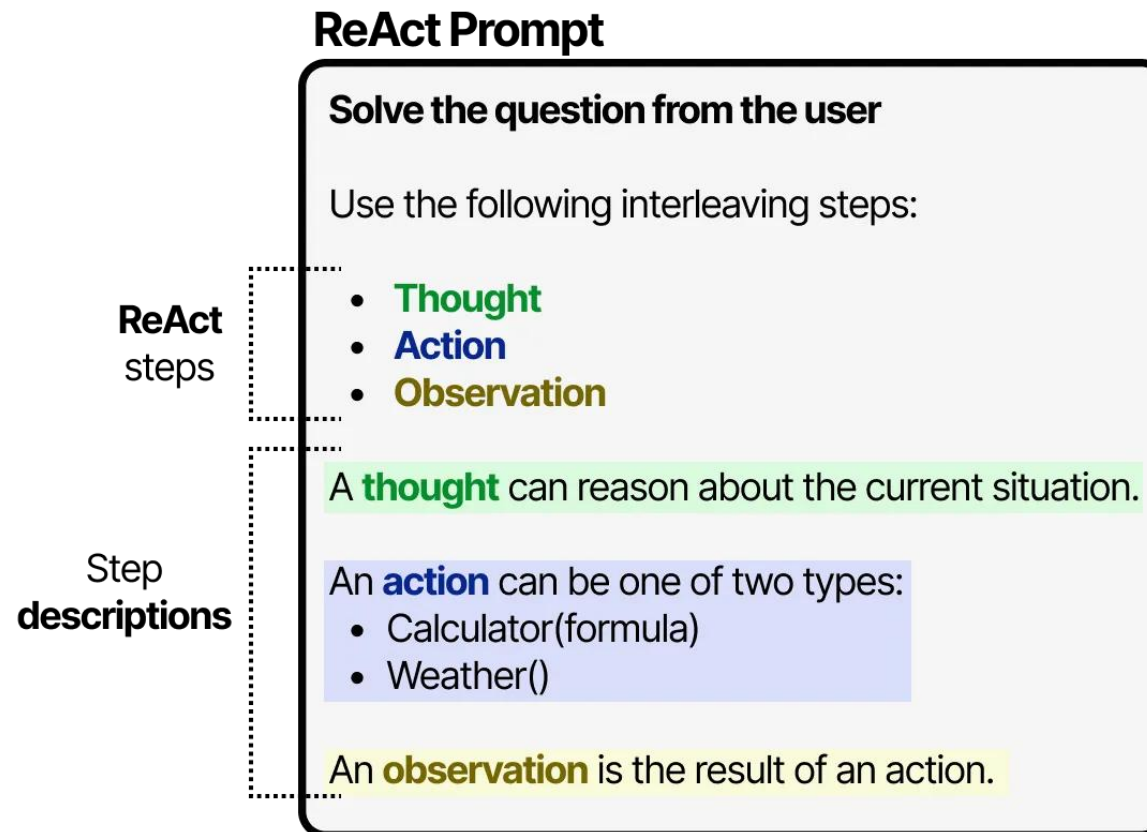
ReAct: Reason and Act

- One of the first techniques to combine both processes is called ReAct (Reason and Act)
- ReAct does so through careful prompt engineering.



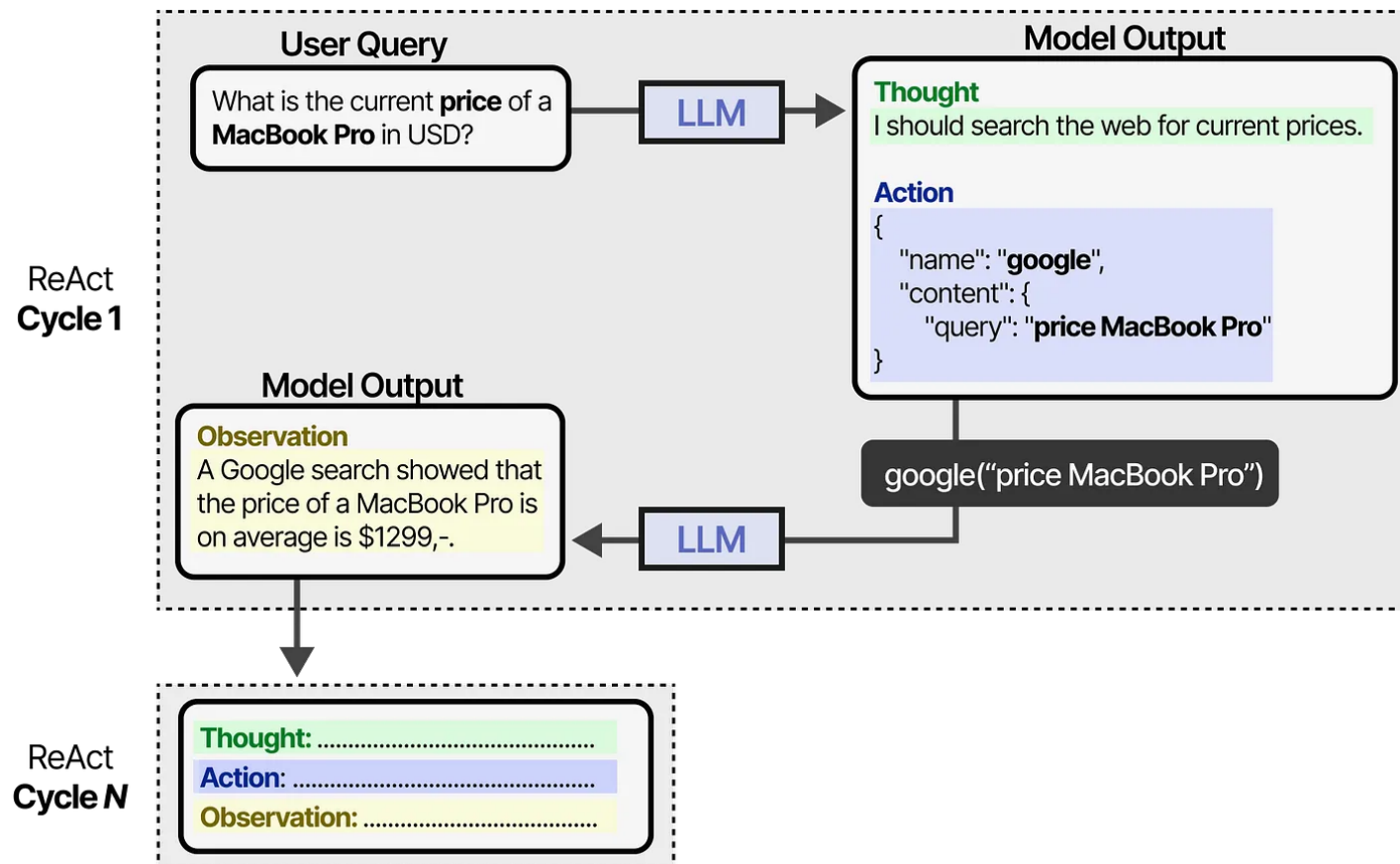
ReAct: Reason and Act

- The ReAct prompt describes three steps:
 - **Thought** - A reasoning step about the current situation
 - **Action** - A set of actions to execute (e.g., tools)
 - **Observation** - A reasoning step about the result of the action



ReAct: Reason and Act

- The LLM uses this prompt (which can be used as a system prompt) to steer its behaviors to work in cycles of thoughts, actions, and observations.



Reflecting

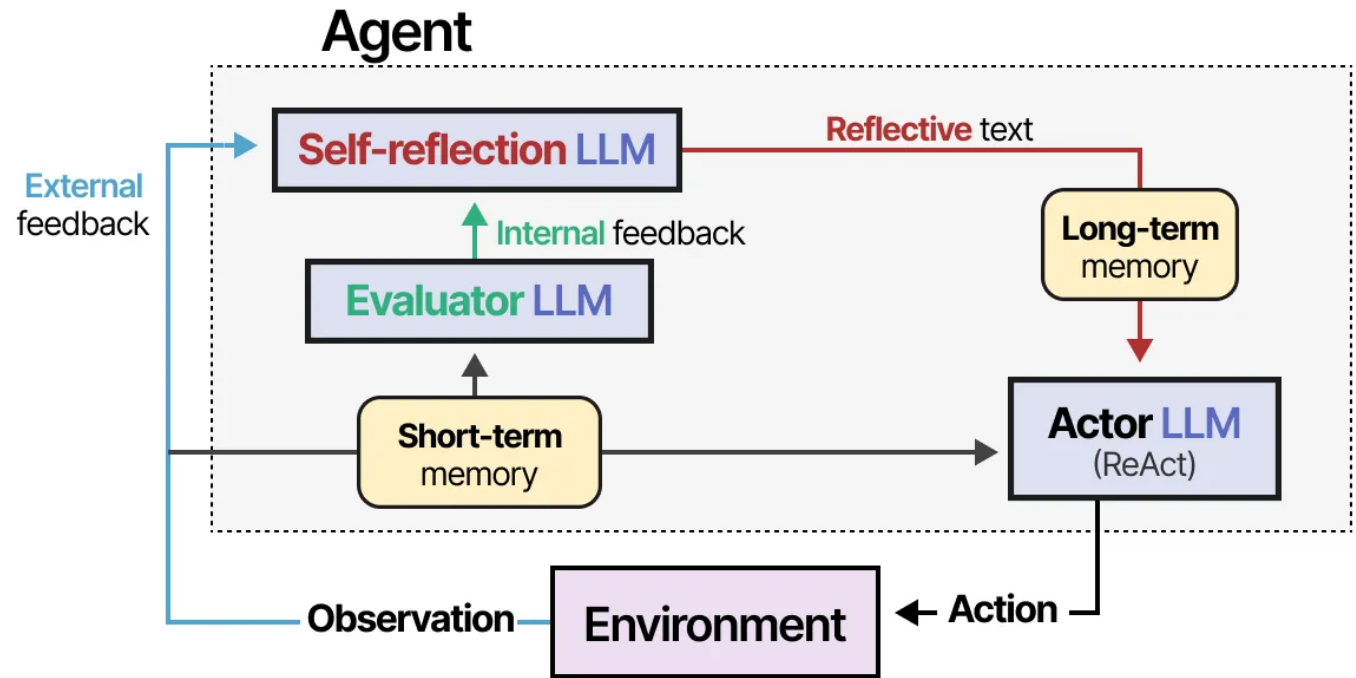
- Nobody, not even LLMs with ReAct, will perform every task perfectly. Failing is part of the process as long as you can reflect on that process.
- This process is missing from ReAct and is where Reflexion comes in. Reflexion is a technique that uses verbal reinforcement to help agents learn from prior failures.

The method assumes three LLM roles:

Actor — Chooses and executes actions based on state observations. We can use methods like Chain-of-Thought or ReAct.

Evaluator — Scores the outputs produced by the Actor.

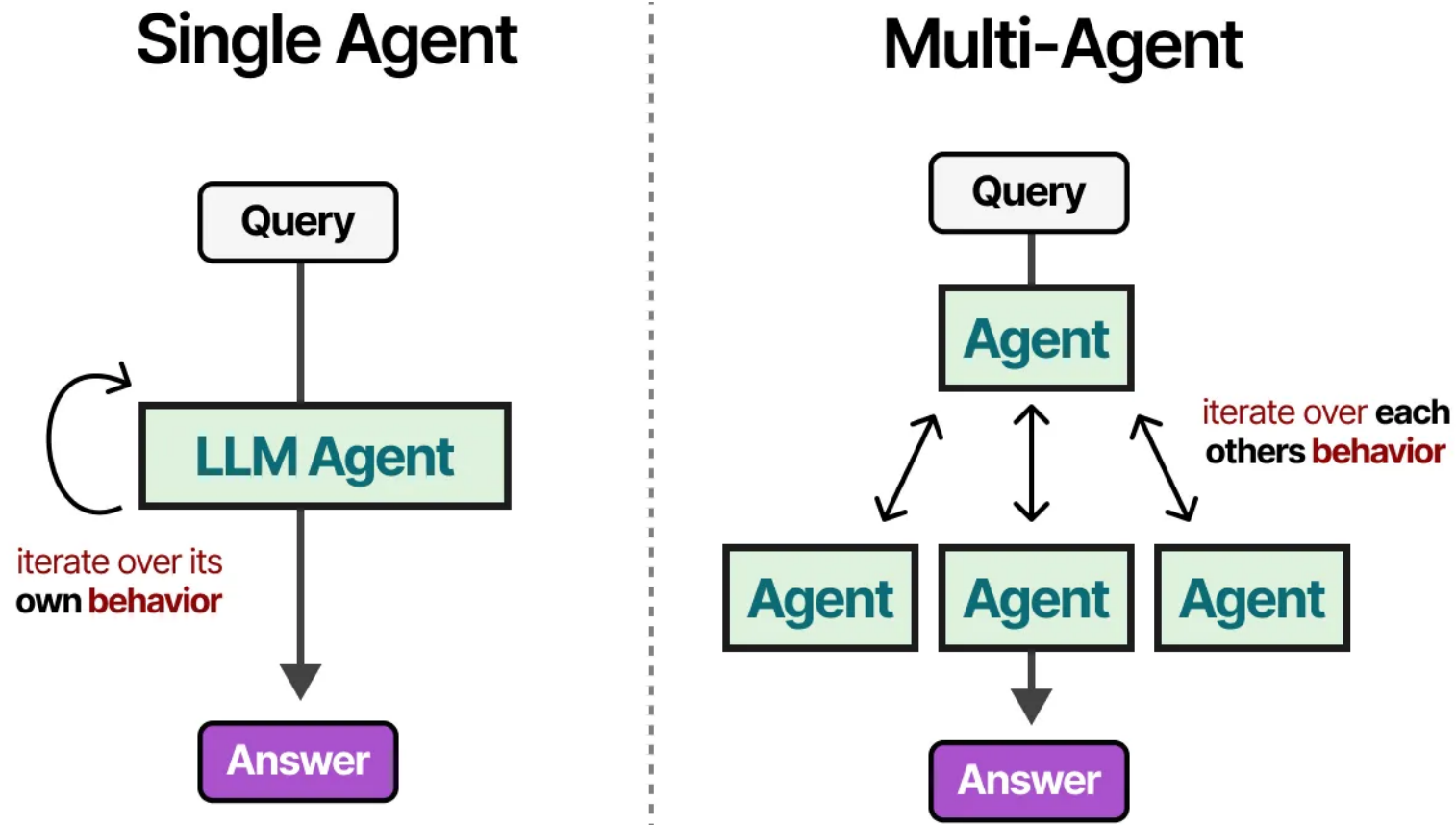
Self-reflection — Reflects on the action taken by the Actor and scores generated by the Evaluator.



Memory modules are added to track actions and self-reflections.

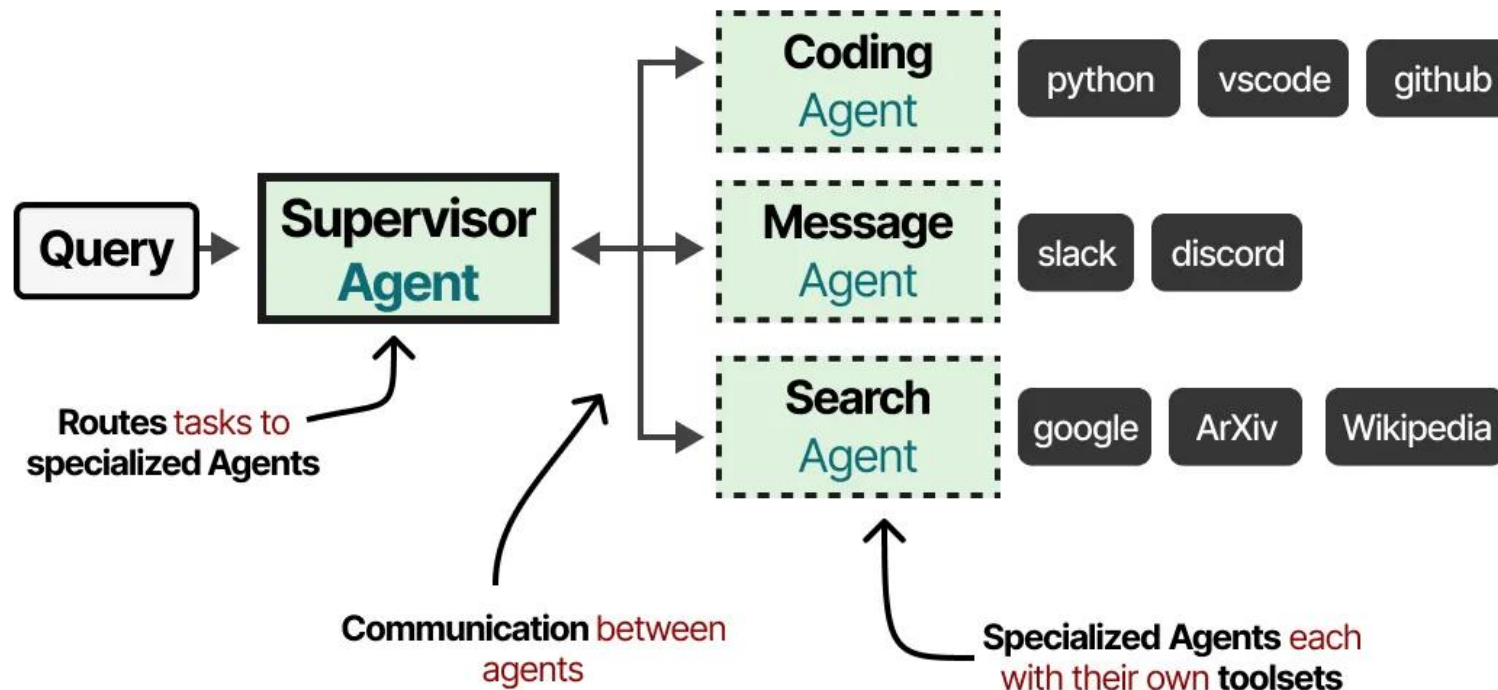
Multi-Agent Collaboration

- We can look towards Multi-Agents, frameworks where multiple agents (each with access to tools, memory, and planning) are interacting with each other and their environments.



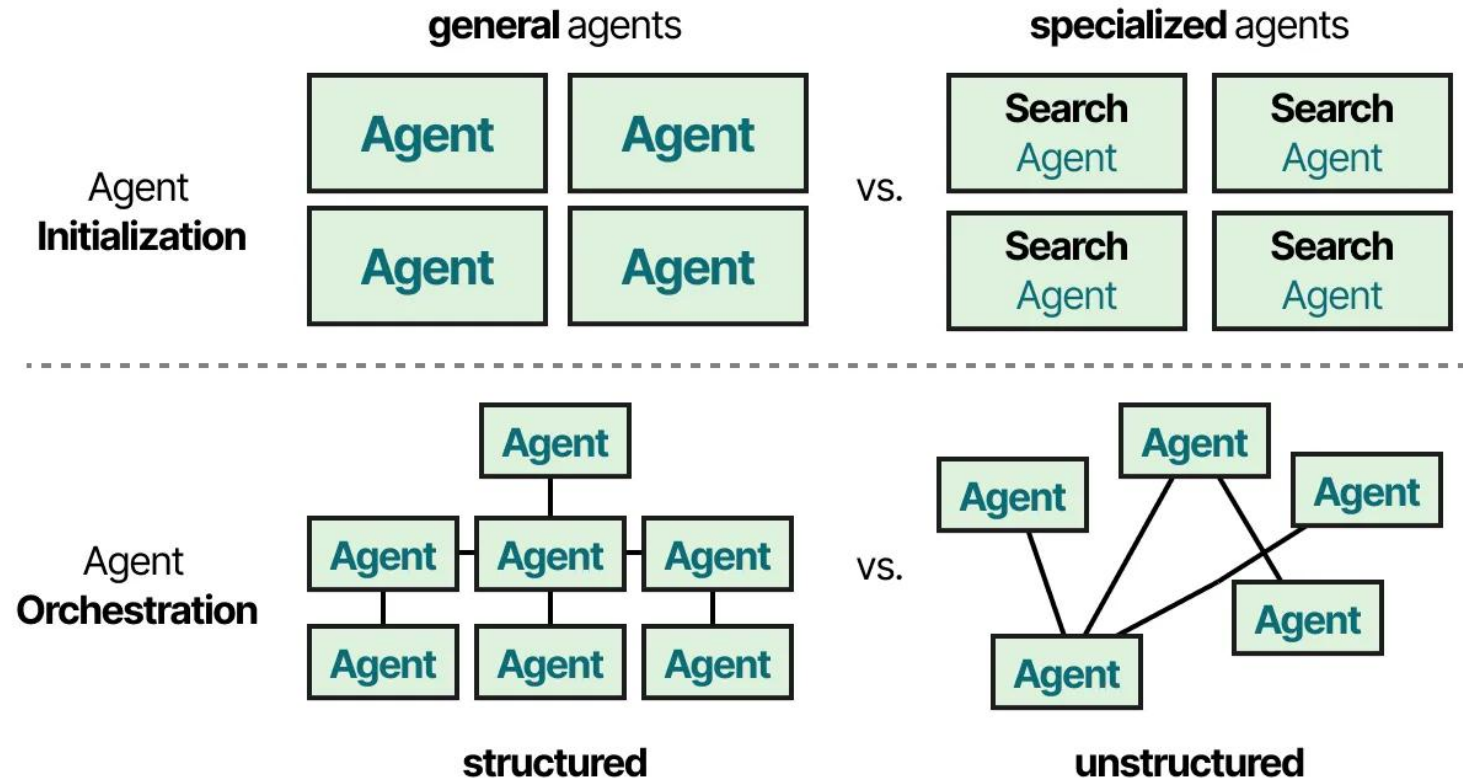
Multi-Agent Collaboration

- These Multi-Agent systems usually consist of specialized Agents, each equipped with their own toolset and overseen by a supervisor. The supervisor manages communication between Agents and can assign specific tasks to the specialized Agents.
- Each Agent might have different types of tools available, but there might also be different memory systems.



Multi-Agent Collaboration

- In practice, there are dozens of Multi-Agent architectures with two components at their core:
 - **Agent Initialization** — How are individual (specialized) Agents created?
 - **Agent Orchestration** — How are all Agents coordinated?



Summary

- **LLM Agent Definition:** An "Augmented LLM" that acts as the "brain" (or effector) in a classic agent framework. It is given access to external **Tools** (actuators), a **Memory** module, and a **Planning** component to autonomously solve complex tasks.
- **Short-Term Memory:** This is how an agent remembers the current conversation. It's typically implemented by fitting the entire conversation and action history into the LLM's **context window**.
- **Long-Term Memory:** To remember information across sessions, agents use an external **Vector Database**. Past conversations are stored as embeddings, and the agent uses **RAG** (Retrieval-Augmented Generation) to retrieve this relevant information when needed.
- **Tools & Function Calling:** Tools are external APIs (e.g., `search()`, `run_code()`, `calculator()`) that allow the agent to interact with the world. The LLM is trained to output structured **JSON** (Function Calling) to autonomously decide *which* tool to use and *when*.
- **Planning:** This is the agent's ability to decompose a complex user request into a series of smaller, actionable steps. This requires the LLM to have "reasoning" capabilities.
- **ReAct (Reason + Act):** The key technique that combines reasoning and action. The LLM is prompted to repeat a cycle: 1. **Thought** (private reasoning about what to do next), 2. **Action** (choosing and calling a tool), and 3. **Observation** (analyzing the tool's output to inform the next thought).
- **Advanced Agent Frameworks:** This concept can be extended with **Reflexion** (using an "Evaluator" LLM to learn from failures) or **Multi-Agent Collaboration**, where a "Supervisor" agent delegates tasks to a team of specialized agents to solve the problem.

LG AI Research Special Lecture

딥러닝 자연어처리 기초와 LLM 에이전트

Lecture 06 – LLM을 움직이는 힘, AI 하드웨어와 GPU



중앙대학교 AI학과

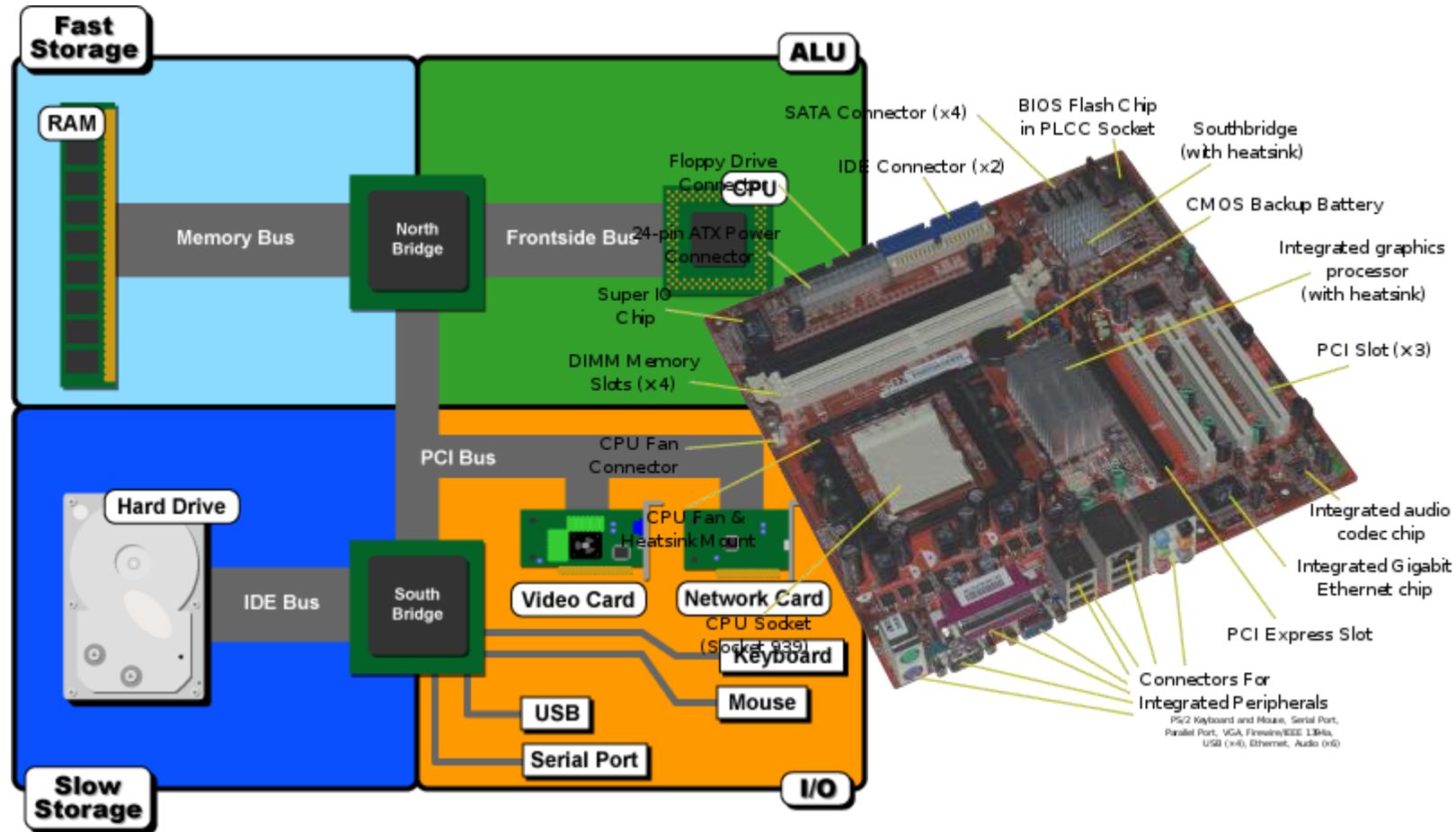
이환희

Outline

LLM을 움직이는 힘, AI 하드웨어와 GPU

- Compute Internals
 - CPU Internals
 - GPU Computing
 - CPU vs GPU
- Floating-Point Number
- Size Guide for LLM Models & GPU VRAM

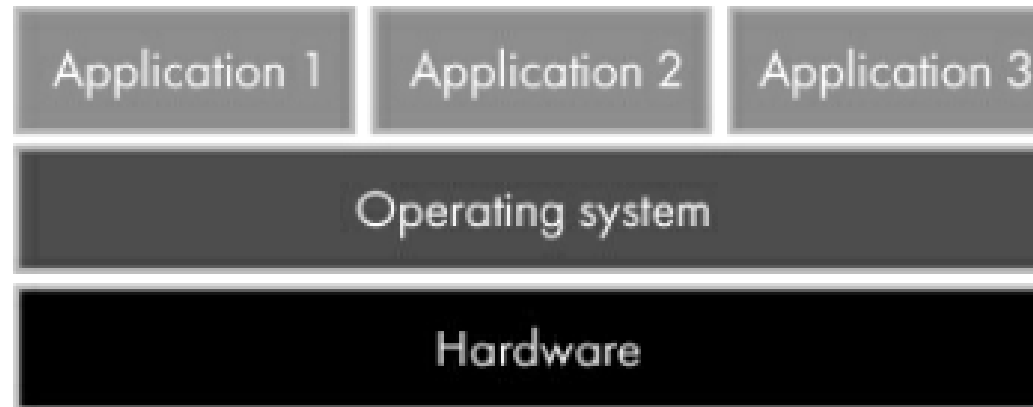
Computer Internal



by Unknown Author is licensed under [CC BY-SA](https://creativecommons.org/licenses/by-sa/4.0/)

Operating System (OS)

- Layer between Hardware and Software



Operating System (OS)

- An operating system is responsible for
 - **Resource management**
 - **Process Management**
 - **Facilitating device I/O**
 - **Providing a set of system services for applications.**
 - **User Interface**
 - **Security and Permission Management**

Operating System Kernel

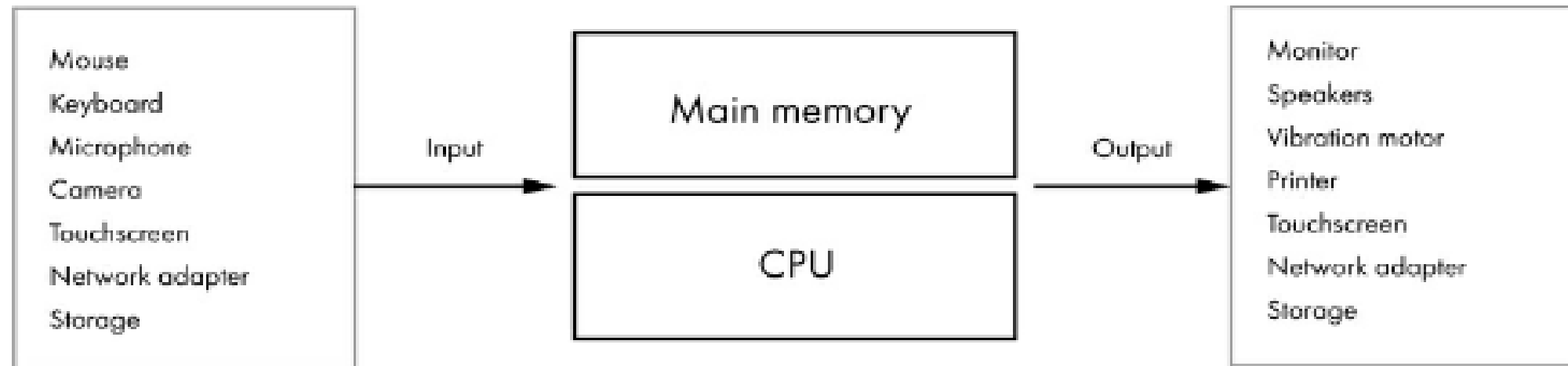
- The operating system kernel allows multiple programs to run in parallel and share hardware resources. Kernel is the core part of an operating system, but it alone provides no way for end users to interact with the system.
- Operating systems also include **non-kernel components** that are needed for a system to be of use. This includes the shell, a user interface for working with the kernel. The terms shell and kernel are part of a metaphor for operating systems, where the OS is thought of as a nut or seed. The kernel is at the core; a shell surrounds it.

Hardware Interface

- The **shell** can be either a command line interface (CLI) or a graphical user interface (GUI). Some examples of shells are the Windows shell GUI (including the desktop, Start menu, taskbar, and File Explorer), and the Bash shell CLI found on Linux and Unix systems.
- When it comes to interacting with hardware, the kernel acts in partnership with device drivers. A device driver, or simply driver, is software designed to interact with specific hardware. Operating systems typically include a set of device drivers for common hardware and also provide a mechanism for installing additional drivers.

Input/Output Operations

- Computers need to interact with external environment
- I/O Devices
- OS has software device drivers for each device

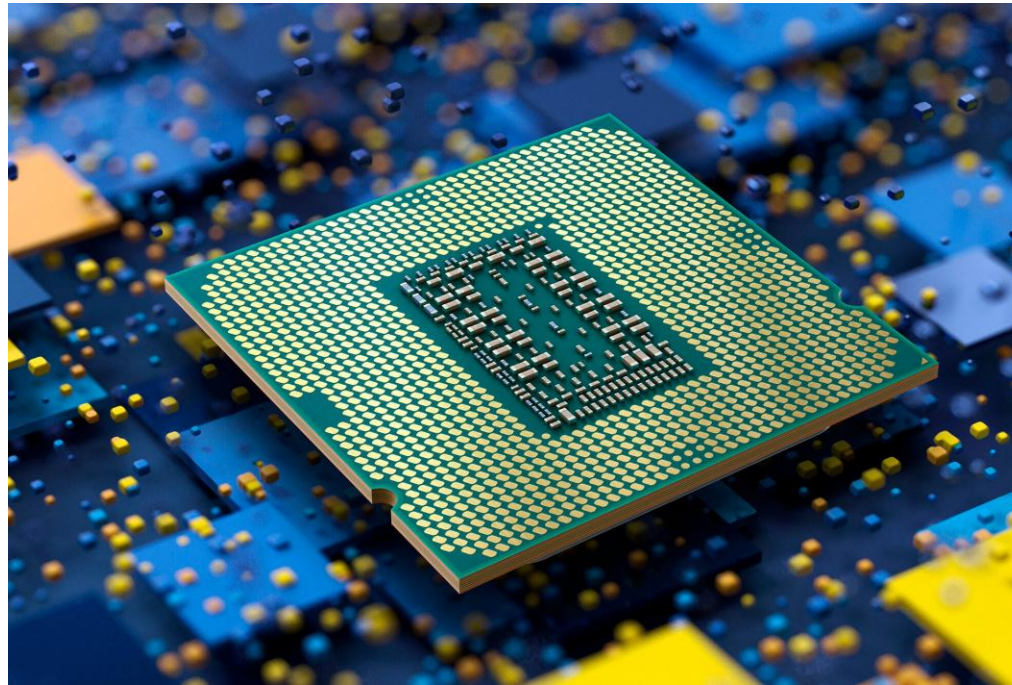


Processors

- CPU – Central Processing Units
- GPU – Graphics Processing Units
- FPGA – Field Programmable Gate Arrays
- ASIC – Application Specific Integrated Circuits

What is CPU?

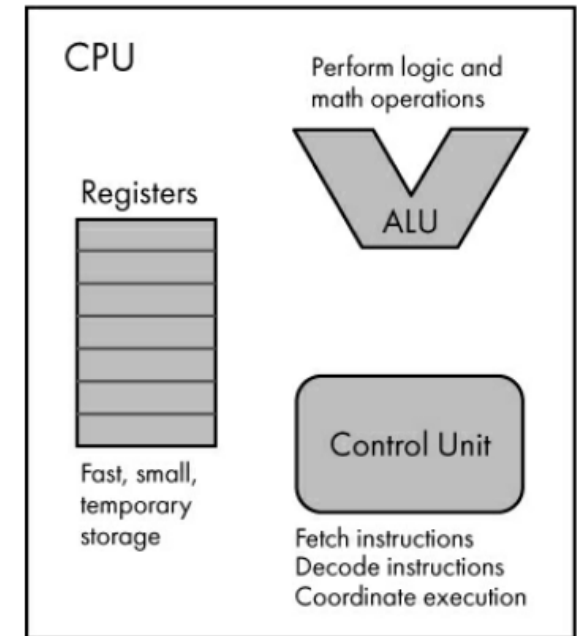
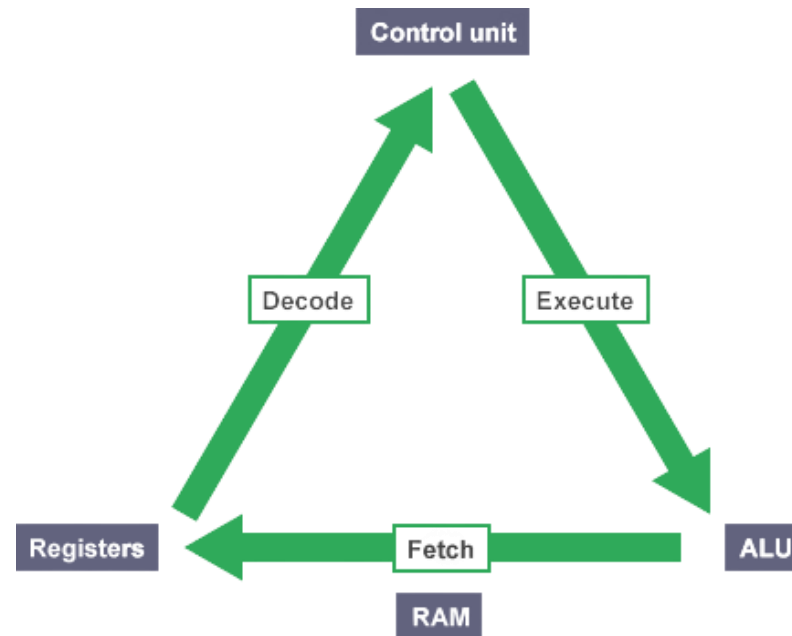
- A “**central processing unit**” (also called processor) is the brain of the computer system that initiates and executes the instructions of a computer program.



Program execution in CPU

- In serial processing, the processor executes the program instructions sequentially, one after the other. After completing that, it executes the next instruction in a sequential manner.
- CPU follows the steps below to process a sequence of program instructions that stored in main memory(RAM).

- 1- Fetch the instruction
- 2- Decode the instruction
- 3- Execute the instruction



CPU Cores

- PHYSICAL AND LOGICAL CORES - Not all cores are equally capable of parallelism. A physical core is a hardware implementation of a core within a CPU. Logical cores represent the ability of a single physical core to run multiple threads at once (one thread per logical core). Intel refers to this capability as hyper-threading.
- A computer with two physical cores, each with two logical cores, has a total of four logical cores and run four threads at once. Logical cores cannot achieve the full parallelism of physical cores.

CPU

- Latency – beginning to end duration of performing a single computation (*Tuomanen, 2018)
- CPUs are engineered to reduce the latency of a single computation.
- Computations are sequential
- Fewer cores that are restricted with the number of processes it can compute and it handles those processes very fast

What is GPU?

- GPU stands for “**Graphic Processing Unit**” that is a piece of hardware designed to accelerate graphics rendering tasks.
- GPU computing is the use of the GPU as a co-processor to accelerate CPUs for general-purpose scientific and engineering computing.
- It can be used to execute the instructions parallelly on multiple GPU cores, thus accelerating the processing.

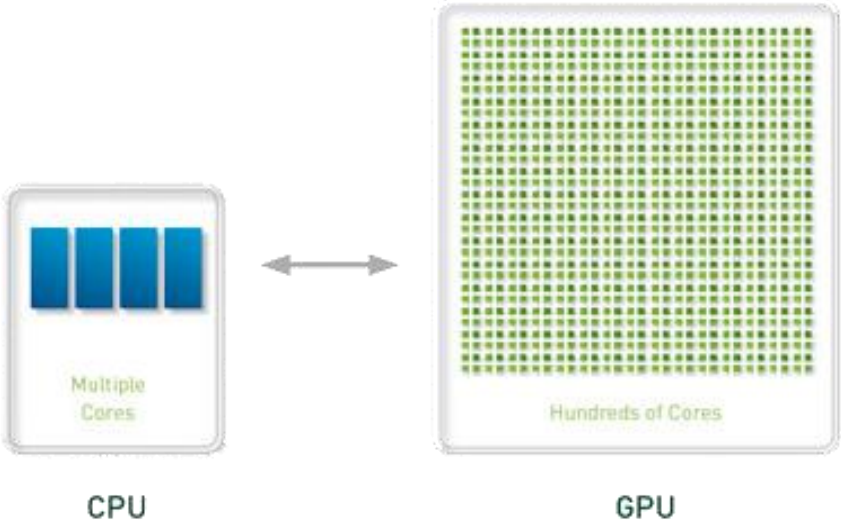


GPU

- Each core is much simpler than that of the CPU and each core on its own is not as fast as a CPU
- It's the number of cores that make a difference. GPU has hundreds to thousands of cores compared to a CPU which usually has 1 to 6 cores
- Computations are done in parallel, asynchronously.
- Still relies on CPU for managing and passing data
- Programs must be rewritten to enable parallel processing

Differences CPU and GPU

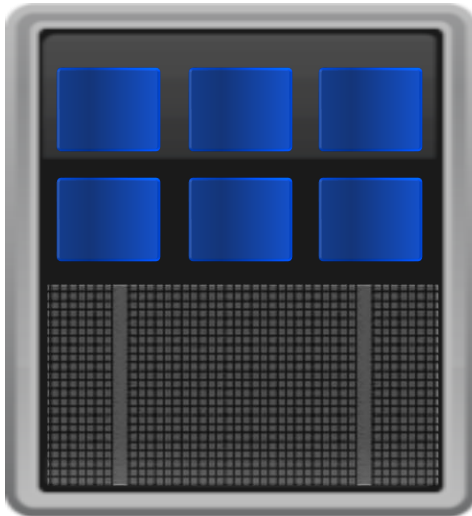
CPU	GPU
Several cores	Many cores
Designed to minimize latency	Designed to maximize throughput
Good for serial processing	Good for parallel processing
Multiple type of data	Single type of data
Can do a handful of operations at once	Can do thousands of operations at once



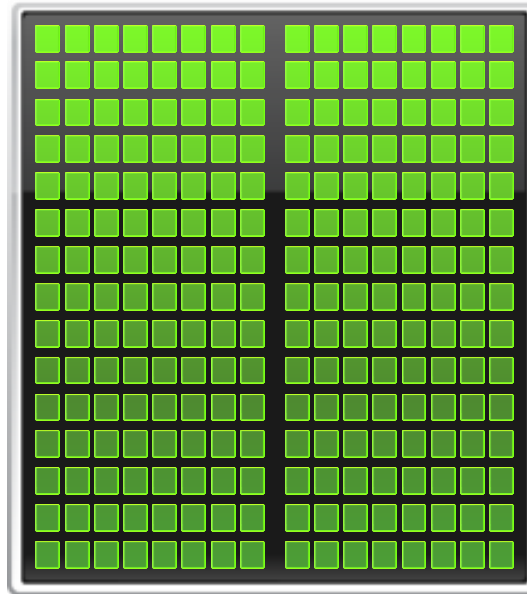
Accelerated Computing

10x Performance & 5x Energy Efficiency for HPC

CPU
Optimized for
Serial Tasks



GPU Accelerator
Optimized for
Parallel Tasks



CPU Strengths

- Very large main memory
- Very fast clock speeds
- Latency optimized via large caches
- Small number of threads can run very quickly

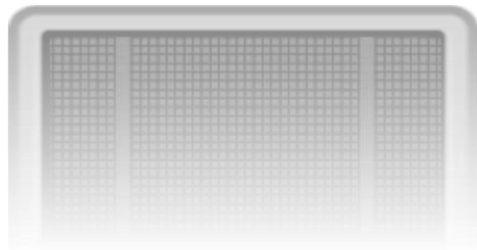
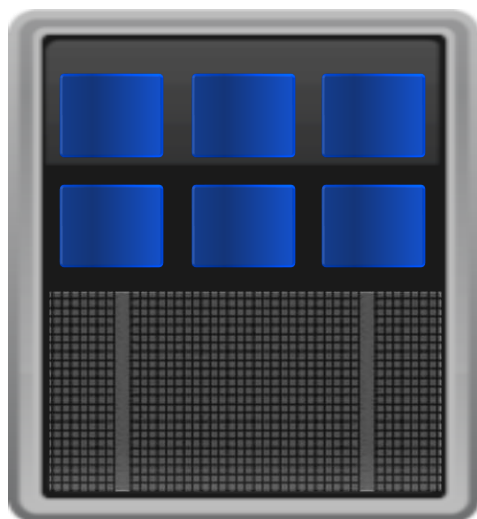
CPU Weaknesses

- Relatively low memory bandwidth
- Cache misses very costly
- Low performance/watt

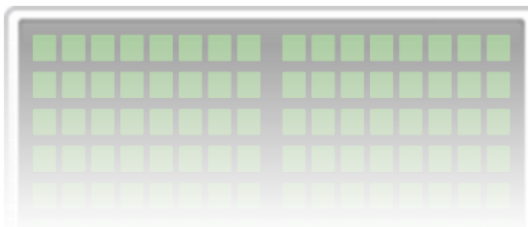
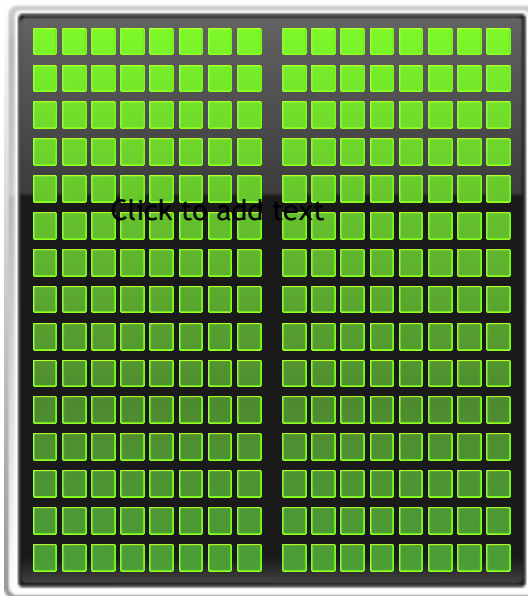
Accelerated Computing

10x Performance & 5x Energy Efficiency for HPC

CPU
Optimized for
Serial Tasks



GPU Accelerator
Optimized for
Parallel Tasks



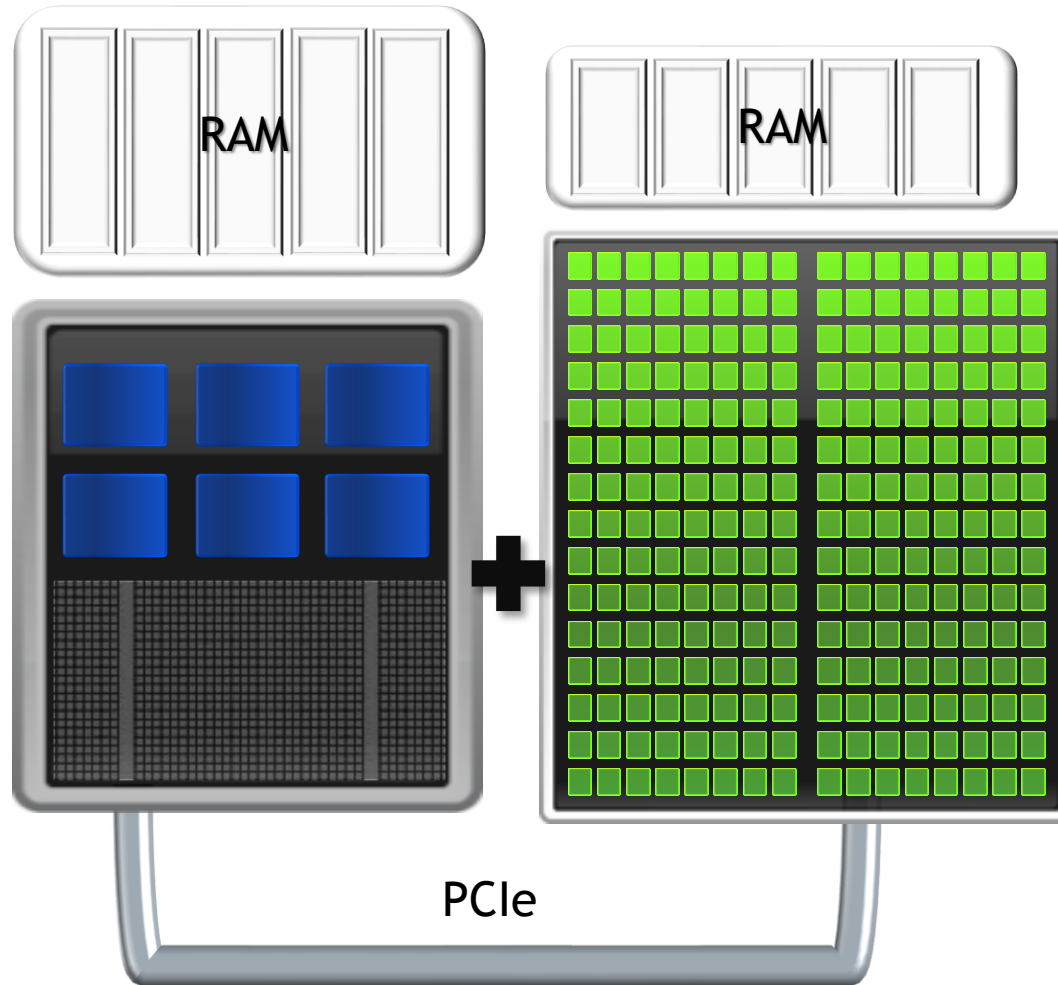
GPU Strengths

- High bandwidth main memory
- Significantly more compute resources
- Latency tolerant via parallelism
- High throughput
- High performance/watt

GPU Weaknesses

- Relatively low memory capacity
- Low per-thread performance

Accelerator Nodes



CPU and GPU have distinct memories

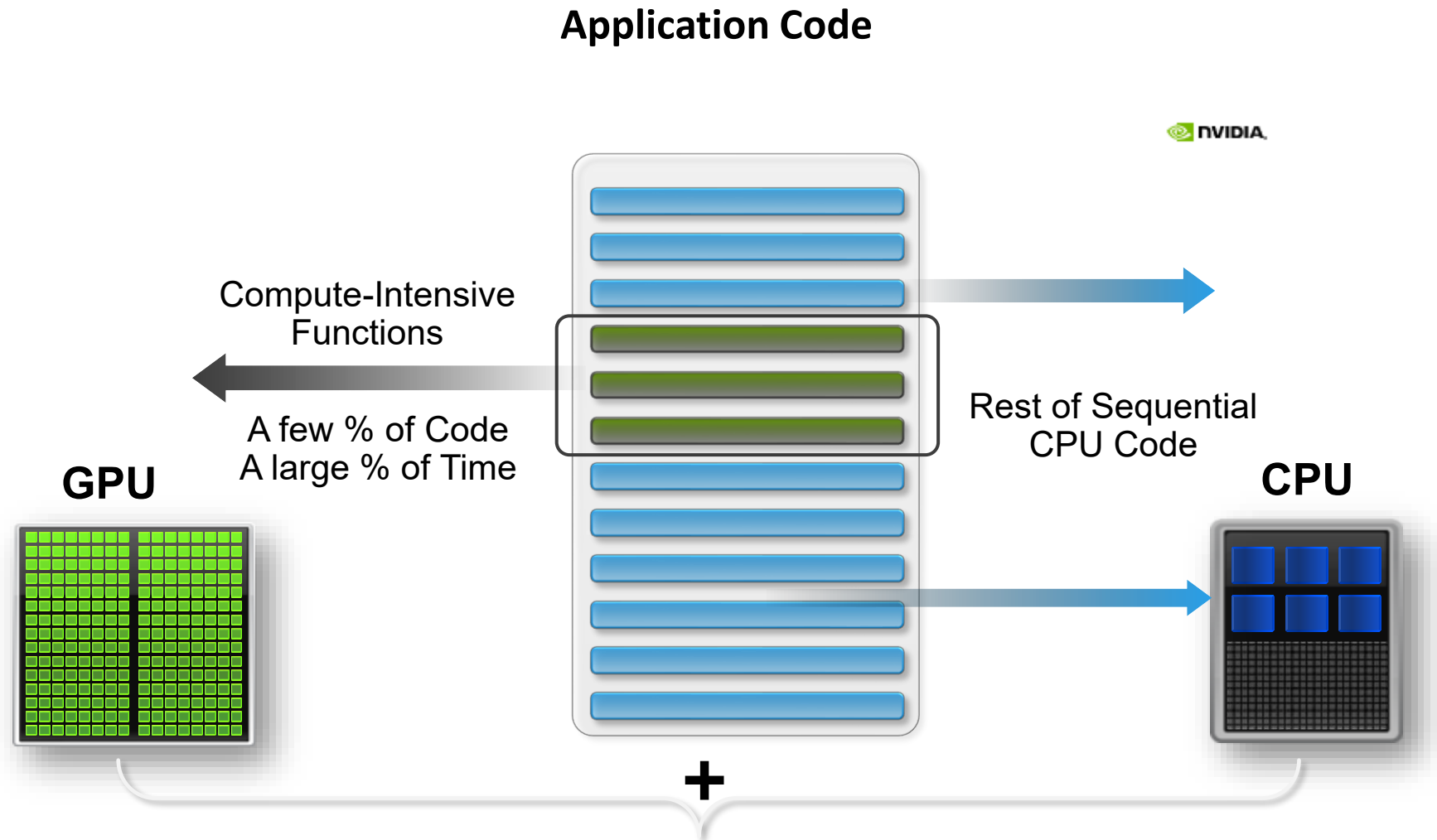
- CPU generally larger and slower
- GPU generally smaller and faster

CPU and GPU communicate via PCIe

- Data must be copied between these memories over PCIe
- PCIe Bandwidth is much lower than either memories

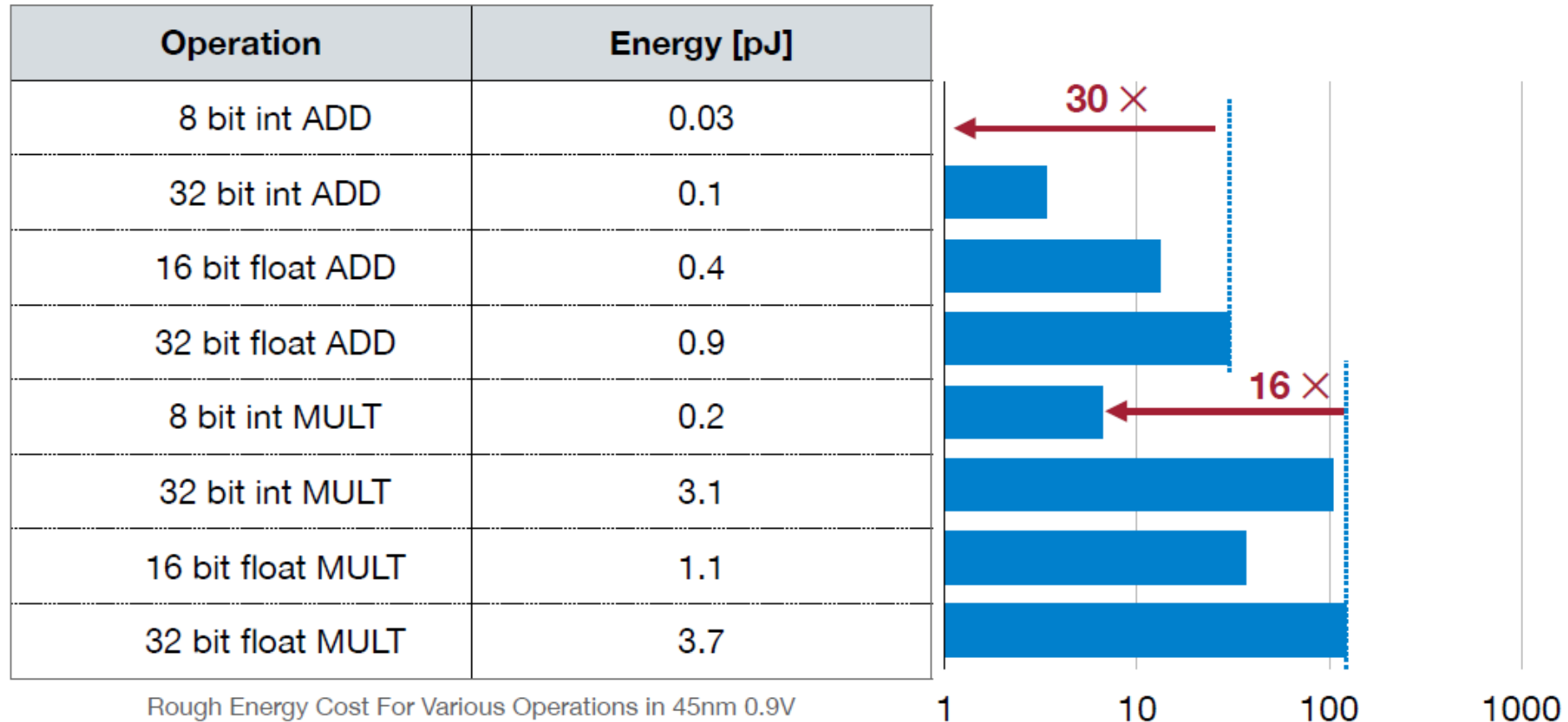
Emerging Tech - Nvlink

Heterogeneous Programming



Low Bit-Width Operations are Cheap

- Less Bit-Width → Less Energy



Data Types: Integer

- Unsigned Integer

- n-bit Range $[0, 2^n - 1]$

- Signed Integer

- Sign-Magnitude Representation

$$[-2^{n-1} + 1, 2^{n-1} - 1]$$

- n-bit Range: $[-2^{n-1} + 1, 2^{n-1} - 1]$

- Both 000...00 and 100...00 represent 0

- Two's Complement $[-2^{n-1}, 2^{n-1} - 1]$

- n-bit Range: $[-2^{n-1}, 2^{n-1} - 1]$

- 000...00 represents 2^{n-1}

- 100...00 represents -2^{n-1}

0	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

× × × × × × × ×

$$2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = 49$$

Sign Bit

1	0	1	1	0	0	0	1
---	---	---	---	---	---	---	---

× × × × × × × ×

$$-2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

1	1	0	0	1	1	1	1
---	---	---	---	---	---	---	---

× × × × × × × ×

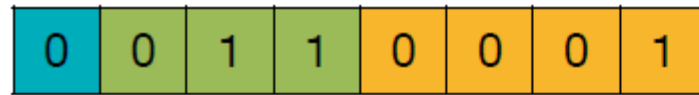
$$-2^7 + 2^6 + 2^5 + 2^4 + 2^3 + 2^2 + 2^1 + 2^0 = -49$$

Fixed-Point Number

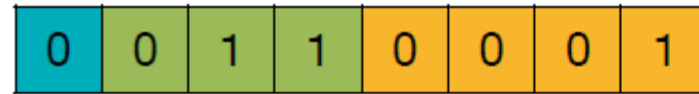


Integer . Fraction

“Decimal” Point



$$\begin{array}{cccccccc} \times & \times & \times & \times & \times & \times & \times & \times \\ -2^3 & +2^2 & +2^1 & +2^0 & +2^{-1} & +2^{-2} & +2^{-3} & +2^{-4} \end{array} = 3.0625$$

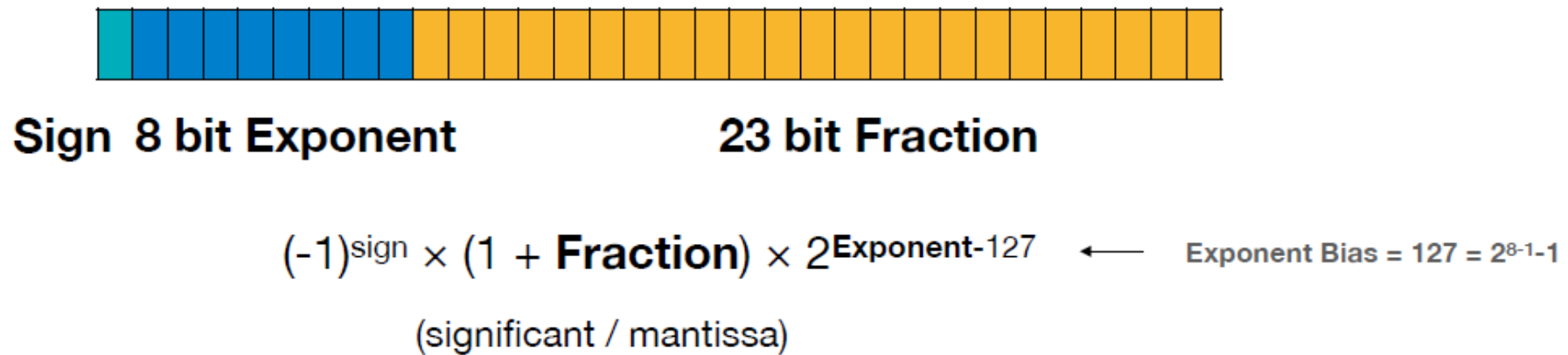


$$\begin{array}{cccccccc} \times & \times & \times & \times & \times & \times & \times & \times \\ (-2^7 & +2^6 & +2^5 & +2^4 & +2^3 & +2^2 & +2^1 & +2^0) \end{array} \times 2^{-4} = 49 \times 0.0625 = 3.0625$$

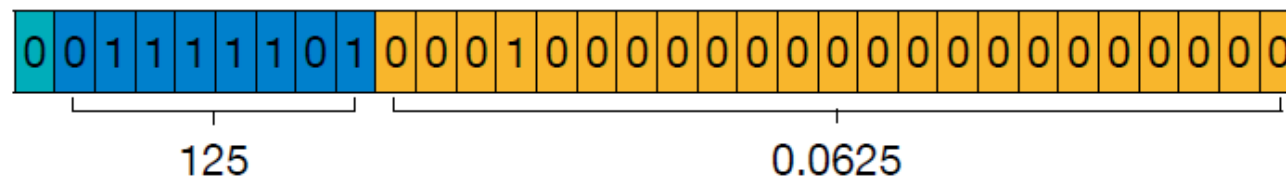
(using 2's complement representation)

Floating-Point Number

- **Example:** 32-bit floating-point number in IEEE 754



$$0.265625 = 1.0625 \times 2^{-2} = (1 + \underline{0.0625}) \times 2^{\underline{125}-127}$$



Floating-Point Number

- Exponent Width -> Range; Fraction Width -> Precision

IEEE 754 Single Precision 32-bit Float (IEEE FP32)



IEEE Half Precision 16-bit Float (IEEE FP16)



Brain Float (BF16)



Nvidia TensorFloat (TF32)



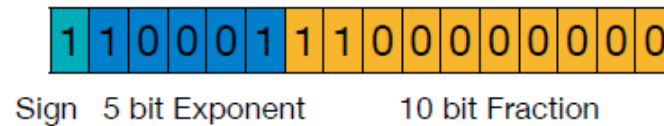
AMD 24-bit Float (AMD FP24)



Exponent (bits)	Fraction (bits)	Total (bits)
8	23	32
5	10	16
8	7	16
8	10	19
7	16	24

Numeric Data Types

- **Question:** What is the following IEEE half precision (IEEE FP16) number in decimal?



Exponent Bias = 15_{10}

- Sign: -
- Exponent: $10001_2 - 15_{10} = 17_{10} - 15_{10} = 2_{10}$
- Fraction: $1100000000_2 = 0.75_{10}$
- Decimal Answer = $-(1 + 0.75) \times 2^2 = -1.75 \times 2^2 = -7.0_{10}$

LLM Models & GPU VRAM: A Sizing Guide

- An analysis of VRAM requirements based on model size (parameters), precision, and context length.
- To run an LLM (for inference or training), all the necessary data must be loaded into the GPU's high-speed VRAM. If VRAM is insufficient, the model cannot run or must "offload" data to slower system RAM, dramatically reducing speed.
- **Key VRAM Consumers:**
 - **Model Weights:** The single largest component.
 - **Optimizer States:** (During full fine-tuning) Can be 2x or more the size of the weights (e.g., using AdamW).
 - **Activations & Gradients:** Intermediate calculations during the forward/backward pass.
 - **KV Cache:** (Crucial for Inference). This stores the state of the sequence. It grows linearly with the context size (sequence length) and batch size.

Model Precision

- The VRAM footprint is determined by the precision used to store each parameter.
- FP32 (Full Precision): 1 parameter = 4 bytes
 - e.g., Llama 3 8B (FP32) = 8B * 4 bytes $\approx$ 32 GB
- FP16 / BF16 (Half Precision): 1 parameter = 2 bytes (Today's Standard)
 - e.g., Llama 3 8B (FP16) = 8B * 2 bytes $\approx$ 16 GB
- INT8 (Quantized): 1 parameter = 1 byte
 - e.g., Llama 3 8B (INT8) = 8B * 1 byte $\approx$ 8 GB
- INT4 (e.g., QLoRA / GPT-Q): 1 parameter = 0.5 bytes (Heavily Quantized)
 - e.g., Llama 3 8B (INT4) $\approx$ 4 GB (+ $\sim$ 1-2GB overhead)
- Loading a model like Llama 3 8B in its standard FP16 precision requires at least 16GB of VRAM for the weights alone.

GPU Example : 16GB (e.g., Google Colab T4/L4, RTX 4060Ti 16GB)

- **Features:** Highly accessible, common in cloud instances (Google Colab) and consumer gaming cards.
- **Models** (e.g., Mistral 7B, Llama 3 8B):
 - **Inference (FP16):** Possible. ($8B * 2 \text{ bytes} \approx 16GB$). This is a perfect fit, but leaves little room for a long context (KV Cache).
 - **Full Fine-Tuning (FP16):** Impossible. ($16GB \text{ weights} + 32GB+ \text{optimizer} + \text{activations} > 16GB$).
 - **PEFT (LoRA / QLoRA):** Possible. Loading the model in 4-bit (QLoRA, $\approx 5-6GB$) leaves ample VRAM for fine-tuning.
- **Context Impact:** At 16GB, a long context (e.g., 32k tokens) during FP16 inference can easily cause an OOM (Out of Memory) error due to the large KV Cache.

VRAM for Full Fine-Tuning

- Training is the process of "updating the model's weights." It requires everything from inference, plus all the intermediate data needed to calculate the weight updates.
- **Model Weights:** (e.g., Llama 3 8B (FP16) $\approx$ **16GB**)
- **Gradients:**
 - These are the "instructions" for how much each weight needs to be updated.
 - They require VRAM equal to the size of the model weights. (e.g., **16GB**)
- **Optimizer States:**
 - **This is the main reason VRAM usage explodes.**
 - A standard AdamW optimizer requires **2x the size of the model weights**. (e.g., $16\text{GB} * 2 = \mathbf{32GB}$)
- **Activations:**
 - Unlike inference, for backpropagation, **the activations from *all layers* must be stored in VRAM** throughout the training step. This can be a significant amount, growing with sequence length.
- **Full-Tuning VRAM for 8B $\approx$ (Weights: 16GB) + (Gradients: 16GB) + (Optimizer States: 32GB) + (Activations: α) $\approx$ 64GB+**

Summary

- **CPU**, the computer's "brain," has a few powerful cores designed for serial tasks to minimize **latency** (the time for one operation), making it ideal for running the OS and sequential logic.
- **GPU** is a massively parallel co-processor with thousands of simpler cores designed to maximize **throughput** (total operations at once), which is essential for the matrix math in deep learning.
- In "Heterogeneous Programming," the CPU (host) runs the main application and offloads the small, compute-intensive parts of the code to the GPU (accelerator) for parallel execution.
- This is critical because LLMs rely on **Floating-Point Numbers**, and GPUs are optimized for these calculations; using lower precision (e.g., FP16 or INT8 instead of FP32) saves VRAM, energy, and time.
- **GPU VRAM** is the primary bottleneck for running LLMs, as the model's **Weights**, **Optimizer States** (during training), and **KV Cache** (during inference) must all fit into this high-speed memory.
- This has practical consequences: a Llama 3 8B model requires ~16GB of VRAM in standard FP16 precision, making it impossible to "full fine-tune" (which needs 64GB+) on a 16GB GPU, necessitating techniques like quantization (e.g., QLoRA).